MASTER THESIS · CONCEPT REFERENCE

Privacy on Sui
ZK Proofs, TEEs & Confidential Transactions

A PRINTABLE, CHAPTER-BY-CHAPTER REFERENCE FOR ANONYMOUS CREDENTIALS AND LOW-VALUE PRIVATE PAYMENTS ON SUI — EACH CONCEPT EXPLAINED TWICE, SIDE BY SIDE: AN INTUITIVE PICTURE AND THE TECHNICAL DETAIL, WITH SCHEMAS.

Alexandre Mourot · EPFL × Mysten Labs · generated from the study-plan corpus

Contents

Each concept is laid out in two columns — **Intuitive** (analogy, schema, key points, thesis relevance) on the left, **Technical** (formal definitions, equations, code/citations) on the right — followed by history, limitations, and exercises. Code/source citations point to MystenLabs repositories (e.g. confidential-transfers / contra @ 27c382f). Excludes Rust, Post-Quantum, audit, and the ZK-book modules.

CHAPTER 1

ZK Proof Systems & Crypto Fundamentals

Crypto Primitives + ZKP Fundamentals

Hash Functions

INTUITIVE

A fingerprint scanner for data. Just as every person has a unique fingerprint that cannot be reverse-engineered back into a person, a hash function produces a unique fixed-size digest for any input. Change a single bit and the entire fingerprint changes beyond recognition.

KEY POINTS

- Deterministic: same input always produces the same output
- One-way (preimage resistant): given H(x), finding x is computationally infeasible
- Collision resistant: finding two inputs with the same hash is infeasible
- Avalanche effect: flipping one input bit changes roughly half the output bits
- Used in Merkle trees, commitment schemes, digital signatures, and proof systems

WHY IT MATTERS

Hash functions are embedded in every layer of the thesis system. Sui uses SHA-256 and Blake2b for transaction hashing and Merkle tree construction. In anonymous credentials, hashes bind attributes inside Pedersen commitments and form the backbone of Fiat-Shamir transforms that make ZKPs non-interactive for on-chain verification.

THESIS ANGLE

In your thesis, Poseidon hash is used inside ZK circuits for the credential commitment tree on Sui. When a user's BBS+ credential is issued, a Poseidon hash of the credential commitment is inserted into an on-chain Merkle tree. During payment verification, the ZK circuit proves membership in this tree — Poseidon's ZK-friendly design keeps the circuit under 1000 constraints per hash, making on-chain verification affordable.

TECHNICAL

A cryptographic hash function is a deterministic, efficiently computable function $H : \{0,1\}^* \to \{0,1\}^n$ that maps arbitrary-length inputs to fixed-length n-bit outputs and satisfies three security properties parameterized by the security parameter λ.

PREIMAGE RESISTANCE

For a randomly sampled $y \in \{0,1\}^n$, no PPT adversary $\mathcal{A}$ can find x such that $H(x) = y$ with non-negligible probability:

$$\Pr\left[\mathcal{A}(y) = x \ : \ H(x) = y\right] \leq \text{negl}(\lambda)$$

This is the one-way property: the function is easy to compute in the forward direction but computationally infeasible to invert. Generic preimage attacks require $O(2^n)$ evaluations.

SECOND PREIMAGE RESISTANCE

Given x, no PPT adversary can find $x' \neq x$ such that $H(x) = H(x')$:

$$\Pr\left[\mathcal{A}(x) = x' \ : \ x' \neq x \wedge H(x) = H(x')\right] \leq \text{negl}(\lambda)$$

Second preimage resistance is strictly weaker than collision resistance: if you can find collisions, you can find second preimages, but not necessarily vice versa.

COLLISION RESISTANCE

No PPT adversary can find any pair (x, x') with $x \neq x'$ and $H(x) = H(x')$:

$$\Pr\left[\mathcal{A}() = (x, x') \ : \ x \neq x' \wedge H(x) = H(x')\right] \leq \text{negl}(\lambda)$$

By the birthday paradox, a generic collision search succeeds after $O(2^{n/2})$ evaluations. For $n = 256$, this gives a 2^{128} security bound.

BIRTHDAY BOUND ANALYSIS

Given q random queries to H, the probability of at least one collision is:

$$\Pr[\text{collision}] \approx 1 - e^{-q^2/(2 \cdot 2^n)}$$

Setting this to $1/2$ yields $q \approx 1.177 \cdot 2^{n/2}$. This birthday bound is tight for ideal hash functions and dictates the minimum output length: for 128-bit security we need $n \geq 256$.

MERKLE-DAMGARD CONSTRUCTION

Most classical hash functions (SHA-256, SHA-1) use the Merkle-Damgard paradigm: the input is padded and split into blocks $m_1, m_2, \ldots, m_\ell$. A compression function $f : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n$ is iterated:

$$h_0 = \text{IV}, \quad h_i = f(h_{i-1}, m_i), \quad H(m) = h_\ell$$

If f is collision resistant, then H is collision resistant. However, Merkle-Damgard is vulnerable to length-extension attacks (mitigated in SHA-3/Keccak via the sponge construction).

RANDOM ORACLE MODEL VS STANDARD MODEL

The Random Oracle Model (ROM) treats H as a truly random function: for each new query x, the output $H(x)$ is uniformly random and independent of all previous outputs. Security proofs in the ROM are stronger but rely on an idealization. The standard model proves security using only concrete properties (collision resistance, etc.) without idealizing H. The Fiat-Shamir transform is proven secure in the ROM; standard-model alternatives exist but are less efficient.

HISTORY Ralph Merkle (Stanford University) (1979). *Merkle's original hash tree patent (US 4,309,569) was filed in 1979 but not granted until 1982. He was 27 years old and the concept was considered too abstract to be useful. Today Merkle trees secure every major blockchain.*

LIMITATIONS

- SHA-256 requires ~27,000 R1CS constraints in a Groth16 circuit because its bitwise operations (rotations, XOR, modular additions) are expensive over arithmetic fields. Poseidon needs only ~300 constraints for equivalent security, making it 90x more ZK-friendly.
- Hash functions are one-way by construction but not hiding for small message spaces. H(m) for a known small domain can be brute-forced (e.g., hashing all 10^9 possible SSNs reveals the preimage in seconds).
- Collision resistance degrades with output length via the birthday bound: a 256-bit hash provides only 128-bit collision resistance (2^128 evaluations), which is marginal for post-quantum security where Grover's algorithm halves the preimage resistance to 128 bits.

EXERCISES

1. **calculation** — If a hash function has a 256-bit output, how many evaluations are needed on average to find a collision (birthday attack)? Express as a power of 2.
2. **conceptual** — Why does Sui use Poseidon hash inside ZK circuits (e.g., zkLogin) instead of SHA-256, even though SHA-256 is more widely studied?
3. **comparison** — Compare SHA-256, Poseidon, and Pedersen hash in terms of constraint cost, algebraic friendliness, and typical use cases in ZK systems.

Commitment Schemes

INTUITIVE

A sealed envelope placed inside a glass box. Everyone can see the envelope exists (the commitment), but nobody can read what is inside until you break the seal and open it (the reveal). Once sealed, you cannot swap the contents for something else.

KEY POINTS

- Hiding: the commitment C reveals nothing about the message m
- Binding: once committed, the committer cannot open to a different value
- Pedersen commitments are additively homomorphic: Commit(a) + Commit(b) = Commit(a+b)
- Randomness r is essential for hiding; without it, commitments become deterministic and breakable

WHY IT MATTERS

Pedersen commitments are the core primitive in BBS+ anonymous credentials. Each credential attribute is committed individually, allowing selective disclosure: the holder reveals only chosen attributes while keeping others hidden inside commitments. The homomorphic property enables efficient range proofs and attribute predicate proofs without revealing underlying values.

THESIS ANGLE

Your credential system uses Pedersen commitments to hide attribute values. When a user presents an age credential, C = g^age · h^r keeps the exact age private. The homomorphic property enables range proofs: prove age ≥ 18 without revealing the exact value. In the payment layer, Pedersen commitments hide transaction amounts while enabling balance verification: sum of input commitments = sum of output commitments.

TECHNICAL

A commitment scheme is a pair of PPT algorithms $(\mathrm{Com}, \mathrm{Open})$. The committer computes $c = \mathrm{Com}(m; r)$ for message m and randomness r, producing commitment c. Later, the committer reveals (m, r) and the verifier checks $\mathrm{Open}(c, m, r) \stackrel{?}{=} 1$. The scheme must satisfy hiding and binding.

HIDING PROPERTY

Computational hiding: for all PPT adversaries $\mathcal{A}$ and all m_0, m_1:

$$\{\mathrm{Com}(m_0; r) : r \leftarrow \mathcal{R}\} \approx_c \{\mathrm{Com}(m_1; r) : r \leftarrow \mathcal{R}\}$$

Perfect (information-theoretic) hiding means the distributions are identical, not just computationally indistinguishable. Pedersen commitments achieve perfect hiding because for any commitment c and any message m', there exists exactly one r' such that $c = g^{m'} h^{r'}$.

BINDING PROPERTY

Computational binding: no PPT adversary can find (m_0, r_0, m_1, r_1) with $m_0 \neq m_1$ and $\mathrm{Com}(m_0; r_0) = \mathrm{Com}(m_1; r_1)$:

$$\Pr\left[\mathrm{Com}(m_0; r_0) = \mathrm{Com}(m_1; r_1) \wedge m_0 \neq m_1\right] \leq \mathrm{negl}(\lambda)$$

For Pedersen commitments, binding relies on the discrete logarithm problem: if $g^{m_0} h^{r_0} = g^{m_1} h^{r_1}$, then $g^{m_0 - m_1} = h^{r_1 - r_0}$, and since $m_0 \neq m_1$ we get $\log_g h = (m_0 - m_1)(r_1 - r_0)^{-1}$, breaking DLP.

PEDERSEN COMMITMENT

Let $\mathbb{G}$ be a cyclic group of prime order p with generators g, h where $\log_g h$ is unknown. The Pedersen commitment to $m \in \mathbb{Z}_p$ with randomness $r \in \mathbb{Z}_p$ is:

$$C = g^m \cdot h^r$$

This is perfectly hiding (for any C and m', set $r' = (\log_g C - m') \cdot (\log_g h)^{-1}$) and computationally binding under DLP.

HOMOMORPHIC PROPERTY

Pedersen commitments are additively homomorphic. Given $C_1 = g^{m_1} h^{r_1}$ and $C_2 = g^{m_2} h^{r_2}$:

$$C_1 \cdot C_2 = g^{m_1 + m_2} \cdot h^{r_1 + r_2} = \mathrm{Com}(m_1 + m_2; r_1 + r_2)$$

This enables proving linear relations on committed values without opening them. For example, to prove $m_3 = m_1 + m_2$, show that $C_1 \cdot C_2 / C_3$ is a commitment to 0.

VECTOR PEDERSEN COMMITMENTS

For multi-attribute credentials, we commit to a vector $\mathbf{m} = (m_1, \ldots, m_k)$ using independent generators $g_1, \ldots, g_k, h$:

$$C = \prod_{i=1}^{k} g_i^{m_i} \cdot h^r = g_1^{m_1} \cdots g_k^{m_k} \cdot h^r$$

This generalizes the scalar Pedersen commitment and is the foundation for BBS+ credential commitments where each m_i is a credential attribute.

IMPOSSIBILITY: PERFECT HIDING + PERFECT BINDING

A fundamental result: no commitment scheme can be simultaneously perfectly hiding and perfectly binding. If perfectly hiding (all messages have identical commitment distributions), then for any commitment c there exist valid openings to different messages, so binding cannot be information-theoretic. The Pedersen commitment trades perfect hiding for computational binding; hash-based commitments ($c = H(m \| r)$) trade perfect binding for computational hiding.

HISTORY Torben Pedersen (Aarhus University) (1991). *Pedersen commitments are so fundamental that they appear in the specifications of Monero (RingCT), Mimblewimble/Grin, Zcash, and nearly every confidential transaction scheme. The original paper has over 3,000 citations.*

LIMITATIONS

- Computationally binding only: if a quantum computer solves the discrete log problem, an adversary could open a Pedersen commitment to any value they choose, completely breaking binding.
- Requires NOTHING-UP-MY-SLEEVE generation of generators g, h: if anyone knows the discrete log log_g(h), they can open any commitment to any value (breaking binding), since C = g^m * h^r = g^(m + r*log_g(h)) gives full freedom in choosing m' and r'.
- Not compact for large attribute sets: a credential with L attributes requires L separate Pedersen commitments (one per attribute), each needing its own randomness, which increases storage and communication overhead linearly.

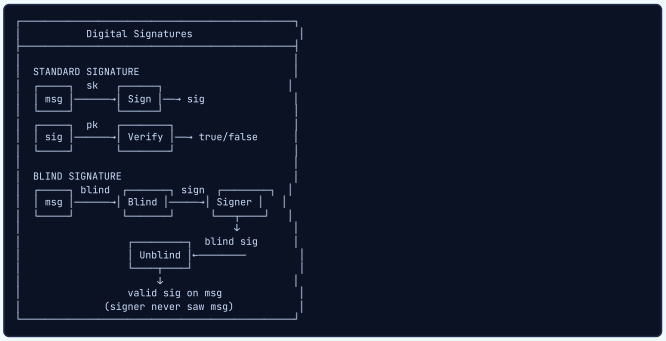
EXERCISES

1. **calculation** — On a toy group where operations are integers mod 11, with generators g=2 and h=7, compute the Pedersen commitment C = g^m * h^r (mod 11) for m=3 and r=4. Then compute a second commitment for m=5 and r=2, and verify that the product of the two commitments equals a commitment to m=8 and r=6.
2. **conceptual** — Why is randomness r essential for the hiding property of Pedersen commitments? What attack becomes possible if the committer uses r=0?
3. **design** — In the thesis credential system, how would you use Pedersen commitments to enable range proofs on an age attribute (prove age >= 18 without revealing exact age)? Outline the approach.

Digital Signatures

INTUITIVE

A wax seal on a medieval letter. Anyone can inspect the seal to verify it matches your signet ring (public key), but only you possess the ring (private key) to create it. Blind signatures are like signing a letter inside a carbon-paper envelope: the signer stamps the envelope, the signature transfers through, but the signer never sees the letter's content.



KEY POINTS

- Authentication: proves the signer's identity
- Non-repudiation: signer cannot deny having signed
- Integrity: any modification to the signed message invalidates the signature
- Blind signatures enable privacy: the issuer signs without seeing the content
- BBS+ signatures extend this to multi-message signing with selective disclosure

WHY IT MATTERS

Anonymous credentials rely on specialized signature schemes. BBS+ signatures allow an issuer to sign a vector of attributes, and the holder can later selectively disclose any subset. On Sui, credential issuance uses blind signing so the issuer never learns the full credential. Verification happens on-chain using pairing-based signature checks.

THESIS ANGLE

BBS+ signatures (a specialized multi-message signature scheme) are the core of your credential layer. The issuer signs the user's attribute vector (name, age, nationality, KYC level) with a single BBS+ signature. Blind issuance ensures the issuer doesn't learn all attributes. The user can later derive unlinkable proofs from this signature, showing only selected attributes to different verifiers on Sui.

TECHNICAL

A digital signature scheme is a triple of PPT algorithms $(\mathrm{KeyGen}, \mathrm{Sign}, \mathrm{Verify})$:

$$\mathrm{KeyGen}(1^\lambda) \to (sk, pk), \quad \mathrm{Sign}(sk, m) \to \sigma, \quad \mathrm{Verify}(pk, m, \sigma) \to \{0,1\}$$

with correctness: $\mathrm{Verify}(pk, m, \mathrm{Sign}(sk, m)) = 1$ for all m.

EUF-CMA SECURITY

Existential Unforgeability under Chosen Message Attack (EUF-CMA): the adversary has access to a signing oracle $\mathcal{O}_{\mathrm{Sign}}$ and must produce a valid signature on a message m^* never queried to the oracle:

$$\Pr\left[\mathrm{Verify}(pk, m^*, \sigma^*) = 1 \ : \ m^* \notin Q\right] \leq \mathrm{negl}(\lambda)$$

where Q is the set of queried messages. This is the standard security notion for digital signatures.

SCHNORR SIGNATURE

Setup: group $\mathbb{G}$ of prime order p, generator g, hash H. Key pair: $sk = x \xleftarrow{\$} \mathbb{Z}_p$, $pk = g^x$.

$$\mathbf{Sign}(sk, m): \ k \xleftarrow{\$} \mathbb{Z}_p, \ R = g^k, \ e = H(R \| pk \| m), \ s = k - e \cdot x \mod p. \text{ Output } \sigma = (R, s).$$

$$\mathbf{Verify}(pk, m, \sigma): \ e = H(R \| pk \| m), \ g^s \cdot pk^e \stackrel{?}{=} R$$

Correctness: $g^s \cdot pk^e = g^{k-ex} \cdot g^{ex} = g^k = R$. Security reduces to the DLP in the ROM (via the forking lemma).

BLS SIGNATURE

Uses a pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ and a hash-to-curve $H : \{0,1\}^* \to \mathbb{G}_1$. Key pair: $sk = x \in \mathbb{Z}_p$, $pk = [x]g_2 \in \mathbb{G}_2$.

$$\mathrm{Sign}(sk, m): \ \sigma = [x]H(m) \in \mathbb{G}_1$$

$$\mathrm{Verify}(pk, m, \sigma): \ e(\sigma, g_2) \stackrel{?}{=} e(H(m), pk)$$

Correctness: $e([x]H(m), g_2) = e(H(m), g_2)^x = e(H(m), [x]g_2)$. Security reduces to CDH in the ROM. Key advantage: signatures aggregate: $\sigma_{\mathrm{agg}} = \sum_i \sigma_i$, verified with a single pairing equation.

BLIND SIGNATURES (CHAUM)

RSA-based blind signatures: the signer has RSA key (n, e, d). The user blinds message m with random r:

$$m' = m \cdot r^e \mod n$$

The signer signs: $\sigma' = (m')^d \mod n$. The user unblinds: $\sigma = \sigma'/r = (m \cdot r^e)^d / r = m^d \cdot r - r = m^d \mod n$.

$$\sigma = \sigma' \cdot r^{-1} \mod n$$

The signer never sees m and cannot link σ to the blinding session. This is the foundation for unlinkable credential issuance: the issuer signs without learning the credential content.

MULTI-SIGNATURES AND AGGREGATE SIGNATURES

BLS aggregation: given n signatures $\sigma_i = [x_i]H(m_i)$, the aggregate is $\sigma_{\mathrm{agg}} = \sum_i \sigma_i \in \mathbb{G}_1$. Verification checks:

$$e(\sigma_{\mathrm{agg}}, g_2) \stackrel{?}{=} \prod_i e(H(m_i), pk_i)$$

Same-message multi-signatures (all sign the same m): $\sigma_{\mathrm{multi}} = \sum_i \sigma_i$, verify against $pk_{\mathrm{agg}} = \sum_i pk_i$:

$$e(\sigma_{\mathrm{multi}}, g_2) \stackrel{?}{=} e(H(m), pk_{\mathrm{agg}})$$

Aggregate signatures compress n signatures into one group element, saving bandwidth and on-chain storage.

HISTORY Whitfield Diffie & Martin Hellman (Stanford); RSA by Rivest, Shamir, Adleman (MIT) (1976). *Chaum's blind signature patent expired before its commercial potential was realized. His company DigiCash (1990) went bankrupt in 1998, but the blind signature concept became foundational for anonymous credentials and privacy-preserving identity 25 years later.*

LIMITATIONS

- Standard ECDSA signatures are linkable: the same public key produces verifiable signatures that can be correlated across different contexts, destroying user privacy. BBS+ signatures solve this with unlinkable derived proofs.
- Blind signatures require an interactive protocol between signer and user (at least 2 rounds), which adds latency and complexity to credential issuance. Threshold blind signing (t-of-n issuers) multiplies this to 2 rounds per signer.
- BBS+ signatures are pairing-based (BLS12-381), making them ~10x slower to verify than ECDSA. A single BBS+ verification takes ~5ms on a modern CPU due to pairing computations, vs ~0.5ms for Ed25519.

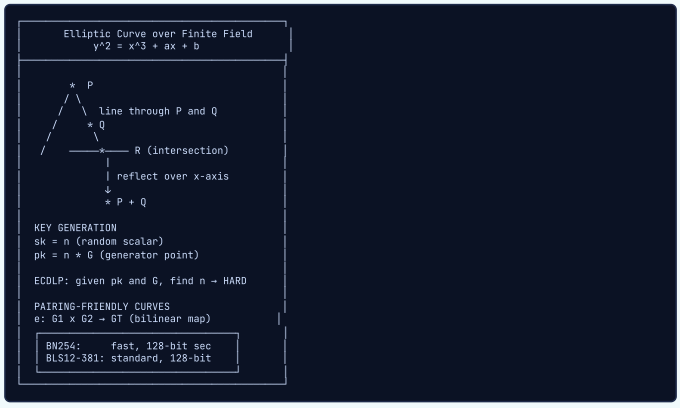
EXERCISES

1. **conceptual** — Explain the difference between a standard digital signature, a blind signature, and a BBS+ signature. When would you use each in an identity system?
2. **comparison** — Compare the verification cost of Ed25519, ECDSA (secp256k1), and BBS+ (BLS12-381) in terms of operations and approximate wall-clock time. Why does Sui use Ed25519 for transactions but the thesis uses BBS+ for credentials?
3. **design** — Design a blind issuance protocol for BBS+ credentials: how does the user get a BBS+ signature on their attributes without the issuer learning all attribute values?

Elliptic Curve Cryptography

INTUITIVE

Clock arithmetic on a curved surface. Imagine adding points on a curve by drawing lines between them: it is easy to compute P + P + ... + P = nP (scalar multiplication), but given only nP and P, finding n is astronomically hard. This one-way trapdoor is the foundation of modern public-key cryptography.



KEY POINTS

- The Elliptic Curve Discrete Log Problem (ECDLP) is the security foundation: given G and nG, finding n is infeasible
- Key pairs: private key is a scalar n, public key is the point nG on the curve
- Pairing-friendly curves (BN254, BLS12-381) enable bilinear maps e(P,Q) used in BBS+ and Groth16
- Bilinear pairings satisfy e(aP, bQ) = e(P, Q)^(ab), enabling signature aggregation and ZKP verification
- BLS12-381 is the current standard for ZKP-friendly applications due to its security and efficiency

WHY IT MATTERS

BBS+ signatures operate over pairing-friendly curves, specifically BLS12-381. The bilinear pairing enables the core selective disclosure mechanism: a verifier can check that hidden attributes are correctly signed without seeing them. On Sui, Groth16 proof verification uses BN254 pairings natively. Understanding EC math is essential for grasping how credentials are issued, held, and verified.

THESIS ANGLE

Your entire system runs on BLS12-381 — the same pairing-friendly curve used by Sui's cryptography module. BBS+ signatures require bilinear pairings e(A, $[x]_2$) for verification. Groth16 proofs for on-chain verification also use BLS12-381. This curve choice means credential operations and ZK verification share the same algebraic foundation, simplifying your Sui Move verifier contract.

TECHNICAL

An elliptic curve over a finite field $\mathbb{F}_p$ (with $p > 3$) is the set of points satisfying the short Weierstrass equation together with the point at infinity $\mathcal{O}$:

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}, \quad 4a^3 + 27b^2 \neq 0$$

The set $E(\mathbb{F}_p)$ forms an abelian group under the chord-and-tangent addition law with $\mathcal{O}$ as identity.

GROUP LAW (POINT ADDITION)

For $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ with $P \neq \pm Q$, their sum $R = P + Q = (x_3, y_3)$ is:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

For point doubling $P = Q$:

$$\lambda = \frac{3x_1^2 + a}{2y_1}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

Special cases: $P + \mathcal{O} = P$, $P + (-P) = \mathcal{O}$ where $-P = (x_1, -y_1)$.

SCALAR MULTIPLICATION AND ECDLP

Scalar multiplication: $[n]P = \underbrace{P + P + \cdots + P}_{n \text{ times}}$. Efficiently computed in $O(\log n)$ group operations via double-and-add.

$$\textbf{ECDLP}: \text{ Given } P, Q = [n]P \in E(\mathbb{F}_p), \text{ find } n \in \mathbb{Z}_r$$

Best known algorithms: Pollard's rho in $O(\sqrt{r})$ group operations, where $r = |E(\mathbb{F}_p)|$ is the group order. For $r \approx 2^{256}$, this gives $\approx 2^{128}$ security. No subexponential algorithms known for generic elliptic curves (unlike factoring or finite field DLP).

BILINEAR PAIRINGS

A bilinear pairing is a map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ where $\mathbb{G}_1, \mathbb{G}_2$ are elliptic curve subgroups of order r and $\mathbb{G}_T$ is a multiplicative subgroup of $\mathbb{F}_{p^k}^*$ (k is the embedding degree). Properties:

$$\textbf{Bilinearity}: \ e([a]P, [b]Q) = e(P, Q)^{ab}$$

$$\textbf{Non-degeneracy}: \ e(G_1, G_2) \neq 1_{\mathbb{G}_T} \text{ for generators } G_1, G_2$$

$$\textbf{Computability}: \ e \text{ can be computed efficiently (Miller's algorithm)}$$

Pairings enable "multiplication in the exponent": we can check $e([a]P, [b]Q) = e([c]P, [d]Q)$ iff $ab = cd$ without knowing a, b, c, d.

PAIRING-FRIENDLY CURVES

Not all elliptic curves support efficient pairings. The embedding degree k must be small enough for $\mathbb{F}_{p^k}$ arithmetic to be practical. Key curves:

$$\textbf{BN254}: \ k = 12, \ r \approx 2^{254}, \ \text{security} \approx 100\text{-bit}$$

$$\textbf{BLS12-381}: \ k = 12, \ r \approx 2^{255}, \ \text{security} \approx 128\text{-bit}$$

BN254 was originally estimated at 128-bit security but extended tower NFS attacks reduced this to ~100 bits. BLS12-381 provides a conservative 128-bit level. Both use embedding degree 12, meaning $\mathbb{G}_T \subset \mathbb{F}_{p^{12}}^*$.

SECURITY ASSUMPTIONS ON PAIRINGS

Key hardness assumptions in pairing groups:

$$\textbf{CDH}: \text{ Given } (g, [a]g, [b]g), \text{ compute } [ab]g$$

$$\textbf{DDH}: \text{ Distinguish } (g, [a]g, [b]g, [ab]g) \text{ from } (g, [a]g, [b]g, [c]g)$$

$$\textbf{DBDH}: \text{ Distinguish } e(g,g)^{abc} \text{ from } e(g,g)^z \text{ given } ([a]g, [b]g, [c]g)$$

Note: DDH is easy in pairing groups (use the pairing to check), so pairing-based schemes rely on CDH or stronger assumptions. The q-SDH assumption (used in Groth16) is: given $(g, [\tau]g, [\tau^2]g, \ldots, [\tau^q]g)$, find $(c, [1/(\tau + c)]g)$.

HISTORY Neal Koblitz (University of Washington) and Victor Miller (IBM Research) (1985). *When Koblitz first proposed ECC in 1985, the NSA was skeptical. By 2005, NSA endorsed Suite B with ECC as the preferred algorithm. By 2018, over 80% of TLS connections used ECDHE key exchange.*

LIMITATIONS

- Not quantum-safe: Shor's algorithm breaks ECDLP in polynomial time on a sufficiently large quantum computer. A 256-bit elliptic curve key would require ~2,300 logical qubits to break, which is expected within 10-20 years.
- Pairing computation is expensive: a single e(P,Q) on BLS12-381 takes ~1.5ms, compared to ~0.05ms for a scalar multiplication. Groth16 verification (3 pairings) costs ~4.5ms, which sets the floor for on-chain verification time.
- Curve choice creates ecosystem lock-in: BLS12-381 proofs cannot be verified in contracts designed for BN254 (different field arithmetic), and Sui's native groth16 module currently supports only BN254, BLS12-381, and BN254 alt_bn128.

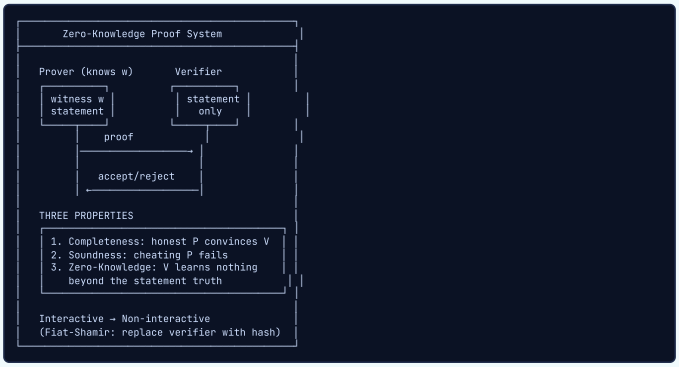
EXERCISES

1. **calculation** — On the tiny elliptic curve y^2 = x^3 + 7 over F_17 (secp256k1 equation, tiny field), the point G = (15, 13) has order 18. Compute 2G using the point doubling formula. The tangent slope is lambda = (3*x1^2 + a) / (2*y1) mod 17.
2. **conceptual** — What is the bilinear property of pairings, and why is it essential for BBS+ signature verification?

Zero-Knowledge Proofs (Fundamentals)

INTUITIVE

Proving you know the exit of a maze without showing the path. In Ali Baba's cave, two tunnels meet at a locked door inside. You enter one tunnel, the verifier shouts which tunnel to exit from. If you know the secret (the door's code), you always exit correctly. After many rounds, the verifier is convinced you know the secret without ever seeing the code.



KEY POINTS

- Completeness: an honest prover with a valid witness always convinces the verifier
- Soundness: a dishonest prover without a valid witness cannot convince the verifier (except with negligible probability)
- Zero-knowledge: the verifier learns nothing beyond the truth of the statement
- Fiat-Shamir heuristic transforms interactive proofs into non-interactive ones by replacing the verifier's challenge with a hash
- Non-interactive ZKPs (NIZKs) are essential for blockchain: no back-and-forth, just submit proof on-chain

WHY IT MATTERS

ZKPs are the heart of the thesis. Anonymous credential presentation is fundamentally a zero-knowledge proof: the holder proves they possess a valid credential with certain attributes without revealing the credential or other attributes. On Sui, these proofs must be non-interactive (Fiat-Shamir) so they can be verified in a single transaction without multi-round interaction.

THESIS ANGLE

ZKPs are the privacy backbone of your thesis. During credential presentation, a ZK proof convinces the Sui verifier that the user holds a valid BBS+ credential with certain attributes, without revealing the credential itself. During payment, another ZK proof shows the user has sufficient balance without revealing the amount. The proof is posted on-chain — anyone can verify, but no one learns private data.

TECHNICAL

An interactive proof system for a language L with NP relation R is a pair of interactive algorithms (P, V) where P is a PPT prover with access to a witness w and V is a PPT verifier. The system satisfies completeness, soundness, and zero-knowledge for the relation $R = \{(x, w) : x \in L\}$.

COMPLETENESS

If the statement is true and both parties are honest, the verifier always accepts:

$$\forall (x, w) \in R : \ \Pr\left[\langle P(x, w), V(x)\rangle = 1\right] = 1$$

Some definitions allow a negligible completeness error: $\geq 1 - \text{negl}(\lambda)$. For practical ZK systems, perfect completeness (probability exactly 1) is standard.

SOUNDNESS

No cheating prover P^* can convince the verifier to accept a false statement:

$$\forall P^*, \forall x \notin L : \ \Pr\left[\langle P^*(x), V(x)\rangle = 1\right] \leq \text{negl}(\lambda)$$

Computational soundness (argument): holds against PPT provers only. Statistical soundness (proof): holds against unbounded provers. SNARKs are arguments (computational soundness); STARKs can achieve statistical soundness.

ZERO-KNOWLEDGE PROPERTY

There exists a PPT simulator Sim that, given only the statement x (no witness), produces a transcript indistinguishable from a real interaction:

$$\text{View}_V\left[P(x, w) \leftrightarrow V(x)\right] \approx \text{Sim}(x)$$

Perfect ZK: distributions are identical. Statistical ZK: statistically close. Computational ZK: computationally indistinguishable. The simulator captures the intuition that the proof reveals nothing beyond the statement's truth.

HVZK VS MALICIOUS-VERIFIER ZK

Honest-Verifier ZK (HVZK): the simulator works only when the verifier follows the protocol (sends random challenges). Malicious-Verifier ZK: the simulator works for any verifier strategy V^*. HVZK is sufficient for many applications because the Fiat-Shamir transform replaces the verifier with a hash function, which is "honest" by construction. Converting HVZK to full ZK in the interactive setting requires additional techniques (e.g., coin-flipping protocols).

KNOWLEDGE SOUNDNESS AND EXTRACTORS

A proof of knowledge (PoK) has a stronger property: there exists a PPT extractor Ext that, given oracle access to any successful prover P^*, can extract the witness w:

$$\Pr\left[\langle P^*(x), V(x)\rangle = 1\right] > \epsilon \implies \Pr\left[\text{Ext}^{P^*}(x) = w : (x, w) \in R\right] > \epsilon - \text{negl}(\lambda)$$

For Sigma protocols, extraction uses the rewinding technique: run P^* to get transcript (a, e_1, z_1), then rewind and send different challenge e_2 to get (a, e_2, z_2). The witness is extracted from the two transcripts.

PROOF OF KNOWLEDGE VS PROOF OF MEMBERSHIP

Proof of membership: proves $x \in L$ (the statement is true). Proof of knowledge: proves the prover knows a witness w for x. These are distinct: a proof of membership for "$\exists w : f(w) = x$" shows that such w exists but does not guarantee the prover possesses it. In credential systems, we need proofs of knowledge: the holder must demonstrate they actually possess the credential secret key, not just that one exists.

FIAT-SHAMIR HEURISTIC

Transforms any public-coin interactive proof into a non-interactive one by replacing the verifier's random challenges with hash evaluations:

$$e = H(x \| a) \quad (\text{hash of statement and commitment})$$

The resulting NIZK proof is $\pi = (a, z)$ where z is computed using $e = H(x \| a)$. Security: in the Random Oracle Model, if the interactive protocol is HVZK with special soundness, then the Fiat-Shamir transform yields a secure NIZK proof of knowledge. Caveat: Fiat-Shamir is not generically secure in the standard model (known counterexamples by Goldwasser-Kalai).

HISTORY Shafi Goldwasser, Silvio Micali, Charles Rackoff (MIT/University of Toronto) (1985). *The original GMR paper was rejected from STOC 1984 because reviewers found the zero-knowledge concept 'too theoretical to be interesting.' It was accepted the following year and became one of the most influential papers in theoretical computer science.*

LIMITATIONS

- Interactive ZKPs require multiple rounds of communication between prover and verifier, making them impractical for blockchain where the verifier is a smart contract. The Fiat-Shamir transform fixes this but relies on the random oracle model (hash behaves as truly random), which is a heuristic assumption, not a proof.
- Computational vs statistical zero-knowledge: most practical ZK systems (Groth16, PLONK) achieve computational ZK, meaning a computationally bounded adversary cannot extract information. A quantum adversary might break computational ZK for pairing-based systems.
- Soundness is only guaranteed with overwhelming probability (1 - 1/2^k for k-bit challenge). With a 128-bit challenge, the probability of a cheating prover succeeding is 2^-128, but it is never exactly zero. Proof-of-knowledge extraction requires rewinding, which does not work in concurrent settings without careful protocol design.

EXERCISES

1. **conceptual** — Explain the Ali Baba cave analogy for zero-knowledge proofs. What does each element of the analogy (cave, tunnels, door, code) correspond to in the formal definition?
2. **comparison** — Compare interactive ZKPs, NIZKs (Fiat-Shamir), and ZK-SNARKs in terms of: number of rounds, proof size, verification time, and trust assumptions. Which is best for on-chain verification?
3. **design** — In the thesis, why must the ZKP for credential presentation be non-interactive? What would happen if you used an interactive protocol on Sui?

Sigma Protocols

INTUITIVE

A three-step dance between prover and verifier. Step 1: the prover commits to a random value (like showing a locked safe). Step 2: the verifier issues a random challenge (like asking to open a specific compartment). Step 3: the prover responds correctly (opening it). This three-move structure is the simplest form of interactive zero-knowledge proof.

KEY POINTS

- Three-move structure: commitment (a), challenge (e), response (z)
- Schnorr protocol: the canonical Sigma protocol for proving knowledge of a discrete logarithm
- Honest-verifier zero-knowledge (HVZK): secure when the verifier follows the protocol honestly
- AND/OR compositions allow proving compound statements without revealing which branch is satisfied
- Made non-interactive via Fiat-Shamir: e = H(a || statement) replaces the verifier's random challenge

WHY IT MATTERS

Sigma protocols are the proof mechanism inside BBS+ credential presentation. When a holder presents a credential on Sui, they run a composed Sigma protocol: an AND-proof that they know the issuer's signature and that the hidden attributes satisfy certain predicates. The Schnorr protocol proves knowledge of the secret key binding the credential to the holder, preventing credential transfer.

THESIS ANGLE

Your BBS+ selective disclosure uses Sigma protocols internally. When proving knowledge of hidden attributes, the prover runs a Schnorr-like protocol: commit to randomized values, receive a challenge (Fiat-Shamir), respond. The OR-composition of Sigma protocols could enable 'prove I'm from EU OR I have premium KYC' without revealing which condition is met — useful for flexible credential policies on Sui.

TECHNICAL

A Sigma protocol for relation R is a three-move interactive proof (a, e, z) between prover $P(x, w)$ and verifier $V(x)$: (1) P sends commitment a, (2) V sends random challenge $e \xleftarrow{\$} \mathcal{E}$, (3) P sends response z. The protocol satisfies special soundness and special HVZK.

SCHNORR IDENTIFICATION PROTOCOL

Relation: $R = \{(h, x) : h = g^x\}$. Prover knows x and wants to prove knowledge of $\log_g h$.

$$\textbf{Step 1 (Commitment)}: \ P \text{ picks } k \xleftarrow{\$} \mathbb{Z}_p, \text{ sends } a = g^k$$

$$\textbf{Step 2 (Challenge)}: \ V \text{ sends } e \xleftarrow{\$} \mathbb{Z}_p$$

$$\textbf{Step 3 (Response)}: \ P \text{ sends } z = k + e \cdot x \mod p$$

$$\textbf{Verification}: \ g^z \stackrel{?}{=} a \cdot h^e$$

Correctness: $g^z = g^{k+ex} = g^k \cdot (g^x)^e = a \cdot h^e$.

SPECIAL SOUNDNESS

Given two accepting transcripts with the same commitment but different challenges:

$$(a, e_1, z_1) \text{ and } (a, e_2, z_2) \text{ with } e_1 \neq e_2$$

From the verification equations:

$$g^{z_1} = a \cdot h^{e_1} \quad \text{and} \quad g^{z_2} = a \cdot h^{e_2}$$

Dividing: $g^{z_1 - z_2} = h^{e_1 - e_2}$, so:

$$x = \frac{z_1 - z_2}{e_1 - e_2} \mod p$$

This extraction formula is the key to proving knowledge soundness. The extractor rewinds the prover to obtain two transcripts with the same first message.

SPECIAL HONEST-VERIFIER ZERO-KNOWLEDGE

Simulator $\mathrm{Sim}(h)$: pick $e \xleftarrow{\$} \mathbb{Z}_p$ and $z \xleftarrow{\$} \mathbb{Z}_p$, then compute:

$$a = g^z \cdot h^{-e}$$

The transcript (a, e, z) is accepting ($g^z = a \cdot h^e$ by construction) and has the same distribution as real transcripts when e is uniform. This proves SHVZK: the simulator produces transcripts without knowing the witness x, just by choosing (e, z) first and computing a backward.

AND COMPOSITION

To prove knowledge of x_1 AND x_2 for $h_1 = g^{x_1}$ and $h_2 = g^{x_2}$: run both Sigma protocols in parallel with the same challenge e.

$$P \text{ sends } (a_1, a_2), \ V \text{ sends } e, \ P \text{ sends } (z_1, z_2)$$

$$\text{Verify: } g^{z_1} \stackrel{?}{=} a_1 \cdot h_1^e \text{ AND } g^{z_2} \stackrel{?}{=} a_2 \cdot h_2^e$$

Sharing the same challenge ensures the prover cannot cheat on either sub-protocol independently.

OR COMPOSITION (CRAMER-DAMGARD-SCHOENMAKERS)

Prove knowledge of x_1 OR x_2 without revealing which. Suppose the prover knows x_1 (WLOG):

$$P \text{ computes real commitment } a_1 = g^{k_1} \text{ and simulates } (a_2, e_2, z_2) \text{ for the unknown branch}$$

$$V \text{ sends challenge } e$$

$$P \text{ sets } e_1 = e - e_2 \mod p, \ z_1 = k_1 + e_1 \cdot x_1$$

$$\text{Verify: both sub-checks pass AND } e_1 + e_2 = e$$

The verifier cannot distinguish which branch was real and which was simulated. This is fundamental for anonymous credentials: prove membership in group A OR group B without revealing which.

EQUALITY OF DISCRETE LOGS

Prove $\log_g h_1 = \log_f h_2$ (same secret x used for two different bases g, f). Relation: $h_1 = g^x$ and $h_2 = f^x$.

$$P : k \xleftarrow{\$} \mathbb{Z}_p, \ a_1 = g^k, \ a_2 = f^k$$

$$V : e \xleftarrow{\$} \mathbb{Z}_p$$

$$P : z = k + ex \mod p$$

$$\text{Verify: } g^z \stackrel{?}{=} a_1 \cdot h_1^e \text{ AND } f^z \stackrel{?}{=} a_2 \cdot h_2^e$$

This is used in credential systems to prove that the same secret key is embedded in different commitments or across different presentations (preventing credential sharing).

HISTORY Claus-Peter Schnorr (Goethe University Frankfurt) (1989). *Schnorr patented his signature scheme (US Patent 4,995,082, filed 1989). This patent blocked adoption for years and prevented Schnorr signatures from being selected for DSS (NIST chose DSA instead as a workaround). The patent expired in 2008, and Schnorr signatures finally entered widespread use via Ed25519 (Bernstein, 2011).*

LIMITATIONS

- Honest-verifier zero-knowledge (HVZK) only: if the verifier deviates from the protocol (e.g., choosing challenges adaptively), security may break. Achieving full ZK against malicious verifiers requires additional techniques (e.g., commitment to the challenge).
- Nonce reuse is catastrophic: if the prover uses the same random k in two Schnorr proofs with different challenges e1 and e2, the secret x can be extracted as x = (z1 - z2) / (e1 - e2). This exact attack extracted the PlayStation 3 ECDSA private key in 2010.
- OR-composition leaks which branch is NOT satisfied to the prover (since the prover simulates one branch). This is fine when the prover is honest, but in some MPC settings it can be problematic. The simulator for the OR-proof also has a specific structure that limits composability with other protocols.

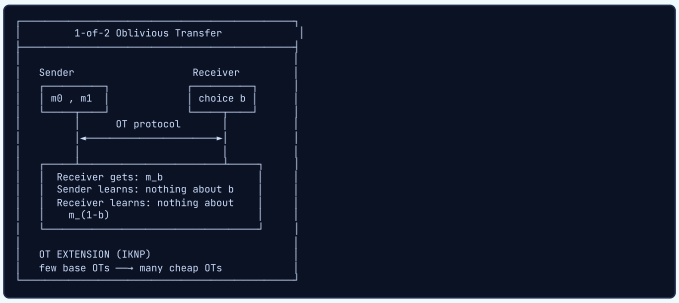
EXERCISES

1. **calculation** — In a Schnorr protocol over Z_23* with generator g=5, the prover's secret is x=7 (so pk = 5^7 mod 23 = 17). The prover picks random k=3, computes commitment a = g^k = 5^3 mod 23. The verifier sends challenge e=4. Compute the response z and verify the proof.
2. **conceptual** — Explain why nonce reuse in a Schnorr signature/proof is catastrophic. If a prover uses the same k for two different challenges e1 and e2, how does an attacker recover the secret x?
3. **design** — How would you use an OR-composition of Sigma protocols to prove 'I am an EU citizen OR I have premium KYC status' without revealing which condition is met?

Oblivious Transfer (OT)

INTUITIVE

A vending machine where the seller does not see what you picked, and you only get the item you chose — neither learns the other's secret. Like choosing a card from a deck face-down: dealer does not know which card you took, you do not see the other cards.



KEY POINTS

- Foundation of MPC — all MPC can be built from OT
- 1-of-2 OT: sender has 2 messages, receiver picks one without sender knowing which
- OT extension: few 'real' OTs produce many cheap OTs (IKNP protocol)
- Base OT from Diffie-Hellman assumption, extended OT is nearly free
- Variants: 1-of-n OT, k-of-n OT, string OT

WHY IT MATTERS

OT enables private credential issuance — the issuer can sign a credential without learning which attributes the user chose to include. Combined with blind signatures, OT is a building block for privacy-preserving identity systems.

THESIS ANGLE

OT could enable private credential issuance in your system. The issuer holds signing capabilities for multiple attribute types, and the user selects which attributes to include without the issuer learning the selection. This is especially relevant for selective attribute enrollment: a user might want to include their nationality but not their employer in a credential, without revealing this choice to the issuer.

TECHNICAL

1-of-2 OT: Sender S has (m_0, m_1), Receiver R has $b \in \{0,1\}$. After protocol: R learns m_b, S learns nothing about b, R learns nothing about m_{1-b}.

DIFFIE-HELLMAN OT (SIMPLEST PROTOCOL)

Sender picks a, publishes $A = g^a$. Receiver: if $b = 0$, $B = g^r$; if $b = 1$, $B = A \cdot g^r$. Sender computes $k_0 = B^a$, $k_1 = (B/A)^a$. Encrypt: $e_i = \text{Enc}(k_i, m_i)$. Receiver decrypts m_b with $k_b = A^r$.

OT EXTENSION (IKNP)

Base OTs: κ real OTs (128). Extend to n OTs using pseudorandom generator. Cost: $O(n)$ symmetric operations + κ base OTs. Core idea: transpose a random $n \times \kappa$ matrix, use correlation to derive keys.

SECURITY DEFINITIONS

Sender security: $\text{View}_S(m_0, m_1, b) \approx \text{View}_S(m_0, m_1, b')$ for any b, b'. Receiver security: $\text{View}_R(m_0, m_1, b) \approx \text{Sim}(m_b)$ — receiver only learns m_b.

VARIANTS

1-of-n OT (receiver picks 1 of n), k-of-n OT, random OT (random messages, used in preprocessing), string OT (messages are strings, not bits).

HISTORY Michael Rabin (Harvard University) (1981). *Joe Kilian proved in 1988 that OT is 'complete' for secure computation: any two-party computation can be built from OT alone. This means OT is the cryptographic equivalent of a universal logic gate.*

LIMITATIONS

- Base OT requires public-key operations (typically Diffie-Hellman), making the first ~128 OTs expensive (~1ms each). OT extension (IKNP) amortizes this but still requires those initial base OTs.
- OT is inherently interactive (at least 2 rounds for 1-of-2 OT), which is problematic for blockchain settings where the receiver might be a smart contract that cannot participate in real-time interaction.
- Malicious security for OT (protecting against cheating parties) roughly doubles the communication cost compared to semi-honest security. Protocols like ALSZ13 add consistency checks that increase computation by ~40%.

EXERCISES

1. **conceptual** — Explain the difference between Rabin's original OT (1981) and 1-of-2 OT (Even-Goldreich-Lempel, 1985). Why is 1-of-2 OT more useful for MPC?
2. **comparison** — OT extension (IKNP 2003) turns k base OTs into n >> k extended OTs. If base OT costs 1ms of public-key crypto each, and extended OT costs 1 microsecond of symmetric-key crypto each, what is the total time for 1 million OTs?

Multi-Party Computation (MPC)

INTUITIVE

Three millionaires want to know who is richest without revealing their exact wealth. MPC lets them compute the answer collaboratively — each inputs their secret, the protocol outputs only the final result, and no one learns anyone else's input.

KEY POINTS

- Compute any function on private inputs without a trusted third party
- Two main paradigms: garbled circuits (2-party, constant rounds) and secret sharing (n-party, interaction per gate)
- Semi-honest vs malicious security models
- Practical applications: threshold signatures, private auctions, salary benchmarking
- MPC-in-the-head: use MPC simulation for ZK proofs (Ligero, MPC-in-the-head paradigm)

WHY IT MATTERS

MPC can distribute trust in credential issuance — no single issuer sees all attributes. Threshold BBS+ signatures use MPC so multiple authorities jointly sign credentials. For your thesis, MPC could enable distributed TEE key management.

THESIS ANGLE

Threshold BBS+ issuance uses MPC: 3 out of 5 issuers jointly sign a credential, so no single issuer sees all attributes or holds the full signing key. This distributes trust in your identity system. MPC-in-the-head could also provide an alternative ZK proof system for credential verification, avoiding the trusted setup of Groth16 at the cost of larger proofs.

TECHNICAL

n parties $P_1, \ldots, P_n$ with inputs $x_1, \ldots, x_n$. Compute $y = f(x_1, \ldots, x_n)$. Security: $\exists \mathrm{Sim} : \mathrm{View}_{P_i}[\pi] \approx \mathrm{Sim}(x_i, y)$ — each party's view is simulatable from their input and output alone.

GMW PROTOCOL (SECRET SHARING BASED)

Additive sharing: $x = x^{(1)} + \cdots + x^{(n)}$. Addition: local (add shares). Multiplication: $[x \cdot y]$ requires interaction — use Beaver triples $[a], [b], [c = ab]$ preprocessed via OT.

YAO'S GARBLED CIRCUITS

For 2-party. Garbler creates garbled circuit $\tilde{C}$, sends to evaluator. Evaluator gets their input labels via OT. Evaluates circuit gate-by-gate. Constant rounds (3).

SPDZ PROTOCOL

Malicious security n-party. Preprocessing: generate Beaver triples with MAC $\alpha \cdot x + \beta$. Online phase: $O(n)$ communication per multiplication gate. MAC check at end detects cheating.

MPC-IN-THE-HEAD

Simulate n-party MPC locally, commit to each party's view, open subset for verification $\rightarrow$ ZK proof. Basis for Ligero, KKW, Banquet signature schemes.

THRESHOLD CRYPTOGRAPHY

(t, n)-threshold signing: t parties jointly sign, $< t$ learn nothing about secret key. Threshold BBS+: distributed credential issuance.

HISTORY Andrew Yao (Stanford University / Tsinghua University) (1982). *Yao received the Turing Award in 2000 for his foundational contributions to computation theory, including MPC. The Millionaires Problem he posed in 1982 is still the most common way MPC is explained to newcomers, 44 years later.*

LIMITATIONS

- Communication complexity scales at least linearly with the number of parties (O(n^2) for naive protocols). A 10-party GMW protocol sends O(n^2 * |C|) field elements where |C| is the circuit size, making large computations with many parties impractical.
- Malicious security is 3-20x more expensive than semi-honest security depending on the protocol. SPDZ achieves malicious security via MAC-based verification, adding ~3x overhead. Garbled circuits with cut-and-choose (Lindell 2013) add ~40x.
- MPC assumes network availability and synchrony for liveness. If enough parties go offline (more than n-t for a t-threshold protocol), the computation halts. This is problematic for blockchain-based MPC where node availability is not guaranteed.

EXERCISES

1. **conceptual** — What is the difference between semi-honest and malicious security in MPC? Why does the thesis care about this distinction for threshold BBS+ issuance?
2. **comparison** — Compare garbled circuits (Yao) and secret sharing (GMW/BGW) for MPC. Which approach is better for 2-party computation, and which for 5-party computation? Why?
3. **design** — Design a threshold BBS+ signing protocol for 3-of-5 issuers. What does each issuer contribute, and how is the final signature assembled without any single issuer seeing the full signing key?

Secret Sharing

INTUITIVE

A treasure map torn into 5 pieces — you need at least 3 pieces to find the treasure, but any 2 pieces reveal absolutely nothing. Shamir's scheme uses polynomial interpolation: the secret is the y-intercept of a random polynomial, and each share is a point on the curve.

KEY POINTS

- Shamir (t,n): secret is f(0), shares are f(1),...,f(n) on degree-(t-1) polynomial
- Additive sharing: secret = sum of shares (simpler, t=n only)
- Information-theoretic security: fewer than t shares reveal nothing (not even computationally)
- Foundation for MPC protocols (SPDZ, BGW)
- Verifiable secret sharing (VSS): detect cheating dealers

WHY IT MATTERS

Secret sharing enables threshold credential issuance — t-of-n issuers must cooperate to sign a credential, preventing single-point compromise. For TEE key management, secret sharing distributes the enclave sealing key across multiple TEE instances.

THESIS ANGLE

In your thesis architecture, the issuer's BBS+ signing key could be Shamir-shared among multiple TEE instances. If one TEE is compromised, the attacker gets one share — useless alone. Threshold reconstruction (e.g., 3-of-5) ensures the system remains operational even if 2 TEE instances fail, while keeping the full signing key safe from any single point of compromise.

TECHNICAL

(t, n)-secret sharing: dealer splits s into shares $(s_1, \ldots, s_n)$ s.t. any t shares reconstruct s, and any $< t$ shares reveal nothing about s.

SHAMIR'S SECRET SHARING

Choose random polynomial $f(x) = s + a_1 x + \cdots + a_{t-1} x^{t-1}$ over $\mathbb{F}_p$. Shares: $s_i = f(i)$ for $i = 1, \ldots, n$. Reconstruct via Lagrange interpolation: $s = f(0) = \sum_{i \in S} s_i \cdot \lambda_i$ where $\lambda_i = \prod_{j \in S, j \neq i} \frac{j}{j - i}$.

ADDITIVE SECRET SHARING

$s = s_1 + s_2 + \cdots + s_n \mod p$. Choose $s_1, \ldots, s_{n-1}$ randomly, set $s_n = s - \sum_{i=1}^{n-1} s_i$. Requires ALL shares (t=n). Addition is free, multiplication needs interaction.

VERIFIABLE SECRET SHARING (VSS)

Feldman VSS: dealer publishes $g^{a_0}, g^{a_1}, \ldots, g^{a_{t-1}}$. Each party verifies: $g^{s_i} = \prod_{j=0}^{t-1} (g^{a_j})^{i^j}$. Detects cheating dealer.

PROACTIVE SECRET SHARING

Periodically refresh shares without changing secret. Prevents mobile adversary (attacker who corrupts different parties over time).

HISTORY Adi Shamir (Weizmann Institute of Science) (1979). *Adi Shamir is the 'S' in RSA (Rivest-Shamir-Adleman). He published both RSA and secret sharing within a year of each other (1978-1979), establishing two pillars of modern cryptography before age 27.*

LIMITATIONS

- Shamir's scheme requires synchronous communication for reconstruction: all t shareholders must be online simultaneously to contribute their shares. Asynchronous secret sharing (Cachin et al., 2002) exists but adds significant complexity.
- Share refresh is necessary for long-lived secrets: if an adversary compromises shares over time (mobile adversary model), they might accumulate t shares from different epochs. Proactive secret sharing (Herzberg et al., 1995) addresses this but requires periodic resharing among all parties.
- Linear secret sharing schemes (including Shamir) reveal the secret to the reconstructing party. Threshold computation (MPC over shares) is needed if the reconstructed secret should never exist in cleartext — adding MPC overhead on top of the sharing scheme.

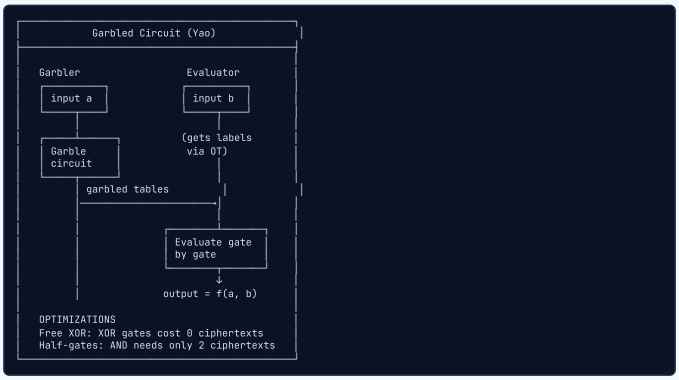
EXERCISES

1. **calculation** — Construct a (2,3) Shamir secret sharing of secret s=42 over Z_97. Choose random polynomial f(x) = 42 + 17x (mod 97). Compute shares f(1), f(2), f(3). Then reconstruct s from shares at x=1 and x=3 using Lagrange interpolation.
2. **conceptual** — What is the difference between Shamir's (t,n) threshold scheme and additive secret sharing? When would you use each?
3. **design** — In the thesis, the issuer's BBS+ signing key is Shamir-shared among 5 TEE instances with threshold 3. Describe a proactive resharing protocol that defends against a mobile adversary who compromises different TEEs over time.

Garbled Circuits

INTUITIVE

Imagine encrypting every wire in a circuit with a different random key, then shuffling (garbling) the truth tables so they work with encrypted values. One party garbles the circuit and sends it; the other evaluates it on their encrypted input. Neither sees the other's input.



KEY POINTS

- Yao's protocol: one party garbles, other evaluates (constant rounds, 2-party)
- Each wire has two random labels (0-label and 1-label)
- Garbled gate: encrypt each output label under the two input labels
- Point-and-permute optimization: 1 bit tells which row to decrypt
- Free XOR: XOR gates cost nothing (labels differ by global offset)
- Half-gates: AND gates need only 2 ciphertexts instead of 4

WHY IT MATTERS

Garbled circuits enable efficient 2-party computation for credential operations. A user and verifier can jointly evaluate a policy function on the user's private credential without revealing attributes to the verifier.

THESIS ANGLE

Garbled circuits could enable a privacy-preserving credential policy evaluation between user and verifier on Sui. The verifier garbles a policy circuit ('age ≥ 18 AND country ∈ EU AND risk_score < 5'), the user evaluates it on their private credential attributes via OT, and both learn only the boolean result. This keeps the full policy logic private to the verifier and attributes private to the user.

TECHNICAL

Garbler G creates $\tilde{C}$, a garbled version of circuit C. Evaluator E evaluates $\tilde{C}$ on garbled inputs to get garbled output. G learns nothing about E's input; E learns nothing beyond $C(x, y)$.

WIRE LABELS

Each wire w has two labels $(W_w^0, W_w^1) \in \{0,1\}^\kappa$. Label W_w^b represents bit b on wire w.

GARBLED GATE

For AND gate with input wires a, b and output wire c:

$$\tilde{g} = \left\{\mathrm{Enc}_{W_a^i, W_b^j}(W_c^{i \wedge j})\right\}_{i,j \in \{0,1\}}$$

Four ciphertexts, randomly permuted (point-and-permute: add select bits to labels).

FREE XOR OPTIMIZATION

Choose global $\Delta \in \{0,1\}^\kappa$. Set $W_w^1 = W_w^0 \oplus \Delta$ for all wires. XOR gates: $W_c^0 = W_a^0 \oplus W_b^0$ — no encryption needed, completely free.

HALF-GATES

AND gate needs only 2 ciphertexts (instead of 4). Garbler half-gate + evaluator half-gate combined. Best known optimization.

INPUT HANDLING

Garbler sends their garbled input directly. Evaluator gets their garbled input via OT: for each input bit, OT transfers the correct label.

HISTORY Andrew Yao (Stanford University) (1986). *The term 'garbled circuit' was actually coined years after Yao's paper. Yao never used the word 'garbled' in his original publication. Beaver, Micali, and Rogaway (1990) introduced the term in their formalization of Yao's protocol.*

LIMITATIONS

- Garbled circuits are one-time use: each garbled circuit can only be evaluated once securely (reuse reveals wire label correlations). For repeated evaluations, new garbled circuits must be generated, making the amortized cost high.
- Communication cost is O(|C| * kappa) where |C| is the circuit size and kappa is the security parameter (~128 bits). For a circuit with 1 million AND gates, this is ~32MB of garbled tables that must be transmitted.
- Garbled circuits are 2-party only. Extending to n > 2 parties requires BMR (Beaver-Micali-Rogaway) protocol, which adds an O(n) factor to the garbled table size and requires interaction during garbling.

EXERCISES

1. **calculation** — A garbled AND gate with point-and-permute has 4 rows in the garbled table. With the half-gates optimization, it has 2 rows. If a circuit has 500,000 AND gates and 300,000 XOR gates (free), and each row is 128 bits (16 bytes), what is the total garbled circuit size with: (a) basic garbling, (b) half-gates?
2. **conceptual** — Why are garbled circuits 'one-time use' and what goes wrong if you reuse the same garbled circuit with different inputs?

Fully Homomorphic Encryption (FHE)

INTUITIVE

A magical glove box — you put your encrypted data inside, perform operations on it while it is still encrypted, and take out an encrypted result. Decrypt to get the answer. The box (server) never sees your data, but processes it correctly.

KEY POINTS

- Compute arbitrary functions on encrypted data without decryption
- Leveled FHE: supports bounded-depth circuits. Fully HE: bootstrapping refreshes noise for unlimited depth
- Lattice-based (LWE/RLWE): quantum-resistant
- Key schemes: BFV (integer arithmetic), CKKS (approximate real numbers), TFHE (boolean circuits, fast bootstrapping)
- Current state: approximately 10,000x overhead vs plaintext, improving rapidly

WHY IT MATTERS

FHE could enable private credential verification without TEE — the verifier encrypts the policy, user evaluates it homomorphically on their credential. Too slow today for real-time use, but a long-term alternative to TEE-based privacy.

THESIS ANGLE

FHE represents a future alternative to TEEs in your architecture. Instead of running credential verification inside an SGX enclave, a Sui node could verify BBS+ proofs homomorphically on encrypted credentials. Currently too slow (~10,000x overhead), but lattice-based FHE is quantum-safe — if TEE trust assumptions weaken and FHE performance improves, it could replace the TEE layer in your post-quantum migration strategy.

TECHNICAL

$(\mathrm{KeyGen}, \mathrm{Enc}, \mathrm{Dec}, \mathrm{Eval})$ where $\mathrm{Eval}(\mathrm{pk}, f, \mathrm{Enc}(m)) = \mathrm{Enc}(f(m))$ for any function f, without decrypting.

LEARNING WITH ERRORS (LWE)

Public key: $(A, b = As + e)$ where $A \in \mathbb{Z}_q^{n \times m}$, $s \in \mathbb{Z}_q^n$ secret, e small error. Encrypt: $(c_1, c_2) = (rA,\ rb + \lfloor q/2 \rfloor \cdot m)$. Decrypt: $m = \lfloor (c_2 - c_1 \cdot s)/(q/2) \rceil$.

NOISE GROWTH

Each homomorphic operation increases noise. Addition: $\|e_{\mathrm{add}}\| \leq \|e_1\| + \|e_2\|$. Multiplication: $\|e_{\mathrm{mult}}\| \approx \|e_1\| \cdot \|e_2\|$. After too many multiplications, noise exceeds threshold $\rightarrow$ decryption fails.

BOOTSTRAPPING (GENTRY)

Homomorphically evaluate the decryption circuit to refresh noise. $\mathrm{Eval}(\mathrm{pk}, \mathrm{Dec}, \mathrm{Enc}(\mathrm{Enc}(m))) = \mathrm{Enc}(m)$ with fresh noise. Enables unlimited depth but expensive (~100ms per bootstrap).

KEY SCHEMES

BFV: integer arithmetic, batching via CRT. BGV: modulus switching for noise management. CKKS: approximate computation on real/complex numbers (ML-friendly). TFHE: boolean gates, fast bootstrapping (~10ms), used for boolean circuits.

SIMD BATCHING

Pack n plaintexts into one ciphertext using CRT/NTT. Single operation processes all n values. Amortized cost $O(1)$ per plaintext element.

HISTORY Craig Gentry (IBM Research / Stanford PhD) (2009). *Gentry's PhD thesis was titled 'A Fully Homomorphic Encryption Scheme' and ran to 209 pages. His advisor was Dan Boneh. When he presented the result, the cryptography community was stunned because most researchers believed practical FHE was decades away.*

LIMITATIONS

- Performance overhead is still 10,000-1,000,000x compared to plaintext computation, depending on the operation. A single bootstrapping operation (noise refresh) takes 10-100ms on modern hardware. TFHE achieves ~13ms per gate-level bootstrap.
- Ciphertext expansion: encrypting 1 bit produces ~4KB-32KB of ciphertext depending on the scheme and parameters. This makes FHE impractical for large datasets or bandwidth-constrained settings like on-chain storage.
- FHE supports only computation (addition and multiplication). Branching on encrypted data (if-then-else) requires evaluating both branches and selecting the result homomorphically, doubling the cost for each conditional. Complex control flow becomes exponentially expensive.

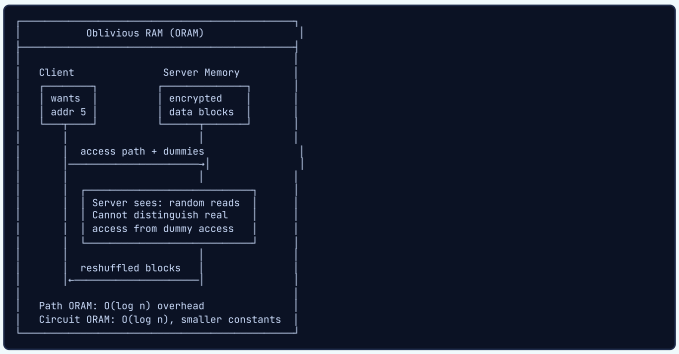
EXERCISES

1. **conceptual** — Explain why 'bootstrapping' was Gentry's key insight for fully homomorphic encryption. What happens to a ciphertext without bootstrapping after many operations?
2. **comparison** — Compare BFV, CKKS, and TFHE schemes. For each, state the data type it operates on, a typical use case, and the main tradeoff.
3. **design** — If FHE performance improves 1000x (reaching ~10x plaintext overhead), could it replace TEEs in the thesis architecture? What would the credential verification flow look like?

Oblivious RAM (ORAM)

INTUITIVE

A librarian who fetches books for you, but shuffles the entire library after every request so that an observer cannot tell which book you asked for — even if they watch every shelf access. Expensive (read the whole library each time) but perfectly hides your access pattern.



KEY POINTS

- Hides memory access patterns from the server/observer
- Path ORAM: store data in a binary tree, access a random path each time, O(log n) overhead
- Circuit ORAM: O(log n) with smaller constants
- Critical for TEE: even if enclave memory is encrypted, access patterns leak information
- Practical overhead: 10-100x, often prohibitive

WHY IT MATTERS

ORAM is essential for TEE security — without it, memory access patterns in SGX enclaves leak credential attributes during verification. Your thesis may need ORAM or ORAM-like techniques when credentials are stored and accessed inside TEEs.

THESIS ANGLE

In your TEE layer, the enclave processes credential verification requests. Without ORAM, an OS-level attacker observing memory access patterns during BBS+ verification could infer which credential attributes are being checked (different attributes cause different memory accesses). Path ORAM inside the SGX enclave hides these patterns at ~10x overhead, protecting attribute privacy even against side-channel adversaries.

TECHNICAL

Client accesses server memory such that access pattern $\mathrm{AP}(\vec{x})$ is computationally indistinguishable from $\mathrm{AP}(\vec{y})$ for any two request sequences $\vec{x}, \vec{y}$ of same length.

PATH ORAM

Server stores N blocks in a binary tree of depth $L = \lceil \log N \rceil$. Each block mapped to a random leaf. Access: read entire path from root to leaf, find block, re-encrypt and shuffle stash, write back. Bandwidth: $O(L \cdot B) = O(B \log N)$ per access where B is block size.

POSITION MAP

Maps block ID $\rightarrow$ leaf. Stored recursively in smaller ORAMs. $O(\log N)$ levels of recursion, each with N/B entries.

STASH MANAGEMENT

Client-side buffer holding blocks that don't fit on their path. Stash size: $O(\log N)$ with high probability. If stash overflows $\rightarrow$ security breach.

CIRCUIT ORAM

Improves over Path ORAM with reverse-lexicographic eviction. Amortized $O(\log N)$ with smaller stash. Better for sequential access patterns.

TEE + ORAM

TEE encrypts data but access patterns visible on memory bus. ORAM inside TEE hides access patterns. ZeroTrace: Path ORAM inside SGX enclave. Overhead: ~10x for TEE + ORAM vs ~100x for pure ORAM.

HISTORY Oded Goldreich and Rafail Ostrovsky (Technion / UCLA) (1996). *Goldreich and Ostrovsky's lower bound proof shows that any ORAM scheme must have Omega(log n) overhead, meaning Path ORAM's O(log n) is asymptotically optimal. No future ORAM scheme can beat this logarithmic barrier.*

LIMITATIONS

- Practical overhead of 10-100x makes ORAM prohibitive for performance-critical applications. Path ORAM with 1 million blocks requires accessing ~20 blocks per real access (log2(10^6) = 20), and each block access involves re-encryption and reshuffling.
- Client-side storage for the position map grows as O(n * log(n) / B) where B is the block size. For 1GB of server data with 4KB blocks, the position map is ~5MB, which must fit in trusted memory (e.g., SGX enclave's limited EPC of 128MB).
- ORAM protects access patterns but not access timing or volume. An adversary can still observe when accesses happen and how many accesses occur. Hiding timing requires constant-rate access (dummy accesses when idle), further increasing overhead.

EXERCISES

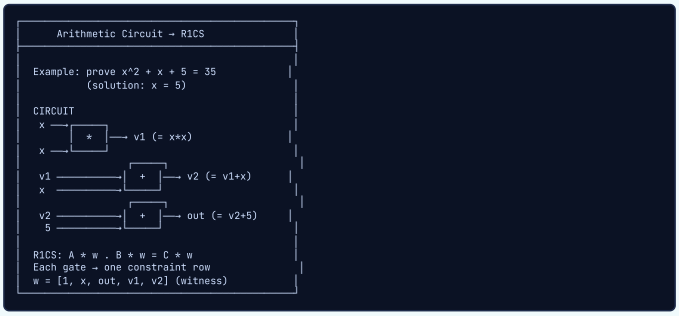
1. **calculation** — In Path ORAM with n = 2^20 blocks (approximately 1 million), the tree has depth log2(n) = 20. Each bucket holds Z = 4 blocks and each block is 4KB. What is the bandwidth overhead per real access (read one path + write one path)?
2. **conceptual** — Why is ORAM specifically important for TEE-based credential verification? What information leaks through memory access patterns in an SGX enclave, even though the data is encrypted?
3. **design** — Given that full ORAM has 100x overhead, propose a practical middle ground for the thesis TEE layer that protects against the most likely access pattern attacks without full ORAM cost.

Proving Systems (SNARKs, STARKs, PLONK)

Arithmetic Circuits & R1CS

INTUITIVE

Turning a math problem into a circuit board. Each gate computes one operation (add or multiply), wires carry values between gates, and the entire circuit proves the computation is correct. If any wire carries the wrong value, the circuit fails. R1CS is the standard blueprint format for describing these circuits.



KEY POINTS

- Arithmetic circuits express computations as DAGs of addition and multiplication gates over a finite field
- R1CS (Rank-1 Constraint System): each gate becomes a constraint A_i * w . B_i * w = C_i * w
- The witness w contains all intermediate values plus inputs and outputs
- Flattening: complex expressions are broken into elementary gate operations
- R1CS is the input format for Groth16; PLONK uses a different arithmetization (PLONKish)

WHY IT MATTERS

Every ZKP about anonymous credentials must first be expressed as an arithmetic circuit. The credential verification logic (pairing checks, attribute predicates, range proofs) is flattened into R1CS constraints. The circuit size directly determines proving time and on-chain verification cost on Sui.

THESIS ANGLE

Your credential verification circuit in R1CS encodes: 'I know a BBS+ signature σ on attributes (a1,...,aL) where the signature verifies against issuer public key ipk, the credential is in the Merkle tree with root R, the nullifier is correctly derived, and attributes satisfy the disclosure policy.' This circuit has ~100K constraints for a 10-attribute credential — the R1CS representation is what Groth16 actually proves.

TECHNICAL

An arithmetic circuit C over a finite field $\mathbb{F}$ is a directed acyclic graph where each internal node is a gate ($+$ or $\times$ over $\mathbb{F}$) and edges carry field elements. The circuit computes $C : \mathbb{F}^n \to \mathbb{F}$. R1CS (Rank-1 Constraint System) represents the circuit as:

$$(A \cdot \mathbf{z}) \circ (B \cdot \mathbf{z}) = C \cdot \mathbf{z}$$

where $A, B, C \in \mathbb{F}^{m \times n}$ are constraint matrices, $\mathbf{z}$ is the extended witness vector, and $\circ$ denotes the Hadamard (element-wise) product.

FLATTENING TO ELEMENTARY GATES

Any polynomial computation is decomposed into a sequence of elementary operations. Example: $y = x^3 + x + 5$ becomes:

$$v_1 = x \cdot x \quad \text{(gate 1: squaring)}$$

$$v_2 = v_1 \cdot x \quad \text{(gate 2: cubing)}$$

$$v_3 = v_2 + x \quad \text{(gate 3: addition)}$$

$$y = v_3 + 5 \quad \text{(gate 4: add constant)}$$

Each multiplication gate becomes one R1CS constraint. Addition gates are free (absorbed into the linear combinations). The total number of constraints equals the number of multiplication gates.

WITNESS VECTOR

The extended witness vector $\mathbf{z} = (1, x_1, \ldots, x_\ell, w_1, \ldots, w_m)$ contains:

$$\text{Constant } 1, \quad \text{public inputs } x_i, \quad \text{private witness values } w_j$$

The public inputs are known to both prover and verifier. The private witness values are known only to the prover. The verifier checks the constraints are satisfied using the public inputs and the proof (without seeing the private witness).

R1CS CONSTRAINT FORMAT

Each constraint (corresponding to one multiplication gate) has the form:

$$\langle \mathbf{a}_i, \mathbf{z} \rangle \cdot \langle \mathbf{b}_i, \mathbf{z} \rangle = \langle \mathbf{c}_i, \mathbf{z} \rangle$$

where $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i$ are row vectors of A, B, C. Each inner product $\langle \mathbf{a}_i, \mathbf{z} \rangle$ computes a linear combination of witness values. The constraint says: (linear combo 1) $\times$ (linear combo 2) $=$ (linear combo 3).

QAP (QUADRATIC ARITHMETIC PROGRAM)

R1CS is converted to QAP by polynomial interpolation. For each row index j of the witness, define polynomials $A_j(x), B_j(x), C_j(x)$ such that $A_j(\omega^i) = A_{i,j}$ for constraint i. The QAP satisfiability condition is:

$$\left(\sum_j z_j A_j(x)\right) \cdot \left(\sum_j z_j B_j(x)\right) - \left(\sum_j z_j C_j(x)\right) = H(x) \cdot T(x)$$

where $T(x) = \prod_{i=1}^{m}(x - \omega^i)$ is the vanishing polynomial and $H(x)$ is the quotient. The key insight: $T(x)$ divides the left-hand side iff all R1CS constraints are satisfied.

CIRCUIT COMPLEXITY

The circuit size (number of constraints) directly determines:

$$\text{Proving time: } O(n \log n) \text{ field operations (Groth16)}$$

$$\text{CRS size: } O(n) \text{ group elements}$$

$$\text{Prover memory: } O(n)$$

Verification is independent of circuit size for Groth16 ($O(1)$ pairings). Optimizing circuit size is critical: one SHA-256 hash costs ~27,000 constraints, one Poseidon hash costs ~250. A credential verification circuit with BLS12-381 pairing checks can reach 100K-1M constraints.

HISTORY Numerous contributors; R1CS formalized in the SNARKs literature (Ben-Sasson, Chiesa et al.) (2013). *The first production R1CS circuit (Zcash Sprout, 2016) had about 1.2 million constraints and took ~40 seconds to prove. Modern circuits for zkLogin on Sui have similar constraint counts but prove in ~2 seconds thanks to hardware improvements and better libraries.*

LIMITATIONS

- R1CS only supports rank-1 constraints (A*w . B*w = C*w), meaning each constraint encodes exactly one multiplication. Operations like range checks or hash functions that require many multiplications result in bloated constraint systems — SHA-256 needs ~27,000 constraints.
- Flattening complex expressions into elementary gates can dramatically increase the number of intermediate variables. A polynomial of degree d requires d-1 multiplication gates and d-1 auxiliary variables, even if the algebraic expression is compact.
- R1CS is not universally efficient: PLONKish arithmetization (custom gates + lookup tables) can express the same computation with 5-10x fewer constraints for hash-heavy circuits. The choice of arithmetization format directly impacts proving time.

EXERCISES

1. **calculation** — Flatten the expression y = x^3 + x + 5 into R1CS constraints. Introduce intermediate variables and write each constraint in the form A * B = C. How many constraints do you need?
2. **conceptual** — Why is R1CS the 'input format' for Groth16 but not for PLONK? What does PLONKish arithmetization offer that R1CS cannot?
3. **design** — Estimate the R1CS constraint count for a credential verification circuit that checks: (1) BBS+ signature verification, (2) Merkle tree membership (depth 20, Poseidon hash), (3) nullifier derivation (1 Poseidon hash), and (4) one range proof (age >= 18, 8-bit range).

ZK-SNARKs (Groth16)

INTUITIVE

A magical seal that compresses an entire book into a single stamp. Anyone can verify the stamp matches the book in milliseconds, but the stamp reveals nothing about the book's content. The catch: creating the stamp mold requires a one-time trusted ceremony, and if the ceremony secret (toxic waste) leaks, fake stamps become possible.

KEY POINTS

- Trusted setup: a per-circuit MPC ceremony generates proving and verifying keys; the secret randomness (toxic waste) must be destroyed
- Constant proof size (192 bytes, 3 group elements) regardless of circuit complexity
- O(1) verification time: only 3 pairing checks needed, making it the fastest to verify on-chain
- Non-universal: each new circuit requires a new trusted setup ceremony
- Sui natively supports Groth16 verification over BN254 via the sui::groth16 module

WHY IT MATTERS

Groth16 is the primary candidate for on-chain credential verification on Sui. Its constant proof size and O(1) verification map directly to minimal gas costs. The sui::groth16 native module enables direct proof verification in Move smart contracts. The trusted setup requirement means each credential circuit version needs a ceremony, which is a deployment consideration for the thesis system.

THESIS ANGLE

Groth16 is your on-chain verification workhorse. The BBS+ selective disclosure proof is wrapped in a Groth16 proof for gas efficiency on Sui: instead of verifying multiple pairing equations, the Sui Move contract verifies a single Groth16 proof (3 pairings, ~50ms, ~200K gas). The per-circuit setup is acceptable since your credential verification circuit is stable — one ceremony covers all presentations.

TECHNICAL

Groth16 is a pairing-based zk-SNARK producing proofs $\pi = ([A]_1, [B]_2, [C]_1) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$ for an R1CS/QAP relation. It achieves the minimal proof size (3 group elements) and verification cost (3 pairings + 1 multi-exponentiation) among known SNARKs, at the cost of a per-circuit trusted setup.

TRUSTED SETUP (CRS GENERATION)

The setup ceremony samples toxic waste $\tau, \alpha, \beta, \gamma, \delta \xleftarrow{\$} \mathbb{Z}_p^*$ and produces the Common Reference String (CRS). The proving key contains:

$$\text{pk} = \left\{ [\tau^i]_1, [\tau^i]_2, [\alpha]_1, [\beta]_1, [\beta]_2, [\delta]_1, [\delta]_2,\ \left[\tfrac{\beta A_i(\tau) + \alpha B_i(\tau) + C_i(\tau)}{\delta}\right]_1,\ \left[\tfrac{\tau^i T(\tau)}{\delta}\right]_1 \right\}$$

The verifying key:

$$\text{vk} = \left\{ [\alpha]_1, [\beta]_2, [\gamma]_2, [\delta]_2,\ \left[\tfrac{\beta A_i(\tau) + \alpha B_i(\tau) + C_i(\tau)}{\gamma}\right]_1 \right\}$$

The toxic waste $(\tau, \alpha, \beta, \gamma, \delta)$ must be destroyed. If any participant in the MPC ceremony is honest (destroys their share), the CRS is secure.

PROVING ALGORITHM

Given witness $\mathbf{z}$ and proving key pk, the prover samples $r, s \xleftarrow{\$} \mathbb{Z}_p$ and computes:

$$[A]_1 = [\alpha + \sum_j z_j A_j(\tau) + r\delta]_1$$

$$[B]_2 = [\beta + \sum_j z_j B_j(\tau) + s\delta]_2$$

$$[C]_1 = \Big[\sum_{j \in \text{private}} z_j \cdot \tfrac{\beta A_j(\tau) + \alpha B_j(\tau) + C_j(\tau)}{\delta} + H(\tau) \cdot \tfrac{T(\tau)}{\delta} + As + Br - rs\delta \Big]_1$$

The randomizers r, s ensure the proof is zero-knowledge (different proofs for the same witness are unlinkable).

VERIFICATION ALGORITHM

Given proof $\pi = ([A]_1, [B]_2, [C]_1)$, public inputs $(x_1, \ldots, x_\ell)$, and verifying key vk, check:

$$e([A]_1, [B]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e\Big(\sum_{i=0}^{\ell} x_i [\tfrac{\beta A_i(\tau) + \alpha B_i(\tau) + C_i(\tau)}{\gamma}]_1,\ [\gamma]_2\Big) \cdot e([C]_1, [\delta]_2)$$

This is 3 pairings plus a multi-scalar multiplication of size $\ell + 1$ (number of public inputs). Verification is $O(\ell)$ in public inputs but constant in circuit size.

PROOF SIZE AND VERIFICATION COST

Proof structure: $[A]_1 \in \mathbb{G}_1$, $[B]_2 \in \mathbb{G}_2$, $[C]_1 \in \mathbb{G}_1$.

$$\text{BN254: } 2 \times 32 + 1 \times 64 = 128 \text{ bytes (uncompressed), or } 192 \text{ bytes}$$

$$\text{BLS12-381: } 2 \times 48 + 1 \times 96 = 192 \text{ bytes (compressed)}$$

Verification cost: 3 pairings (~4.5 ms on BLS12-381, ~1.5 ms on BN254) + MSM of public inputs. This is constant regardless of whether the circuit has 1,000 or 10,000,000 constraints.

PER-CIRCUIT TRUSTED SETUP

The CRS is circuit-specific: the polynomials $A_j(x), B_j(x), C_j(x)$ are derived from the specific R1CS instance. Changing even one constraint requires a new setup. MPC ceremonies (Powers of Tau + phase 2) mitigate the trust assumption: any one honest participant ensures security. Zcash's Sapling ceremony had 90+ participants. Perpetual Powers of Tau (Hermez): a phase-1 ceremony that is reusable across circuits; only phase 2 is circuit-specific.

HISTORY Jens Groth (University College London) (2016). *Groth16 proofs are exactly 192 bytes (3 elements of G1/G2 on BN254) regardless of whether the circuit has 100 or 10 million constraints. This constant size is why Groth16 dominates on-chain verification despite being 8 years old.*

LIMITATIONS

- Trusted setup is circuit-specific: every new circuit (or circuit modification) requires a new MPC ceremony. Zcash's Sapling ceremony (2018) involved 90 participants over several months. If the toxic waste (secret tau) is not destroyed, an adversary can forge proofs for any statement.
- Not quantum-safe: Groth16 relies on pairing-based assumptions (Knowledge of Exponent, q-SDH) that Shor's algorithm breaks. A sufficiently large quantum computer could forge Groth16 proofs, breaking all on-chain verification.
- Proving time is O(n log n) where n is the number of constraints, with large constants due to FFT and multi-scalar multiplication. A 1 million constraint circuit takes ~5-15 seconds to prove on a modern laptop. This limits real-time applications where the user must generate a proof interactively.

EXERCISES

1. **conceptual** — What is 'toxic waste' in Groth16's trusted setup, and why is it called the 'most dangerous secret in cryptography'? How do MPC ceremonies mitigate this risk?
2. **calculation** — Groth16 verification consists of 3 pairing checks. If each pairing on BN254 takes 1.5ms, and the verifier also needs to compute a multi-scalar multiplication of the public inputs (taking 0.2ms per input), what is the total verification time for a circuit with 5 public inputs?
3. **comparison** — Compare Groth16 with Pinocchio (the predecessor SNARK). What did Groth16 improve, and what remained the same?

ZK-STARKs

INTUITIVE

Like SNARKs but with transparent setup. No secret ceremony is needed: all parameters are derived from public randomness. The tradeoff is larger proofs, like writing a longer but self-evident proof instead of a short proof that requires trusting the exam board.

KEY POINTS

- Transparent setup: no trusted ceremony, all parameters are public (no toxic waste risk)
- Quantum-resistant: relies only on collision-resistant hash functions, not elliptic curve assumptions
- Larger proofs (~50-200 KB vs 192 bytes for Groth16) and slower verification
- FRI (Fast Reed-Solomon Interactive Oracle Proofs) is the core commitment scheme
- Used by StarkNet and zkSync; not natively supported on Sui for verification

WHY IT MATTERS

STARKs are relevant to the thesis as a future-proofing consideration. While Groth16 is preferred for current on-chain verification on Sui, STARKs offer quantum resistance which may matter for long-lived identity credentials. The thesis should acknowledge the SNARK-to-STARK migration path and its implications for credential longevity.

THESIS ANGLE

STARKs offer a quantum-safe alternative for your post-quantum migration. While Groth16 is broken by quantum computers (pairing-based), STARKs rely only on hash functions. The tradeoff: ~100KB proofs vs 288B for Groth16, meaning higher gas costs on Sui. Your migration strategy could use STARKs for high-value transactions (where quantum safety matters more) and keep Groth16 for low-value payments.

TECHNICAL

A ZK-STARK (Scalable Transparent ARgument of Knowledge) is a proof system for computations expressed as an Algebraic Intermediate Representation (AIR) over $\mathbb{F}_p$. It produces proofs of size $O(\log^2 n)$ with verification time $O(\log^2 n)$, using only collision-resistant hash functions (no algebraic assumptions, no trusted setup).

ALGEBRAIC INTERMEDIATE REPRESENTATION (AIR)

An AIR instance defines an execution trace as a matrix $T \in \mathbb{F}^{n \times w}$ with n rows (steps) and w columns (registers), along with transition constraints:

$$f_i(T[j,*], T[j+1,*]) = 0 \quad \forall j \in [n-1],\ \forall i$$

Each column is interpolated as a polynomial $t_k(x)$ of degree $< n$ over a domain $D = \{\omega^0, \omega^1, \ldots, \omega^{n-1}\}$ where ω is a primitive n-th root of unity. The constraints become polynomial identities that must hold on D.

FRI PROTOCOL (FAST REED-SOLOMON IOP)

FRI proves that a committed function is close to a polynomial of bounded degree. The key mechanism is recursive folding:

$$f_0(x) \to f_1(x) \to \cdots \to f_k(x)$$

At each step, split $f_i(x) = g_i(x^2) + x \cdot h_i(x^2)$ and fold with random challenge α_i:

$$f_{i+1}(x) = g_i(x) + \alpha_i \cdot h_i(x)$$

Each fold halves the degree: $\deg(f_{i+1}) = \deg(f_i)/2$. After $\log_2 d$ rounds (where d is the initial degree), the result is a constant. Commitments use Merkle trees of evaluations.

STARK PROOF STRUCTURE

The full STARK proof consists of:

$$\pi = (\text{trace commitment},\ \text{composition poly commitment},\ \text{FRI layers},\ \text{query responses})$$

1. Commit to trace polynomials (Merkle root). 2. Combine constraints into a composition polynomial $C(x)$ using random challenges. 3. Run FRI on $C(x)$ to prove low degree. 4. Answer s random query points with evaluations + Merkle proofs. Soundness error: $\leq (d/|\mathbb{F}|)^s + \text{FRI soundness}$ where s is the number of queries.

PROOF AND VERIFICATION COMPLEXITY

For a computation with n steps:

$$\text{Prover time: } O(n \log n) \text{ (dominated by FFTs and Merkle trees)}$$

$$\text{Proof size: } O(\log^2 n) \text{ (FRI layers + Merkle paths)}$$

$$\text{Verifier time: } O(\log^2 n) \text{ (check FRI queries + Merkle paths)}$$

Concretely, for $n = 2^{20}$ (~1M steps): proof size ~100-200 KB, verification ~50 ms. Compare with Groth16: 192 bytes, ~10 ms verification. STARKs trade proof size for transparency and quantum resistance.

TRANSPARENCY AND QUANTUM SAFETY

STARKs are transparent: the only cryptographic assumption is collision resistance of the hash function used in Merkle commitments. No structured reference string, no pairing groups, no number-theoretic assumptions. Quantum safety: Grover's algorithm provides at most a square-root speedup for finding hash collisions ($2^{n/2} \to 2^{n/3}$ for n-bit hashes). For 256-bit hashes, this gives ~85-bit post-quantum security. Doubling the hash output to 512 bits restores 128-bit post-quantum security.

HISTORY Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev (Technion / StarkWare) (2018). *The name STARK stands for 'Scalable Transparent ARgument of Knowledge,' a deliberate contrast with SNARKs (Succinct Non-interactive ARgument of Knowledge). The 'T' for Transparent emphasizes no trusted setup, and the 'S' for Scalable refers to quasi-linear prover time.*

LIMITATIONS

- Proof size is 50-200KB (vs 192 bytes for Groth16), making on-chain verification storage-expensive. Posting a STARK proof on Ethereum costs ~500K-1M gas for calldata alone, compared to ~300K total for Groth16 verification.
- Verification time is O(log^2 n) with large constants. For a circuit of size 2^20, STARK verification takes ~50-100ms (compared to ~5ms for Groth16). This makes STARKs impractical for Sui's per-transaction verification model.
- STARKs require a specific arithmetization (AIR) that is less expressive than R1CS for some computations. Converting existing R1CS circuits to AIR format requires non-trivial rewriting, and the STARK ecosystem has fewer developer tools than SNARKs (no equivalent to Circom's large library ecosystem).

EXERCISES

1. **conceptual** — Explain the FRI protocol at a high level. How does it prove that a committed polynomial has low degree, and why is this useful for STARKs?
2. **comparison** — A credential system must choose between Groth16 and STARKs for on-chain verification on Sui. Create a decision matrix comparing: proof size, verification time, setup trust, quantum safety, and proving time.

PLONK

INTUITIVE

A universal proving factory. Unlike Groth16, which needs a new setup ceremony for each circuit (like building a new factory per product), PLONK's single setup works for any circuit up to a maximum size. Change the circuit, keep the setup. The factory just reconfigures its production line.

KEY POINTS

- Universal setup: one SRS (Structured Reference String) works for any circuit up to a max size
- Updatable: anyone can add their own randomness to the SRS, strengthening trust assumptions
- Permutation argument: enforces that wires connecting gates carry consistent values (copy constraints)
- KZG polynomial commitments enable efficient proof generation and verification
- Custom gates allow encoding complex operations (hash functions, range checks) more efficiently than R1CS

WHY IT MATTERS

PLONK is a strong alternative for the thesis if the credential circuit needs to evolve. A universal setup means new attribute schemas or predicate types do not require new ceremonies. PLONK-based systems like Halo2 are used in privacy protocols (Zcash Orchard). If Sui adds native PLONK verification, it could simplify credential system upgrades.

THESIS ANGLE

PLONK's universal setup is attractive for your system: as you iterate on the credential circuit during your thesis, you don't need a new trusted ceremony for each circuit change. The lookup argument (plookup) could efficiently implement the 'country $\in$ EU' set membership check inside the circuit. PLONK proofs are ~400B (vs Groth16's 288B) with ~5ms verification — a reasonable tradeoff for development flexibility.

TECHNICAL

PLONK (Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge) is a universal zk-SNARK using a structured reference string $\text{srs} = \left([1]_1, [\tau]_1, \ldots, [\tau^n]_1, [\tau]_2\right)$ that works for any circuit of size $\leq n$. It uses polynomial commitments (KZG) and a permutation argument for copy constraints.

PLONKISH ARITHMETIZATION

PLONK uses a gate equation with selector polynomials. For wire values a, b, c at each gate:

$$q_L \cdot a + q_R \cdot b + q_O \cdot c + q_M \cdot (a \cdot b) + q_C = 0$$

Selectors encode the gate type: addition gate ($q_L = q_R = 1, q_O = -1, q_M = 0$), multiplication gate ($q_M = 1, q_O = -1, q_L = q_R = 0$), constant gate ($q_C = c, q_O = -1$), etc. Custom gates: add more selector columns (e.g., q_{range} for range checks, q_{bool} for boolean constraints $a(a-1) = 0$).

COPY CONSTRAINTS (PERMUTATION ARGUMENT)

Wires connecting gates must carry consistent values. This is enforced by a permutation $\sigma : [3n] \to [3n]$ that maps each wire to the next wire carrying the same value. The grand product argument proves the permutation holds:

$$Z(\omega^0) = 1$$

$$Z(\omega^{i+1}) = Z(\omega^i) \cdot \frac{(a_i + \beta\omega^i + \gamma)(b_i + \beta k_1\omega^i + \gamma)(c_i + \beta k_2\omega^i + \gamma)}{(a_i + \beta\sigma_1(\omega^i) + \gamma)(b_i + \beta\sigma_2(\omega^i) + \gamma)(c_i + \beta\sigma_3(\omega^i) + \gamma)}$$

If the permutation is satisfied, $Z(\omega^n) = 1$ (the product telescopes to 1).

KZG POLYNOMIAL COMMITMENT

Kate-Zaverucha-Goldberg (KZG) commitments: commit to polynomial $f(x)$ of degree $\leq n$ using the SRS:

$$[f(\tau)]_1 = \sum_{i=0}^{n} c_i [\tau^i]_1 \quad (\text{one MSM of size } n)$$

Opening proof at point z: compute quotient $q(x) = \frac{f(x) - f(z)}{x - z}$ and commit:

$$\pi = [q(\tau)]_1$$

Verification: check the pairing equation $e([f(\tau)]_1 - [f(z)]_1, [1]_2) \stackrel{?}{=} e(\pi, [\tau - z]_2)$. Opening proof is a single $\mathbb{G}_1$ element (48 bytes on BLS12-381). Batch opening: open multiple polynomials at the same point with a single proof using random linear combination.

LINEARIZATION TRICK

The verifier needs to check a polynomial identity involving products of committed polynomials. Directly checking would require the verifier to know the polynomials. The linearization trick: evaluate all but one polynomial at a random point ζ, then check a linear (not quadratic) polynomial identity at ζ. This reduces the number of opening proofs the verifier must check, keeping verification efficient.

LOOKUP ARGUMENTS (PLOOKUP)

Plookup extends PLONK with table lookups: prove that a value v belongs to a predefined table $T = \{t_1, \ldots, t_N\}$ without encoding the check as arithmetic constraints. The argument sorts the concatenation of the lookup values and table, then uses a grand product to verify the sorting is valid:

$$\prod_i \frac{(1+\beta) \cdot (\gamma + f_i) \cdot (\gamma(1+\beta) + t_i + \beta t_{i+1})}{(\gamma(1+\beta) + s_i + \beta s_{i+1})} = 1$$

where f is the lookup column, t is the table, and s is the sorted concatenation. Lookups are especially useful for range checks, XOR tables, and non-algebraic operations.

UNIVERSAL AND UPDATABLE SRS

PLONK's SRS is universal: $\text{srs} = ([1]_1, [\tau]_1, \ldots, [\tau^n]_1, [\tau]_2)$ works for any circuit of size $\leq n$. Only the preprocessing (selector and permutation polynomials) is circuit-specific, and it does not require a new ceremony. Updatable: any party can add their randomness τ' via $[\tau^i]_1 \mapsto [(\tau')^i \cdot \tau^i]_1$. Security holds if any one participant is honest (stronger trust model than Groth16's fixed ceremony).

HISTORY Ariel Gabizon, Zachary Williamson, Oana Ciobotaru (Aztec Network / Protocol Labs) (2019). *The name PLONK is a backronym: 'Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge.' The 'O' stands for 'Oecumenical' (meaning universal), chosen to make the acronym spell a word meaning 'cheap wine' in British slang.*

LIMITATIONS

- PLONK proofs are ~400-700 bytes (vs 192 for Groth16) and require ~15-25ms verification (vs ~5ms for Groth16). While still practical, this 3-5x overhead accumulates in high-throughput blockchain settings where every millisecond of verification affects TPS.
- The universal SRS still requires a trusted setup ceremony (powers of tau). While the SRS is reusable across circuits and updatable (anyone can add randomness), the initial ceremony must be conducted honestly. If compromised, all circuits using that SRS are insecure.
- KZG polynomial commitments (used in PLONK) are not quantum-safe (based on discrete log). Replacing KZG with FRI (hash-based) gives Plonky2/Plonky3, which are quantum-safe but with larger proofs (~100KB) and slower verification.

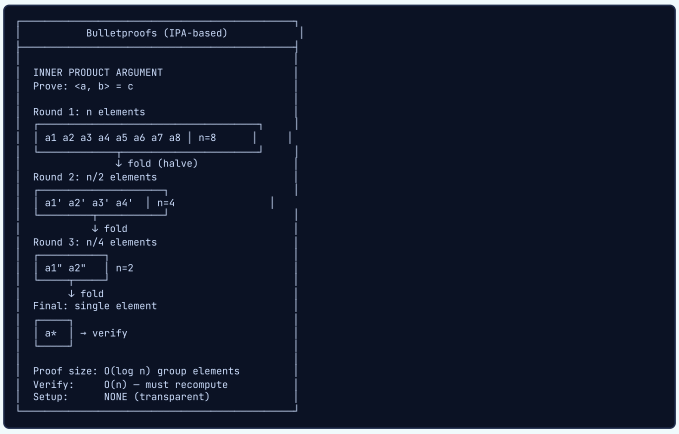
EXERCISES

1. **conceptual** — What is the 'permutation argument' in PLONK, and why is it needed? What problem does it solve that R1CS handles differently?
2. **comparison** — Compare Groth16, PLONK, and Halo2 as candidates for the thesis credential circuit. Consider: setup requirements, proof size, verification time, ability to upgrade the circuit, and ecosystem maturity.
3. **design** — The thesis credential circuit may need updates (new attribute types, new predicate functions). Design a strategy using PLONK's universal setup to handle circuit evolution without new ceremonies.

Bulletproofs

INTUITIVE

A Russian nesting doll proof. Each layer of the proof halves its size through a recursive inner product argument, compressing a large statement into a logarithmically small proof. No trusted setup ceremony is needed at all, but opening each doll takes time, making verification slower.



KEY POINTS

- No trusted setup: completely transparent, based on discrete log assumption only
- Logarithmic proof size: O(log n) group elements, much smaller than the statement but larger than SNARKs
- Linear verification time O(n): verifier must recompute the inner product, making it slower than Groth16
- Primary use case: range proofs (proving a value is in [0, 2^n) without revealing it)
- Used in Monero for confidential transaction amounts; also in Zcash Sapling for Pedersen hash commitments

WHY IT MATTERS

Bulletproofs are relevant for range proofs in the thesis credential system: proving an age attribute is above 18, or a payment amount is within bounds, without revealing the exact value. However, their O(n) verification cost makes them unsuitable for direct on-chain use on Sui. The thesis may use Bulletproof range proofs inside a Groth16 wrapper for efficient on-chain verification.

THESIS ANGLE

Bulletproofs handle range proofs in your payment layer: prove that the transaction amount v ∈ [0, 2^64) without revealing v. No trusted setup needed (unlike Groth16 range proofs). Aggregated Bulletproofs let you prove ranges for multiple outputs in one proof. Monero's adoption of Bulletproofs validates their practicality for private payments — your system applies the same technique on Sui.

TECHNICAL

Bulletproofs is a non-interactive zero-knowledge proof system based on the inner product argument. For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_p^n$ and generators $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$, it proves:

$$P = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + \langle \mathbf{a}, \mathbf{b} \rangle \cdot Q \quad \text{and} \quad \langle \mathbf{a}, \mathbf{b} \rangle = c$$

with proof size $O(\log n)$ group elements and no trusted setup.

PEDERSEN VECTOR COMMITMENT

The prover commits to vectors $\mathbf{a}$ and $\mathbf{b}$ using independent generators $\mathbf{G} = (G_1, \ldots, G_n)$ and $\mathbf{H} = (H_1, \ldots, H_n)$, plus an additional generator Q for the inner product value:

$$P = \sum_{i=1}^{n} a_i G_i + \sum_{i=1}^{n} b_i H_i + \langle \mathbf{a}, \mathbf{b} \rangle \cdot Q$$

This commitment is perfectly hiding (the generators are independent) and computationally binding under the discrete log assumption.

RECURSIVE HALVING PROTOCOL

Split vectors into halves: $\mathbf{a} = (\mathbf{a}_L, \mathbf{a}_R)$, similarly for $\mathbf{b}, \mathbf{G}, \mathbf{H}$. Compute cross-terms:

$$L = \langle \mathbf{a}_L, \mathbf{G}_R \rangle + \langle \mathbf{b}_R, \mathbf{H}_L \rangle + \langle \mathbf{a}_L, \mathbf{b}_R \rangle \cdot Q$$

$$R = \langle \mathbf{a}_R, \mathbf{G}_L \rangle + \langle \mathbf{b}_L, \mathbf{H}_R \rangle + \langle \mathbf{a}_R, \mathbf{b}_L \rangle \cdot Q$$

Send (L, R) to verifier, receive challenge x. Fold:

$$\mathbf{a}' = x\mathbf{a}_L + x^{-1}\mathbf{a}_R, \quad \mathbf{b}' = x^{-1}\mathbf{b}_L + x\mathbf{b}_R$$

$$\mathbf{G}' = x^{-1}\mathbf{G}_L + x\mathbf{G}_R, \quad \mathbf{H}' = x\mathbf{H}_L + x^{-1}\mathbf{H}_R$$

New commitment: $P' = x^2 L + P + x^{-2} R$. Recurse on half-size vectors. After $\log_2 n$ rounds, vectors reduce to scalars.

PROOF STRUCTURE AND SIZE

The proof consists of:

$$\pi = \big((L_1, R_1), (L_2, R_2), \ldots, (L_{\log n}, R_{\log n}),\ a, b\big)$$

where (L_i, R_i) are group elements from each round and (a, b) are the final scalar values. Total size:

$$2 \log_2 n \text{ group elements} + 2 \text{ scalars} = 2 \log_2 n \cdot 32 + 2 \cdot 32 \text{ bytes (Curve25519)}$$

For $n = 64$ (64-bit range proof): $2 \cdot 6 + 2 = 14$ elements, ~672 bytes.

RANGE PROOF CONSTRUCTION

Prove $v \in [0, 2^n)$ without revealing v. Write v in binary: $v = \sum_{i=0}^{n-1} b_i \cdot 2^i$ where $b_i \in \{0, 1\}$. Set:

$$\mathbf{a}_L = (b_0, b_1, \ldots, b_{n-1}), \quad \mathbf{a}_R = \mathbf{a}_L - \mathbf{1}^n$$

The constraints are: (1) $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$ (binary decomposition), (2) $\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{0}$ (each bit is 0 or 1, since $b(b-1) = 0$). These are encoded as an inner product relation and proved using the halving protocol.

AGGREGATED RANGE PROOFS

Prove m values $v_1, \ldots, v_m \in [0, 2^n)$ simultaneously. The proof size grows only logarithmically in m:

$$|\pi| = 2\lceil \log_2(mn) \rceil \text{ group elements} + \text{constant overhead}$$

For $m = 16$ values with $n = 64$-bit range: ~928 bytes (vs ~672 bytes for one). This sublinear aggregation is key for confidential transactions: prove all transaction outputs are in range with a single compact proof.

HISTORY Benedikt Bunz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, Greg Maxwell (2017). *Before Bulletproofs, Monero's range proofs used Borromean ring signatures, producing ~13KB per output. Bulletproofs compressed this to ~0.7KB (single output), saving Monero ~$100M per year in transaction fees at 2018 volumes.*

LIMITATIONS

- Verification time is O(n) where n is the number of generators (bit length of the range). Verifying a 64-bit range proof requires ~64 multi-scalar exponentiations, taking ~3-5ms. Batch verification amortizes this (128 proofs in ~50ms) but single-proof verification is 5-10x slower than Groth16.
- Not quantum-safe: Bulletproofs rely on the discrete log assumption (same as Pedersen commitments). Shor's algorithm breaks the binding of the underlying commitment and the soundness of the inner product argument.
- Proving time is O(n) with significant constants due to multi-scalar multiplication at each round. A 64-bit range proof takes ~20-50ms to generate, compared to ~500ms for an equivalent Groth16 range circuit. However, Groth16's proof is constant-size while Bulletproofs grow logarithmically.

EXERCISES

1. **calculation** — A Bulletproof for a 64-bit range proof produces 2*ceil(log2(64)) = 2*6 = 12 group elements plus 5 scalars and a few additional elements. If each group element is 32 bytes and each scalar is 32 bytes, estimate the proof size. Compare with Groth16's 192 bytes.
2. **conceptual** — Explain the inner product argument at a high level. Why does it produce a logarithmic-size proof for a linear-size statement?
3. **design** — The thesis payment layer needs range proofs for transaction amounts. Compare two approaches: (a) Bulletproofs directly on-chain, (b) Bulletproof range proof wrapped inside a Groth16 proof. Which should the thesis use and why?

CHAPTER 2

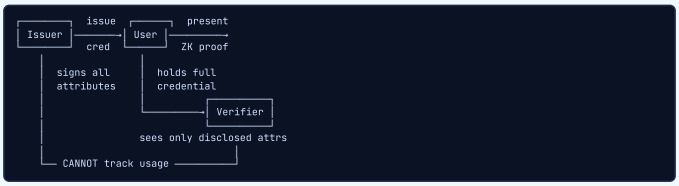
Anonymous Credentials & TEE Foundations

Anonymous Credentials (CL, BBS+, VCs)

Anonymous Credentials

INTUITIVE

A bouncer at a club who can verify you are over 21 without seeing your name, address, or photo. You prove a property about yourself without revealing your identity. The bouncer never learns who you are, only that you meet the entry requirement.



KEY POINTS

- Unlinkability: issuer cannot track where or when you use your credential
- Selective disclosure: reveal only the attributes needed for each interaction
- Minimal disclosure principle: never share more than strictly required
- Multi-show unlinkability: using the same credential twice appears as two different users

WHY IT MATTERS

Your thesis builds an anonymous credential system on Sui. Users receive credentials (e.g., KYC-verified, accredited investor) and prove properties without revealing identity, enabling private payments and compliant DeFi.

THESIS ANGLE

In your thesis, anonymous credentials are the bridge between identity and privacy on Sui. A government issues a credential containing (name, age, nationality, KYC_level) signed with BBS+. When the user wants to make a private payment, they present a zero-knowledge proof that they hold a valid credential with KYC_level >= 2 — the Sui verifier learns only this fact, not the user's identity. The same credential works at different merchants without creating a traceable profile.

TECHNICAL

A tuple of PPT algorithms $(\text{IssuerSetup}, \text{Obtain}, \text{Show}, \text{Verify})$ where: $\text{IssuerSetup}(1^\lambda) \to (isk, ipk)$; $\text{Obtain}(\text{user}, isk, (a_1, \ldots, a_n)) \to \sigma$; $\text{Show}(\sigma, (a_1, \ldots, a_n), S) \to \pi$ for disclosed set $S \subseteq \{1, \ldots, n\}$; $\text{Verify}(ipk, \{a_i\}_{i \in S}, \pi) \to \{0, 1\}$.

KEY GENERATION AND CREDENTIAL ISSUANCE

The issuer generates a key pair (isk, ipk). A credential on attributes $(a_1, \ldots, a_n) \in \mathcal{M}^n$ is a signature σ computed under isk. The issuance protocol Obtain may be interactive (blind issuance) so the issuer does not learn all attributes, or non-interactive if the issuer knows all attributes.

SHOW PROTOCOL AND SELECTIVE DISCLOSURE

The Show protocol produces a zero-knowledge proof π that the user holds a valid signature σ on some attribute vector $(a_1, \ldots, a_n)$, revealing only the subset $\{a_i\}_{i \in S}$ for a disclosed set $S \subseteq [n]$. The verifier learns nothing about $\{a_i\}_{i \notin S}$ or σ itself beyond validity.

UNLINKABILITY

Two presentations Show_1 and Show_2 derived from the same credential are computationally indistinguishable from presentations of two different credentials. Formally, for any PPT distinguisher $\mathcal{D}$:

$$|\Pr[\mathcal{D}(\pi_1, \pi_2) = 1 \mid \text{same } \sigma] - \Pr[\mathcal{D}(\pi_1, \pi_2) = 1 \mid \text{diff } \sigma]| \leq \text{negl}(\lambda)$$

ONE-SHOW VS MULTI-SHOW CREDENTIALS

One-show credentials embed a serial number revealed on use, enabling double-spend detection: if used twice, the two presentations leak the user identity. Multi-show credentials (like BBS+) allow unlimited unlinkable presentations. The choice depends on the application: e-cash requires one-show, identity credentials require multi-show.

FORMAL SECURITY GAMES

Anonymity (CCA-like): adversary chooses two users with valid credentials, receives a presentation from one, and must guess which. Advantage must be negligible. Unforgeability (EUF-CMA analog): adversary with access to issuance oracle cannot produce a valid presentation for an attribute vector never issued. Formally:

$$\Pr[\text{Verify}(ipk, \{a_i\}_{i \in S}, \pi^*) = 1 \mid (a_1, \ldots, a_n) \notin Q] \leq \text{negl}(\lambda)$$

where Q is the set of issued credential attribute vectors.

HISTORY David Chaum (DigiCash) (1985). *DigiCash went bankrupt in 1998 — Chaum later said it was 'too early for the market.' The lineage runs Chaum 1985 → Brands 1993 (efficient blind signatures) → CL 2001 → BBS 2004 → BBS+ 2009 → SAAC 2025 (your thesis builds on this chain).*

LIMITATIONS

- Revocation is fundamentally hard: revoking a credential without breaking unlinkability requires cryptographic accumulators, which add O(n) witness update cost for n credentials or require a trusted update server
- Credential transfer prevention relies on the assumption that users will not share their secret key — there is no cryptographic mechanism to prevent a user from giving their full credential to someone else
- Multi-show unlinkability means the issuer cannot detect abuse (e.g., one credential shared by 100 users) without adding linkability, creating a privacy-accountability tension

EXERCISES

1. **conceptual** — Explain why multi-show unlinkability is both a feature and a risk. How does your thesis address the credential-sharing problem?
2. **design** — Design a credential issuance flow for your Sui system where a government issues a KYC credential. What attributes would you include, and which would be disclosed for a DeFi lending protocol?

CL Signatures (Camenisch-Lysyanskaya)

INTUITIVE

A magic stamp that can be broken into pieces. The stamp proves the whole document is valid, but you can show just one piece (one attribute) without revealing the rest. The catch: the stamp is large and heavy (RSA-based, big keys).

KEY POINTS

- Multi-message signing: one signature covers multiple attributes simultaneously
- Attribute-based credentials: each message slot encodes one user attribute
- RSA-based: large key sizes (~2048 bits), slower than pairing-based schemes
- Efficient for selective disclosure but less compact than BBS+
- Foundation of IBM Idemix (Identity Mixer) system

WHY IT MATTERS

CL signatures were the first practical anonymous credential scheme. Understanding CL helps you appreciate why BBS+ is preferred for Sui: CL keys are too large for on-chain storage and verification is expensive in circuits.

THESIS ANGLE

CL signatures were the original anonymous credential scheme (IBM's Idemix). For your thesis comparison, CL uses RSA groups (~256B keys, slow modular exponentiation) while BBS+ uses pairings (~48B keys, faster operations). Your thesis argues for BBS+ over CL: BBS+ is 5x faster for selective disclosure, has smaller proofs, and aligns with BLS12-381 already used by Sui — making on-chain verification significantly cheaper.

TECHNICAL

Signature scheme based on the Strong RSA assumption. Given a special RSA modulus $n = pq$ where p, q are safe primes, the public key is $(n, S, Z, R_1, \ldots, R_L)$ with $S, Z, R_i \in QR_n$ (quadratic residues mod n). A signature on messages $(m_1, \ldots, m_L)$ is a triple (A, e, v) satisfying the CL signature equation.

CL03 SIGNATURE EQUATION

A signature on $(m_1, \ldots, m_L)$ is a triple (A, e, v) such that:

$$A^e \equiv Z \cdot S^v \cdot \prod_{i=1}^{L} R_i^{m_i} \pmod{n}$$

where e is a prime in $[2^{\ell_e - 1}, 2^{\ell_e}]$ and v is chosen from a range ensuring statistical ZK. The issuer computes $A = \left(\frac{Z}{S^v \cdot \prod R_i^{m_i}}\right)^{1/e} \bmod n$, which requires knowledge of p, q to compute e-th roots.

PROOF OF KNOWLEDGE OF A CL SIGNATURE

To prove possession of a valid signature without revealing it, the holder executes a Sigma protocol proving:

$$\text{PK}\{(A, e, v, m_1, \ldots, m_L) : A^e \equiv Z \cdot S^v \cdot \prod R_i^{m_i} \pmod{n}\}$$

This is a proof of knowledge of an RSA representation. The prover first randomizes the signature to prevent linkability, then runs an interactive (or Fiat-Shamir transformed) proof.

SIGNATURE RANDOMIZATION

Given (A, e, v), choose random r' and compute:

$$A' = A \cdot S^{r'}, \quad v' = v - e \cdot r'$$

Then $(A')^e = A^e \cdot S^{e \cdot r'} = Z \cdot S^v \cdot \prod R_i^{m_i} \cdot S^{e \cdot r'} = Z \cdot S^{v'} \cdot \prod R_i^{m_i} \cdot S^{e \cdot r' + v - v}$. Substituting $v' = v - er'$:

$$(A')^e = Z \cdot S^{v'} \cdot \prod R_i^{m_i}$$

So (A', e, v') is a valid signature on the same messages, unlinkable to the original.

SELECTIVE DISCLOSURE WITH CL

To reveal a subset $D \subseteq [L]$ of messages, the prover separates the signature equation:

$$A^e = Z \cdot S^v \cdot \prod_{i \in D} R_i^{m_i} \cdot \prod_{i \notin D} R_i^{m_i} \pmod{n}$$

The disclosed $\{m_i\}_{i \in D}$ are sent in the clear. The hidden $\{m_i\}_{i \notin D}$ along with (A, e, v) are proved in zero knowledge via a Sigma protocol.

PREDICATE PROOFS (RANGE PROOFS)

To prove $m_i \geq 18$ without revealing m_i, decompose $m_i - 18$ into a sum of four squares (Lagrange's theorem: every non-negative integer is a sum of four squares):

$$m_i - 18 = a_1^2 + a_2^2 + a_3^2 + a_4^2$$

Then prove knowledge of a_1, a_2, a_3, a_4 satisfying this decomposition, embedded in the CL proof framework. More efficient range proofs use binary decomposition or Bulletproofs.

REVOCATION VIA RSA ACCUMULATORS

An RSA accumulator over a set of valid credential identifiers $V = \{x_1, \ldots, x_k\}$:

$$\text{acc} = g^{\prod_{i \in V}(x_i + s)} \bmod n$$

where s is a secret and $g \in QR_n$. A membership witness for x_i is:

$$w_i = g^{\prod_{j \in V, j \neq i}(x_j + s)} \bmod n$$

Verification: $w_i^{(x_i + s)} \equiv \text{acc} \pmod{n}$. The holder proves in ZK that their credential identifier is accumulated, without revealing which one.

HISTORY Jan Camenisch and Anna Lysyanskaya (IBM Zurich / Brown University) (2001). *CL signatures became the core of IBM's Identity Mixer (Idemix) system, deployed in the Hyperledger Fabric blockchain. Lysyanskaya was only 25 when the first CL paper was published — she completed her PhD at MIT under Ron Rivest (the 'R' in RSA).*

LIMITATIONS

- RSA-based: public keys are ~2048-4096 bits (256-512 bytes), compared to 48 bytes for BBS+ on BLS12-381 — prohibitively expensive for on-chain storage on Sui where storage costs ~76 MIST per byte
- Modular exponentiation over RSA groups is ~10x slower than elliptic curve scalar multiplication, making CL proof generation take 200-500ms vs ~50ms for BBS+
- CL requires a trusted RSA modulus generation ceremony (n = p*q where p,q must be secret) — if the factors leak, all credentials become forgeable
- Not compatible with pairing-based SNARKs (Groth16): encoding RSA operations inside a BLS12-381 arithmetic circuit requires expensive non-native field arithmetic, inflating circuit size by 100x+

EXERCISES

1. **comparison** — Compare the key sizes and proof generation times of CL vs BBS+ signatures. Why does this make CL impractical for on-chain verification on Sui?
2. **conceptual** — Why does the RSA trusted setup for CL signatures pose a different risk than the Groth16 trusted setup ceremony?

BBS+ Signatures

INTUITIVE

A holographic badge with hidden layers. The full badge contains all your info, but you can activate specific layers while keeping others invisible. Unlike the heavy CL stamp, this badge is lightweight and becoming the industry standard.

KEY POINTS

- Pairing-based cryptography: bilinear pairings on elliptic curves (BLS12-381)
- Unlinkable proofs: two presentations of the same credential cannot be linked
- Compact: smaller keys and signatures than CL (~48-96 bytes per group element)
- W3C standardization candidate for Verifiable Credentials Data Integrity
- Proof of knowledge of signature without revealing the signature itself

WHY IT MATTERS

BBS+ is the primary candidate for your thesis credential system. It uses the same pairing-friendly curves as Groth16 (BLS12-381), making it efficient to verify BBS+ signature proofs inside SNARKs on Sui.

THESIS ANGLE

BBS+ is THE signature scheme for your credential layer. The issuer signs attributes (a1,...,a10) with one BBS+ signature. During payment on Sui, the user randomizes the signature (A' = A^r) making each presentation unlinkable, then proves in zero-knowledge that specific attributes satisfy the verifier's policy. The pairing equation e(A', X~) = e(A_bar, g~_2) is what your Groth16 circuit encodes for on-chain verification.

TECHNICAL

A pairing-based signature scheme on a message vector $(m_1, \ldots, m_L) \in \mathbb{Z}_p^L$. Defined over a Type-3 pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ with generators $g_1 \in \mathbb{G}_1$ and $\tilde{g}_2 \in \mathbb{G}_2$. Security rests on the q-Strong Diffie-Hellman (q-SDH) assumption.

KEY GENERATION

Choose $x \xleftarrow{\$} \mathbb{Z}_p$ and $h_0, h_1, \ldots, h_L \xleftarrow{\$} \mathbb{G}_1$. The secret key is $sk = x$. The public key is:

$$pk = (\tilde{X} = \tilde{g}_2^{\,x},\ h_0, h_1, \ldots, h_L) \in \mathbb{G}_2 \times \mathbb{G}_1^{L+1}$$

Here h_i are public generators in $\mathbb{G}_1$, one per message slot plus h_0 for the blinding factor s.

SIGNING

To sign $(m_1, \ldots, m_L) \in \mathbb{Z}_p^L$, choose $e, s \xleftarrow{\$} \mathbb{Z}_p$ and compute:

$$B = g_1 \cdot h_0^{\,s} \cdot \prod_{i=1}^{L} h_i^{m_i}$$

$$A = B^{1/(x+e)}$$

The signature is $\sigma = (A, e, s) \in \mathbb{G}_1 \times \mathbb{Z}_p^2$. Computing $B^{1/(x+e)}$ requires the secret key x.

VERIFICATION (PAIRING EQUATION)

Given $pk = (\tilde{X}, h_0, \ldots, h_L)$, signature (A, e, s), and messages $(m_1, \ldots, m_L)$, verify:

$$e(A,\ \tilde{X} \cdot \tilde{g}_2^{\,e}) = e\left(g_1 \cdot h_0^{\,s} \cdot \prod_{i=1}^{L} h_i^{m_i},\ \tilde{g}_2\right)$$

This holds because $e(B^{1/(x+e)}, \tilde{g}_2^{\,x+e}) = e(B, \tilde{g}_2)$ by bilinearity. The pairing check requires two pairing evaluations and one multi-exponentiation in $\mathbb{G}_1$.

PROOF OF KNOWLEDGE (SELECTIVE DISCLOSURE)

To prove knowledge of a valid BBS+ signature while disclosing only $\{m_i\}_{i \in D}$:

Step 1: Randomize. $r_1, r_2 \xleftarrow{\$} \mathbb{Z}_p, \quad A' = A^{r_1}, \quad \bar{A} = A'^{-e} \cdot B^{r_1}$

where $B = g_1 \cdot h_0^{s} \cdot \prod h_i^{m_i}$. Then $\bar{A} = A'^{-e} \cdot (A^{x+e})^{r_1} = A'^{x} \cdot A'^{e} \cdot A'^{-e} = (A')^{x}$. So: $e(A', \tilde{X}) = e(\bar{A}, \tilde{g}_2)$.

Step 2: Sigma protocol.

Prove knowledge of $(e, r_2, s', \{m_i\}_{i \notin D})$ satisfying:

$$\bar{A}/d = A'^{-e} \cdot h_0^{r_2} \cdot \prod_{i \notin D} h_i^{m_i}$$

where $d = g_1 \cdot \prod_{i \in D} h_i^{m_i}$ is computable by the verifier, and $s' = s + r_1 \cdot r_2$. The proof uses Schnorr-style commitments and a Fiat-Shamir challenge.

UNLINKABILITY VIA RANDOMIZATION

Each presentation uses a fresh random r_1, producing a different $A' = A^{r_1}$. Since r_1 is uniform in $\mathbb{Z}_p^*$, the distribution of A' is uniform over $\mathbb{G}_1 \setminus \{\mathcal{O}\}$, independent of A. Two presentations from the same credential are information-theoretically unlinkable (perfect ZK in the randomization step).

MULTI-MESSAGE SUPPORT

BBS+ natively supports L messages in a single signature through the generators $h_1, \ldots, h_L$. The signature size is constant (one $\mathbb{G}_1$ element + two scalars) regardless of L. The proof size grows linearly with the number of hidden attributes: each hidden m_i adds one scalar to the proof (the Schnorr response). Disclosed attributes add no proof overhead.

HISTORY Dan Boneh, Xavier Boyen, Hovav Shacham (Stanford University) (2004). *BBS+ is now under IRTF standardization by the Crypto Forum Research Group (draft-irtf-cfrg-bbs-signatures). The W3C also adopted it for the VC Data Integrity BBS Cryptosuite (2023). Boneh went on to become one of the most cited cryptographers alive, co-authoring the BLS signature scheme used by Ethereum 2.0 and Sui.*

LIMITATIONS

- Requires pairing-friendly curves (BLS12-381): scalar multiplication on BLS12-381 is 10-20x slower than on non-pairing curves like Curve25519, and pairing computation itself costs ~1ms per evaluation
- Proof size grows linearly with the number of hidden attributes: each hidden message adds ~48 bytes (one G1 element) to the proof, so a credential with 20 hidden attributes produces a ~1KB proof
- No post-quantum security: BBS+ relies on the hardness of the Discrete Logarithm Problem in pairing groups, which is broken by Shor's algorithm — a quantum computer with ~4000 logical qubits could forge signatures
- The IRTF standardization is still in draft status (not finalized as of 2025), meaning the exact API and serialization format may change, creating integration risk for early adopters

EXERCISES

1. **calculation** — A BBS+ credential has 15 attributes. During a Sui DeFi interaction, the user discloses 3 attributes and hides 12. Estimate the proof size in bytes, knowing each hidden attribute adds ~48 bytes (one G1 point) and the base proof overhead is ~240 bytes.
2. **design** — Why does your thesis wrap BBS+ verification inside a Groth16 circuit instead of verifying BBS+ directly on-chain? What are the tradeoffs?
3. **conceptual** — BBS+ has no post-quantum security. How would you future-proof your thesis credential system against quantum computers?

Selective Disclosure

INTUITIVE

A redacted document at a government hearing. You show the relevant paragraphs and black out the rest, but the official stamp on the page still proves the entire original document was authentic and unmodified.

KEY POINTS

- Prove predicates (age > 18) without revealing the exact value
- Range proofs: demonstrate a value falls within a range without disclosure
- Attribute equality: prove same attribute across two credentials without revealing it
- Combine multiple credentials in a single presentation
- Foundation for privacy-preserving identity verification

WHY IT MATTERS

Selective disclosure is the key feature for your thesis use cases. A user can prove 'I am KYC-verified and accredited' to a DeFi protocol on Sui without revealing their name, nationality, or any other personal data.

THESIS ANGLE

Selective disclosure is the UX differentiator of your system. A user with credential (name, age=25, country=CH, KYC=3) can prove 'age >= 18 AND country in EU/CH' to a Sui DeFi protocol without revealing their exact age (25), name, or KYC level. For payments below the threshold, they only prove 'valid credential exists.' This flexibility means one credential covers convenience stores, DeFi protocols, and regulated exchanges with different disclosure levels.

TECHNICAL

Given a signature σ on messages $(m_1, \ldots, m_L)$, produce a proof π that convinces a verifier of the validity of σ while revealing only $\{m_i\}_{i \in D}$ for a disclosed set $D \subseteq [L]$. Formally: $\pi \leftarrow \text{Prove}(pk, \sigma, (m_1, \ldots, m_L), D)$ and $\text{Verify}(pk, \{m_i\}_{i \in D}, \pi) \to \{0, 1\}$.

BBS+ SELECTIVE DISCLOSURE SPLIT

The core technique splits the multi-exponentiation in the signature equation into disclosed and hidden parts:

$$\prod_{i=1}^{L} h_i^{m_i} = \underbrace{\prod_{i \in D} h_i^{m_i}}_{\text{disclosed (verifier computes)}} \cdot \underbrace{\prod_{i \notin D} h_i^{m_i}}_{\text{hidden (proved in ZK)}}$$

The verifier computes $d = g_1 \cdot \prod_{i \in D} h_i^{m_i}$ from the disclosed values and checks the proof against this. The prover demonstrates knowledge of the hidden exponents via Schnorr proofs.

PREDICATE PROOFS (RANGE PROOFS)

To prove $m_i \in [a, b]$ without revealing m_i, embed a range proof in the Sigma protocol. Using Bulletproofs: prove $m_i - a$ and $b - m_i$ are both non-negative by decomposing into n-bit representations and proving each bit is binary:

$$m_i - a = \sum_{j=0}^{n-1} b_j \cdot 2^j, \quad b_j \in \{0, 1\}$$

Bulletproof range proof size: $O(\log n)$ group elements, where n is the bit length. For a 64-bit range proof: ~672 bytes.

EQUALITY PROOF ACROSS CREDENTIALS

To prove $m_i^{(1)} = m_j^{(2)}$ (same value in two different credentials) without revealing it, use a combined Schnorr proof. If both credentials use BBS+, the prover shows:

$$\text{PK}\{(\alpha) : C_1 = h_i^{\alpha} \ \wedge \ C_2 = h_j'^{\alpha}\}$$

where C_1, C_2 are commitments to $m_i^{(1)}$ and $m_j^{(2)}$ respectively, extracted from the respective BBS+ proofs. A single Schnorr response α binds both.

SET MEMBERSHIP PROOF

To prove $m_i \in \{v_1, \ldots, v_k\}$ (e.g., country in an allowed list) without revealing which value:

$$\text{PK}\{(m_i) : \prod_{j=1}^{k} (m_i - v_j) = 0\}$$

This can be realized via a polynomial evaluation proof or an OR-composition of Sigma protocols (prove $m_i = v_1 \vee m_i = v_2 \vee \ldots$). For small sets, OR-composition is efficient. For large sets, use an accumulator-based membership proof.

NON-MEMBERSHIP (INEQUALITY) PROOF

To prove $m_i \neq v$ or $m_i \notin S$ (e.g., not on a blocklist), use accumulator non-membership witnesses. For RSA accumulators, a non-membership witness for $x \notin V$ uses Bezout coefficients:

$$a \cdot x + b \cdot \prod_{v \in V} v = \gcd(x, \prod_{v \in V} v) = 1$$

The holder proves knowledge of (a, b) satisfying this relation in ZK. This is computationally harder than membership proofs.

HISTORY Stefan Brands (CWI Amsterdam / Credentica) (1993). *Microsoft acquired Brands' company Credentica in 2008 and released U-Prove as an open specification, but it never achieved mainstream adoption. Meanwhile, the EU Digital Identity Wallet (eIDAS 2.0, mandated by 2026) chose the BBS+/SD-JWT approach for selective disclosure — validating the concept 30 years after Brands' original work.*

LIMITATIONS

- Range proofs (e.g., age >= 18) are expensive: a Bulletproof range proof for a 64-bit value adds ~672 bytes and ~50ms verification time; encoding range checks in Groth16 adds ~10K constraints per predicate
- Set membership proofs ('country in {CH, DE, FR, ...}') scale linearly with set size when done naively — a Merkle proof approach reduces this to O(log n) but adds tree management complexity
- The verifier's disclosure policy must be fixed before proof generation — dynamic policies (where the verifier decides what to ask after seeing partial data) are not supported without interactive protocols
- Correlation attacks: even with selective disclosure, if a user always discloses the same unusual combination of attributes (e.g., age=97 AND country=LI), the verifier can re-identify them statistically

EXERCISES

1. **design** — Design three disclosure policies for your Sui payment system: (1) a coffee shop, (2) a DeFi lending protocol, (3) a regulated exchange. What attributes does each verifier need?
2. **conceptual** — How do correlation attacks threaten selective disclosure, and what countermeasures can your thesis system employ?

W3C Verifiable Credentials & DIDs

INTUITIVE

A digital passport system where YOU hold your documents in your own wallet, not in a government database. You choose what to share with each service. No central authority can revoke your ability to present credentials or track where you use them.

KEY POINTS

- Self-sovereign identity: users control their own credentials without intermediaries
- DID methods: did:web, did:key, did:sui (potential) map identifiers to public keys
- JSON-LD vs JWT: two serialization formats for VCs with different tradeoffs
- Holder-controlled presentation: user decides what to share and with whom
- Interoperability: standard format enables cross-platform credential exchange

WHY IT MATTERS

Your thesis maps the VC/DID model onto Sui. DIDs could be Sui addresses, VCs are issued off-chain (or anchored on-chain), and VPs are ZK proofs verified by Sui smart contracts. This bridges W3C standards with blockchain verification.

THESIS ANGLE

Your system implements W3C Verifiable Credentials on Sui. Each user has a DID (did:sui:<wallet_object_id>) resolved from a Sui object. Credentials follow the VC data model with BBS+ as the proof mechanism. The VP (Verifiable Presentation) sent to a Sui smart contract contains the selective disclosure proof. This standards compliance means your thesis system could interoperate with the EU Digital Identity Wallet (eIDAS 2.0) — a massive real-world adoption vector.

TECHNICAL

A Verifiable Credential is a tamper-evident data structure: $\text{VC} = (\text{issuer}, \text{subject}, \text{claims}, \text{proof})$ where $\text{claims} = \{(k_i, v_i)\}_{i=1}^{n}$ are attribute key-value pairs and proof is a cryptographic proof of integrity (digital signature, ZK proof, or both). A DID (Decentralized Identifier) is a URI of the form **`did:method:identifier`** that resolves to a DID Document containing public keys and service endpoints.

DID RESOLUTION

DID resolution is a function $\text{resolve} : \text{DID} \to \text{DIDDocument}$ where the DID Document contains: verification methods (public keys), authentication methods, service endpoints, and controller relationships. The resolution method depends on the DID method: **`did:web`** fetches from a web server, **`did:key`** derives from the identifier itself, **`did:sui`** reads a Sui on-chain object.

SD-JWT (SELECTIVE DISCLOSURE JWT)

SD-JWT hashes individual claims before inclusion in the JWT payload. For each claim (k_i, v_i):

$$H_i = \text{SHA-256}(\text{salt}_i \| k_i \| v_i)$$

The JWT body contains $\{H_1, \ldots, H_n\}$ signed by the issuer. To disclose claim i, the holder reveals $(\text{salt}_i, k_i, v_i)$ and the verifier checks $H_i = \text{SHA-256}(\text{salt}_i \| k_i \| v_i)$. Undisclosed claims remain hidden (preimage resistance of SHA-256). Limitation: SD-JWT presentations are linkable (same JWT signature shown each time).

BBS+ INTEGRATION WITH VCS

VC claims are mapped to a BBS+ message vector: $m_i = \text{Hash}(k_i \| v_i)$ for each claim. The issuer signs $(m_1, \ldots, m_L)$ using BBS+, producing $\sigma = (A, e, s)$. The VC proof field contains the BBS+ signature (or a derived proof). For presentation: the holder runs the BBS+ selective disclosure protocol, producing an unlinkable ZK proof that replaces the original signature in the Verifiable Presentation. W3C BBS Data Integrity Cryptosuite (draft) standardizes this flow.

VERIFIABLE PRESENTATIONS

A Verifiable Presentation VP packages one or more VCs (or derived proofs) with a holder binding proof:

$$\text{VP} = (\{\text{VC}_i^{\text{derived}}\}_{i=1}^{k},\ \text{holderProof})$$

The holder proof binds the presentation to the holder's DID (prevents replay). With BBS+, each $\text{VC}_i^{\text{derived}}$ is a selective disclosure proof, and the holder proof is a Schnorr proof of knowledge of the DID's private key.

DID METHODS FOR SUI

A potential **`did:sui`** method: \(\texttt{did:sui:} \) where the DID Document is stored as a Sui object. Resolution calls $\text{sui\_getObject}(\text{object\_id})$ and parses the object data into a DID Document. Advantages: on-chain key rotation (update the Sui object), Sui's object model maps naturally to DID Document lifecycle, and verification is on-chain (smart contract reads the DID object directly).

HISTORY W3C Credentials Community Group (Manu Sporny, Dave Longley, et al.) (2019). *There are now 100+ registered DID methods (did:web, did:key, did:ion, did:ethr, etc.) but no dominant standard has emerged. The EU's eIDAS 2.0 regulation (2024) mandates that all EU member states offer digital identity wallets by 2026, using VC-compatible formats — this is the largest-scale deployment of verifiable credentials in history, targeting 450 million EU citizens.*

LIMITATIONS

- The VC ecosystem is fragmented: JSON-LD credentials (used by W3C VC Data Integrity) and JWT credentials (used by OpenID4VC) have different serialization, proof formats, and tooling — interoperability between them requires translation layers
- DID resolution depends on the DID method: did:web relies on DNS/HTTPS (centralized), did:key has no revocation, did:ethr requires an Ethereum node — no DID method satisfies all requirements (decentralized + resolvable + revocable + privacy-preserving)
- The VC Data Model does not specify a standard for selective disclosure — this is left to proof suites (BBS+, SD-JWT), creating a 'proof format war' between competing approaches
- Storing VCs on-chain (for availability) conflicts with GDPR's right to erasure: personal data committed to an immutable blockchain cannot be deleted, even if encrypted

EXERCISES

1. **design** — Design a did:sui DID method for your thesis. What would the DID document contain, how would it be resolved, and where would it be stored on Sui?
2. **comparison** — Compare how your thesis system handles VCs vs. how the EU Digital Identity Wallet (eIDAS 2.0) handles them. Where could they interoperate?

Revocation Mechanisms

INTUITIVE

A 'banned users' list that does not reveal who is on it. To use your credential, you prove 'I am NOT on this list' without showing which entry you checked. The list can be updated without reissuing credentials to honest users.

KEY POINTS

- Accumulator-based: compact cryptographic structure for set membership proofs
- Non-membership proofs: prove your ID is NOT in the revoked set
- Privacy-preserving: revocation check does not reveal which credential is being checked
- Credential validity periods: time-based expiry as a complementary mechanism
- Dynamic accumulators allow adding/removing members without reissuing all witnesses

WHY IT MATTERS

On Sui, the revocation accumulator can be stored as a shared object, updated by the issuer. Users fetch the latest accumulator and generate a non-membership proof as part of their credential presentation ZK circuit.

THESIS ANGLE

Your thesis uses an on-chain RSA accumulator for revocation. When an issuer revokes a credential (e.g., expired passport), they update the accumulator on Sui. During each payment, the user includes a non-revocation proof: 'my credential is NOT in the revoked set.' This proof is bundled into the Groth16 circuit alongside the credential proof — one on-chain verification covers both validity and non-revocation, keeping gas costs constant.

TECHNICAL

A tuple of algorithms $(\text{Setup}, \text{Add}, \text{Revoke}, \text{ProveNonRevoked}, \text{VerifyNonRevoked})$ where: $\text{Setup}(1^\lambda) \to (\text{acc}_0, sk_{\text{acc}})$; $\text{Add}(sk_{\text{acc}}, \text{acc}, x) \to (\text{acc}', w_x)$; $\text{Revoke}(sk_{\text{acc}}, \text{acc}, x) \to \text{acc}'$; $\text{ProveNonRevoked}(w_x, \text{acc}) \to \pi$; $\text{VerifyNonRevoked}(\text{acc}, x, \pi) \to \{0, 1\}$.

RSA ACCUMULATOR

Let $n = pq$ be an RSA modulus with p, q safe primes, $g \in QR_n$. For a set of valid credential identifiers $V = \{x_1, \ldots, x_k\}$ mapped to distinct primes p_i:

$$\text{acc} = g^{\prod_{i=1}^{k} p_i} \bmod n$$

Membership witness for x_i (with prime representative p_i):

$$w_i = g^{\prod_{j \neq i} p_j} \bmod n$$

Verification: $w_i^{p_i} \equiv \text{acc} \pmod n$.

NON-MEMBERSHIP PROOF

To prove $x \notin V$ (credential not revoked), use the Bezout coefficient method. Since p_x (prime for x) is coprime to $\prod_{j \in V} p_j$, by the extended Euclidean algorithm:

$$a \cdot p_x + b \cdot \prod_{j \in V} p_j = 1$$

The non-membership witness is $(g^b \bmod n, a)$. Verification checks:

$$(g^b)^{\prod_{j \in V} p_j} \cdot \text{acc}^{a \cdot p_x} \neq \text{acc} \quad \text{...wait, more precisely:}$$

$$\text{acc}^a \cdot (g^b)^{p_x} = g^{a \cdot \prod p_j + b \cdot p_x} = g^1 = g$$

The verifier checks $\text{acc}^a \cdot d^{p_x} = g$ where $d = g^b$. In the ZK setting, the holder proves knowledge of (a, d) satisfying this without revealing p_x.

DYNAMIC ACCUMULATOR UPDATES

Adding element x_{k+1} (prime p_{k+1}):

$$\text{acc}' = \text{acc}^{p_{k+1}} \bmod n$$

Existing witnesses update: $w_i' = w_i^{p_{k+1}} \bmod n$. Revoking element x_r (removing from accumulated set): the issuer with p, q computes

$$\text{acc}' = \text{acc}^{1/p_r} \bmod n$$

using knowledge of $\phi(n)$. Existing witnesses for non-revoked elements must be updated: $w_i' = w_i^{1/p_r} \bmod n$, or batch update via the new accumulator value.

MERKLE-BASED REVOCATION

Alternative approach: maintain a Merkle tree over revoked identifiers. Non-revocation proof = Merkle non-membership proof (sorted tree or sparse Merkle tree). For a sparse Merkle tree with 2^{256} leaves, a non-membership proof is a Merkle path of length 256 showing the leaf at the credential's position is empty (default hash). Proof size: $256 \times 32 = 8192$ bytes (can be compressed with pruning).

PRIVACY-PRESERVING STATUS LISTS

W3C StatusList2021: a bitstring where position i indicates whether credential i is revoked. Privacy issue: the verifier learns which position the holder checks. ZK overlay: the holder proves in zero knowledge that their credential's bit is 0 (not revoked) without revealing which bit position. This requires a ZK proof over the status list, e.g., proving $\text{StatusList}[\text{idx}] = 0$ inside a SNARK with the index as private input.

HISTORY Josh Benaloh and Michael de Mare (Yale University) (1993). *The simplest revocation approach — a revocation list (CRL) — was proposed in X.509 certificates in 1988. It took 15 years to develop privacy-preserving revocation that does not reveal WHICH credential is being checked. Even today, most deployed credential systems use non-private revocation (OCSP, CRLs) because accumulator-based revocation is complex to implement correctly.*

LIMITATIONS

- Witness updates are expensive: when k credentials are revoked in an epoch, each remaining user must update their witness — naive approach is O(k) multiplications per user per epoch, though batching reduces this to O(1) amortized with an update server
- RSA accumulators require a trusted setup (same problem as CL): the accumulator modulus n = p*q must have secret factors. Pairing-based accumulators (Nguyen 2005) avoid this but are less efficient
- Revocation latency vs. privacy tradeoff: checking revocation in real-time requires the accumulator to be updated on-chain after every revocation, creating a timing side-channel (the verifier learns WHEN the accumulator last changed)
- On Sui, the accumulator is a shared object — concurrent revocations by multiple issuers create contention, as shared object transactions go through Sui's consensus path (~400ms) rather than the fast path (~200ms)

EXERCISES

1. **design** — Design the revocation flow for your Sui credential system. Where is the accumulator stored, who updates it, and how does the user prove non-revocation inside the Groth16 circuit?
2. **conceptual** — Why can you NOT simply publish a list of revoked credential IDs on Sui? What privacy property would this break?

TEEs (SGX, Attestation, Trust Model)

Trusted Execution Environments (TEE)

INTUITIVE

A transparent vault inside your computer. Code runs inside where NOBODY (not even the OS or hardware owner) can see or tamper with the data. The vault has a window showing results but not internals. Even the datacenter operator with physical access cannot peek inside.

KEY POINTS

- Hardware-based isolation: CPU enforces memory encryption and access control
- Confidentiality: data inside the TEE is encrypted even in RAM
- Integrity: any tampering with enclave code or data is detected
- Defense in depth: protects against compromised OS, hypervisor, and physical attacks
- Attestation: TEE can prove to remote parties what code it is running

WHY IT MATTERS

In your thesis, TEEs can run credential issuance or ZK proof generation in a protected environment. This enables a hybrid trust model where TEE handles performance-critical operations while ZKPs provide the mathematical guarantee.

THESIS ANGLE

In your thesis, TEEs provide the fast path for credential verification. When low latency matters (point-of-sale payments), the merchant's TEE verifies the BBS+ credential in ~1ms instead of waiting for on-chain ZKP verification (~500ms proof generation + block time). The TEE guarantees the merchant's software honestly checks the credential without seeing the hidden attributes — hardware-enforced privacy at payment-terminal speed.

TECHNICAL

A hardware-isolated computation environment defined by a tuple of operations $\mathrm{TEE} = (\mathrm{Init}, \mathrm{Execute}, \mathrm{Attest}, \mathrm{Seal})$ with the guarantee that for all PPT adversaries $\mathcal{A}$ controlling the OS, hypervisor, and co-located processes: $\Pr[\mathcal{A} \text{ reads or modifies enclave state}] \leq \mathrm{negl}(\lambda)$.

THREAT MODEL

The adversary $\mathcal{A}$ controls the entire software stack: operating system, hypervisor, BIOS/UEFI, and all other processes. The adversary may also have physical access to the machine (bus probing, cold-boot attacks). The ONLY trusted component is the CPU package itself (including its microcode and key material fused at manufacturing). Formally: the trusted computing base (TCB) is reduced to the CPU die.

ISOLATION GUARANTEE

For any adversary $\mathcal{A}$ and enclave program P with state st:

$$\Pr[\mathcal{A}^{\mathrm{Execute}(P,\cdot)}(\mathrm{st}) \neq \mathrm{st}] \leq \mathrm{negl}(\lambda) \quad (\text{integrity})$$

$$\Pr[\mathcal{A}^{\mathrm{Execute}(P,\cdot)} \text{ learns } \mathrm{st}] \leq \mathrm{negl}(\lambda) \quad (\text{confidentiality})$$

Integrity means the adversary cannot alter the enclave's execution. Confidentiality means the adversary learns nothing about the enclave's internal state.

MEMORY ENCRYPTION

The Memory Encryption Engine (MEE) encrypts all data leaving the CPU to DRAM using AES-128-CTR (SGX) or AES-128-XTS (TDX/SEV). Each cache line (64 bytes) is encrypted with a unique tweak derived from the physical address. This prevents bus-snooping and cold-boot attacks from reading enclave memory. The encryption keys are generated at boot and stored in CPU registers (never in DRAM).

INTEGRITY TREE

A Merkle tree is maintained over enclave memory pages to detect tampering. Each page's MAC (message authentication code) is stored in the tree. On every cache line fetch from DRAM, the CPU verifies the MAC against the integrity tree. Replay attacks (serving stale data) are detected via version numbers at each tree node. Tree depth: $O(\log(\mathrm{EPC\ size}/\mathrm{page\ size}))$, typically ~20 levels.

UC-SECURE IDEAL FUNCTIONALITY

In the Universal Composability (UC) framework, a TEE realizes an ideal functionality $\mathcal{F}_{\mathrm{att}}$ for attested computation:

$$\mathcal{F}_{\mathrm{att}}(\mathrm{prog}, \mathrm{input}) \rightarrow (\mathrm{output}, \sigma_{\mathrm{att}})$$

where σ_{att} is an attestation binding the output to the program prog and the hardware identity. Any protocol using $\mathcal{F}_{\mathrm{att}}$ composes securely with other UC-secure protocols, enabling modular security proofs for TEE-based systems.

HISTORY Multiple vendors (ARM TrustZone 2004, Intel SGX 2015, AMD SEV 2016) (2004). *The concept of a 'secure coprocessor' dates back to the IBM 4758 cryptographic coprocessor (1998), which self-destructed (zeroized keys) if tampered with physically. Modern TEEs achieve similar goals in software using CPU microcode, without the dramatic hardware self-destruction.*

LIMITATIONS

- TEE security fundamentally depends on trusting the CPU manufacturer (Intel, AMD, ARM) — a compromised or backdoored manufacturing process invalidates all security guarantees, and there is no way to independently verify silicon integrity
- Side-channel attacks have repeatedly broken TEE confidentiality: Foreshadow/L1TF (2018), Plundervolt (2019), AEPIC Leak (2022), WireTap (CCS 2025) — each new CPU generation brings new attack surfaces
- Performance overhead: SGX enclave transitions (ECALL/OCALL) cost ~8,000-14,000 CPU cycles each, and the EPC memory limit (128-256MB) causes expensive page swapping for large workloads
- TEEs provide confidentiality and integrity but NOT availability: a malicious OS can refuse to schedule the enclave, starve it of resources, or simply kill the process

EXERCISES

1. **conceptual** — Why does your thesis use TEEs as a 'performance optimization' rather than a 'trust anchor'? What happens if every TEE in the system is compromised simultaneously?
2. **comparison** — Compare Intel SGX, AMD SEV, and ARM TrustZone across three dimensions: isolation granularity, memory encryption, and attestation model.

Intel SGX Enclaves

INTUITIVE

A diplomatic embassy inside a foreign country. The embassy (enclave) operates under its own rules, and the host country (OS) cannot enter or spy on it, even though it physically surrounds it. Communication happens only through official channels (ECALL/OCALL).

KEY POINTS

- Per-process enclaves: each application creates its own isolated enclave
- EPC (Enclave Page Cache): hardware-encrypted memory region, limited to ~128MB
- ECALL/OCALL interface: structured entry/exit points between trusted and untrusted code
- Sealing: encrypt data to a key derived from enclave identity for persistent storage
- Memory Encryption Engine (MEE): encrypts all data leaving the CPU to DRAM

WHY IT MATTERS

SGX enclaves can host a credential issuer or a ZK prover on Sui validators. The enclave ensures that secret keys (issuer signing key, user private data) never leave the protected environment, even if the host machine is compromised.

THESIS ANGLE

Your TEE layer uses SGX enclaves on Sui validator nodes or payment processors. The enclave loads the BBS+ verification logic and issuer public keys, sealed to MRENCLAVE. Users submit encrypted credential presentations to the enclave via ECALL. The enclave verifies the BBS+ proof, checks the policy, and returns only accept/reject — the credential attributes never leave the encrypted enclave memory, even if the validator node's OS is compromised.

TECHNICAL

An SGX enclave E is a process-level isolated execution environment with a cryptographic measurement $\text{MRENCLAVE} = H(\text{code} \| \text{data} \| \text{layout} \| \text{config})$ computed by the CPU during enclave creation (ECREATE + EADD + EEXTEND + EINIT sequence). H is SHA-256 applied incrementally to each page added to the enclave.

ENCLAVE PAGE CACHE (EPC)

The EPC is a reserved region of physical memory (Processor Reserved Memory, PRM) whose contents are encrypted by the MEE. SGX1: EPC size fixed at boot, typically 128MB (93.5MB usable after metadata). SGX2 (EDMM): dynamic memory management, EPC up to 512GB. When EPC is full, pages are encrypted and evicted to regular DRAM (EWB instruction), then loaded back on demand (ELD instruction). Each evicted page carries a MAC and version number to detect tampering and replay.

ECALL / OCALL INTERFACE

ECALL (Enclave Call): untrusted application enters the enclave through a predefined entry point table (ECALL table). The CPU switches to enclave mode, saves untrusted registers, and begins execution at the specified entry point. OCALL (Outside Call): enclave calls back to untrusted code. Enclave state is saved (encrypted), control transfers to untrusted handler. Return value is NOT trusted (enclave must validate). The ECALL/OCALL boundary is the attack surface: all OCALL return values and ECALL parameters must be sanitized (Iago attacks: malicious OS provides crafted return values to manipulate enclave logic).

SEALING (PERSISTENT ENCRYPTION)

Data persistence across enclave restarts via sealing:

$$\text{ct} = \text{AES-GCM}(K_{\text{seal}}, \text{data})$$

where the seal key K_{seal} is derived by the EGETKEY instruction from:

$$K_{\text{seal}} = \text{KDF}(K_{\text{root}},\ \text{policy} \| \text{SVN} \| \text{CPUSVN} \| \text{KE\_ID})$$

K_{root} is a per-CPU fused key that never leaves the hardware. Two sealing policies determine the identity used in derivation.

SEALING POLICIES: MRENCLAVE VS MRSIGNER

MRENCLAVE policy: K_{seal} is bound to the exact enclave code hash. Only the identical enclave binary can unseal the data. Pros: maximum isolation. Cons: any code update (even a bug fix) changes MRENCLAVE, losing access to sealed data.

$$K_{\text{seal}}^{\text{enclave}} = \text{KDF}(K_{\text{root}}, \text{MRENCLAVE})$$

MRSIGNER policy: K_{seal} is bound to the developer's signing key hash. Any enclave signed by the same developer key can unseal. Enables seamless upgrades.

$$K_{\text{seal}}^{\text{signer}} = \text{KDF}(K_{\text{root}}, \text{MRSIGNER} \| \text{ProdID})$$

KEY DERIVATION VIA EGETKEY

EGETKEY is a leaf function of the ENCLU instruction that derives enclave-specific keys from the CPU root key. It takes a KEYREQUEST structure specifying: key type (seal, report, provisioning), key policy (MRENCLAVE or MRSIGNER), SVN (Security Version Number), and optional key ID. The derived key is deterministic: same inputs on same CPU always produce the same key. This enables persistent sealed storage and local attestation report generation.

HISTORY Intel Corporation (2015). *Intel removed SGX from consumer CPUs in 2022 after years of side-channel attacks embarrassed the 'impenetrable enclave' marketing. However, cloud providers (Azure Confidential Computing, Alibaba Cloud) doubled down on SGX for servers. The WireTap attack (CCS 2025) later showed that even DCAP attestation keys could be extracted with <$1000 of equipment — the cat-and-mouse game continues.*

LIMITATIONS

- EPC (Enclave Page Cache) is limited to 128-256MB: workloads exceeding this trigger page swapping through the untrusted OS, adding ~1000x latency per swapped page and creating page-fault side channels observable by the OS
- ECALL/OCALL transitions cost 8,000-14,000 cycles each (~3-5 microseconds) — a chatty interface design with frequent enclave transitions can degrade throughput by 10-100x compared to native execution
- Intel deprecated SGX in consumer CPUs (12th-gen Alder Lake, 2022), limiting deployment to Xeon server processors — this means end-user devices cannot run SGX enclaves, restricting TEE-based credential verification to server infrastructure
- Sealing binds data to either MRENCLAVE (code identity) or MRSIGNER (developer identity) — sealed data becomes unrecoverable if the enclave binary is updated (MRENCLAVE changes) unless migration logic is explicitly implemented

EXERCISES

1. **design** — Design the ECALL interface for an SGX enclave that verifies BBS+ credentials in your Sui payment system. What functions would you expose, and what data crosses the enclave boundary?
2. **conceptual** — Why did Intel deprecate SGX in consumer CPUs but keep it in Xeon server processors? What does this mean for your thesis architecture?

Remote Attestation

INTUITIVE

A doctor credential verification before sharing your medical records. You verify the doctor identity, their hospital legitimacy, and that their computer is secure. The TEE proves: 'I am running exactly this code, on genuine Intel hardware, completely untampered.'

KEY POINTS

- MRENCLAVE: hash of enclave code and initial data (identity of WHAT is running)
- MRSIGNER: hash of the enclave signer key (identity of WHO built it)
- Quote generation: enclave produces a signed statement of its measurements
- IAS (Intel Attestation Service): centralized verification by Intel
- DCAP (Data Center Attestation Primitives): decentralized verification without calling Intel

WHY IT MATTERS

Remote attestation is critical for your thesis trust model. A Sui smart contract (or off-chain verifier) can verify that a TEE-based credential issuer is running the correct code, bridging hardware trust guarantees into the blockchain protocol.

THESIS ANGLE

Before sending their credential to a TEE, the user verifies via remote attestation that the enclave is running the correct verification code (MRENCLAVE matches the audited binary). The attestation report includes a fresh nonce from the user, preventing replay. On Sui, the attestation could be verified on-chain once, and subsequent users trust the attested enclave — amortizing the attestation cost across many credential verifications.

TECHNICAL

A protocol Π_{att} between an enclave E (prover) and a remote verifier V: E proves to V that it is running a specific program (identified by MRENCLAVE) on genuine hardware, with integrity guarantees. Formally, V obtains a statement $(\text{MRENCLAVE}, \text{MRSIGNER}, \text{ProdID}, \text{SVN}, \text{userData})$ authenticated by the hardware manufacturer's root of trust.

LOCAL ATTESTATION (INTRA-PLATFORM)

Enclave A verifies enclave B on the same platform:

$$\text{REPORT}_B = (\text{MRENCLAVE}_B, \text{MRSIGNER}_B, \text{userData}_B, \text{MAC}_B)$$

where $\text{MAC}_B = \text{CMAC}(K_{\text{report}(A,B)}, \text{REPORT\_BODY}_B)$. The report key $K_{\text{report}(A,B)}$ is derived from the CPU root key and target enclave A's identity. Enclave A retrieves the same key via EGETKEY and verifies the MAC. This establishes a secure channel between two enclaves on the same CPU without external trust.

EPID-BASED REMOTE ATTESTATION (SGX1)

Step 1: Application enclave generates REPORT (via EREPORT instruction). Step 2: Quoting Enclave (QE, Intel-provided) verifies REPORT locally, then signs it using Enhanced Privacy ID (EPID) group signature:

$$\text{QUOTE} = \text{EPID.Sign}(sk_{\text{member}}, \text{REPORT})$$

EPID is a group signature scheme: all members of a group (all SGX CPUs) share a group public key, but each has a unique member private key. Signatures are unlinkable across sessions. Step 3: Verifier sends QUOTE to Intel Attestation Service (IAS), which validates the EPID signature, checks revocation lists, and returns a signed attestation report. Step 4: Verifier checks IAS report signature (Intel's public key) and MRENCLAVE value.

DCAP (DATA CENTER ATTESTATION PRIMITIVES, SGX2)

DCAP replaces EPID with ECDSA (P-256) quotes, removing the Intel online dependency:

$$\text{QUOTE}_{\text{DCAP}} = \text{ECDSA.Sign}(sk_{\text{QE}}, \text{REPORT})$$

The QE's signing key is certified by a Provisioning Certification Enclave (PCE), which holds a key certified by Intel's Provisioning Certification Key (PCK). Certificate chain: QUOTE signature ← QE cert ← PCK cert ← Intel Root CA. Verification is fully offline: the verifier caches the PCK certificate chain and validates locally.

ATTESTATION REPORT CONTENTS

The REPORT/QUOTE body contains:

MRENCLAVE	(32B)	enclave code measurement
MRSIGNER	(32B)	enclave signer key hash
ISV_PROD_ID	(2B)	product identifier
ISV_SVN	(2B)	security version number
REPORT_DATA	(64B)	user-defined data

The REPORT_DATA field is critical for binding: typically set to $H(pk_{\text{enclave}} \| \text{nonce})$ to bind the attestation to an ephemeral key pair, enabling a secure channel establishment after attestation.

ATTESTATION-KEY BINDING FOR SECURE CHANNELS

The enclave generates a key pair (sk_E, pk_E) and sets REPORT_DATA = $H(pk_E \| \text{nonce})$. After attestation, the verifier is convinced that pk_E belongs to a genuine enclave running the attested code. A TLS session using pk_E as the server certificate ensures: (1) authenticity (attestation proves code identity), (2) confidentiality (TLS encrypts the channel), (3) freshness (nonce prevents replay of old attestations). This pattern (RA-TLS) is used by Gramine and other SGX frameworks.

HISTORY Trusted Computing Group (TCG) / Intel (2003). *EPID (Enhanced Privacy ID) uses group signatures — all SGX platforms share a group key, so the attestation verifier cannot distinguish individual CPUs. This was designed for privacy, but it also means Intel cannot selectively revoke a single compromised CPU without revoking the entire group. DCAP switched to ECDSA (per-platform keys) to fix this, but at the cost of revealing platform identity.*

LIMITATIONS

- IAS (Intel Attestation Service) is centralized: if Intel's servers are down or Intel decides to revoke attestation for a platform, the enclave cannot prove its integrity — this is a single point of failure for any system relying on IAS
- DCAP eliminates the Intel dependency but introduces a new problem: per-platform ECDSA keys reveal platform identity, breaking the privacy guarantees that EPID provided — a verifier can track which physical machine generated each attestation
- Attestation reports prove code identity (MRENCLAVE) at load time but NOT runtime behavior: if the enclave code has a bug that leaks secrets through a side channel, the attestation still shows 'correct code is running'
- The WireTap attack (CCS 2025) demonstrated that DCAP attestation keys can be extracted with <$1000 of hardware equipment, enabling attestation forgery — an attacker can produce fake attestation reports for code that is not actually running in an enclave

EXERCISES

1. **design** — Design the attestation verification flow for your Sui system. Should attestation be verified on-chain or off-chain? How do you handle attestation expiry?
2. **conceptual** — After the WireTap attack (CCS 2025) showed DCAP key extraction is possible, how should your thesis adjust its trust model for TEE attestation?

Side-Channel Attacks

INTUITIVE

Cracking a safe by listening to the clicks. Even though you cannot see inside the TEE, you can observe timing, power consumption, or cache behavior to leak secrets. The vault walls are strong, but information leaks through vibrations.

KEY POINTS

- Spectre/Meltdown: speculative execution attacks that bypass memory isolation
- Cache timing attacks: infer secret data by measuring cache hit/miss patterns
- Foreshadow (L1TF): reads SGX-protected memory via L1 cache side channel
- Mitigations: ORAM (oblivious RAM), constant-time code, cache partitioning
- Residual risk: new side channels are discovered regularly, no complete fix exists

WHY IT MATTERS

Side-channel risk is why your thesis does not rely on TEEs alone. The hybrid architecture uses ZKPs as the trust anchor (math-based, no side channels) and TEEs as a performance optimization. If the TEE is compromised, the ZKP layer still guarantees correctness.

THESIS ANGLE

Side channels are the main threat to your TEE layer. During BBS+ verification inside SGX, the pairing computation accesses different memory locations depending on the credential attributes. A cache-timing attack could infer which attributes are present. Your thesis mitigates this with constant-time BBS+ implementation — all attribute slots are processed regardless of disclosure policy, ensuring uniform memory access patterns.

TECHNICAL

An attack that exploits observable physical or micro-architectural side effects of computation to extract secret information. Formally, an adversary $\mathcal{A}$ with access to a leakage oracle $\mathcal{L}(\text{trace})$ that reveals partial information about the execution trace (memory access patterns, branch outcomes, timing) can reconstruct secrets with non-negligible probability.

TIMING ATTACKS

If execution time depends on secret data:

$$T(x) = T_{\text{base}} + f(x_{\text{secret}})$$

the adversary measures $T(x)$ across many inputs and correlates timing with hypotheses about x_{secret}. Example: RSA decryption with square-and-multiply has different timing for 0-bits vs 1-bits of the exponent. A single secret-dependent branch can leak the entire key over $O(n)$ observations where n is the key bit-length.

CACHE ATTACKS: PRIME+PROBE

The adversary and enclave share the L1/L2 cache (SGX does not partition caches). Prime: adversary fills a cache set with known data. Enclave executes: enclave memory accesses evict adversary's lines from specific cache sets. Probe: adversary measures access time to each cache set. Slow access = enclave used that set. Resolution: cache-set granularity (64B line, 64 sets in L1 = 4096B resolution). This reveals which cache lines the enclave accessed, leaking memory access patterns.

SPECTRE AND SPECULATIVE EXECUTION

Spectre exploits branch prediction: the CPU speculatively executes instructions beyond a mispredicted branch, accessing memory that would be forbidden on the correct path. Although speculative results are rolled back architecturally, they leave cache footprints (micro-architectural state). The adversary trains the branch predictor, triggers speculative execution in the enclave, then uses cache timing (Flush+Reload) to read the speculatively accessed data. SGX is particularly vulnerable because the adversary controls the OS and can manipulate branch predictor state.

FORESHADOW (L1 TERMINAL FAULT)

Foreshadow (CVE-2018-3615) exploits L1 terminal faults in SGX: when a page table entry is marked not-present, the CPU still speculatively reads from the L1 cache using the physical address. If the enclave data is in L1, the speculative read succeeds and the data can be extracted via cache side channel. This allows reading arbitrary EPC pages from L1 cache, completely breaking SGX confidentiality. Mitigated by Intel microcode updates (L1 cache flush on enclave exit).

FORMAL LEAKAGE MODEL

The leakage function for a program execution trace τ captures observable side effects:

$$\mathcal{L}(\tau) = \{\text{mem\_access\_pattern}(\tau),\ \text{branch\_outcomes}(\tau),\ \text{timing}(\tau)\}$$

A program is side-channel resistant if its leakage is independent of secret inputs:

$$\forall x_1, x_2 \in \text{Secret} : \mathcal{L}(\tau(x_1)) = \mathcal{L}(\tau(x_2))$$

Achieving this requires: no secret-dependent branches (constant-time conditionals), no secret-dependent memory access (constant-time table lookups), and no secret-dependent timing.

ORAM (OBLIVIOUS RAM)

ORAM makes memory access patterns independent of the actual data accessed. For N data blocks:

$$\text{Path ORAM} : O(\log N) \text{ overhead per access}$$

Each access reads and rewrites an entire tree path, shuffling blocks randomly. The adversary sees a random-looking access pattern regardless of the actual access sequence. Practical overhead: 10-100x slowdown depending on implementation. For SGX: ORAM protects against page-level access pattern leakage but is typically too slow for production use. Used in research prototypes (Obliviate, ZeroTrace).

HISTORY Paul Kocher (Cryptography Research Inc.) (1996). *The Foreshadow attack was independently discovered by two teams simultaneously: one from KU Leuven (Belgium) and one from the University of Michigan/University of Adelaide. Intel assigned it CVE-2018-3615 and patched it with microcode updates, but the fix reduced SGX performance by 20-40% due to L1 cache flushing on every enclave transition.*

LIMITATIONS

- Side channels are fundamentally unpatchable at the hardware level: each fix introduces new performance overhead (Foreshadow fix: 20-40% SGX slowdown from L1 cache flushing), and new attack vectors are discovered every 1-2 years
- Constant-time implementations prevent timing attacks but do not protect against cache-line attacks, power analysis, or electromagnetic emanation — defense requires addressing ALL channels simultaneously
- ORAM (Oblivious RAM) hides memory access patterns but adds 10-100x overhead: each memory access becomes O(log n) accesses to random locations, making it impractical for performance-critical credential verification
- Microarchitectural attacks (Spectre, MDS, LVI) exploit CPU design choices that are shared across all Intel/AMD processors — they cannot be fixed without fundamental CPU redesign, only mitigated with performance-costly software workarounds

EXERCISES

1. **conceptual** — During BBS+ verification inside SGX, which specific operations leak information through cache timing, and how would you make them constant-time?
2. **comparison** — Rank the following SGX attacks by severity for your thesis system: Foreshadow, Plundervolt, AEPIC Leak, WireTap. Which one is most dangerous for credential confidentiality?

Confidential Computing Landscape

INTUITIVE

SGX is one brand of armored truck, but the vault industry has many manufacturers. AMD SEV-SNP, ARM CCA, and RISC-V Keystone each build their vaults differently — some encrypt entire rooms (VMs), others partition buildings in half (secure world). Choosing a single vendor is a single point of failure, so your system must speak multiple vault dialects.

KEY POINTS

- AMD SEV-SNP: encrypts entire VM memory with per-VM keys, adds integrity protection (SNP = Secure Nested Paging) against hypervisor tampering
- ARM CCA (Confidential Compute Architecture, ARMv9 2021): introduces Realms as a new isolation primitive, separate from TrustZone secure world
- Intel TDX (Trust Domain Extensions): VM-level confidential computing, successor to SGX for multi-tenant cloud workloads
- RISC-V Keystone: open-source TEE framework using Physical Memory Protection (PMP), auditable hardware with no vendor trust
- Vendor diversity reduces single-point-of-failure risk: a vulnerability in one TEE platform does not compromise the entire system

WHY IT MATTERS

Your thesis benefits from TEE portability. If the credential verification logic is designed to run on any TEE (not just SGX), the system can migrate across cloud providers and hardware platforms, reducing vendor lock-in and improving resilience against platform-specific attacks.

THESIS ANGLE

For production deployment, your thesis should abstract the TEE layer behind a common interface: ConfidentialRuntime.verify(presentation) -> {accept, attestation}. The first implementation targets SGX (most mature tooling), but AMD SEV-SNP on Azure and ARM CCA on mobile are the migration paths. When a Foreshadow-class attack hits Intel, operators switch to AMD SEV-SNP nodes without changing the Sui smart contract — it verifies any valid attestation chain from a whitelisted set of TEE vendors.

TECHNICAL

A comparison of trust assumptions: TEE security relies on hardware manufacturer integrity and absence of physical/side-channel attacks. ZKP security relies on computational hardness assumptions (e.g., DLP, q-SDH, knowledge-of-exponent). A hybrid model achieves security under the disjunction: $\text{Secure}_{\text{hybrid}} \iff \text{Secure}_{\text{TEE}} \vee \text{Secure}_{\text{ZKP}}$.

ZKP TRUST ASSUMPTIONS

ZKP security rests on well-studied computational assumptions:

$$\text{Discrete Log}: \text{Given } (g, g^x), \text{ computing } x \text{ is hard}$$

$$q\text{-SDH}: \text{Given } (g, g^x, \ldots, g^{x^q}), \text{ computing } (g^{1/(x+c)}, c) \text{ is hard}$$

These assumptions are believed to hold as long as $P \neq NP$ (more precisely, as long as no sub-exponential algorithm exists for DLP in the relevant groups). Post-quantum: DLP and pairing-based assumptions are broken by Shor's algorithm. Lattice-based ZKPs (e.g., based on Module-LWE) resist quantum attacks but are less efficient.

TEE TRUST ASSUMPTIONS

TEE security requires:

$$\text{Trust}_{\text{TEE}} = \text{HW\_honest} \wedge \neg\text{physical\_access} \wedge \text{side\_channel\_mitigated}$$

Hardware honest: CPU manufacturer did not insert backdoors. No physical access: adversary cannot decap the CPU die (cost: ~$100K+ for advanced attacks). Side-channel mitigated: known side channels are patched and residual leakage is acceptable. Each assumption is empirical (not mathematical) and can be violated by supply chain attacks, nation-state adversaries, or novel micro-architectural discoveries.

HYBRID SECURITY MODEL

The hybrid architecture provides security under EITHER assumption:

$$\text{Security}_{\text{hybrid}} = \text{Security}_{\text{ZKP}} \cup \text{Security}_{\text{TEE}}$$

Meaning: the system remains secure if at least one of the two mechanisms is unbroken. If TEE is compromised (side-channel attack): ZKP layer ensures correctness and privacy (adversary cannot forge proofs or link presentations). If ZKP assumption is broken (e.g., quantum computer): TEE still provides confidentiality (computation happens inside enclave, outputs are encrypted). This is a defense-in-depth approach with graceful degradation.

TEE-ASSISTED ZKP PROVING

A key construction for the thesis: use TEE to generate ZK proofs efficiently.

$$\text{TEE}: (\text{witness}, \text{circuit}) \to \text{proof}$$

The witness (private data: credential attributes, user identity) never leaves the enclave. The ZKP (Groth16 proof) is published on-chain and verified by a Sui Move contract. Trust analysis: even if the TEE is compromised, the adversary learns the witness but CANNOT forge a proof for a false statement (ZKP soundness holds independently). If ZKP is broken, the TEE still prevents witness extraction (confidentiality holds).

UC COMPOSABILITY OF HYBRID MODEL

In the UC framework, the hybrid protocol realizes an ideal functionality $\mathcal{F}_{\text{hybrid}}$ that composes $\mathcal{F}_{\text{att}}$ (TEE attestation) with $\mathcal{F}_{\text{ZK}}$ (zero-knowledge proof). If both sub-functionalities are UC-realized, the composition theorem guarantees that $\mathcal{F}_{\text{hybrid}}$ is UC-secure. Caveat: $\mathcal{F}_{\text{att}}$ assumes no side-channel leakage, which weakens the UC guarantee. Practical solution: treat TEE as an efficiency optimization, not a security requirement. The system must be secure even with $\mathcal{F}_{\text{att}}$ replaced by a no-op.

PERFORMANCE COMPARISON

For a credential presentation (prove attributes satisfy a policy):

$$\begin{aligned} &\text{TEE-only}: && \sim 1\text{ms compute} + 200\text{ms attestation} \\ &\text{ZKP-only (Groth16)}: && \sim \text{1-5s prove}, \sim 5\text{ms verify} \\ &\text{Hybrid (TEE proves)}: && \sim 500\text{ms prove (in TEE)}, \sim 5\text{ms verify} \end{aligned}$$

TEE accelerates ZKP proving by ~2-10x through optimized native code and access to hardware AES-NI. On-chain verification cost is identical (Groth16: 3 pairings regardless of prover). Latency-sensitive use case (point-of-sale): TEE-only path (~200ms total). High-assurance use case (DeFi transaction): ZKP path with on-chain verification.

HISTORY AMD (SEV 2016, SEV-SNP 2020), ARM (TrustZone 2004, CCA 2021), Keystone (MIT/Berkeley 2020) (2016). *ARM TrustZone runs on over 10 billion devices (every modern smartphone has it), making it the most deployed TEE by volume. Yet it has the weakest isolation model — no memory encryption by default, and the secure world shares physical memory with the normal world. Samsung Knox, which secures 100M+ enterprise phones, is built on TrustZone despite its known limitations.*

LIMITATIONS

- AMD SEV-SNP still trusts the AMD Platform Security Processor (PSP), a closed-source ARM Cortex-A5 coprocessor inside every EPYC CPU — PSP firmware vulnerabilities (CVE-2021-26311, CVE-2023-31315) can undermine all SEV-SNP guarantees
- ARM CCA Realms are too new for production: ARMv9 CCA-capable hardware (Arm Cortex-X4, 2023+) has limited server deployment, and the attestation infrastructure is still maturing compared to Intel DCAP
- RISC-V Keystone relies on Physical Memory Protection (PMP) which provides coarser granularity than SGX page-level encryption — a single PMP entry protects a contiguous memory range, limiting the number of concurrent enclaves to the hardware PMP entry count (typically 8-16)

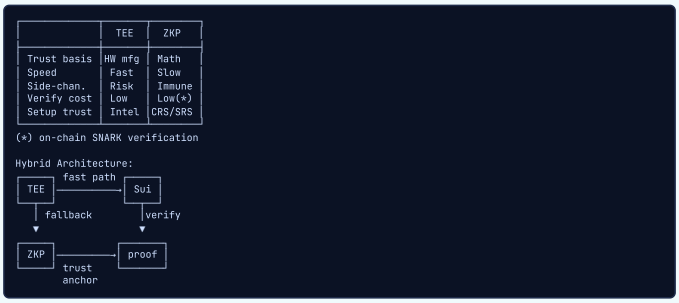
EXERCISES

1. **design** — Design a vendor-agnostic TEE abstraction layer for your Sui credential system. What interface would allow switching between SGX, SEV-SNP, and ARM CCA without changing the smart contract?
2. **comparison** — For your thesis, rank AMD SEV-SNP, ARM CCA, and RISC-V Keystone as alternatives to SGX. Which is the best migration target if Intel deprecates SGX entirely?

Trust Model: TEE vs ZKP

INTUITIVE

TEE is trusting a bank vault (hardware manufacturer guarantee). ZKP is trusting math itself (no hardware trust needed). Combining both is belt AND suspenders: if the vault is cracked, the math still holds. If the math is too slow, the vault accelerates it.



KEY POINTS

- TEE trusts hardware vendor (Intel/AMD): security depends on manufacturing integrity
- ZKP trusts mathematical hardness assumptions: no hardware dependency
- TEE is fast but vulnerable to side-channel and physical attacks
- ZKP is slow (proving takes seconds-minutes) but provides trustless verification
- Hybrid approach: TEE for speed, ZKP as fallback trust anchor when TEE is insufficient

WHY IT MATTERS

This is the central architectural insight of your thesis. On Sui, TEEs accelerate credential issuance and proof generation while ZKP circuits provide the on-chain verification. If a TEE is compromised, users can fall back to pure ZK proofs, maintaining the system security guarantees.

THESIS ANGLE

Your thesis makes an explicit architectural choice: TEE for speed, ZKP for trust. Low-value payments (coffee, transit) use TEE verification (~1ms, trusting hardware). High-value payments or DeFi interactions use on-chain Groth16 (~500ms, trustless math). If an SGX vulnerability is discovered, the system gracefully degrades to ZKP-only mode — slower but still secure. This dual-path architecture is the core contribution of your thesis.

TECHNICAL

A comparison of trust assumptions: TEE security relies on hardware manufacturer integrity and absence of physical/side-channel attacks. ZKP security relies on computational hardness assumptions (e.g., DLP, q-SDH, knowledge-of-exponent). A hybrid model achieves security under the disjunction: $\text{Secure}_{\text{hybrid}} \iff \text{Secure}_{\text{TEE}} \vee \text{Secure}_{\text{ZKP}}$.

ZKP TRUST ASSUMPTIONS

ZKP security rests on well-studied computational assumptions:

$$\text{Discrete Log} : \text{Given } (g, g^x), \text{ computing } x \text{ is hard}$$

$$q\text{-SDH} : \text{Given } (g, g^x, \ldots, g^{x^q}), \text{ computing } (g^{1/(x+c)}, c) \text{ is hard}$$

These assumptions are believed to hold as long as $P \neq NP$ (more precisely, as long as no sub-exponential algorithm exists for DLP in the relevant groups). Post-quantum: DLP and pairing-based assumptions are broken by Shor's algorithm. Lattice-based ZKPs (e.g., based on Module-LWE) resist quantum attacks but are less efficient.

TEE TRUST ASSUMPTIONS

TEE security requires:

$$\text{Trust}_{\text{TEE}} = \text{HW\_honest} \wedge \neg\text{physical\_access} \wedge \text{side\_channel\_mitigated}$$

Hardware honest: CPU manufacturer did not insert backdoors. No physical access: adversary cannot decap the CPU die (cost: ~$100K+ for advanced attacks). Side-channel mitigated: known side channels are patched and residual leakage is acceptable. Each assumption is empirical (not mathematical) and can be violated by supply chain attacks, nation-state adversaries, or novel micro-architectural discoveries.

HYBRID SECURITY MODEL

The hybrid architecture provides security under EITHER assumption:

$$\text{Security}_{\text{hybrid}} = \text{Security}_{\text{ZKP}} \cup \text{Security}_{\text{TEE}}$$

Meaning: the system remains secure if at least one of the two mechanisms is unbroken. If TEE is compromised (side-channel attack): ZKP layer ensures correctness and privacy (adversary cannot forge proofs or link presentations). If ZKP assumption is broken (e.g., quantum computer): TEE still provides confidentiality (computation happens inside enclave, outputs are encrypted). This is a defense-in-depth approach with graceful degradation.

TEE-ASSISTED ZKP PROVING

A key construction for the thesis: use TEE to generate ZK proofs efficiently.

$$\text{TEE} : (\text{witness}, \text{circuit}) \to \text{proof}$$

The witness (private data: credential attributes, user identity) never leaves the enclave. The ZKP (Groth16 proof) is published on-chain and verified by a Sui Move contract. Trust analysis: even if the TEE is compromised, the adversary learns the witness but CANNOT forge a proof for a false statement (ZKP soundness holds independently). If ZKP is broken, the TEE still prevents witness extraction (confidentiality holds).

UC COMPOSABILITY OF HYBRID MODEL

In the UC framework, the hybrid protocol realizes an ideal functionality $\mathcal{F}_{\text{hybrid}}$ that composes $\mathcal{F}_{\text{att}}$ (TEE attestation) with $\mathcal{F}_{\text{ZK}}$ (zero-knowledge proof). If both sub-functionalities are UC-realized, the composition theorem guarantees that $\mathcal{F}_{\text{hybrid}}$ is UC-secure. Caveat: $\mathcal{F}_{\text{att}}$ assumes no side-channel leakage, which weakens the UC guarantee. Practical solution: treat TEE as an efficiency optimization, not a security requirement. The system must be secure even with $\mathcal{F}_{\text{att}}$ replaced by a no-op.

PERFORMANCE COMPARISON

For a credential presentation (prove attributes satisfy a policy):

$$\begin{aligned} &\text{TEE-only} : && \sim 1\text{ms compute} + 200\text{ms attestation} \\ &\text{ZKP-only (Groth16)} : && \sim \text{1-5s prove}, \sim 5\text{ms verify} \\ &\text{Hybrid (TEE proves)} : && \sim 500\text{ms prove (in TEE)}, \sim 5\text{ms verify} \end{aligned}$$

TEE accelerates ZKP proving by ~2-10x through optimized native code and access to hardware AES-NI. On-chain verification cost is identical (Groth16: 3 pairings regardless of prover). Latency-sensitive use case (point-of-sale): TEE-only path (~200ms total). High-assurance use case (DeFi transaction): ZKP path with on-chain verification.

HISTORY Fan Zhang et al. (Cornell/IC3, Town Crier) and Raymond Cheng et al. (Oasis Labs, Ekiden) (2016). *Town Crier's SGX-based oracle was broken by the Foreshadow attack (2018) just two years after publication, vindicating the need for a ZKP fallback. The Oasis Network (launched 2020, from the Ekiden team) adopted the TEE+ZKP hybrid model specifically because pure TEE trust had proven fragile. Secret Network chose TEE-only and suffered when SGX vulnerabilities were disclosed — a cautionary tale your thesis explicitly addresses.*

LIMITATIONS

- The hybrid model adds architectural complexity: two verification paths (TEE fast path + ZKP fallback) means double the code surface, double the audit surface, and complex state management for path switching
- Defining the threshold between TEE-path and ZKP-path is policy-dependent and subjective: a $50 payment might be 'low-value' for one user and 'high-value' for another — the threshold must be configurable per-verifier without creating discrimination
- During TEE compromise recovery, there is a window where the system operates in degraded mode (ZKP-only): proof generation time increases from ~1ms to ~2-5 seconds, potentially causing transaction timeouts for latency-sensitive applications
- The ZKP fallback assumes users have sufficient client-side compute for proof generation: mobile devices with limited CPUs may struggle with Groth16 proving (~10-30 seconds on a phone vs. ~2 seconds on a laptop), creating a UX disparity

EXERCISES

1. **design** — Design the path-selection logic for your Sui payment system. Given a payment request, how does the system decide whether to use the TEE fast path or the ZKP trustless path?
2. **conceptual** — If a critical SGX zero-day is published tomorrow, describe the emergency response procedure for your thesis system. What happens to in-flight transactions?
3. **comparison** — Compare the trust assumptions of your thesis system vs. a pure ZKP system (like Zcash) vs. a pure TEE system (like Secret Network). What are the tradeoffs?

CHAPTER 3

Confidential Transactions & Range Proofs

Pedersen Commitments

INTUITIVE

A sealed envelope with a numeric lock. You write a number on a piece of paper, add a random 'salt' number, seal both in an envelope, and lock it with a special combination lock. Nobody can open the envelope (hiding), but you can later prove what number is inside without opening it. The mathematical magic: sealed envelopes can be added together, and the sum envelope contains the sum of the numbers inside.

KEY POINTS

- Perfectly hiding: given C, an adversary cannot determine v (information-theoretic)
- Computationally binding: committer cannot open to a different value (relies on discrete log)
- Homomorphic: sum of commitments = commitment to sum of values
- The coin IS the commitment — UTXOs are curve points, not numbers
- Used in: Confidential Transactions, Mimblewimble, Bulletproofs, your thesis payment system

WHY IT MATTERS

Pedersen commitments are the foundation of the confidential payment layer in your thesis. Every transaction amount is stored as a commitment on Sui. The homomorphic property lets validators verify conservation of value (inputs = outputs) without learning any amounts. Combined with Bulletproof range proofs, this prevents inflation attacks while maintaining privacy.

THESIS ANGLE

In your thesis system on Sui, when Alice sends 100 tokens to Bob, the transaction contains: C_in = 500*G + r1*H (Alice's input), C_out1 = 100*G + r2*H (Bob gets), C_out2 = 400*G + r3*H (Alice's change). Validators check C_in - C_out1 - C_out2 = 0*G + (r1-r2-r3)*H = commitment to zero. Balance verified, amounts hidden. The excess blinding factor (r1-r2-r3) serves as a transaction kernel signature.

TECHNICAL

A Pedersen commitment scheme operates over an elliptic curve group $\mathbb{G}$ of prime order p with two independent generators $G, H \in \mathbb{G}$ such that $\log_G(H)$ is unknown. To commit to a value $v \in \mathbb{Z}_p$, the committer samples a blinding factor $r \xleftarrow{\$} \mathbb{Z}_p$ and computes:

$$C = v \cdot G + r \cdot H$$

The commitment C is a single curve point. Opening requires revealing (v, r) and checking $C \stackrel{?}{=} v \cdot G + r \cdot H$. Pedersen (1991) introduced this scheme, which achieves perfectly hiding and computationally binding properties.

COMPUTATIONAL BINDING (DL ASSUMPTION)

Suppose an adversary finds two valid openings (v_0, r_0) and (v_1, r_1) with $v_0 \neq v_1$ for the same commitment C. Then:

$$v_0 \cdot G + r_0 \cdot H = v_1 \cdot G + r_1 \cdot H$$

$$(v_0 - v_1) \cdot G = (r_1 - r_0) \cdot H$$

Since $v_0 \neq v_1$, we have $v_0 - v_1 \neq 0$, so:

$$H = \frac{v_0 - v_1}{r_1 - r_0} \cdot G$$

This yields $\log_G(H) = (v_0 - v_1)(r_1 - r_0)^{-1}$, breaking the discrete log assumption. Therefore, any adversary breaking binding can be used to solve the DLP in $\mathbb{G}$.

INFORMATION-THEORETIC HIDING

For any commitment $C \in \mathbb{G}$ and any value $v' \in \mathbb{Z}_p$, there exists exactly one blinding factor r' such that $C = v' \cdot G + r' \cdot H$:

$$r' = (C - v' \cdot G) \cdot H^{-1}$$

(where H^{-1} denotes the scalar inverse of the discrete log). Since every value has a unique consistent blinding factor, the commitment C reveals zero information about v information-theoretically. Even an unbounded adversary cannot distinguish $\mathrm{Com}(v_0; r_0)$ from $\mathrm{Com}(v_1; r_1)$.

HOMOMORPHIC PROPERTY PROOF

Given two commitments $C_1 = v_1 \cdot G + r_1 \cdot H$ and $C_2 = v_2 \cdot G + r_2 \cdot H$, their elliptic curve sum is:

$$C_1 + C_2 = (v_1 + v_2) \cdot G + (r_1 + r_2) \cdot H$$

This is a valid commitment to $v_1 + v_2$ with blinding factor $r_1 + r_2$. More generally, for scalars a, b:

$$a \cdot C_1 + b \cdot C_2 = (av_1 + bv_2) \cdot G + (ar_1 + br_2) \cdot H$$

This additive homomorphism is the key property enabling confidential transaction balance verification: $\sum C_{\text{in}} - \sum C_{\text{out}} = 0 \cdot G + r_{\text{excess}} \cdot H$ proves conservation of value without revealing any amounts.

NOTHING-UP-MY-SLEEVE GENERATOR CONSTRUCTION

The security of Pedersen commitments requires that $\log_G(H)$ is unknown to all parties. The standard technique uses hash-to-curve: given a canonical generator G, compute $H = \mathrm{hash\_to\_curve}(\texttt{"Pedersen\_H"})$. Since the hash function is modeled as a random oracle, finding $\log_G(H)$ requires solving the DLP. This nothing-up-my-sleeve construction was used in Maxwell's Confidential Transactions proposal (2015) and in Grin/Beam implementations.

HISTORY Torben Pedersen (Aarhus University) (1991). *The scheme is so elegant it fits in a single equation (C = vG + rH), yet it underpins billions of dollars in confidential transactions across Monero, Mimblewimble, and enterprise blockchains today.*

LIMITATIONS

- Computationally binding only -- if quantum computers break discrete log, an adversary could open a commitment to any value they choose
- Requires NOTHING-UP-MY-SLEEVE generation of H -- if anyone knows log_G(H), binding breaks completely (can commit to v then open as v', since v*G + r*H = v'*G + r'*H is solvable when the discrete log relation is known)
- Homomorphic property is double-edged: enables CT balance verification but also enables algebraic manipulation if range proofs are absent (attacker can commit to negative values that sum to zero)

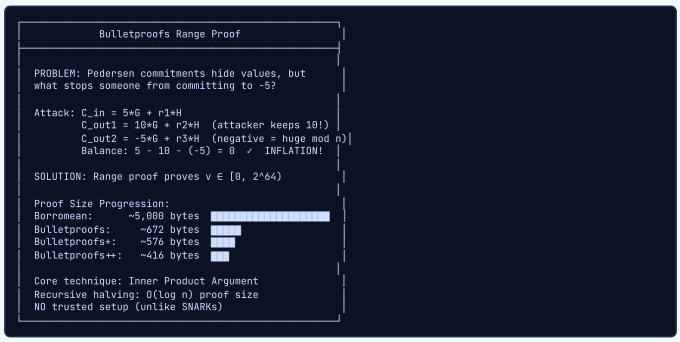
EXERCISES

1. **calculation** — On a toy curve where G=1 and H=7 (think of these as integers mod 11 for simplicity), compute the Pedersen commitment C = v*G + r*H for v=7, r=3. Then compute a second commitment for v=4, r=5. Verify the homomorphic property by adding both commitments and comparing with a direct commitment to v=11, r=8.
2. **conceptual** — Explain precisely why knowledge of log_G(H) = x (i.e., H = x*G) breaks the binding property of Pedersen commitments. Give a concrete attack: how would an adversary open commitment C to two different values?
3. **comparison** — Compare Pedersen commitments (C = vG + rH) with hash-based commitments (C = Hash(v || r)) for use in Confidential Transactions. Consider: hiding guarantee, binding guarantee, homomorphism, proof size, and quantum resistance. Which is better for CT and why?

Bulletproofs

INTUITIVE

A magic weighing scale. Imagine you need to prove your luggage weighs between 0 and 23 kg for a flight, but you do not want anyone to know the exact weight (maybe it is full of gold bars). A Bulletproof is like a special certificate from a trusted scale that says 'weight is in the valid range' without printing the actual number. The certificate itself is tiny — just a few hundred bytes — no matter how precise the range is.



KEY POINTS

- Proves a committed value is in [0, 2^64) without revealing it
- O(log n) proof size via inner product argument — ~672 bytes for 64-bit
- No trusted setup (unlike Groth16/PLONK) — relies only on discrete log assumption
- Aggregation: multiple range proofs compressed together (8 proofs ≈ 866 bytes, not 8x672)
- Bulletproofs++ (2024): reciprocal range proofs, ~416 bytes — 38% smaller
- Advisor preference: Bulletproofs over SNARKs for range proofs specifically (simpler, no toxic waste)

WHY IT MATTERS

Bulletproofs are the range proof mechanism your advisor specifically recommended. Every confidential output in your payment system needs a Bulletproof to prevent inflation. The no-trusted-setup property is critical — a compromised trusted setup in a currency means someone can print money. Bulletproofs++ is the target implementation.

THESIS ANGLE

In your system, every transaction output includes a Bulletproof++ range proof (~416 bytes) proving the committed amount is non-negative. This is verified by Sui validators as part of transaction validation. The fastcrypto library already has Bulletproofs support — your thesis extends it with Bulletproofs++ and integrates it with the ElGamal auditor key pattern for selective compliance.

TECHNICAL

Bulletproofs (Bunz, Bootle, Boneh, Poelstra, Wuille, Maxwell, 2018) are short non-interactive zero-knowledge arguments of knowledge for arithmetic circuits, with a primary application as range proofs. Given a Pedersen commitment $C = v \cdot G + r \cdot H$, a Bulletproof proves:

$$v \in [0, 2^n)$$

without revealing v or r. The proof size is $O(\log n)$ group elements (concretely, $2\lceil \log_2 n \rceil + 4$ group elements and 5 scalars for a single range proof). The core technique is an inner product argument that recursively halves the witness size.

REDUCTION FROM RANGE PROOF TO INNER PRODUCT

To prove $v \in [0, 2^n)$, decompose v into bits $v = \sum_{i=0}^{n-1} a_i \cdot 2^i$ where $a_i \in \{0, 1\}$. Define $\mathbf{a}_L = (a_0, \ldots, a_{n-1})$ and $\mathbf{a}_R = \mathbf{a}_L - \mathbf{1}^n$. The bit constraint $a_i \in \{0, 1\}$ is equivalent to:

$$\langle \mathbf{a}_L, \mathbf{a}_R \rangle = 0$$

since $a_i(a_i - 1) = 0 \iff a_i \in \{0, 1\}$. Combined with the constraint $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$, the range proof reduces to proving an inner product relation on committed vectors.

INNER PRODUCT ARGUMENT (RECURSIVE HALVING)

The inner product argument proves $\langle \mathbf{a}, \mathbf{b} \rangle = c$ for committed vectors of length n. In each round, split $\mathbf{a} = (\mathbf{a}_{\text{lo}}, \mathbf{a}_{\text{hi}})$ and $\mathbf{b} = (\mathbf{b}_{\text{lo}}, \mathbf{b}_{\text{hi}})$, each of length $n/2$. Compute cross-terms:

$$L = \langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle \cdot U + \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + \langle \mathbf{b}_{\text{hi}}, \mathbf{H}_{\text{lo}} \rangle$$

$$R = \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle \cdot U + \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + \langle \mathbf{b}_{\text{lo}}, \mathbf{H}_{\text{hi}} \rangle$$

The verifier sends challenge x (or it is derived via Fiat-Shamir). The prover folds:

$$\mathbf{a}' = x \cdot \mathbf{a}_{\text{lo}} + x^{-1} \cdot \mathbf{a}_{\text{hi}}$$

$$\mathbf{b}' = x^{-1} \cdot \mathbf{b}_{\text{lo}} + x \cdot \mathbf{b}_{\text{hi}}$$

After $\log_2 n$ rounds, the vectors reduce to single scalars, yielding $2 \log_2 n$ group elements (the L_i, R_i pairs) plus 2 final scalars.

PROOF SIZE ANALYSIS

For a 64-bit range proof ($n = 64$):

$$\text{Proof size} = 2\lceil \log_2 64 \rceil + 4 \text{ group elements} + 5 \text{ scalars}$$

$$= 2 \cdot 6 + 4 = 16 \text{ group elements} + 5 \text{ scalars}$$

On secp256k1 (33-byte points, 32-byte scalars): $16 \times 33 + 5 \times 32 = 528 + 160 = 688$ bytes. Compare with Borromean ring signatures (Maxwell, 2015): $2 + 32 \cdot 64 = 2050$ bytes for 64-bit range. Aggregation of m proofs adds only $2\lceil \log_2 m \rceil$ group elements: 8 proofs cost ~866 bytes total, not 8 times 688.

BULLETPROOFS++ IMPROVEMENTS

Bulletproofs++ (Eagen, Flaminia, Ganesh, 2024) introduces two main improvements. First, the reciprocal range proof technique: instead of decomposing v into bits, prove that v is a sum of digits in a chosen base b, using a reciprocal argument $\sum 1/(X - d_i)$ to enforce digit membership. Second, the norm argument replaces the inner product argument with a more efficient ℓ_2-norm argument. Combined, BP++ achieves ~416 bytes for a 64-bit range proof (38% smaller than original Bulletproofs) and ~30% faster verification.

HISTORY Benedikt Bunz (Stanford), Jonathan Bootle (UCL), Dan Boneh (Stanford), Andrew Poelstra (Blockstream), Pieter Wuille (Blockstream), Greg Maxwell (Blockstream) (2018). *Monero adopted Bulletproofs in October 2018 (just months after publication), reducing average transaction sizes by ~80%. Grin and Beam both launched on January 15, 2019 -- the same day -- as independent Mimblewimble implementations using Bulletproofs.*

LIMITATIONS

- Verification is O(n) group operations -- slower than SNARK verification which is O(1), making Bulletproofs less competitive for large general-purpose circuits
- No succinctness for general computation -- the inner product argument is efficient for range proofs and specific R1CS instances, but not practical as a general-purpose ZK proof system
- Prover time is O(n) with larger constants than some SNARK provers, and proving is single-threaded in naive implementations (parallelization requires multi-scalar multiplication optimizations)
- Not quantum-secure -- relies on the discrete log assumption on elliptic curves, which Shor's algorithm breaks

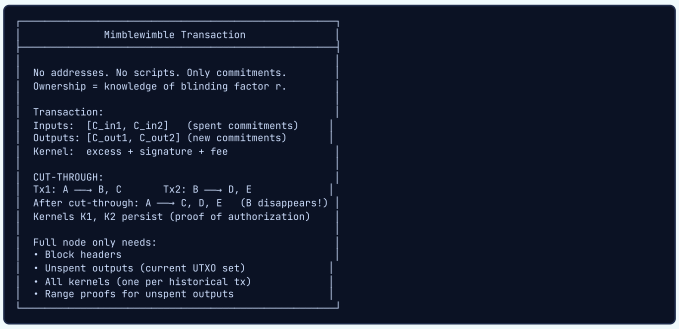
EXERCISES

1. **calculation** — Bulletproof proof size for a single 64-bit range proof is 2*ceil(log2(64))*32 + 9*32 = 2*6*32 + 288 = 672 bytes. Aggregated proofs share the inner product argument: for m proofs, size = 2*ceil(log2(64*m))*32 + 9*32. Compute the proof size for 1, 4, and 8 aggregated 64-bit range proofs. What is the per-proof cost at each level?
2. **conceptual** — Explain why the absence of a trusted setup is critical for a cryptocurrency range proof system. What specific attack becomes possible if a trusted setup is compromised in the context of Confidential Transactions?
3. **comparison** — Compare Bulletproofs, Groth16, and STARKs for use as range proof systems in Confidential Transactions. Build a comparison table with these dimensions: proof size, verification time, prover time, trusted setup, quantum resistance, and suitability for on-chain CT.

Mimblewimble

INTUITIVE

A restaurant where everyone eats in the dark. In a normal restaurant (Bitcoin), everyone can see who ordered what and how much they paid. In the Mimblewimble restaurant, there are no names on the bill, no menu items listed, and at the end of the night, if everyone ate in the dark but the kitchen's total food used matches the total money collected, the books balance. Even better: if Alice passed food to Bob who passed it to Carol, the intermediate step is erased from the records entirely (cut-through).



KEY POINTS

- Extends CT: no addresses at all, ownership proved by blinding factor knowledge
- Interactive transaction construction (sender + receiver must coordinate — biggest UX hurdle)
- Cut-through: intermediate outputs cancel algebraically, dramatic blockchain size reduction
- Kernels persist forever: proof of authorization even after cut-through
- Implementations: Grin (Rust, clean), Beam (C++, auditable), Litecoin MWEB (opt-in)
- Limitations: interactivity, no scripting, mempool graph leakage

WHY IT MATTERS

Mimblewimble is a reference architecture, not the thesis target. The key insights to borrow: coins as commitments (not values), kernel excess as transaction authorization, and the idea that addresses are unnecessary when ownership is cryptographic. Your thesis applies these ideas on Sui's account model rather than UTXO, which requires a different design (closer to Aptos's Twisted ElGamal approach).

THESIS ANGLE

Your thesis does not use Mimblewimble directly (Sui is account-based, not UTXO). However, the core idea — replacing plaintext amounts with Pedersen commitments and using range proofs — is exactly what the advisor recommended. The Aptos approach (Twisted ElGamal on account balances) adapts these Mimblewimble ideas to an account model, which is closer to what you will build on Sui.

TECHNICAL

Mimblewimble (Tom Elvis Jedusor, 2016; refined by Andrew Poelstra, 2016) is a UTXO-based confidential transaction protocol where every output is a Pedersen commitment $C = v \cdot G + r \cdot H$. There are no addresses and no scripts. Ownership is proved solely by knowledge of the blinding factor r. A valid transaction satisfies:

$$\sum_i C_{\text{in},i} - \sum_j C_{\text{out},j} - \text{fee} \cdot G = r_{\text{excess}} \cdot H$$

The excess $r_{\text{excess}} \cdot H$ is called the transaction kernel and is accompanied by a Schnorr signature proving knowledge of r_{excess}. Cut-through allows intermediate outputs to be algebraically eliminated from the blockchain.

TRANSACTION KERNEL (EXCESS AND SIGNATURE)

Define the kernel excess as:

$$E = \sum_i C_{\text{in},i} - \sum_j C_{\text{out},j} - \text{fee} \cdot G$$

If values balance ($\sum v_{\text{in}} = \sum v_{\text{out}} + \text{fee}$), then:

$$E = (\sum r_{\text{in}} - \sum r_{\text{out}}) \cdot H = r_{\text{excess}} \cdot H$$

The kernel is the pair (E, σ) where σ is a Schnorr signature proving knowledge of r_{excess} such that $E = r_{\text{excess}} \cdot H$. This serves as transaction authorization: only someone who knows both the input and output blinding factors can produce a valid kernel signature.

SCHNORR MULTI-SIGNATURE FOR KERNEL

In practice, sender and receiver jointly construct the kernel via an interactive multi-signature protocol. The sender knows r_{in} and the receiver chooses r_{out}. Neither party alone knows $r_{\text{excess}} = r_{\text{in}} - r_{\text{out}}$. They use a 2-of-2 Schnorr multi-signature:

$$\text{Each party picks nonce } k_i, \text{ shares } R_i = k_i \cdot H$$

$$R = R_1 + R_2, \quad e = H_{\text{sig}}(R \| E \| \text{msg})$$

$$s_i = k_i + e \cdot r_i^{\text{partial}}, \quad s = s_1 + s_2$$

The combined (R, s) is a valid Schnorr signature on the kernel excess E.

CUT-THROUGH ALGEBRA

Consider two transactions:

$$\text{Tx}_1 : C_A \to C_B + C_C \quad (\text{kernel } K_1)$$

$$\text{Tx}_2 : C_B \to C_D + C_E \quad (\text{kernel } K_2)$$

Output C_B from Tx1 is spent in Tx2. After cut-through, C_B is removed from both the output set and the input set:

$$\text{Merged} : C_A \to C_C + C_D + C_E \quad (\text{kernels } K_1, K_2)$$

Balance still holds because:

$$C_A - C_C - C_D - C_E = (C_A - C_B - C_C) + (C_B - C_D - C_E)$$

$$= K_1 + K_2 = (r_{\text{excess},1} + r_{\text{excess},2}) \cdot H$$

The kernels are preserved; only the intermediate output C_B disappears. A new node syncing the chain only needs: block headers, the current UTXO set, all kernels, and range proofs for unspent outputs.

HISTORY Anonymous ('Tom Elvis Jedusor'), formalized by Andrew Poelstra (Blockstream) (2016). *The name 'Mimblewimble' is the tongue-tying curse from Harry Potter -- fitting for a protocol that prevents the blockchain from 'speaking' transaction details. The mysterious author has never been identified.*

LIMITATIONS

- Interactive transactions -- sender and receiver must exchange data to co-construct a transaction (Grin's Slatepack protocol helps, but this remains a major UX hurdle compared to Bitcoin's send-to-address model)
- No scripting capability -- cannot natively support multisig, time-locked contracts, atomic swaps, or DeFi without workarounds like scriptless scripts via adaptor signatures, which are limited in expressiveness
- Transaction graph leakage in the mempool before cut-through -- a surveillance node connecting to many peers can observe the original transaction graph before block inclusion (Dandelion++ mitigates by randomizing propagation paths but does not fully solve the problem)
- Quantum vulnerability -- the entire security model rests on Pedersen commitments and Bulletproofs, both of which break if discrete log is broken by a quantum computer

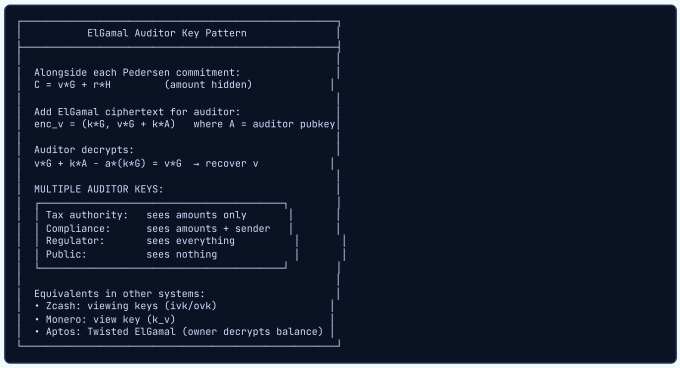
EXERCISES

1. **calculation** — Demonstrate cut-through on a 3-transaction chain. Tx1: Alice (input C_A = 10G + r1*H) sends to Bob (C_B = 7G + r2*H) with change C_A2 = 3G + r3*H. Tx2: Bob (input C_B = 7G + r2*H) sends to Carol (C_C = 5G + r4*H) with change C_B2 = 2G + r5*H. Tx3: Carol (input C_C = 5G + r4*H) sends to Dave (C_D = 5G + r6*H). After cut-through, which outputs remain? Which are eliminated?
2. **conceptual** — Why does Mimblewimble require interactive transaction construction (sender and receiver must communicate), while Bitcoin does not? What specific role does the receiver play in building the transaction?
3. **design** — How would you add time-locked contracts to Mimblewimble without a scripting language? Consider scriptless scripts with adaptor signatures. Describe a mechanism where Bob can only claim an output after block height 1000, with Alice able to reclaim it before then. How does this relate to your thesis's payment system on Sui?

ElGamal Auditor Keys

INTUITIVE

A special X-ray machine at airport security. Regular passengers see nothing in your sealed luggage (privacy). But security officers with the right clearance can X-ray specific bags when needed. Different officers have different clearance levels: customs sees the contents, tax authority sees the value, regular staff sees nothing. The ElGamal auditor key is like giving each authority their own X-ray machine that only reveals what they need to see.



KEY POINTS

- Encrypt (v, r) to auditor's public key alongside each Pedersen commitment
- Auditor decrypts and verifies C = v*G + r*H — sees actual amounts, cannot spend
- Multiple auditor keys: different disclosure levels for different authorities
- Common pattern in enterprise: Zcash viewing keys, Monero view keys
- Advisor requirement: 'Ability to have auditors come in. Special keys of the machine to audit.'
- Twisted ElGamal (Aptos): ciphertext = (v*G + r*PK, r*G) — owner can decrypt own balance

WHY IT MATTERS

This is a core advisor requirement for your thesis. The system must support selective auditability: a regulator can be given a key that reveals transaction amounts without revealing sender identity. This is implemented as an ElGamal envelope attached to each confidential output. The TEE component in your thesis adds proportional disclosure: low-value payments get aggregate audit only, high-value gets detailed records.

THESIS ANGLE

In your thesis, each confidential transaction on Sui includes an ElGamal envelope: enc_v = (k*G, v*G + k*A_regulator). The regulator can decrypt all amounts but cannot see who transacted (anonymous credentials handle identity). For GDPR proportionality, the TEE processes low-value transactions and only produces aggregate stats for the auditor, while high-value or suspicious transactions get full ElGamal envelopes. This is the commercially friendly tradeoff the advisor wants.

TECHNICAL

The ElGamal auditor key pattern encrypts transaction values to an auditor's public key alongside each Pedersen commitment. Given an auditor's key pair $(a, A = a \cdot G)$ and a transaction value v, the sender encrypts:

$$R = k \cdot G, \quad C_{\text{enc}} = v \cdot G + k \cdot A$$

where $k \xleftarrow{\$} \mathbb{Z}_p$ is a random ephemeral key. The ciphertext (R, C_{enc}) is published alongside the Pedersen commitment. Only the auditor, knowing a, can recover $v \cdot G$ and extract v. This is an elliptic curve variant of ElGamal encryption (ElGamal, 1985).

DECRYPTION AND VALUE RECOVERY

The auditor computes:

$$C_{\text{enc}} - a \cdot R = (v \cdot G + k \cdot A) - a \cdot (k \cdot G)$$

$$= v \cdot G + k \cdot a \cdot G - a \cdot k \cdot G = v \cdot G$$

This yields the point $v \cdot G$, not v directly. For small values ($v < 2^{40}$), the auditor recovers v via Baby-step Giant-step (BSGS): precompute $\{j \cdot G : 0 \le j < 2^{20}\}$ and search for $v \cdot G - j \cdot G = i \cdot (2^{20} \cdot G)$ with $0 \le i < 2^{20}$. Total: $O(2^{20})$ storage and $O(2^{20})$ group operations, recovering values up to 2^{40} in ~1 second.

TWISTED ELGAMAL VARIANT (APTOS)

The Twisted ElGamal variant (used in Aptos confidential transactions) restructures the ciphertext so the owner can decrypt their own balance efficiently:

$$C_{\text{tw}} = (v \cdot G + r \cdot \text{PK},\ r \cdot G)$$

where $\text{PK} = \text{sk} \cdot G$ is the owner's public key. The owner decrypts:

$$v \cdot G + r \cdot \text{PK} - \text{sk} \cdot (r \cdot G) = v \cdot G$$

then recovers v via BSGS. Crucially, this is also a valid Pedersen commitment (to v with blinding factor r under generator PK), so it supports homomorphic balance verification and Bulletproof range proofs simultaneously.

EQUALITY PROOF (TWO CIPHERTEXTS, SAME VALUE)

To prove that a Pedersen commitment $C = v \cdot G + r \cdot H$ and an ElGamal ciphertext $(R, C_{\text{enc}} = v \cdot G + k \cdot A)$ encrypt the same value v, the prover constructs a sigma protocol:

$$\text{Prove knowledge of } (v, r, k) \text{ such that:}$$

$$C = v \cdot G + r \cdot H \quad \wedge \quad C_{\text{enc}} = v \cdot G + k \cdot A \quad \wedge \quad R = k \cdot G$$

This is a standard conjunction of Schnorr-type proofs sharing the witness component v. The proof size is 3 group elements + 3 scalars (~200 bytes), and verification requires 6 multi-scalar multiplications.

MULTIPLE AUDITOR KEYS PATTERN

For m auditors with keys $A_1, \ldots, A_m$, the sender produces m ElGamal ciphertexts:

$$(R_j = k_j \cdot G,\ C_j = v \cdot G + k_j \cdot A_j) \quad j = 1, \ldots, m$$

Each auditor j can only decrypt their own ciphertext. Optimization: use a single k for all ciphertexts ($R = k \cdot G$) if auditors are non-colluding, reducing the overhead from m points to 1 shared R plus m encrypted values. The equality proof extends to prove all ciphertexts encrypt the same v as the Pedersen commitment.

HISTORY Taher ElGamal (Netscape Communications) (1985). *Taher ElGamal was Chief Scientist at Netscape and is often called the 'father of SSL'. The ElGamal signature scheme (not the encryption) was the basis for the DSA standard adopted by NIST, though ElGamal himself was not involved in that standardization.*

LIMITATIONS

- ElGamal is IND-CPA secure but NOT IND-CCA2 -- ciphertexts are malleable: an attacker can transform enc(v) into enc(2v) without knowing v by doubling both ciphertext components, which is dangerous if ciphertexts are used as inputs to other protocols
- Value recovery requires solving discrete log on v*G -- practical only for small values (brute force or baby-step-giant-step up to about 2^40, which takes ~1 million operations). For transaction amounts this is fine, but it cannot encrypt arbitrary data
- Auditor key is a single point of failure -- compromise of the auditor's private key reveals all past and future audited transactions. No key rotation without re-encrypting all historical ciphertexts
- No forward secrecy -- if the auditor key is compromised at any point in the future, all past transactions encrypted to that key are retrospectively exposed, unlike protocols with ephemeral key exchange

EXERCISES

1. **calculation** — Using toy numbers: generator G=1 (integers mod 23), auditor private key a=5, auditor public key A=a*G=5. Encrypt value v=100 using random nonce k=3. Compute the ElGamal ciphertext (R, E) where R=k*G and E=v*G + k*A. Then show decryption: the auditor computes E - a*R to recover v*G, then brute-forces v.
2. **design** — Design a threshold auditor scheme where 2-of-3 auditors must cooperate to decrypt transaction amounts. The individual auditor keys are a1, a2, a3 with public keys A1, A2, A3. How would you encrypt so that any 2 of the 3 can jointly decrypt? How does this mitigate the single-point-of-failure limitation? How might this apply to your thesis system?
3. **comparison** — Compare three approaches to selective auditability: (1) ElGamal auditor keys as used in your thesis design, (2) Zcash viewing keys (ivk/ovk in Sapling), and (3) Canton Network's observer pattern. For each, describe: what is revealed to the auditor, what remains hidden, the trust model, and the main tradeoff.

CHAPTER 4

Confidential Transfers on Sui (contra)

Twisted ElGamal — Encrypting an Amount You Can Still Add Up

INTUITIVE

Think of each balance as a sealed box on a public shelf. Anyone can see the box and even stack two boxes together, but only the owner has the key to read the number inside. Twisted ElGamal is the lock. A balance/amount m is hidden as a pair: a ciphertext c = r·g + m·h and a decryption handle d = r·pk. Here g and h are two fixed 'pegs' on the Ristretto255 curve whose relationship nobody knows, r is fresh randomness, and pk = x·g is your public key. The magic is two-fold. (1) Homomorphic: stack two boxes (add the curve points componentwise) and you get a sealed box of the SUM — the chain can credit a deposit to your balance without ever decrypting either. (2) Only you decrypt: with your secret x you compute c − d/x = m·h, which strips the randomness and leaves m sitting 'in the exponent' of h. The catch — and the next concept — is that recovering m from m·h means solving a discrete log, which is only feasible for small m.

```
Encrypt amount m under pk = x·g, randomness r:

      ciphertext        decryption handle
   c = r·g + m·h          d = r·pk
   └─ hides m ─┘          └─ binds to your key ─┘

Homomorphic add (deposit lands, chain never decrypts):
   (c1,d1) + (c2,d2) = (c1+c2, d1+d2)  =  Enc(m1+m2)

Decrypt (owner only, secret x):
   c - d/x = (r·g + m·h) - r·(x·g)/x
           = r·g + m·h - r·g
           = m·h          →  solve m = log_h(m·h)

   g = standard generator
   h = hash_to_curve("fastcrypto-blinding-gen-01")  (log_g h unknown)
```

KEY POINTS

- Group is Ristretto255 (sui::ristretto255 / group_ops); two generators g (standard) and h with unknown discrete-log relation — twisted_elgamal.move:11-53
- h is hardcoded: g_from_bytes(x"34ce1477c14558178089500a39c864e0f607b3c1f41ab398400e4a9de6d2c446") = hash_to_curve("fastcrypto-blinding-gen-01") — twisted_elgamal.move:49
- Encryption: c = r·g + m·h (ciphertext), d = r·pk (decryption handle); 'message in the exponent' — twisted_elgamal.move:29-40
- Additively homomorphic: add()/sub()/add_assign() add the curve points componentwise → Enc(m1±m2) — twisted_elgamal.move:67-94
- Decrypt: c − d/x = m·h, then brute-force the discrete log m = log_h(m·h)
- The struct is just two curve points: Encryption { ciphertext, decryption_handle } — twisted_elgamal.move:29
- Twisted ElGamal vs textbook ElGamal: the message rides on the SECOND generator h (a Pedersen-style commitment), which is what makes range proofs over the same commitment composable

WHY IT MATTERS

This is the bridge between Ch 2.2 (Pedersen commitments + Bulletproofs) and a real ledger. c = r·g + m·h IS a Pedersen commitment to m with blinding r; the extra handle d = r·pk is what upgrades a commitment into a decryptable ciphertext for a specific owner. Your thesis can frame the whole system as 'commit-and-prove on an encrypted balance', and twisted ElGamal is why the same commitment can be both spent homomorphically and range-proved.

THESIS ANGLE

In the cryptographic-preliminaries chapter, present twisted ElGamal as the design choice that avoids a SNARK for the payment path: because Enc is additively homomorphic, a deposit is a free curve-point add and only SPENDING needs a proof. Contrast this with a Zcash-style note model (full SNARK per spend) and quantify the trade: cheaper/auditable here, but bounded amount range (next concept) and no full anonymity set.

TECHNICAL

Let `G` be the Ristretto255 prime-order group with two generators `g` (standard) and `h`, where `log_g(h)` is unknown (nothing-up-my-sleeve: `h = hash_to_curve("fastcrypto-blinding-gen-01")`, twisted_elgamal.move:49). A keypair is `x ←$ Z_q`, `pk = x·g`.

Twisted ElGamal (message-in-the-exponent) encrypts `m ∈ Z_q` with fresh `r ←$ Z_q` as

```
c = r·g + m·h      (ciphertext component / Pedersen commitment to m)
d = r·pk            (decryption handle)
```

Decryption with `x`: compute `c - x⁻¹·d = r·g + m·h - r·g = m·h`, then recover `m = log_h(m·h)` by a bounded discrete-log search.

Additive homomorphism: `(c₁,d₁) ⊕ (c₂,d₂) = (c₁+c₂, d₁+d₂) = Enc(m₁+m₂)` under randomness `r₁+r₂`. This is the property that lets the chain credit deposits without decrypting.

Security: IND-CPA hiding reduces to DDH in `G`; binding of the embedded commitment reduces to the discrete-log hardness of `log_g(h)`.

ON-CHAIN TYPE & HOMOMORPHIC OPS (MOVE)

```
// twisted_elgamal.move:29
public struct Encryption has copy, drop, store {
    ciphertext: Element<G>,        // c = r·g + m·h
    decryption_handle: Element<G>, // d = r·pk
}

// add(): componentwise point addition  (twisted_elgamal.move:67)
Encryption {
    ciphertext:        g_add(&e1.ciphertext,        &e2.ciphertext),
    decryption_handle: g_add(&e1.decryption_handle, &e2.decryption_handle),
}
```

`add_assign` / `sub_assign` mutate in place; `add_assign_u64(e, a)` folds a PUBLIC value via `e.ciphertext += a·h` (no handle change, twisted_elgamal.move:97).

THE BLINDING GENERATOR H (VERIFY IT IS NUMS)

```
// twisted_elgamal.move:49-53
public(package) fun h(): Element<G> {
    g_from_bytes(
        &x"34ce1477c14558178089500a39c864e0f607b3c1f41ab398400e4a9de6d2c446"
    )
}
```

Soundness of the embedded Pedersen commitment requires `log_g(h)` to be unknown. The constant is the hash-to-curve image of the domain string `"fastcrypto-blinding-gen-01"` — a thesis security note: a reviewer should independently recompute this point to confirm it is nothing-up-my-sleeve.

TRIVIAL ENCRYPTIONS (PUBLIC VALUES ENTERING THE SYSTEM)

`encrypt_zero()` = `(0, 0)` (both identity); `encrypt_trivial(a)` = `(a·h, 0)` — a deterministic, randomness-free encryption used when a known public amount is wrapped in (twisted_elgamal.move:119-138). Because `d = 0`, a trivial encryption is NOT hiding — consistent with the fact that `wrap` reveals the amount.

HISTORY Twisted ElGamal popularized by the Zether confidential-payment paper (Bünz, Agrawal, Zamani, Boneh); implemented for Sui by Mysten Labs in the `contra` package (2026). *The blinding generator h is literally pinned in the contract as a 32-byte constant derived from the ASCII string "fastcrypto-blinding-gen-01" — so the whole scheme's hiding property rests on nobody ever finding log_g(h) of that one hashed string.*

LIMITATIONS

- Decryption is a discrete-log search → only small messages are recoverable (handled by the u16-limb trick, next concept)
- Hides amounts only — sender, receiver, and timing of every transfer stay fully public (no anonymity set)
- A trivial encryption of a known public value uses d = identity (encrypt_trivial, twisted_elgamal.move:128) — fine, but it means 'public-into-confidential' wraps are not hidden
- Security reduces to DDH on Ristretto255 + the hardcoded h being nothing-up-my-sleeve; a bad h (known log_g h) would silently break hiding/soundness

EXERCISES

1. **hands-on** — Open twisted_elgamal.move:35-94 and trace new(), add(), sub(): confirm that addition is just two g_add calls — the homomorphism is one line per component.
2. **concept** — Write out by hand why c − d/x = m·h cancels the randomness. Where does the secret x have to be inverted, and why can't the chain do this decryption itself?
3. **concept** — c = r·g + m·h is also a Pedersen commitment. Which value is the 'message' and which is the 'blinding'? Why does that ordering matter for the range proof in concept 3?

u16 Limbs & Bounded Aggregation — Keeping a u64 Decryptable

INTUITIVE

Decrypting needs a discrete-log search, which is only practical up to roughly 2^32. But balances are u64 (up to ~1.8e19) — far too big to brute-force directly. The fix is the same trick a child uses to read a big number: split it into digits. Instead of base-10 digits, `contra` uses four base-65536 digits (u16 'limbs'): amount = l0 + l1·2^16 + l2·2^32 + l3·2^48. Each limb is its own little sealed box, and each fits in [0, 2^16), so a precomputed baby-step/giant-step table cracks it in milliseconds. The subtlety: when you keep stacking encrypted deposits homomorphically, a limb that started below 2^16 grows. Stack k deposits and a limb can reach k·2^16. To stay under the 2^32 decryptable ceiling the contract counts how many values have been folded in (an `upper_bound`) and refuses new deposits once it hits ~2^16 — you must `merge` to reset. This is why incoming money piles up in a 'pending' box that you periodically sweep into your 'active' box.

```
u64 amount as four u16 limbs (each separately encrypted):

  amount = l0 + l1·2^16 + l2·2^32 + l3·2^48

  Enc:     [l0]  [l1]    [l2]    [l3]    ← 4 Twisted-ElGamal ciphertexts

  each limb ∈ [0,2^16)  → BSGS table cracks log_h in ms

Bounded aggregation (why a 'pending' balance exists):

  start: limb ≤ 2^16
  fold k deposits homomorphically → limb ≤ k·2^16
  cap k at 0xFFFF  ⇒  limb ≤ 0xFFFF·0xFFFF < 2^32  (still decryptable)

  upper_bound counts folds; one slot reserved (0xFFFE)
  too many deposits, no merge  →  abort EBalancesFull (code 10)
  fix: merge() pending → active, resets the bound
```

KEY POINTS

- EncryptedAmount = four Encryption limbs l0..l3; value = l0 + 2^16·l1 + 2^32·l2 + 2^48·l3 — encrypted_amount.move:36-44
- Each limb encrypts a u16 so decryption stays a feasible discrete-log search (~2^32 ceiling per limb) — baby-step/giant-step over a precomputed table
- EncryptedBalance<T> = { amount: EncryptedAmount, upper_bound: u16 }; upper_bound counts how many u16-bounded values were folded in — balance.move:42-45
- max_upper_bound() = 0xFFFF keeps every limb < 2^32; has_deposit_slot reserves one slot so the live cap is 0xFFFE — balance.move:115-122, contra.move:977-981
- Exceeding the cap aborts EBalancesFull (code 10); recovery = merge() to reset the bound — contra.move:97, 686-693
- collapse() recombines the four limbs into one Encryption via collapse_limbs (l0 + 2^16 l1 + ...) — encrypted_amount.move:180, 241
- merge_public() folds a known public value, merge_encrypted() folds an encrypted coin, each bumps upper_bound by 1 — balance.move:144-161

WHY IT MATTERS

Limb decomposition is the systems-engineering reality behind 'just use Bulletproofs'. Your Ch 2.2 study treats a range proof as proving m ∈ [0, 2^n); here n = 16 PER LIMB, and four limbs reconstruct the u64. The bounded-aggregation cap is a beautiful example of a cryptographic invariant (limbs stay < 2^32) enforced by a plain integer counter — a pattern worth highlighting when your thesis argues about which invariants need ZK vs which can be cheap bookkeeping.

THESIS ANGLE

Use the limb cap as a concrete 'practicality' data point in your evaluation chapter: the decryption cost (BSGS table size) directly bounds how big a single limb may grow, which in turn bounds how many deposits an account can receive before a mandatory merge. That is a real UX/throughput constraint your design must either inherit or improve (e.g. larger limbs + bigger tables, or a different decryption strategy).

TECHNICAL

Decryption recovers `m` via `log_h(m·h)`, feasible only for small `m` (baby-step/giant-step, ~ 2^{32} ceiling). A `u64` amount is therefore split into four base-2^{16} limbs:

$$\text{amount} = l_0 + l_1 \cdot 2^{16} + l_2 \cdot 2^{32} + l_3 \cdot 2^{48}, \quad \text{each } l_i \in [0,\ 2^{16})$$

An `EncryptedAmount` is four independent `Encryption` limbs (encrypted_amount.move:39). An `EncryptedBalance<T>` pairs the amount with an `upper_bound: u16` counting how many `u16`-bounded values were folded in (balance.move:42).

Aggregation invariant. Each homomorphic fold adds two limbs; starting from limbs $\le 2^{16}$, folding `k` values gives limbs $\le k \cdot 2^{16}$. Capping $k \le$ `0xFFFF` keeps every limb $\le$ `0xFFFF`$\cdot$`0xFFFF` $< 2^{32}$, i.e. inside the BSGS window (balance.move:115). `has_deposit_slot` reserves one slot, so the live cap is `0xFFFE` (contra.move:977).

LIMB STRUCTS & RECOMBINATION

```
// encrypted_amount.move:39
public struct EncryptedAmount has copy, drop, store {
    l0: Encryption, l1: Encryption, l2: Encryption, l3: Encryption,
}
// balance.move:42
public struct EncryptedBalance<phantom T> has store {
    amount: EncryptedAmount,
    upper_bound: u16,   // folded-value counter (growth bound)
}

// collapse_limbs: l0 + 2^16 l1 + 2^32 l2 + 2^48 l3   (encrypted_amount.move:241)
g_add(l0, &g_add(
    &g_mul(&scalar_from_u64(1 << 16), l1),
    &g_add(&g_mul(&scalar_from_u64(1 << 32), l2),
           &g_mul(&scalar_from_u64(1 << 48), l3))))
```

THE CAP AND ITS ENFORCEMENT

```
// balance.move:115-122
public(package) fun max_upper_bound(): u16 { 0xFFFF }
public(package) fun max_upper_bound_minus_1(): u16 { 0xFFFE }

// contra.move:977-981 — slot check before accepting a deposit
let cap  = balance::max_upper_bound_minus_1();
let used = self.active.upper_bound() + self.pending.upper_bound();
cap > used   // false → EBalancesFull (code 10)
```

Recovery is `merge()` (contra.move:686): it folds pending into active and the post-merge balance is re-stated to `upper_bound = 1`, freeing slots. This is a cryptographic invariant (limbs $< 2^{32}$) enforced by a plain `u16` counter, not by a proof.

WHY THREE BALANCES FOLLOW FROM THE CAP

A spend proof commits to a snapshot of `active`. If deposits added directly into `active`, a concurrent deposit would invalidate an in-flight proof. So incoming value is quarantined: `pending` (encrypted transfers) and `public_balance` (wrapped coins), both swept by `merge`. The `upper_bound` compared against the cap is the SUM of active + pending (contra.move:979), so a flood of un-merged deposits is what triggers `EBalancesFull`.

HISTORY Limb-encoding for ElGamal-in-the-exponent is folklore in confidential-payment systems (Zether, Solana confidential transfers); contra uses four u16 limbs (2026). *The whole reason your encrypted balance has THREE sub-balances (active / pending-encrypted / pending-public) is this limb cap: deposits can't safely auto-add into the balance you're about to prove a statement about, so they queue in 'pending' until you sweep them.*

LIMITATIONS

- Hard cap of ~65534 un-merged deposits before EBalancesFull — a busy receiver must merge often
- Decryption is still O(table) work per limb; very large per-limb values (after heavy aggregation) approach the 2^32 BSGS limit
- Four limbs = four ciphertexts = four range-proof commitments per amount → proof size and verify cost scale with limbs
- The cap is enforced by a u16 counter, not by the proof — an issuer override (set_balance_by_issuer) sets upper_bound = 1 and can desync accounting if misused

EXERCISES

1. **hands-on** — In encrypted_amount.move:241-257 read collapse_limbs and confirm the 1, 2^16, 2^32, 2^48 weighting. Then in balance.move:115-122 explain why 0xFFFF·0xFFFF is the exact safety margin.
2. **concept** — Why does an incoming transfer credit `pending` (contra.move:648) instead of `active`? Tie your answer to 'a spend proof commits to a snapshot of active'.
3. **hands-on** — Find where EBalancesFull is raised (contra.move:521-522 and 637) and describe the two situations (wrap vs receive) that can hit the cap.

Range Proofs + Sigma NIZKs — Proving 'No Inflation, No Overdraft' Without Revealing Anything

INTUITIVE

Encryption hides the amounts; proofs stop people from cheating with hidden amounts. Two cheats matter: making a NEGATIVE amount (which would mint money via wrap-around in the curve's scalar field) and spending more than you have. `contra` blocks both with zero-knowledge proofs the chain checks before trusting any encrypted number. A WellFormedProof bundles (a) Bulletproofs — compact range proofs that every limb really is in [0, 2^16) so nobody sneaks in a giant/negative value, and (b) per-limb sigma proofs (Schnorr-style) that each ciphertext is a genuine encryption under the claimed key. Separately, when you spend, a Chaum–Pedersen DDH proof shows 'my new encrypted balance equals my old balance minus exactly the amount I'm sending' — a subtraction the verifier checks on ciphertexts it cannot read. All these proofs are made non-interactive with Fiat–Shamir (hash the transcript to get the challenge), using Blake2b256 so the TypeScript prover and the Move verifier compute the identical challenge.

```
What the chain checks on a spend (it never sees plaintexts):

  WellFormedProof  =  Bulletproof(s)        +  ConsistencyProof / amount
                      └ each limb ∈[0,2^16)    └ each limb is valid Enc under pk
                        (no negative/inflation)   (Schnorr/ElGamal sigma)

  Balance equality (DdhProof, Chaum-Pedersen):
     prove   old_active - amount  ==  new_active     (on ciphertexts)
     i.e. knows x with X = x·g AND X' = x·h  →  same plaintext re-encrypted

  Fiat-Shamir:  challenge = Blake2b256( g,h, statement, commitments )
     same hash in nizk.ts (prover) and nizk.move (verifier)  ⇒ proofs verify on-chain

  Three sigma proof types (nizk.move):
     DdhProof           balance equality / re-key
     ElGamalProof       ciphertext well-formedness (per limb)
     KeyConsistencyProof viewing-key encrypted correctly to auditors
```

KEY POINTS

- WellFormedProof = vector of Bulletproofs (chunked) + one ConsistencyProof (4 ElGamal sigma proofs) per amount — encrypted_amount.move:56-70
- Range check delegates to the framework: sui::rangeproofs::verify_bulletproofs_ristretto255, version 0 (Bünz et al. 2018), LIMB_BITS = 16 — encrypted_amount.move:19-29, 299
- One Bulletproof covers at most 8 amounts (32 limb commitments = native cap); batch_sizes() greedily chunks n amounts, e.g. n=20 → [8,8,4] — encrypted_amount.move:25-29, 311
- Three sigma proofs in nizk.move: DdhProof (Chaum–Pedersen DDH), ElGamalProof (encryption well-formedness), KeyConsistencyProof (auditor key encryption) — nizk.move:34-64
- Fiat–Shamir over Blake2b256; the bases g,h are bound into the challenge so the same proof type is reusable across contexts — nizk.move:90-133, repo CLAUDE.md note
- Domain separation tags: DST_DDH=0x01, DST_ELGAMAL=0x02, DST_KEY_CONSISTENCY=0x03; session_id (20-byte derived address) ‖ protocol_id — contra.move:107-111, 984-995
- Client builds proofs in WASM: @contra/bulletproofs-wasm wraps fastcrypto::bulletproofs, byte-compatible with the on-chain verifier (Merlin/STROBE transcript)
- into_well_formed() verifies then wraps an amount as WellFormedEncryptedAmount; aborts EWellFormedProofFailed on failure — encrypted_amount.move:132-153

WHY IT MATTERS

This concept is where Ch 2.2 (Bulletproofs inner-product argument) and Ch 2.5 (sigma protocols, Fiat–Shamir, soundness) meet a deployed verifier. Note the architectural choice: Bulletproofs for the RANGE statement (efficient for [0, 2^n)) but cheap bespoke sigma proofs for the LINEAR statements (equality, well-formedness, key consistency) — you don't pay SNARK costs for things a Schnorr proof handles. That 'right proof for the right relation' split is a strong thesis talking point.

THESIS ANGLE

In your proof-systems chapter, dissect the KeyConsistencyProof (nizk.move:58-64): it proves the 8 32-bit limbs of a 256-bit secret key are each correctly ElGamal-encrypted to every auditor key AND sum to the public key. That is a non-trivial AND-composition of DDH statements with a linear-combination check — a clean worked example of building a custom sigma protocol, which you can reproduce/benchmark against a generic SNARK to argue when hand-rolled sigmas win.

TECHNICAL

Encrypted amounts are untrusted until accompanied by proofs. `contra` uses two proof families, each for the relation it is efficient at:

- **Bulletproofs** (Bünz et al. 2018, version 0) for the RANGE relation $l_i \in [0, 2^{16})$ per limb — blocks negative/overflow amounts that would mint value. Verified on-chain by `sui::rangeproofs::verify_bulletproofs_ristretto255` (encrypted_amount.move:299).
- **Sigma (Schnorr) NIZKs** for LINEAR relations — cheap, bespoke, Fiat–Shamir over Blake2b256 (nizk.move).

A `WellFormedProof` = a (chunked) vector of Bulletproofs + one `ConsistencyProof` (four `ElGamalProof`s, one per limb) per amount (encrypted_amount.move:56-70). `into_well_formed` verifies and wraps into a `WellFormedEncryptedAmount` or aborts `EWellFormedProofFailed` (code 4).

THE THREE SIGMA RELATIONS (NIZK.MOVE)

```
DdhProof  (Chaum-Pedersen): knows x s.t.  x_g = x·g  AND  x_h = x·h
    → used for balance EQUALITY: prove old_active - amount == new_active
      and for re-keying a balance to a new pk      (nizk.move:34, 90)

ElGamalProof: knows (r,m) s.t.  c = r·g + m·h  AND  d = r·pk
    → per-limb well-formedness (valid Enc under pk) (nizk.move:43, 117)

KeyConsistencyProof: the 8×32-bit limbs u_i of a 256-bit secret key
    satisfy  C_i = r_i·g + u_i·h,  D_ij = r_i·pk_j (every auditor j),
    and  (∑_i u_i·2^{32i})·g == sender_public_key    (nizk.move:58, 138)
```

`verify_elgamal` checks two equations: $z_1 \cdot pk = c \cdot e_2 + a$ and $c \cdot e_1 + b = z_1 \cdot g + z_2 \cdot h$ (nizk.move:129-132), the standard Schnorr verification with challenge `c` and responses z_1, z_2.

BULLETPROOF BATCHING & LIMB LAYOUT

```
// encrypted_amount.move:19-29
BULLETPROOFS_VERSION = 0   // Bünz et al. 2018
LIMB_BITS            = 16  // each limb proved in [0, 2^16)
MAX_BATCH_SIZE       = 8   // native caps 32 commitments = 8 amounts × 4 limbs

// batch_sizes(n): greedy chunking (encrypted_amount.move:311)
//   n=7  → [4,2,1]   n=8 → [8]   n=16 → [8,8]   n=20 → [8,8,4]

// per chunk, the 4·chunk limb ciphertexts are flattened for the verifier
// (encrypted_amount.move:299-305)
rangeproofs::verify_bulletproofs_ristretto255(
    range_proof, LIMB_BITS,
    &vector::tabulate!(4*chunk, |j| *amounts[start + j/4][j%4].ciphertext()),
    BULLETPROOFS_VERSION)
```

FIAT–SHAMIR & DOMAIN SEPARATION

Challenges are `Blake2b256` over the transcript, with the bases `g,h` bound in (nizk.move:99-112) so a proof for one relation cannot be replayed for another. The client (nizk.ts via helpers.ts) hashes the SAME BCS-encoded transcript, so TS-built proofs verify on-chain. Domain tags (contra.move:107-111):

```
DST_DDH = 0x01   DST_ELGAMAL = 0x02   DST_KEY_CONSISTENCY = 0x03
session_id = 20-byte derived TokenAccount address          (contra.move:984)
dst        = session_id ‖ protocol_id  (21 bytes)          (contra.move:991)
// SDK reserves protocol-id 100 for the selective-disclosure decryption proof
```

Range proofs use the **Merlin/STROBE** transcript (via `@contra/bulletproofs-wasm` → `fastcrypto::bulletproofs`), byte-compatible with the on-chain verifier.

HISTORY Bulletproofs (Bünz, Bootle, Boneh, Poelstra, Wuille, Maxwell, 2018); Chaum–Pedersen DDH proofs (1992); Fiat–Shamir (1986) (2018). *The TS prover and the Move verifier must hash IDENTICAL bytes or every proof fails — so the repo pins Blake2b256 for the sigma transcripts and Merlin/STROBE for Bulletproofs on BOTH sides; a single byte-ordering mismatch silently rejects all transactions.*

LIMITATIONS

- Bulletproofs verify time is linear in the aggregate; the 8-amount/proof cap forces chunking for large batches
- Range proofs prove [0, 2^16) per limb, not the full statement — soundness of 'no inflation' relies on the limb decomposition + collapse being correct, not on a single proof
- Sigma proofs are only as sound as the Fiat–Shamir transform (random-oracle model); a transcript-binding bug (missing field in the hash) can break soundness — the repo's own TODOs note unfinished DST separation
- Proofs are built client-side in WASM and assembled into large PTBs by hand (no high-level byte-array entry points yet) — a real integration burden

EXERCISES

1. **hands-on** — Read verify_well_formed_range_proofs (encrypted_amount.move:286-306). How are the 4 limb ciphertexts per amount laid out for verify_bulletproofs_ristretto255? Why 4·chunk commitments?
2. **hands-on** — In nizk.move:117-133 read verify_elgamal and write the two verification equations it checks. Identify the Schnorr response z1, z2 and the challenge c.
3. **concept** — Why bind g and h into the Fiat–Shamir challenge (nizk.move:99-112) even though they're fixed constants? What attack does omitting them enable? (Hint: proof-reuse across relations.)

Accounts, Balances & the Auth Model — Where the Hidden Money Lives

INTUITIVE

Picture a bank branch (one shared ConfidentialToken<T> per coin type) with a vault (Pool<T>) holding the real coins, and a wall of safe-deposit boxes. Each customer has ONE base Account (created once per address) and, inside it, a per-token drawer (TokenAccount<T>, stored as a dynamic field). Each drawer has three compartments: active (what you can spend), pending-encrypted (transfers others sent you, not yet swept), and pending-public (coins you wrapped in, not yet swept). To do anything you must present an Auth<T> — an unforgeable, single-use permission slip that says 'this operation is allowed for this owner'. There are exactly three ways to mint one: as the transaction sender (normal wallet), as an object that owns the account (e.g. a payment-channel smart contract proving custody by holding the object's UID), or with a witness that the token issuer's contract hands out after its own checks (the KYC/permissioned path). The whole design keeps deposits out of `active` so that a spend proof, which commits to a snapshot of `active`, can never be invalidated by money arriving mid-flight.

KEY POINTS

- Singletons TokenRegistry + AccountRegistry shared at init; everything else is a derived object → deterministic addresses, no indexing — contra.move:117-121, 205-210
- ConfidentialToken<T> { is_active, freeze_admins, policy, auditors }; one per coin type — contra.move:127-133
- Pool<T> is a derived object of the token that custodies wrapped Coin<T> as a Sui address balance (low contention) — contra.move:140-142
- Account { id, owner } is key-only (no store); per-token state is a dynamic field TokenAccount<T> — contra.move:145-160
- TokenAccount holds THREE balances: active, pending (encrypted), public_balance (wrapped) plus pk, is_frozen, accepts_deposits — contra.move:151-160
- Auth<T> is drop-only and phantom-typed so it can't be stored or reused across tokens — policy.move:35-39
- Three Auth constructors: authorize_as_sender / authorize_as_object / authorize_with_witness — contra.move:216-237
- Policy is a u32 bitmap of permissioned operations gated by a witness TypeName; default is permissionless (Option::none) — policy.move:26-70

WHY IT MATTERS

The Auth/Policy layer is the single most thesis-relevant seam in the whole codebase: it is the generic hook where an anonymous-credential check could authorize an operation. Today `authorize_with_witness` lets an issuer's contract gate register/wrap/unwrap behind ANY logic (KYC list, allowlist, rate limit). Your thesis could replace that 'issuer checks a list' with 'a ZK credential proof authorizes the witness' — confidential amounts (this system) + anonymous authorization (your contribution) = private payments with private identity.

THESIS ANGLE

In your architecture chapter, draw exactly this object graph and mark the trust/authorization boundary at Auth<T>. Then show your extension: a Move module that verifies a Groth16 credential proof (à la zkLogin) and, on success, mints an Auth<T> via authorize_with_witness. Cite contra.move:223-230 as the integration point and argue why the phantom-T, drop-only Auth is a safe capability to gate this way.

TECHNICAL

State is built entirely from Sui derived objects + dynamic fields, so addresses are deterministic and no indexer is needed. Two shared singletons are created at `init` (contra.move:205): `TokenRegistry` and `AccountRegistry`. Per coin type `T` there is one `ConfidentialToken<T>` (claimed via `TokenKey<T>()`) and a derived `Pool<T>` custodying the wrapped `Coin<T>` reserve as a Sui address balance. Per address there is one `Account`; per-token state is a dynamic field `TokenAccount<T>` with three balances.

Every state-changing op consumes an `Auth<T>` — a drop-only, phantom-typed capability carrying an operations bitmap + the authenticated owner (policy.move:35). A `Policy` (u32 bitmap gated by a witness `TypeName`) decides which operations need the permissioned path; absence of a policy = fully permissionless.

CORE TYPES (CONTRA.MOVE)

```
// contra.move:127
public struct ConfidentialToken<phantom T> has key {
    id: UID, is_active: bool, freeze_admins: VecSet<address>,
    policy: Option<Policy>, auditors: Auditors }

// contra.move:145, 151
public struct Account has key { id: UID, owner: address }
public struct TokenAccount<phantom T> has store {
    pk: Element<G>, verified_key_encryption, session_id,
    is_frozen: bool, accepts_deposits: bool,
    active:  EncryptedBalance<T>,   // spendable
    pending: EncryptedBalance<T>,   // encrypted deposits
    public_balance: PublicCoin<T>,  // wrapped coins
}
```

THE AUTH CAPABILITY & ITS THREE CONSTRUCTORS

```
// policy.move:35 — drop-only, phantom T (no store ⇒ per-PTB, non-reusable)
public struct Auth<phantom T> has drop { operations: u32, owner: address }

// contra.move
authorize_as_sender(ct, ctx)            // owner = ctx.sender()      :216
authorize_as_object(ct, uid: &mut UID) // owner = addr(UID)          :235
authorize_with_witness<T,W>(ct, op, owner, w: W)  // issuer-gated   :223

// checks
is_allowed(auth, op)        // operation bit set    (policy.move:109)
is_authenticated(auth, who) // auth.owner == who    (policy.move:115)
```

Thesis seam: `authorize_with_witness` mints an `Auth<T>` for `owner` only if the policy's witness type is `W` and `op` is permissioned. A credential-verifying Move module that produces `W` on a valid ZK proof would attach anonymous authorization to confidential amounts.

POLICY BITMAP (PERMISSIONED OPERATIONS)

```
// operation ids (contra.move:103-105)
PERMISSIONED_REGISTER = 0   PERMISSIONED_WRAP = 1   PERMISSIONED_UNWRAP = 2

// policy.move:26 — witness-gated u32 bitmap; MAX_OPERATION_INDEX = 31
public struct Policy has drop, store {
    witness_type: TypeName,             // only the issuer can construct W
    permissioned_operations_bitmap: u32 }

set_policy<T,W>(ct, &mut treasuryCap, ops)   // contra.move:936
// empty ops vector ⇒ permissionless again
```

Confidential transfers between already-registered accounts stay permissionless under any policy — only `register/wrap/unwrap` are gateable.

HISTORY Object-capability + witness patterns are idiomatic Sui Move (Mysten Labs); the three-balance split is contra-specific (2026). *Anyone can create an Account for any address (new_account doesn't check the sender) — it's safe because every actual operation re-checks owner == authenticated sender, and the only way to dispose of an Account is to share it. The capability, not the object, carries the authority.*

LIMITATIONS

- The client SDK (ContraClient) only builds the permissionless flows; issuer-defined permissioned wrappers must be assembled by hand
- Three balances + manual merge is real UX complexity exposed to wallets
- Auth<T> is per-PTB and per-token; cross-token or long-lived authorization needs re-minting each transaction
- Account state in dynamic fields means generic explorers may not render confidential balances without contra-specific decoding

EXERCISES

1. **hands-on** — Read the three Auth constructors (contra.move:216, 235, 223) and policy.move:77-106. What exactly distinguishes is_allowed (operation bitmap) from is_authenticated (owner)?
2. **concept** — Why are deposits credited to `pending` and swept by merge() rather than added to `active` directly? Connect to 'a spend proof commits to a snapshot' from concept 2.
3. **hands-on** — Study apps/payment-channel: how does a Channel<T> shared object authorize spending its own confidential Account via authorize_as_object? This is the canonical as_object() reference.

The Flows — Wrap In, Transfer Hidden, Unwrap Out (and Why Transfers Can 'Soft-Fail')

INTUITIVE

Living a confidential life with this token is a four-verb loop. WRAP: hand the vault a public Coin<T> and get private credit — the amount is VISIBLE (you're crossing the public/private border). TRANSFER: send an encrypted amount account-to-account — the amount is HIDDEN; this is the only step that keeps privacy. UNWRAP: withdraw private credit back to a public Coin<T> — amount VISIBLE again. MERGE: sweep your pending boxes into active so you can spend them. A transfer is a little state machine: batched_transfer debits the sender and produces one sealed coin per receiver, then you call add once per receiver, then finalize. The clever part is failure handling. The SDK builds your spend proof against the balance it SAW; if a deposit lands between 'SDK builds proof' and 'chain runs it', the proof is stale. Rather than abort (and burn gas / scare the user), the chain runs the prepended merge successfully and then SOFT-FAILS the spend: it emits a TryTransferFailedEvent and leaves your funds untouched. You just retry with merge:false and it goes through. So 'my transfer silently did nothing' is usually this race, not a bug.

KEY POINTS

- wrap(): public Coin<T> → pending-public; amount is public; reserve goes to Pool<T> — contra.move:502-529
- batched_transfer() → TransferBatch enum (Ok{coins,...} | BalanceProofFailed); one aggregate Bulletproof over receiver_amounts ++ [new_balance] — contra.move:543-608
- add_to_batch() once per receiver, in order, credits receiver.pending-encrypted; finalize()/try_finalize() closes it — contra.move:615-678
- unwrap() → Coin<T> paid from Pool<T>; amount public; try_unwrap() is the non-aborting variant — contra.move:741-797
- merge() folds pending-encrypted + pending-public into active and resets the growth bound — contra.move:686-693
- Privacy boundary: only confidential transfers hide the amount; wrap/unwrap reveal it (they touch the public coin layer) — top-level README 'Privacy boundary'
- Soft-fail: try_finalize / try_unwrap emit TryTransferFailedEvent / TryUnwrapFailedEvent instead of aborting; the TX SUCCEEDS, so clients must watch events — contra.move:658-672, 770-797, events.move:61-69
- Fix for the merge-then-spend race: retry with merge:false (pending already folded by the first tx's merge) — top-level README

WHY IT MATTERS

These flows are the empirical surface your evaluation chapter measures: gas per wrap/transfer/unwrap, proof-build latency in WASM, and the failure-rate of optimistic merge under concurrent deposits. The wrap/unwrap 'border reveal' is also the key privacy caveat to state plainly — it mirrors the 'shielded pool entry/exit is observable' problem in Zcash, and is exactly where mixing/anonymity-set ideas from Ch 2.4 would extend the design.

THESIS ANGLE

Build a small experiment for the evaluation chapter: drive concurrent deposits while a sender submits transfers and measure how often the optimistic merge soft-fails (TryTransferFailedEvent). Report it as a real-world reliability metric and propose a mitigation (e.g. proofs that bind to active only, or a deposit-epoch). Cite contra.move:658-672 and the README race description.

TECHNICAL

The public API is four verbs plus a state machine for transfers. `wrap` (contra.move:502) deposits a public `Coin<T>` into the receiver's `public_balance` (amount public; reserve → `Pool<T>`). `unwrap` (contra.move:741) withdraws a public coin from the pool (amount public). `merge` (contra.move:686) folds pending → active. A confidential **transfer** is the only amount-hiding step and is modelled as a hot-potato enum `TransferBatch<T>` (contra.move:164) that must be finalized in the same PTB.

Optimistic failure. The SDK defaults `merge:true`, prepending a `merge` to a spend. Because the spend proof is built against the observed balance, a deposit arriving in the build→execute window makes the proof stale: the chain runs the `merge` but the spend SOFT-FAILS (emits an event, no abort, funds intact). Retry with `merge:false` succeeds.

TRANSFER STATE MACHINE (CONTRA.MOVE)

```
// contra.move:164 — no abilities ⇒ hot potato
public enum TransferBatch<phantom T> {
    BalanceProofFailed,
    Ok { sender, sender_pk, coins: vector<EncryptedCoin<T>>, sender_amounts },
}

batched_transfer(sender, auth, ct, deny_list,
    receiver_pks, receiver_amounts,
    well_formed_proofs,            // ONE aggregate Bulletproof over
                                   //   receiver_amounts ++ [new_balance]
    sender_amounts, consistency_proof, new_balance, balance_proof)
  → TransferBatch                                        (contra.move:543)
add_to_batch(batch, receiver, memo, deny_list) ×N        (contra.move:615)
finalize(batch) | try_finalize(batch): bool              (contra.move:676, 658)
```

The sender is debited inside `balance::try_split_batch` (balance.move:188): it checks the sender total equals the sum of receiver commitments, verifies the consistency + balance proofs, then peels one `EncryptedCoin<T>` per receiver. Each `add_to_batch` credits a receiver's `pending` (contra.move:648).

SOFT-FAIL EVENTS (WATCH THESE IN THE CLIENT)

```
// events.move:61-69
TryTransferFailedEvent     // try_finalize: balance proof failed
TryUnwrapFailedEvent       // try_unwrap:   balance proof failed
TrySetPublicKeyFailedEvent // optimistic re-key raced a deposit

// contra.move:770 — try_unwrap returns a ZERO coin instead of aborting
let (success, coin) = account.try_unwrap_internal(...);
if (!success) { events::emit_try_unwrap_failed(); };
coin   // zero-value if the proof failed
```

The transaction SUCCEEDS in all soft-fail cases, so a wallet cannot rely on tx status — it must subscribe to these events. Event type tags use the `events` module segment, e.g. `<pkg>::events::TransferEvent<T>`.

PRIVACY BOUNDARY (WHAT WRAP/UNWRAP LEAK)

Confidential transfers hide the amount but reveal *sender, receiver, timing*. `wrap` / `unwrap` cross into/out of the public coin layer, so that single operation's amount and counterparties are public — analogous to shielded-pool entry/exit observability in Zcash. The `TransferEvent` (events.move:46) even carries the amount twice (under sender and receiver keys); note its `encrypted_amount_sender` is only verified as part of the batch total, not individually range-checked — a documented sharp edge.

HISTORY Mysten Labs (contra); the wrap/unwrap 'shielded pool' shape mirrors Zcash/Zether confidential-balance designs (2026). *The transfer is modelled as a Move enum with no abilities (a 'hot potato') — TransferBatch must be consumed by finalize in the same transaction or the whole PTB aborts, which is how the contract guarantees every produced receiver-coin is actually credited.*

LIMITATIONS

- wrap/unwrap leak the amount and counterparty of that operation — privacy is only intra-domain
- Optimistic merge can soft-fail under concurrent deposits, requiring a retry (extra latency, client must handle the event)
- Each receiver in a batch must be added in order and the batch finalized in the same PTB (hot-potato) — fiddly to assemble by hand
- No high-level entry points: flows are large hand-built PTBs constructing every proof/ciphertext input

EXERCISES

1. **hands-on** — Trace batched_transfer → add_to_batch → finalize (contra.move:543-678). Where is the sender debited, where is each receiver credited, and what happens to the batch if the balance proof failed?
2. **concept** — Explain why try_unwrap returns a ZERO coin + event on failure instead of aborting (contra.move:770-797). What does this buy the wallet UX, and what must the client now do?
3. **hands-on** — Run the kaisho demo (kaisho-wallet.vercel.app) end to end: register → wrap → transfer → unwrap. Note which amounts appear in the explorer and which don't.

Compliance & Auditors — Selective Disclosure Built In

INTUITIVE

A pure privacy coin is a regulatory non-starter; contra ships the escape valves an issuer needs, without breaking the default of privacy. Think of it as 'private by default, with named keyholders'. AUDITORS: at registration a user encrypts their OWN viewing key to the issuer's set of auditor public keys (proving they did so correctly). From then on an auditor can decrypt that one key and READ everything that account does — but can never MOVE funds. Crucially this is PER-ACCOUNT, not per-transaction: you pay the auditor-visibility cost once, not on every transfer. FREEZE/SEIZE: the issuer (via TreasuryCap) and designated freeze-admins (via ManagementCap) can freeze accounts, pause the whole token, or even overwrite a balance to seize/burn — the levers a court order or fraud case demands. SELECTIVE DISCLOSURE: independently, any user can voluntarily reveal one value (a balance, or one transfer's amount) plus a proof that it matches the on-chain ciphertext under their key — convincing an auditor of a single fact without handing over their secret. PERMISSIONED MODE: the issuer can gate register/wrap/unwrap behind their own contract (KYC, screening) while user-to-user transfers stay permissionless.

```
Per-account auditing (pay once at register, not per transfer):

  user secret key x ──encrypt to auditor pks──▶ VerifiedKeyEncryption
        (KeyConsistencyProof: 8×32-bit limbs, each Enc'd to every auditor,
         and the limbs sum to pk)  — auditors.move / nizk.move
        stored on the TokenAccount; auditor decrypts x → reads all activity
        (read-only: auditors never sign protocol txs)

Issuer compliance levers:
   deny list (DenyCapV2)      block send/recv/wrap/unwrap (sender+receiver)
   account_freeze / global    admins freeze; only TreasuryCap unfreezes
   set_balance_by_issuer      overwrite a balance (seize / burn / reset)
   set_policy<T,W>            gate register/wrap/unwrap behind witness W

Selective disclosure (user-initiated, no auditor needed):
   reveal value v + DDH proof  →  verifier checks v matches ciphertext
   under pk, WITHOUT learning x   (protocol-id 100 in the SDK)
```

KEY POINTS

- Per-account auditing: user encrypts their viewing key to all auditor pks at register; KeyConsistencyProof proves it correct — auditors.move:49-186, nizk.move:138
- Auditors are passive readers (decrypt the user key, read state/events); they NEVER sign or move funds — top-level README 'For Auditors'
- update_auditors() rotates the set and bumps a version; recommended_min_version is ADVISORY (not enforced on-chain) — contra.move:953-965, auditors.move:34-35
- Per-account freeze (admins via ManagementCap) vs global pause (is_active); only TreasuryCap unfreezes — contra.move:898-928
- Deny-list integration uses the underlying coin's DenyCapV2 (sender + receiver, next-epoch) — deny_list.move:12-25
- set_balance_by_issuer overwrites active + clears pending/public (seize/burn); can desync supply↔pool, caller's responsibility — contra.move:862-873
- Permissioned mode: set_policy<T,W> gates register/wrap/unwrap behind a witness; user↔user transfers stay permissionless — contra.move:936-943, README 'Permissioned'
- Selective disclosure: SDK decryptWithProof + on-chain verifyDecryption; reveal one value with a DDH proof, secret key stays hidden — repo CLAUDE.md, protocol-id 100

WHY IT MATTERS

This is the compliance-preserving-privacy pillar from your thesis principles, already prototyped. The per-account auditor is a concrete 'designated opening' mechanism — compare it in your related-work chapter to viewing keys (Zcash), auditable decryption (PGC/Solana), and threshold auditors. The selective-disclosure path is literally an attribute proof over an encrypted value: the same machinery your anonymous-credential design needs for 'prove age > 18 without revealing birthdate', here specialized to 'prove balance = v'.

THESIS ANGLE

Dedicate a related-work subsection to contra's auditor design and critique it: per-account auditing is cheap but COARSE (an auditor who can read one account reads ALL its history, with no scoping to a time window or a single transfer). Propose a finer-grained alternative — e.g. per-transfer auditor ciphertext gated by a credential, or threshold auditors so no single party can deanonymize — and position it as a thesis contribution. Cite auditors.move and the AUDITORS.md per-account-vs-per-transaction discussion.

TECHNICAL

`contra` is private-by-default with issuer-controlled openability. Four mechanisms:

- **Per-account auditing.** At register the user encrypts their viewing key to the auditor key set and proves it correct with a `KeyConsistencyProof` + an aggregate Bulletproof; the result is a `VerifiedKeyEncryption` stored on the account (auditors.move:152). Auditors decrypt that one key and read all activity — never sign.
- **Freeze / pause.** Per-account and global, asymmetric authority.
- **Seize.** Direct balance overwrite by the issuer.
- **Selective disclosure.** A user reveals one value + a DDH proof matching the on-chain ciphertext under their key, without exposing the secret.

AUDITOR KEY-CONSISTENCY CHECK

```
// auditors.move:36, 49
public struct Auditors has store {
    pks: vector<Element<G>>, version: u32, recommended_min_version: u32 }
public struct VerifiedKeyEncryption has copy,drop,store {
    ciphertext: vector<MultiRecipientEncryption>, version: u32 }

// auditors.move:152 — verify before storing
proof.verify_key_consistency(dst, sender_pk, auditors.pks(), &ciphertext)
  && rangeproofs::verify_bulletproofs_ristretto255(   // LIMB_BITS = 32
        &range_proof, 32, &limb_commitments, 0)
```

The proof attests the 8×32-bit limbs of the 256-bit viewing key are each ElGamal-encrypted to *every* auditor and sum to the user's `pk` (nizk.move:138). Per-account (not per-transaction) auditing — one ciphertext at register, not one per transfer.

ISSUER LEVERS (ASYMMETRIC AUTHORITY)

```
// contra.move
global_freeze(ct, ctx)         // any freeze admin → is_active = false   :898
global_unfreeze(ct, &TreasuryCap)            // ISSUER only            :906
account_freeze(ct, account, ctx)             // admin freezes account  :913
account_unfreeze(&TreasuryCap, account)      // ISSUER only            :924
set_balance_by_issuer(&mut TreasuryCap, account, new_balance) // seize :862
update_auditors(ct, &ManagementCap, pks, bump_recommended_min)        :953

// deny_list.move:12-25 — underlying coin DenyCapV2 (sender + receiver)
is_sender_denied / is_receiver_denied / is_frozen (global pause)
```

`set_balance_by_issuer` overwrites `active` and clears `pending`/`public_balance`, setting `upper_bound = 1`; it bypasses homomorphic accounting and can desync circulating supply vs the `Pool<T>` reserve — the caller must restore consistency. `recommended_min_version` is advisory: the chain does NOT reject stale-key accounts (auditors.move:34-35).

SELECTIVE DISCLOSURE & THESIS CRITIQUE

Independently of auditors, the SDK's `decryptWithProof` produces `(value, proof)` for any ciphertext (a balance, or a `TransferEvent` amount); a verifier checks `ciphertext.verifyDecryption(pk, value, proof)` — a DDH proof under Fiat–Shamir protocol-id 100. This is an attribute proof over an encrypted value, exactly the primitive an anonymous-credential layer reuses.

Critique to develop: per-account auditing is coarse (whole-history visibility, no per-transfer or time scoping) and single-key (one compromised auditor key deanonymizes every opted-in account). A thesis contribution could add threshold auditors or credential-gated, per-transfer disclosure. See `AUDITORS.md` for the per-account-vs-per-transaction design discussion.

HISTORY Auditable confidential payments line: PGC (Chen, Ma, Tang, Au), Zether auditability, Solana confidential transfers auditor keys; contra's per-account variant by Mysten Labs (2026). *An account registered while the token had NO auditors carries an empty VerifiedKeyEncryption — so enabling auditors later doesn't retroactively open old accounts unless they rotate keys, a subtle and thesis-worthy privacy/compliance corner case.*

LIMITATIONS

- Per-account auditing is coarse: an auditor sees the account's ENTIRE history, not a single scoped transfer or time window
- recommended_min_version is advisory — the chain does NOT reject stale-auditor-key accounts; wallets must enforce rotation
- set_balance_by_issuer bypasses the homomorphic accounting and can desync total supply vs the pool reserve if misused
- Trust in auditors is all-or-nothing (single key per auditor); no built-in threshold so one compromised auditor key deanonymizes every opted-in account
- Whole system is UNAUDITED devnet beta — compliance guarantees are not yet production-grade

EXERCISES

1. **hands-on** — Read verify_key_encryption (auditors.move:152-186): which two proofs are checked (sigma + Bulletproof) before a VerifiedKeyEncryption is stored, and what does each guarantee?
2. **concept** — Contrast per-account vs per-transaction auditing (AUDITORS.md). State one privacy advantage and one cost advantage of contra's choice, then propose where a thesis could improve scoping.
3. **hands-on** — Find set_balance_by_issuer (contra.move:862-873). List the three balances it touches and explain the supply-consistency warning. When is this lever legitimate vs dangerous?

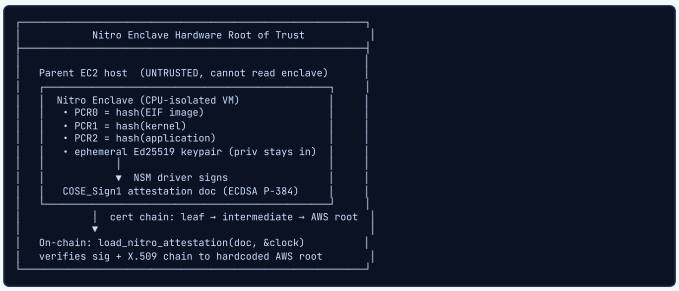
CHAPTER 5

Nautilus — Verifiable Off-Chain Compute (TEE)

TEE Trust Model & Nitro Attestation

INTUITIVE

A Nitro Enclave is a sealed, tamper-evident envelope produced by a notary you already trust (AWS). Inside the envelope, specific code runs in total isolation — even the parent machine cannot peek in. When the enclave wants to prove 'I am exactly this code, and here is my fresh public key', it asks the hardware (the Nitro Secure Module) to stamp a notarized document. The stamp is an ECDSA P-384 signature chaining up to AWS's root certificate. On-chain, Sui already knows AWS's root cert, so it can verify the stamp is genuine — the chain trusts the hardware, not the operator running it.



KEY POINTS

- AWS Nitro Enclave = CPU-isolated VM; the parent host cannot read enclave memory
- PCRs (Platform Configuration Registers) measure the running code: PCR0=EIF image, PCR1=kernel, PCR2=application
- The Nitro Secure Module (NSM) hardware signs the attestation document — the private signing key never leaves the chip
- Attestation is a COSE_Sign1 (RFC 8152) structure, tag=18, alg=-35 (ECDSA P-384), 96-byte signature
- Certificate chain runs leaf → intermediate(s) → AWS root; the root cert is hardcoded into the Sui framework
- On-chain entry point: sui::nitro_attestation::load_nitro_attestation(attestation, &Clock) → NitroAttestationDocument
- Move abort codes: ENotSupportedError=0, EParseError=1, EVerifyError=2, EInvalidPCRsError=3

WHY IT MATTERS

This is the foundational trust assumption of the whole TEE branch of your thesis. Where a ZKP convinces a verifier with pure math (no trusted party), a TEE convinces it with a hardware manufacturer's signature. Your thesis must articulate precisely what each model assumes: Nautilus trusts AWS's CA and the Nitro silicon; Seal/ZK trusts only the soundness of the proof system. Knowing where the trust is rooted lets you reason about which threats each design stops.

THESIS ANGLE

When you write the threat-model chapter, you'll contrast 'who can forge a valid output'. For Nautilus the answer is: anyone who can either (a) extract the ephemeral key from inside a genuine enclave (hardware attack) or (b) forge an AWS-rooted cert chain. You can cite the exact verifier — verify_nitro_attestation in nitro_attestation.rs:118-148 — as the concrete artifact enforcing that boundary.

TECHNICAL

A **Trusted Execution Environment (TEE)** shifts the verifier's trust from the operator of a machine to its **hardware manufacturer**. AWS Nitro Enclaves provide a CPU-isolated VM carved out of a parent EC2 instance with no persistent storage, no interactive access, and a hardware *attestation* channel via the Nitro Secure Module (NSM).

- The enclave's running software is measured into **PCRs** (Platform Configuration Registers): `PCR0` =hash of the EIF image, `PCR1` =kernel, `PCR2` =application.
- The NSM signs an **attestation document** binding the PCRs to an enclave-supplied public key, using **ECDSA P-384**.
- A certificate chain (leaf → intermediate(s) → AWS root) lets a relying party verify the signature came from genuine AWS Nitro hardware.

Sui exposes on-chain verification via the native `sui::nitro_attestation::load_nitro_attestation`, with the AWS root certificate embedded in the framework.

ON-CHAIN ENTRY POINT (MOVE)

The public Move API and its abort codes (nitro_attestation.move):

```
// nitro_attestation.move:52-54
public fun load_nitro_attestation(
    attestation: vector<u8>,
    clock: &Clock,
): NitroAttestationDocument;

// abort codes
const ENotSupportedError: u64 = 0; // feature not on this network
const EParseError: u64       = 1; // CBOR/attestation parse failed
const EVerifyError: u64      = 2; // signature or cert verify failed
const EInvalidPCRsError: u64 = 3; // PCR mismatch in document
```

The public function calls a native `load_nitro_attestation_internal` (nitro_attestation.move:96-99) bound to the Rust verifier.

NATIVE VERIFICATION (RUST)

`verify_nitro_attestation` (nitro_attestation.rs:118-148) extracts the P-384 signature and the leaf certificate's public key, verifies the COSE signature, then validates the certificate chain against the on-chain `Clock`:

```
pub fn verify_nitro_attestation(
    signature: &[u8],
    signed_message: &[u8],
    payload: &AttestationDocument,
    timestamp: u64,
) -> SuiResult<()> {
    let signature = Signature::from_slice(signature)?;
    let cert = X509Certificate::from_der(
        payload.certificate.as_slice())?;
    let pk = SubjectPublicKeyInfo::parsed(cert.1.public_key())?;
    match pk {
        PublicKey::EC(ec) => {
            let vk = VerifyingKey::from_sec1_bytes(ec.data())?;
            vk.verify(signed_message, &signature)?;
        }
        _ => return Err(/* ... */),
    }
    payload.verify_cert(timestamp)?; // chain to AWS root
    Ok(())
}
```

The AWS root is embedded: `include_bytes!("./nitro_root_certificate.pem")` (nitro_attestation.rs:37-44).

CERTIFICATE CHAIN VALIDATION

Chain validation (`verify_cert_chain`, nitro_attestation.rs:720-827) enforces:

- Leaf certificate must carry the `digitalSignature` key-usage bit.
- Intermediate and root CAs must have `keyCertSign` and `basicConstraints(CA=true)`, with `pathLenConstraint` respected.
- Issuer/subject chaining checked at each hop; each cert signed by the previous issuer.
- Validity windows checked against the `Clock` (millisecond precision).

The signed message for COSE_Sign1 is the canonical array `["Signature1", protected, empty, payload]` (nitro_attestation.rs:363-376).

HISTORY Mysten Labs (Nautilus framework) on top of AWS Nitro Enclaves (2025). *Sui literally ships AWS's root certificate inside its source tree — include_bytes!("./nitro_root_certificate.pem") — so the whole chain's trust in any Nitro Enclave reduces to one embedded PEM file.*

LIMITATIONS

- Trust is rooted in AWS and the Nitro silicon — a manufacturer/CA compromise or a hardware side-channel breaks every Nautilus deployment at once
- Attestation only proves WHICH code is running, not that the code is correct or bug-free — a buggy enclave produces validly-signed wrong outputs
- Side-channel attacks (timing, power, micro-architectural) against TEEs are an active research area; the attestation says nothing about side-channel resistance
- Vendor lock-in: the Move verifier is specific to AWS Nitro's COSE/X.509 format; SGX/SEV would need a different verifier
- Certificate validity is time-bounded and checked against the on-chain Clock, so attestation documents are not indefinitely replayable but also expire

EXERCISES

1. **hands-on** — Read sui::nitro_attestation::load_nitro_attestation (nitro_attestation.move:52-54) and trace how it calls the native load_nitro_attestation_internal (nitro_attestation.move:96-99).
2. **hands-on** — Open nitro_attestation.rs:118-148 (verify_nitro_attestation) and list every check performed: signature, leaf cert key usage, cert chain, timestamp.
3. **hands-on** — Find the four Move abort codes (ENotSupportedError=0 … EInvalidPCRsError=3) in nitro_attestation.move and write down which failure each maps to.

Attestation Document Structure (PCRs & COSE_Sign1)

INTUITIVE

The attestation document is the enclave's notarized ID card. It lists who the enclave is (module_id), when it was issued (timestamp), a set of fingerprints of the exact software running (the PCRs), the enclave's fresh public key, and optional custom fields (user_data, nonce). The whole card is sealed in a COSE_Sign1 wrapper — think of it as cryptographic shrink-wrap: a single ECDSA P-384 signature over a canonical CBOR encoding, so any tampering breaks the seal and the on-chain parser rejects it.

KEY POINTS

- NitroAttestationDocument fields: module_id, timestamp, digest, pcrs[], public_key, user_data, nonce (nitro_attestation.move:28-45)
- Each PCREntry is { index: u8, value: vector<u8> }; value is 32/48/64 bytes for SHA256/384/512
- Required PCRs always included: indices 0,1,2,3,4,8; custom PCRs 5-7 and 9-31 included only when nonzero (nitro_attestation.rs:629-651)
- COSE_Sign1 wrapper: protected = {1:-35}, unprotected empty, payload = CBOR(doc), signature = 96 bytes (nitro_attestation.rs:155-168)
- Signature is computed over the canonical array ["Signature1", protected, empty, payload] (nitro_attestation.rs:363-376)
- Hard limits enforced in Rust: MAX_CERT_CHAIN_LENGTH=10, MAX_USER_DATA_LENGTH=512, MAX_PK_LENGTH=1024, MAX_PCRS_LENGTH=32, MAX_CERT_LENGTH=1024 (nitro_attestation.rs:25-34)
- Rust doc has both pcr_vec (legacy [0,1,2,3,4,8]) and pcr_map (all nonzero PCRs) for the upgraded parser (nitro_attestation.rs:382-393)

WHY IT MATTERS

Understanding the document format is what lets you reason about freshness and binding. The nonce field is a freshness proof; user_data is a 512-byte channel to bind application context into the attestation. In a privacy-payment setting you might bind a commitment or a session identifier into user_data so the attestation proves not just 'this enclave' but 'this enclave for THIS request' — exactly the kind of binding argument you'll make for ZK statements too.

THESIS ANGLE

If you need the enclave's attestation to commit to a specific batch of confidential transactions, you can place a Merkle root or BCS-encoded request hash into user_data (≤512 bytes) so the on-chain verifier ties the enclave's identity to the exact input it processed — preventing an attacker from replaying a valid attestation against a different input.

TECHNICAL

The attestation payload is an **AttestationDocument** CBOR-encoded inside a **COSE_Sign1** (RFC 8152, tag 18) envelope. It binds the enclave's measured identity (PCRs), an enclave-supplied `public_key`, and optional application data to a single ECDSA P-384 signature.

- **Required PCRs**: indices 0,1,2,3,4,8 are always present.
- **Custom PCRs**: indices 5-7, 9-31 are included only when nonzero (under `include_all_nonzero_pcrs`).
- PCR values are 32 / 48 / 64 bytes for SHA-256 / SHA-384 / SHA-512.

MOVE & RUST DOCUMENT STRUCTS

Move side (nitro_attestation.move:28-45) and Rust side (nitro_attestation.rs:382-393):

```
// Move
public struct NitroAttestationDocument has drop {
    module_id: vector<u8>,
    timestamp: u64,                 // ms since UNIX epoch
    digest: vector<u8>,             // hash alg (SHA384)
    pcrs: vector<PCREntry>,         // index + value pairs
    public_key: Option<vector<u8>>,// DER key, ≤1024 B
    user_data: Option<vector<u8>>, // custom, ≤512 B
    nonce: Option<vector<u8>>,     // freshness
}
public struct PCREntry has drop {
    index: u8,
    value: vector<u8>,
}

// Rust
pub struct AttestationDocument {
    pub module_id: String,
    pub timestamp: u64,
    pub digest: String,
    pub pcr_vec: Vec<Vec<u8>>,     // legacy [0,1,2,3,4,8]
    pub pcr_map: BTreeMap<u8, Vec<u8>>, // all nonzero
    certificate: Vec<u8>,
    cabundle: Vec<Vec<u8>>,        // CA chain to AWS root
    pub public_key: Option<Vec<u8>>,
    pub user_data: Option<Vec<u8>>,
    pub nonce: Option<Vec<u8>>,
}
```

COSE_SIGN1 WRAPPER

The signed envelope (nitro_attestation.rs:155-168):

```
pub struct CoseSign1 {
    protected: Vec<u8>,   // CBOR map {1: -35} ⇒ ES384
    unprotected: HeaderMap,// always empty
    payload: Vec<u8>,     // CBOR(AttestationDocument)
    signature: Vec<u8>,   // 96 bytes (P-384)
}
```

Tag = 18 marks COSE_Sign1; alg = -35 is the IANA COSE identifier for ES384 (ECDSA + SHA-384 over P-384). Parsing entry point: `CoseSign1::parse_and_validate` (nitro_attestation.rs:285-322).

HARD LIMITS (DOS BOUNDS)

Constants enforced during parsing (nitro_attestation.rs:25-34):

```
const MAX_CERT_CHAIN_LENGTH: usize = 10;
const MAX_USER_DATA_LENGTH:  usize = 512;
const MAX_PK_LENGTH:         usize = 1024;
const MAX_PCRS_LENGTH:       usize = 32;
const MAX_CERT_LENGTH:       usize = 1024;
```

PCR-selection logic lives at nitro_attestation.rs:629-651: required PCRs (0,1,2,3,4,8) always included; nonzero custom PCRs added when `include_all_nonzero_pcrs = true`.

HISTORY AWS (Nitro attestation format) + IETF COSE (RFC 8152) (2020). *The COSE algorithm value -35 is not arbitrary: it is the registered IANA COSE identifier for ES384 (ECDSA with SHA-384 over P-384).*

LIMITATIONS

- Parsing CBOR/COSE on-chain is non-trivial; a malformed document aborts with EParseError=1 rather than giving a precise reason
- user_data and nonce are application-defined — Nautilus does not enforce any semantics, so binding correctness is entirely the dapp's responsibility
- PCR values prove software identity but reveal nothing about runtime data confidentiality; two different secrets in the same code yield identical PCRs
- Document size is bounded (1024-byte cert, 512-byte user_data), so large context cannot be embedded directly — only hashes/commitments fit

EXERCISES

1. **hands-on** — Map each field of the Move NitroAttestationDocument (nitro_attestation.move:28-45) to its Rust counterpart AttestationDocument (nitro_attestation.rs:382-393).
2. **hands-on** — Read nitro_attestation.rs:629-651 and explain the difference between the required PCR set and the include_all_nonzero_pcrs behavior.
3. **hands-on** — Find the five MAX_* constants (nitro_attestation.rs:25-34) and reason about which denial-of-service each bound prevents.

On-Chain Enclave Registration (Cap/Witness Pattern)

INTUITIVE

Registration is the enclave's one-time 'passport check' at the border. The enclave shows up with its notarized attestation document; the Move verifier checks the document's PCR measurements match exactly what the dapp owner pre-registered (the EnclaveConfig), then stamps the passport by extracting the enclave's ephemeral public key and storing it in a shared Enclave object. The Cap<T> is the dapp's signet ring: only the module holding the witness type T can mint a config for that dapp, so nobody can impersonate your enclave's identity.

KEY POINTS

- EnclaveConfig<phantom T> holds the trusted PCR triple, a capability_id, and a version (enclave.move:31-37)
- Pcrs(vector<u8>, vector<u8>, vector<u8>) — three 48-byte SHA-384 digests for PCR0/PCR1/PCR2 (enclave.move:23)
- Cap<phantom T> is minted only via new_cap<T: drop>(witness, ctx) — the witness type T gates who can create a dapp's config (enclave.move:60-64)
- register_enclave<T>(config, document, ctx) verifies PCRs then extracts the ephemeral pk and shares an Enclave<T> object (enclave.move:85-100)
- load_pk asserts document.to_pcrs() == config.pcrs (abort EInvalidPCRs=0), then destroy_some's the public key (enclave.move:163-167)
- Registration is expensive (full attestation + cert-chain verify) but happens ONCE; after that the cheap Ed25519 key is reused
- Enclave registration abort codes: EInvalidPCRs=0, EInvalidConfigVersion=1, EInvalidCap=2, EInvalidOwner=3 (enclave.move:15-18)

WHY IT MATTERS

The Cap/Witness pattern is core idiomatic Sui Move and shows up everywhere in your thesis codebase (TreasuryCap, AdminCap, etc.). Here it solves the identity problem: how does the chain know THIS Enclave object belongs to MY dapp and runs MY code? The answer — phantom type T + capability_id binding — is the same capability-based access-control reasoning you'll apply when designing who may mint credentials or authorize confidential transfers.

THESIS ANGLE

If you build a TEE-backed confidential-payment oracle, you'd define a witness type PAYMENT_ORACLE, mint a Cap<PAYMENT_ORACLE>, and register an EnclaveConfig<PAYMENT_ORACLE> pinned to the PCRs of your reviewed enclave image. Any payment-settlement function would take &Enclave<PAYMENT_ORACLE>, guaranteeing the signature it checks came from your audited code and no other enclave.

TECHNICAL

Registration anchors an enclave's ephemeral public key on-chain **once**, after verifying that the attestation document's PCRs match a pre-registered `EnclaveConfig<T>`. Access control uses the Sui **capability + witness** pattern: a phantom type `T` identifies the dapp, a `Cap<T>` authorizes config creation, and the config's `capability_id` binds them together.

CORE STRUCTS

```
// enclave.move:23
public struct Pcrs(
    vector<u8>, vector<u8>, vector<u8> // PCR0/1/2, 48 B each
) has copy, drop, store;

// enclave.move:31-37
public struct EnclaveConfig<phantom T> has key {
    id: UID,
    name: String,
    pcrs: Pcrs,
    capability_id: ID,  // references Cap<T>
    version: u64,       // bumped on PCR update
}

// enclave.move:48-50
public struct Cap<phantom T> has key, store { id: UID }
```

WITNESS-GATED CAP CREATION

```
// enclave.move:60-64
public fun new_cap<T: drop>(
    _: T,                 // witness consumed (must be `drop`)
    ctx: &mut TxContext,
) → Cap<T> {
    Cap { id: object::new(ctx) }
}
```

Only the module that defines `T` can construct a value of `T`, so only that module can mint its `Cap<T>`. The cap's id is stored as `EnclaveConfig.capability_id`, and access checks assert `cap.id.to_inner() == enclave_config.capability_id` (abort `EInvalidCap = 2`, enclave.move:160).

REGISTRATION FLOW & PCR CHECK

```
// enclave.move:85-100
public fun register_enclave<T>(
    enclave_config: &EnclaveConfig<T>,
    document: NitroAttestationDocument,
    ctx: &mut TxContext,
) {
    let pk = enclave_config.load_pk(&document);
    let enclave = Enclave<T> {
        id: object::new(ctx),
        pk,
        config_version: enclave_config.version,
        owner: ctx.sender(),
    };
    transfer::share_object(enclave);
}

// enclave.move:163-167
fun load_pk<T>(
    enclave_config: &EnclaveConfig<T>,
    document: &NitroAttestationDocument,
) → vector<u8> {
    assert!(document.to_pcrs() == enclave_config.pcrs,
            EInvalidPCRs); // = 0
    (*document.public_key()).destroy_some()
}
```

Registration abort codes (enclave.move:15-18): `EInvalidPCRs=0`, `EInvalidConfigVersion=1`, `EInvalidCap=2`, `EInvalidOwner=3`.

LIMITATIONS

- PCRs must be known and pinned in advance — every code change to the enclave changes PCR2 and forces re-registration with a new attestation
- There is no on-chain notion of 'this PCR set was audited' — the dapp owner is trusted to register PCRs of genuinely reviewed code
- A compromised dapp owner (holder of Cap<T>) can register a malicious enclave config; the Cap is a single point of authority
- The Enclave object is shared, so its public key is globally readable — fine for verification, but the model assumes the ephemeral private key is never extracted

EXERCISES

1. **hands-on** — Read register_enclave<T> (enclave.move:85-100) and load_pk (enclave.move:163-167); identify exactly where the PCR equality check happens.
2. **hands-on** — Study new_cap<T: drop> (enclave.move:60-64) and explain in one sentence why the witness type T must have the drop ability.
3. **hands-on** — List the four enclave-registration abort codes (enclave.move:15-18) and construct, on paper, a transaction that would trigger EInvalidCap=2.

Off-Chain Server: Ephemeral Keypair & Signed Responses

INTUITIVE

Think of the enclave server as a sworn witness who, the moment they enter the courtroom, is handed a brand-new fountain pen (the ephemeral Ed25519 keypair) that exists only for this session and never leaves the room. The court records the pen's unique nib pattern (the public key, via attestation). From then on, every statement the witness signs with that pen is provably theirs. Each response is wrapped in an IntentMessage — a tamper-evident envelope stamped with an intent code and a timestamp — then BCS-serialized and signed, so the on-chain verifier can re-derive the exact bytes and check the signature.

KEY POINTS

- AppState holds eph_kp: Ed25519KeyPair (generated on boot) and an api_key for external services (lib.rs:40-45)
- Keypair created via Ed25519KeyPair::generate(&mut rand::thread_rng()) in main.rs; one keypair per enclave instance (main.rs:16)
- GET /get_attestation calls the NSM driver with NsmRequest::Attestation{ public_key: Some(pk) } and returns the COSE doc (common.rs:82-113)
- POST /process_data wraps output via to_signed_response<T>(kp, payload, timestamp_ms, intent) (common.rs:56-71)
- IntentMessage<T> { intent: u8, timestamp_ms: u64, data: T } mirrors the on-chain Move struct field-for-field (common.rs:26-40)
- Signing payload is bcs::to_bytes(&intent_msg) — BCS guarantees identical bytes in Rust and Move for cross-language verification
- The private key signs only inside the enclave; only the public key is ever exported (via attestation) — rotation requires re-registration

WHY IT MATTERS

This is where the TEE meets your cryptography background: the enclave acts as a single signer whose key is hardware-anchored. The IntentMessage/BCS discipline is the same determinism requirement you face when a ZK circuit and a verifier must agree on serialized inputs byte-for-byte. The 'compute privately inside, sign the result, verify cheaply outside' pattern is the exact shape a TEE-backed confidential-payment processor takes in your thesis.

THESIS ANGLE

Your confidential-payment enclave would receive an encrypted intent, decrypt and route it internally, then call to_signed_response with intent=PAYMENT_SETTLED and a payload carrying the (public) settlement commitment. The client submits that signed message to a Move function that mints/settles only if enclave.verify_signature(...) returns true — the enclave's correctness is enforced cryptographically, not by trusting the operator.

TECHNICAL

The enclave runs a Rust web server holding an **ephemeral Ed25519 keypair** generated at boot; the private key never leaves the enclave. It exposes `GET /get_attestation` (one-time, returns the COSE document containing the public key) and `POST /process_data` (wraps each result in an `IntentMessage`, BCS-serializes, and signs it).

APP STATE & KEYPAIR

```
// lib.rs:40-45
pub struct AppState {
    pub eph_kp: Ed25519KeyPair, // generated on boot (fastcrypto)
    pub api_key: String,        // for external APIs
}

// main.rs:16
let eph_kp = Ed25519KeyPair::generate(
    &mut rand::thread_rng());
```

INTENTMESSAGE & SIGNED RESPONSE

```
// common.rs:26-40 — mirrors Move (enclave.move:53-57)
pub struct IntentMessage<T: Serialize> {
    pub intent: u8,         // application intent code
    pub timestamp_ms: u64, // ms since UNIX epoch
    pub data: T,
}

// common.rs:56-71
pub fn to_signed_response<T: Serialize + Clone>(
    kp: &Ed25519KeyPair,
    payload: T,
    timestamp_ms: u64,
    intent: u8,
) -> ProcessedDataResponse<IntentMessage<T>> {
    let intent_msg =
        IntentMessage::new(payload.clone(), timestamp_ms, intent);
    let signing_payload =
        bcs::to_bytes(&intent_msg).expect("should not fail");
    let sig = kp.sign(&signing_payload);
    ProcessedDataResponse {
        response: intent_msg,
        signature: Hex::encode(sig),
    }
}
```

Because BCS is canonical, the exact bytes signed here are reconstructed independently in Move during verification — the signed bytes are never transmitted.

GET /GET_ATTESTATION (NSM DRIVER)

```
// common.rs:82-113
pub async fn get_attestation(
    State(state): State<Arc<AppState>>,
) -> Result<Json<GetAttestationResponse>, EnclaveError> {
    let pk = state.eph_kp.public();
    let fd = driver::nsm_init();
    let request = NsmRequest::Attestation {
        user_data: None,
        nonce: None,
        public_key: Some(ByteBuf::from(
            pk.as_bytes().to_vec())),
    };
    let response = driver::nsm_process_request(fd, request);
    // returns COSE_Sign1 over a doc containing `pk`
}
```

The NSM signs internally; only the public key crosses the enclave boundary, embedded in the attestation document.

HISTORY Mysten Labs (Nautilus reference server) using fastcrypto Ed25519 (2025). *Because BCS serialization is canonical, the very same byte vector that the Rust server signs is reconstructed independently inside Move — neither side ever transmits the signed bytes; they each derive them from the structured fields.*

LIMITATIONS

- Ephemeral key compromise inside the enclave (e.g., via a memory-disclosure bug in the app code) lets an attacker forge arbitrary signed outputs
- No on-chain freshness enforcement: the server stamps timestamp_ms but the framework does not check it — replay protection is the dapp's job
- Key rotation is heavyweight: a new keypair means a new attestation and a new on-chain registration, so long-lived deployments accumulate registrations
- The enclave is stateless by design, so it cannot prove ordering or 'this is the correct successor state' — only single-output authenticity

EXERCISES

1. **hands-on** — Read to_signed_response<T> (common.rs:56-71) and write down the exact byte sequence that gets signed.
2. **hands-on** — Compare the Rust IntentMessage<T> (common.rs:26-40) with the Move intent message (enclave.move:53-57) and confirm the field order matches.
3. **hands-on** — Trace get_attestation (common.rs:82-113): which NsmRequest variant is used and what does public_key: Some(pk) put into the document?

On-Chain Signature Verification & Usage

INTUITIVE

After the one-time passport check, every later interaction is a quick signature match. The Move verifier takes the structured claim (intent + timestamp + payload), rebuilds the exact bytes the enclave would have signed, and checks them against the public key it already stored at registration. It's like a bank teller who, once your signature card is on file, just compares each cheque to the card — fast, cheap, and no notary needed for every transaction.

KEY POINTS

- verify_signature<T, P: drop>(enclave, intent_scope, timestamp_ms, payload, signature) -> bool (enclave.move:102-112)
- It reconstructs the intent message on-chain, BCS-serializes it, and calls ed25519::ed25519_verify against enclave.pk
- The public key compared against is the one anchored during register_enclave — no per-call attestation needed
- There is NO on-chain timestamp validation; freshness is left entirely to the dapp (a deliberate design choice)
- Example: update_weather<T>(location, temperature, timestamp_ms, sig, &enclave, ctx) verifies WEATHER_INTENT (=0) then mints (weather.move:45-67)
- Failure path: assert!(res, EInvalidSignature) — the dapp defines its own abort code on signature mismatch
- Verification is cheap (one Ed25519 check) versus registration (full cert-chain attestation) — the core cost-amortization of the design

WHY IT MATTERS

The two-phase 'expensive attestation once, cheap signature many times' structure is the performance argument that makes TEE attractive versus per-call ZK proofs in some settings. For your thesis comparison table this is the headline tradeoff: ZK verification cost is paid every proof; Nautilus pays a heavy cert-chain check once at registration and then only an Ed25519 verify per output. Quantifying that is a concrete, citable result for the evaluation chapter.

THESIS ANGLE

A TEE-backed confidential-balance oracle would expose verify_signature(enclave, BALANCE_INTENT, ts, BalanceAttestation{commitment}, sig) inside a Move settlement function. You'd add the freshness check the framework omits — e.g., assert!(ts + WINDOW > clock.timestamp_ms()) — to close the replay gap, and document that as a hardening you contributed over the reference design.

TECHNICAL

After registration, the chain verifies enclave outputs with a single Ed25519 check. `verify_signature` reconstructs the `IntentMessage` on-chain, BCS-serializes it, and verifies the signature against the registered `enclave.pk`. There is **no on-chain timestamp validation** — freshness/replay protection is delegated to the dapp.

VERIFY_SIGNATURE (MOVE)

```
// enclave.move:102-112
public fun verify_signature<T, P: drop>(
    enclave: &Enclave<T>,
    intent_scope: u8,
    timestamp_ms: u64,
    payload: P,
    signature: &vector<u8>,
): bool {
    let intent_message =
        create_intent_message(intent_scope, timestamp_ms, payload);
    let payload = bcs::to_bytes(&intent_message);
    ed25519::ed25519_verify(signature, &enclave.pk, &payload)
}
```

The function returns `bool` — callers MUST `assert!` it. The public key compared against is the one anchored in `register_enclave`; no per-call attestation is needed.

DAPP USAGE PATTERN (WEATHER EXAMPLE)

```
// weather.move:45-67
public fun update_weather<T>(
    location: String,
    temperature: u64,
    timestamp_ms: u64,
    sig: &vector<u8>,
    enclave: &Enclave<T>,
    ctx: &mut TxContext,
): WeatherNFT {
    let res = enclave.verify_signature(
        WEATHER_INTENT, // = 0
        timestamp_ms,
        WeatherResponse { location, temperature },
        sig,
    );
    assert!(res, EInvalidSignature);
    // mint NFT ...
}
```

END-TO-END TEST WALKTHROUGH

The on-chain test (weather.move:69-127) shows the full lifecycle:

```
let payload = x"8444a1013822..."; // attestation CBOR
let document =
    nitro_attestation::load_nitro_attestation(payload, &clock);
config.register_enclave(document, scenario.ctx());

let sig = x"77b6d8be225440d0..."; // Ed25519 from enclave
let nft = update_weather(
    b"San Francisco".to_string(),
    13,
    1744683300000, // timestamp_ms
    &sig,
    &enclave,
    scenario.ctx(),
);
```

Security note: no freshness is enforced by the framework — a dapp that wants replay protection must add e.g. `assert!(timestamp_ms + WINDOW > clock.timestamp_ms())` itself.

LIMITATIONS

- No built-in replay protection: the same (payload, timestamp, sig) can be submitted repeatedly unless the dapp tracks nonces/timestamps
- verify_signature returns bool, not an abort — forgetting the assert! silently accepts unverified data (a real footgun)
- Correctness of the payload is not verified, only its authenticity — the chain trusts that the registered enclave computed the right answer
- Statelessness means the chain cannot detect a forked/rolled-back enclave producing two contradictory valid signatures for the same timestamp

EXERCISES

1. **hands-on** — Read verify_signature<T, P> (enclave.move:102-112) and the example update_weather (weather.move:45-67); note where the result bool is asserted.
2. **hands-on** — Identify the exact line where the framework could (but deliberately does not) validate timestamp_ms, and sketch the assert you'd add.
3. **hands-on** — Trace the full weather walkthrough (weather.move:69-127): registration via load_nitro_attestation, then update_weather with a real signature.

ZKP + TEE Composition (Seal Integration)

INTUITIVE

ZK and TEE are two different ways to make a stranger trust a computation. ZK is a magician proving a card trick is fair using only math you can check yourself — no trusted assistant. TEE is a sealed glass box from a trusted manufacturer: you trust the box, and it shows you the result is genuine. Composing them gives you the best of both: the enclave (TEE) holds and decrypts secrets that Seal encrypted, computes privately inside the box, and emits a signed result — while the math-checkable proof commitments handle the parts you don't want to trust the box for. The thesis question is exactly where to draw that line.

KEY POINTS

- Nautilus ships an optional seal-example feature flag (#[cfg(feature = "seal-example")]) that integrates Seal into the enclave (lib.rs:12-34)
- Two-phase bootstrap: a host-init server provisions/injects a sealed secret before the main computation server starts (main.rs:32-35)
- Secret provisioning is decoupled from attestation — a SEAL_API_KEY is supplied outside and unsealed inside the enclave
- Trust-model contrast: ZK roots in proof soundness (no trusted party); TEE roots in the AWS cert chain + Nitro hardware
- Verification cost contrast: ZK pays per proof; Nautilus pays one cert-chain verify at registration, then a cheap Ed25519 per output
- Composition glue differs: ZK composes via proof commitments; Nautilus composes via the ephemeral key's signature
- Combined claim: a signed enclave output can assert BOTH 'from the registered enclave' AND 'ran a Seal-verified private computation'

WHY IT MATTERS

This is the thesis thesis. Your project lives precisely at the ZKP+TEE seam: deciding which guarantees come from cryptographic soundness and which from hardware attestation, and how privacy-payment flows compose them. Nautilus + Seal is the concrete reference for that composition on Sui, and the Mysten internship is where you'd push it — e.g., using a TEE to make ZK proving practical over secrets, or using ZK to remove the hardware-trust assumption where it matters most.

THESIS ANGLE

A privacy-payment sketch: the payer encrypts their payment intent under Seal; the enclave (and only the enclave) decrypts it, computes the routing/settlement, re-encrypts the output, and signs it with its ephemeral key. On-chain you verify the Ed25519 signature (proves the enclave was used) and, where present, a Seal/ZK commitment (proves the intent stayed private). Your contribution is the protocol and the precise statement of what each layer guarantees.

TECHNICAL

Nautilus (TEE) and Seal (threshold encryption / ZK-adjacent secrets) compose to give a single signed output that proves *both* 'this came from the registered enclave' and 'the enclave ran a Seal-verified private computation'. The two trust models are complementary:

- **ZK (Seal)**: root of trust = proof soundness (offline-verifiable, no trusted party); high per-proof verification cost; hides the entire computation; composes via proof commitments.
- **TEE (Nautilus)**: root of trust = AWS CA + Nitro silicon; one cert-chain verify at registration then cheap Ed25519 per output; hides internal state and attests enclave correctness; composes via ephemeral-key signatures.

FEATURE-GATED SEAL INTEGRATION (RUST)

```
// lib.rs:12-34
#[cfg(feature = "seal-example")]
#[path = "seal-example/mod.rs"]
pub mod seal_example;

// main.rs:32-35 — two-phase bootstrap
#[cfg(feature = "seal-example")]
{
    nautilus_server::app::spawn_host_init_server(
        state.clone()).await?;
}
```

A host-only init server runs in parallel to the main server to inject an encrypted secret (e.g. `SEAL_API_KEY`) before the main computation starts — decoupling secret provisioning from the main attestation.

TRUST-MODEL COMPARISON TABLE

```
Dimension      | ZKP (Seal)          | TEE (Nautilus)
---------------+---------------------+------------------------
Root of trust  | proof (offline)     | AWS CA chain + hardware
Verify cost    | high (per proof)    | 1x cert chain + low
               |                     | (Ed25519 per output)
Privacy        | hides computation   | hides internal state;
               |                     | attests correctness
Composability  | proof commitment    | signature (eph key)
```

COMPOSED PRIVACY-PAYMENT PIPELINE

The intended composition for a privacy-payment flow:

- **1.** Enclave boots; generates ephemeral Ed25519 keypair.
- **2.** Attestation document (carrying the public key) registered on-chain via `register_enclave`.
- **3.** Seal injects the encrypted secret/intent into the enclave; only the enclave can decrypt.
- **4.** Enclave computes routing/settlement privately, re-encrypts output, and signs it via `to_signed_response`.
- **5.** On-chain: `ed25519_verify` proves the enclave was used; an optional Seal/ZK commitment proves the intent stayed private.

Caveat: composing widens the trusted computing base — soundness now depends on *both* the hardware/CA and the proof system. The signature alone proves authenticity, not that the private-compute branch was taken, unless that fact is explicitly committed into the payload or `user_data`.

HISTORY Mysten Labs (Seal + Nautilus on Sui) (2025). *The seal-example is gated behind a Cargo feature flag, so the base Nautilus server compiles without any Seal dependency — the TEE and the ZK/secret layers are deliberately decoupled, mirroring the thesis's modular trust argument.*

LIMITATIONS

- Composing TEE + ZK widens the trusted computing base: you now depend on BOTH the hardware/CA and the proof system being sound
- The seal-example is a reference, not a hardened protocol — secret-injection timing and the host-init server are attack surface
- Privacy of the intent depends on Seal's threshold assumptions; the TEE alone does not provide the cryptographic hiding a ZK proof does
- Reasoning about the combined guarantee is subtle: a valid signature proves authenticity but not that the private-compute branch was actually taken unless explicitly committed

EXERCISES

1. **hands-on** — Read the seal-example feature gate (lib.rs:12-34) and the host-init spawn (main.rs:32-35); describe the two-phase bootstrap order.
2. **hands-on** — Fill in the ZKP-vs-TEE comparison table from the source: root of trust, verification cost, privacy model, composition mechanism.
3. **hands-on** — Sketch (on paper) a privacy-payment flow that uses Seal for the encrypted intent and Nautilus for the signed settlement, labeling which property each layer provides.

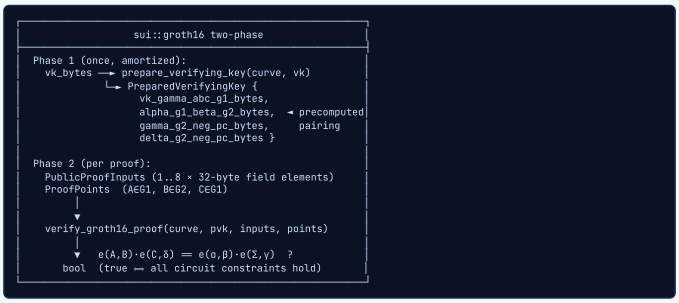
CHAPTER 6

Sui Cryptographic Primitives

sui::groth16 — SNARK Verifier Engine

INTUITIVE

A bouncer with a magic stamp. The prover spent enormous effort building a sealed envelope (the proof) that says 'I know a secret satisfying these constraints' without revealing the secret. groth16 is the bouncer: it does NOT re-do the prover's work — it runs a tiny, fixed-cost check (two elliptic curve pairings) that is mathematically guaranteed to pass only if the secret really exists. Even better, the bouncer pre-stamps part of the check once (prepare_verifying_key) so every subsequent guest is verified faster.



KEY POINTS

- Curve selector: bls12381() → id 0, bn254() → id 1 (Curve { id: u8 })
- Two-phase: prepare_verifying_key(curve, vk) then verify_groth16_proof(...) → bool
- Public inputs capped at MaxPublicInputs = 8 field elements, 32 bytes each
- ProofPoints = (A in G1, B in G2, C in G1); accepts Arkworks-canonical bytes
- Aborts: EInvalidVerifyingKey(0), EInvalidCurve(1), ETooManyPublicInputs(2), EInvalidScalar(3)
- public_proof_inputs_from_bytes validates len % 32 == 0 and count ≤ 8; proof_points_from_bytes does NOT validate
- BN254 is the curve zkLogin and most Circom/snarkjs circuits target on-chain

WHY IT MATTERS

groth16 is the on-chain endpoint of your Veil confidential-transaction pipeline. The off-chain prover (Rust/arkworks or Circom) produces a proof that a transaction is balanced and the spender knows the note's secret; the Move contract calls verify_groth16_proof to accept or reject the state update. The two-phase split matters for your gas budget: prepare the verifying key once at deploy time and store the PreparedVerifyingKey, so every confidential transfer pays only the per-proof pairing cost.

THESIS ANGLE

In Veil you store a PreparedVerifyingKey (built from your circuit's vk) as an immutable object. A confidential transfer entry function reconstructs PublicProofInputs from the on-chain commitments + nullifier and ProofPoints from the user-supplied proof, then asserts verify_groth16_proof(&bn254(), &pvk, &inputs, &points). Only on true do you insert the nullifier and update encrypted balances — the contract never learns the amount.

TECHNICAL

`sui::groth16` provides on-chain verification of **Groth16 zk-SNARK proofs** over **BLS12-381** and **BN254**. Verification is a two-phase pipeline: a one-time `prepare_verifying_key` that precomputes circuit-dependent pairing terms, and a per-proof `verify_groth16_proof` that runs the final pairing equation and returns a `bool`.

- `Curve { id: u8 }` — `bls12381()` → 0, `bn254()` → 1
- `PreparedVerifyingKey` — four byte vectors: `vk_gamma_abc_g1_bytes`, `alpha_g1_beta_g2_bytes`, `gamma_g2_neg_pc_bytes`, `delta_g2_neg_pc_bytes`
- `PublicProofInputs { bytes }` — 1..8 field elements, 32 bytes each, little-endian
- `ProofPoints { bytes }` — three group elements (A∈G1, B∈G2, C∈G1)

File: `sui-framework/sources/crypto/groth16.move` (140 lines).

PUBLIC TYPES, CONSTANTS AND ABORT CODES

```
public struct Curve has copy, drop, store { id: u8 }
public fun bls12381(): Curve { Curve { id: 0 } }
public fun bn254():   Curve { Curve { id: 1 } }

public struct PreparedVerifyingKey has copy, drop, store {
    vk_gamma_abc_g1_bytes:   vector<u8>,
    alpha_g1_beta_g2_bytes:  vector<u8>,
    gamma_g2_neg_pc_bytes:   vector<u8>,
    delta_g2_neg_pc_bytes:   vector<u8>,
}

public struct PublicProofInputs has copy, drop, store { bytes: vector<u8> }
public struct ProofPoints       has copy, drop, store { bytes: vector<u8> }

const MaxPublicInputs:     u64 = 8;
const EInvalidVerifyingKey: u64 = 0;
const EInvalidCurve:        u64 = 1;
const ETooManyPublicInputs: u64 = 2;
const EInvalidScalar:       u64 = 3;
```

PHASE 1 — PREPARE_VERIFYING_KEY (GROTH16.MOVE:97-99)

Canonicalizes an Arkworks verifying key and precomputes the fixed pairing terms. Aborts `EInvalidVerifyingKey` (bad encoding) or `EInvalidCurve` (unsupported curve). Wraps the native `prepare_verifying_key_internal`.

```
public fun prepare_verifying_key(
    curve: &Curve,
    verifying_key: &vector<u8>,
) : PreparedVerifyingKey {
    prepare_verifying_key_internal(curve.id, verifying_key)
}

native fun prepare_verifying_key_internal(
    curve: u8, verifying_key: &vector<u8>,
) : PreparedVerifyingKey;
```

PHASE 2 — VERIFY_GROTH16_PROOF (GROTH16.MOVE:113-128)

Runs the final pairing check, returning `true` iff the proof satisfies all circuit constraints. Aborts `EInvalidCurve` and `ETooManyPublicInputs`. The conceptual equation is `e(A,B) · e(C, δ) == e(α, β) · e(Σ vk_abc·inputs, γ)`.

```
public fun verify_groth16_proof(
    curve: &Curve,
    prepared_verifying_key: &PreparedVerifyingKey,
    public_proof_inputs:    &PublicProofInputs,
    proof_points:           &ProofPoints,
) : bool {
    verify_groth16_proof_internal(
        curve.id,
        &prepared_verifying_key.vk_gamma_abc_g1_bytes,
        &prepared_verifying_key.alpha_g1_beta_g2_bytes,
        &prepared_verifying_key.gamma_g2_neg_pc_bytes,
        &prepared_verifying_key.delta_g2_neg_pc_bytes,
        &public_proof_inputs.bytes,
        &proof_points.bytes,
    )
}
```

CONSTRUCTORS AND VALIDATION (GROTH16.MOVE:44-89)

`public_proof_inputs_from_bytes` **validates** that the byte length is divisible by 32 and the element count ≤ 8 (aborts `EInvalidScalar`, `ETooManyPublicInputs`). `proof_points_from_bytes` is an **opaque wrapper with no validation** — malformed points surface only inside the native.

```
public fun pvk_from_bytes(
    vk_gamma_abc_g1_bytes:  vector<u8>,
    alpha_g1_beta_g2_bytes: vector<u8>,
    gamma_g2_neg_pc_bytes:  vector<u8>,
    delta_g2_neg_pc_bytes:  vector<u8>,
) : PreparedVerifyingKey;            // line 44-56

public fun pvk_to_bytes(pvk: PreparedVerifyingKey)
    : vector<vector<u8>>;            // line 59-66

public fun public_proof_inputs_from_bytes(bytes: vector<u8>)
    : PublicProofInputs;             // line 75-79  (validates % 32, ≤ 8)

public fun proof_points_from_bytes(bytes: vector<u8>)
    : ProofPoints;                   // line 87-89  (no validation)
```

Gas note: a single pairing ≈ 5-10M gas; full Groth16 verification (two pairings) ≈ 20-30M gas. Prepare the key once and store the `PreparedVerifyingKey`.

HISTORY Jens Groth (groth16 scheme); Mysten Labs / fastcrypto (Sui native binding) (2016). *A groth16 proof is only ~200 bytes and verifies in milliseconds, no matter how astronomically complex the statement being proven is.*

LIMITATIONS

- Requires a circuit-specific trusted setup (the toxic-waste ceremony); a leaked setup secret lets an attacker forge proofs
- Hard cap of 8 public inputs (MaxPublicInputs) — larger statements must pack data into fewer field elements or use a different proof system
- Each pairing is expensive (~5-10M gas); full verification (two pairings) is on the order of tens of millions of gas units — batch off-chain where possible
- proof_points_from_bytes performs no validation — malformed points surface only as an EInvalidCurve/abort inside the native, so trust boundaries must be explicit
- Only BLS12-381 and BN254 are supported (EInvalidCurve otherwise)

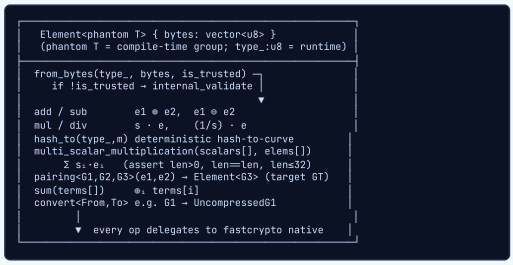
EXERCISES

1. **conceptual** — Why does sui::groth16 split verification into prepare_verifying_key and verify_groth16_proof instead of a single call? What does the first phase compute that the second reuses?
2. **design** — Sketch the Move entry function for a Veil confidential transfer that uses groth16. Which arguments come from chain state vs. the user, and where exactly does the abort happen on a bad proof?

sui::group_ops — Generic Group Arithmetic (Element<T>)

INTUITIVE

A type-safe calculator for elliptic-curve points. The numbers inside are opaque byte blobs that only the native Rust engine understands, but Move's phantom type parameter T acts like a label on each calculator key: you physically cannot add a G1 point to a G2 point, the compiler stops you. Every operation (add, multiply, pairing) is forwarded to fastcrypto with a one-byte 'type_' tag telling the engine which group it is working in.



KEY POINTS

- Core type: Element<phantom T> { bytes: vector<u8> } — phantom T = compile-time group safety
- from_bytes(type_, bytes, is_trusted): validates via internal_validate when !is_trusted, else stores opaquely
- add/sub/mul/div: group law and scalar mul/div; div aborts on scalar = 0 (field inversion failure)
- multi_scalar_multiplication: asserts non-empty, equal lengths, ≤ 32 elements (EInputTooLong)
- pairing<G1,G2,G3>(type_, e1, e2): bilinear map into target group GT
- convert<From,To>: space/time tradeoff projection (e.g. compressed ↔ uncompressed)
- Errors: ENotSupported(0), EInvalidInput(1), EInputTooLong(2), EInvalidBufferLength(3)

WHY IT MATTERS

group_ops is the arithmetic substrate beneath everything pairing-based in your thesis. Pedersen commitments (C = v·G + r·H), commitment homomorphism, and multi-scalar multiplication for aggregating openings all bottom out in these calls. When you write Veil's commitment logic in Move rather than calling a higher-level wrapper, group_ops is the API you use directly, with bls12381's typed constants supplying the generators.

THESIS ANGLE

To form a Veil Pedersen commitment in Move you compute group_ops::add(G1_TYPE, &group_ops::mul(...value·G...), &group_ops::mul(...r·H...)). To verify a batched opening of N commitments you use multi_scalar_multiplication(G1_TYPE, &coeffs, &basis_points) — one native call evaluating Σ $\text{coeff}_i \cdot \text{point}_i$, which is dramatically cheaper than N separate muls and keeps your confidential-transfer verification under the gas ceiling.

TECHNICAL

`sui::group_ops` is the low-level, `Element<T>`-typed elliptic-curve arithmetic layer backing all pairing-based cryptography on Sui. It is a type-safe Move wrapper over fastcrypto native calls: the **phantom type parameter** `T` enforces group discipline at compile time, while a runtime `type_: u8` discriminator tells the native which group it is operating in.

```
public struct Element<phantom T> has copy, drop, store {
    bytes: vector<u8>,
}
```

File: `sui-framework/sources/crypto/group_ops.move` (150 lines).

ACCESSORS, VALIDATION AND THE GROUP LAW

```
public fun bytes<G>(e: &Element<G>): &vector<u8>      // 26-28
public fun equal<G>(e1: &Element<G>, e2: &Element<G>): bool // 30-32

public fun from_bytes<G>(
    type_: u8, bytes: vector<u8>, is_trusted: bool,
) : Element<G> {                                      // 35-38
    if (!is_trusted) {
        assert!(internal_validate(type_, &bytes), EInvalidInput);
    };
    Element<G> { bytes }
}

public fun add<G>(type_: u8, e1: &Element<G>, e2: &Element<G>): Element<G> // 40-42
public fun sub<G>(type_: u8, e1: &Element<G>, e2: &Element<G>): Element<G> // 44-46
public fun mul<S, G>(type_: u8, scalar: &Element<S>, e: &Element<G>): Element<G> // 48-50
public fun div<S, G>(type_: u8, scalar: &Element<S>, e: &Element<G>): Element<G> // 53-55 (aborts if scalar = 0)
```

`div` aborts implicitly when the scalar is zero (fastcrypto field inversion failure). `from_bytes` with `is_trusted = true` skips validation entirely.

HASH_TO, MSM, PAIRING, CONVERT, SUM

```
public fun hash_to<G>(type_: u8, m: &vector<u8>): Element<G> // 57-59

public fun multi_scalar_multiplication<S, G>(   // 64-83
    type_: u8,
    scalars:  &vector<Element<S>>,
    elements: &vector<Element<G>>,
) : Element<G> {
    assert!(scalars.length() > 0, EInvalidInput);
    assert!(scalars.length() == elements.length(), EInvalidInput);
    // aborts EInputTooLong if length > 32 (current limit)
    // returns s1*e1 + s2*e2 + ... + sn*en
}

public fun pairing<G1, G2, G3>(
    type_: u8, e1: &Element<G1>, e2: &Element<G2>,
) : Element<G3>                                     // 85-91  (target group GT)

public fun convert<From, To>(
    from_type_: u8, to_type_: u8, e: &Element<From>,
) : Element<To>                                     // 93-99

public fun sum<G>(type_: u8, terms: &vector<Element<G>>): Element<G> // 101-103
```

ERROR CODES AND HELPERS

```
const ENotSupported:        u64 = 0;
const EInvalidInput:        u64 = 1;
const EInputTooLong:        u64 = 2;
const EInvalidBufferLength:  u64 = 3;

// test/encoding helper (135-149): u64 → 8 big/little-endian bytes
fun set_as_prefix(x: u64, big_endian: bool, buffer: &mut vector<u8>) {
    assert!(buffer.length() > 7, EInvalidBufferLength);
    // ...
}
```

Trust caveat: the phantom `T` and the runtime `type_: u8` are not cross-validated by Move; mismatches abort only inside the native. All real arithmetic is delegated to audited Rust (fastcrypto `internal_*` functions).

HISTORY Mysten Labs (group_ops Move module over fastcrypto) (2023). *Move's phantom types give you compile-time group discipline for free even though, at runtime, every Element is just an indistinguishable vector<u8>.*

LIMITATIONS

- The phantom type T and the runtime type_: u8 are NOT cross-checked by Move — passing a G1 byte buffer with G2_TYPE is the caller's responsibility and aborts only inside the native
- multi_scalar_multiplication is capped at 32 elements (EInputTooLong) — larger batches must be split, partially defeating MSM's efficiency
- All bytes are opaque on-chain; you cannot inspect or branch on point coordinates in Move
- div / scalar inversion aborts on zero with a native (non-named) failure, so guard divisors yourself
- Validation is skipped entirely when is_trusted = true — a footgun if the source is not actually trusted

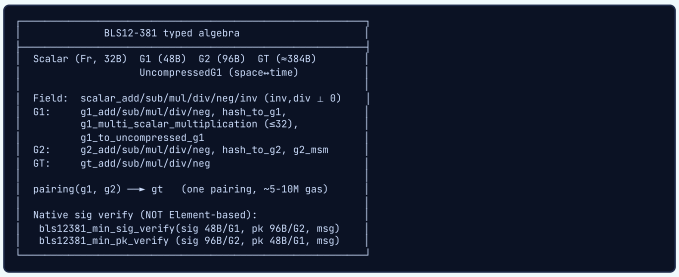
EXERCISES

1. **comparison** — Compare calling group_ops::mul N times and summing, versus a single multi_scalar_multiplication. What does the MSM win on, and what is the hard constraint you must respect?
2. **design** — You need a Pedersen commitment C = v·G + r·H on G1 in Move. Which group_ops calls do you chain, and what supplies G and H?

sui::bls12381 — Typed BLS12-381 Curve Operations

INTUITIVE

The fully-labeled, ready-to-use edition of the group_ops calculator, specialized for the BLS12-381 curve. Instead of passing raw type tags, you get named keys: scalar_add, g1_mul, g2_generator, pairing. Each Scalar, G1, G2, GT is its own Move type so the compiler enforces the algebra, and the module ships the exact generator and identity constants so you never hand-encode a curve point.



KEY POINTS

- Type tags: SCALAR=0, G1=1, G2=2, GT=3, UNCOMPRESSED_G1=4 — sizes 32/48/96/≈384 bytes
- Ships generator + identity byte constants for Scalar, G1, G2, GT (big-endian)
- Field ops scalar_div/scalar_inv and the *_div ops abort on zero (field inversion failure)
- hash_to_g1 / hash_to_g2 use DST BLS_SIG_BLS12381G{1,2}_XMD:SHA-256_SSWU_RO_NUL_
- g1/g2_multi_scalar_multiplication abort when len > 32
- pairing(e1: &Element<G1>, e2: &Element<G2>) → Element<GT> — single on-chain pairing
- Native bls12381_min_sig_verify (sig on G1) and bls12381_min_pk_verify (sig on G2) for raw signature checks

WHY IT MATTERS

BLS12-381 is the native curve for Groth16 over BLS and the standard curve for threshold and aggregatable signatures. For your thesis, this module is how Veil's multi-party / auditor protocols verify BLS signatures on-chain, and how you compose typed commitments and pairing checks without dropping to raw group_ops byte tags. The min_sig / min_pk split lets you choose where the signature lives (small G1 vs small G2) to optimize for verification cost vs. key size.

THESIS ANGLE

If Veil uses a committee (e.g. an auditor set or a threshold decryptor), each member signs an attestation with BLS and you verify the aggregate on-chain with bls12381_min_sig_verify(&agg_sig, &agg_pk, &msg). For commitment aggregation you call g1_multi_scalar_multiplication(&opening_coeffs, &commitment_points) to fold many Pedersen openings into one G1 element, then check a single pairing(folded_g1, vk_g2) against the expected target.

TECHNICAL

`sui::bls12381` provides high-level, typed wrappers over `group_ops` specialized for the **BLS12-381** curve, plus two native signature-verification functions. Each algebraic domain is a distinct Move type — `Scalar` (Fr, 32B), `G1` (48B compressed), `G2` (96B compressed), `GT` (≈384B), `UncompressedG1` — so the compiler enforces correct algebra.

```
const SCALAR_TYPE:          u8 = 0;
const G1_TYPE:              u8 = 1;
const G2_TYPE:              u8 = 2;
const GT_TYPE:              u8 = 3;
const UNCOMPRESSED_G1_TYPE:  u8 = 4;
```

File: `sui-framework/sources/crypto/bls12381.move` (283 lines). Ships byte constants for generators and identities of Scalar / G1 / G2 / GT (big-endian).

NATIVE SIGNATURE VERIFICATION (BLS12381.MOVE:15-31)

These are **not** `Element`-based — they take raw byte vectors. `min_sig` places the signature on G1 (48B) and the key on G2 (96B); `min_pk` reverses it.

```
public native fun bls12381_min_sig_verify(
    signature:  &vector<u8>,   // 48 bytes, point on G1
    public_key: &vector<u8>,   // 96 bytes, point on G2
    msg:        &vector<u8>,
) : bool;  // DST: BLS_SIG_BLS12381G1_XMD:SHA-256_SSWU_RO_NUL_

public native fun bls12381_min_pk_verify(
    signature:  &vector<u8>,   // 96 bytes, point on G2
    public_key: &vector<u8>,   // 48 bytes, point on G1
    msg:        &vector<u8>,
) : bool;  // DST: BLS_SIG_BLS12381G2_XMD:SHA-256_SSWU_RO_NUL_
```

SCALAR FIELD FR (BLS12381.MOVE:80-122)

```
public fun scalar_from_bytes(bytes: &vector<u8>): Element<Scalar>
public fun scalar_from_u64(x: u64):           Element<Scalar>
public fun scalar_zero(): Element<Scalar>
public fun scalar_one():  Element<Scalar>
public fun scalar_add(e1, e2): Element<Scalar>
public fun scalar_sub(e1, e2): Element<Scalar>
public fun scalar_mul(e1, e2): Element<Scalar>
public fun scalar_div(e1, e2): Element<Scalar>  // aborts if e1 = 0
public fun scalar_neg(e):      Element<Scalar>
public fun scalar_inv(e):      Element<Scalar>  // aborts if e = 0
```

G1 / G2 / GT AND PAIRING (BLS12381.MOVE:127-282)

```
// G1 (48-byte compressed), lines 127-178
public fun g1_generator(): Element<G1>
public fun g1_identity():  Element<G1>
public fun g1_add/g1_sub/g1_neg(...): Element<G1>
public fun g1_mul(scalar: Element<Scalar>, e: Element<G1>): Element<G1>
public fun g1_div(scalar, e): Element<G1>       // aborts if scalar = 0
public fun hash_to_g1(m: &vector<u8>): Element<G1>
public fun g1_multi_scalar_multiplication(
    scalars:  &vector<Element<Scalar>>,
    elements: &vector<Element<G1>>,
) : Element<G1>                                 // aborts if len > 32
public fun g1_to_uncompressed_g1(e): Element<UncompressedG1>

// G2 (96-byte compressed), lines 183-229  — symmetric API + hash_to_g2
// GT (≈384-byte), lines 234-261 — gt_add/sub/mul/div/neg (div aborts if scalar = 0)

// Pairing + conversion, lines 266-282
public fun pairing(e1: &Element<G1>, e2: &Element<G2>): Element<GT>
public fun uncompressed_g1_to_g1(e: &Element<UncompressedG1>): Element<G1>
public fun uncompressed_g1_sum(
    terms: &vector<Element<UncompressedG1>>,
) : Element<UncompressedG1>
```

hash_to_g1 DST: `BLS_SIG_BLS12381G1_XMD:SHA-256_SSWU_RO_NUL_`; hash_to_g2 DST: the G2 variant. A single on-chain `pairing` ≈ 5-10M gas.

HISTORY Sean Bowe / Zcash (BLS12-381 curve); Boneh-Lynn-Shacham (BLS signatures) (2017). *min_sig vs min_pk is a literal space tradeoff: put the signature on the 48-byte G1 (cheap to store/transmit, e.g. for high-volume signatures) or on the 96-byte G2, swapping which object is small.*

LIMITATIONS

- A single on-chain pairing costs ~5-10M gas; pairing-heavy logic gets expensive fast
- GT elements are ~384 bytes uncompressed — storing or moving them on-chain is costly
- MSM still capped at 32 elements per call (same fastcrypto limit as group_ops)
- Signature verify natives take raw byte vectors with no length pre-check in Move — wrong-length inputs abort inside the native, not with a friendly Move error
- The phantom-typed API prevents type confusion in Move but not malleability of the underlying encoding — you must still feed canonical, subgroup-checked bytes

EXERCISES

1. **comparison** — When would you choose bls12381_min_sig_verify over bls12381_min_pk_verify? What lives on G1 vs G2 in each, and how does that map to a real cost decision?
2. **design** — Express a Pedersen-style commitment and its homomorphic addition using only bls12381 typed functions, and explain why the addition of two commitments still opens correctly.

sui::poseidon — ZK-Friendly Hash (BN254)

INTUITIVE

A hash function that 'speaks the same language' as a SNARK circuit. Normal hashes (SHA, Keccak) shuffle bits — cheap on a CPU but a nightmare to prove in zero-knowledge, where every bit operation explodes into constraints. Poseidon instead does pure field arithmetic (add and multiply over BN254), so the same Merkle tree you build on-chain with poseidon_bn254 can be proven inside a Groth16 circuit with a tiny number of constraints. It is the hash that lets on-chain state and off-chain proofs agree cheaply.

KEY POINTS

- Signature: public fun poseidon_bn254(data: &vector<u256>): u256
- MAX_INPUTS = 16 scalars per call; at least 1 required
- Every input must be < BN254_MAX (21888242871839275222246405745257275088548364400416034343698204186575808495617)
- Aborts: ENonCanonicalInput(0), EEmptyInput(1), ETooManyInputs(2)
- Inputs are BCS-serialized little-endian u256, hashed by native sponge, parsed back to u256
- Arithmetic-friendly (no bitwise ops) → far cheaper to prove in-circuit than Keccak/SHA
- BN254 field matches Circom/snarkjs and zkLogin's curve, so on-chain and in-circuit hashes agree

WHY IT MATTERS

Poseidon is the backbone of every private Merkle tree in your thesis. Veil's note commitments and nullifier accumulators are Poseidon hashes: the same function computes leaf/root on-chain (poseidon_bn254) and inside the Groth16 circuit that proves membership. Using a ZK-friendly hash is what keeps your membership proofs small enough to verify on Sui — a Keccak Merkle path would blow up the circuit's constraint count by orders of magnitude.

THESIS ANGLE

A Veil note leaf is leaf = poseidon_bn254(&vector[amount, blinding, recipient_pk]). The contract maintains a Poseidon Merkle accumulator; on deposit it recomputes the root with poseidon_bn254 up the path and stores it. When spending, the user proves in Groth16 'I know a leaf in the tree with this nullifier' — and because the in-circuit hash is the identical Poseidon over BN254, the on-chain root and the proven root are guaranteed to match.

TECHNICAL

`sui::poseidon` exposes the **Poseidon** hash over the **BN254 scalar field**. It hashes 1..16 field elements (as `u256`) into a single `u256` digest, and is optimized for zero-knowledge circuits: pure field arithmetic, no bitwise operations, so the same hash that runs on-chain costs only a handful of constraints in a SNARK.

- `public fun poseidon_bn254(data: &vector<u256>): u256`
- `MAX_INPUTS = 16`; at least one input required
- Every input must be strictly less than the BN254 field modulus

File: `sui-framework/sources/crypto/poseidon.move` (53 lines).

CONSTANTS AND ABORT CODES

```
const BN254_MAX: u256 =
  21888242871839275222246405745257275088548364400416034343698204186575808495617u256;
const MAX_INPUTS:        u64 = 16;
const ENonCanonicalInput: u64 = 0;
const EEmptyInput:        u64 = 1;
const ETooManyInputs:     u64 = 2;
```

POSEIDON_BN254 — VALIDATION + NATIVE (POSEIDON.MOVE:38-52)

Rejects empty input (`EEmptyInput`), >16 inputs (`ETooManyInputs`), and any non-canonical element ≥ field modulus (`ENonCanonicalInput`). Each element is BCS-serialized to little-endian `u256` bytes, hashed by the native sponge, and parsed back to `u256`.

```
public fun poseidon_bn254(data: &vector<u256>): u256 {
    assert!(data.length() > 0,           EEmptyInput);
    assert!(data.length() ≤ MAX_INPUTS, ETooManyInputs);
    let b = data.map_ref!(|e| {
        assert!(*e < BN254_MAX, ENonCanonicalInput);
        bcs::to_bytes(e)
    });
    let binary_output = poseidon_bn254_internal(&b);
    bcs::new(binary_output).peel_u256()
}

native fun poseidon_bn254_internal(
    data: &vector<vector<u8>>,
) : vector<u8>;
```

Gas note: a Poseidon hash of up to 16 inputs is on the order of ~100-200K gas (estimate, varies by input count) — cheap relative to a pairing.

HISTORY Grassi, Khovratovich, Rechberger, Roy, Schofnegger (Poseidon) (2019). *The same Poseidon hash can be a one-liner in a Circom circuit and a single native call in Move, which is exactly why on-chain and in-circuit Merkle roots line up bit-for-bit.*

LIMITATIONS

- Limited to BN254 — if your circuit uses BLS12-381's scalar field, this native does not apply
- MAX_INPUTS = 16 per call; wider arities must be hashed in a tree/sponge of multiple calls
- Inputs must be canonical (< field modulus) or it aborts ENonCanonicalInput — caller must reduce
- Younger and less battle-tested than SHA-2/Keccak; security rests on algebraic cryptanalysis assumptions
- Parameter choice (round counts) is security-critical; you rely on Sui shipping vetted constants

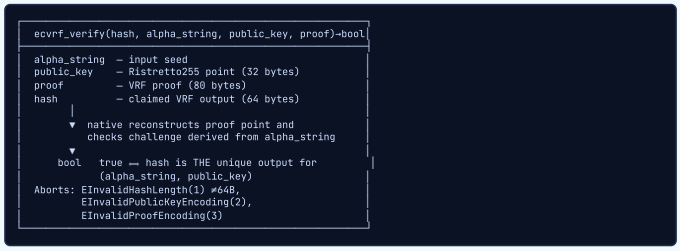
EXERCISES

1. **conceptual** — Why is Poseidon preferred over Keccak256 for the Merkle trees in a ZK privacy payment system, even though Keccak is faster on a normal CPU?
2. **calculation** — You call poseidon_bn254 with a vector containing a u256 equal to BN254_MAX. What happens, and how do you make the input valid?

sui::ecvrf — Verifiable Random Function (Ristretto255)

INTUITIVE

A tamper-proof dice machine with a receipt. A holder of a secret key can produce a random-looking output from any input seed, plus a proof that this exact output is the unique correct one for that seed and their public key. Anyone can check the receipt (ecvrf_verify) but no one — not even the key holder — can choose a different output. Unpredictable before reveal, verifiable after.



KEY POINTS

- Single native: public native fun ecvrf_verify(hash, alpha_string, public_key, proof): bool
- hash = 64-byte VRF output; public_key = 32-byte Ristretto255 point; proof = 80 bytes
- Aborts: EInvalidHashLength(1), EInvalidPublicKeyEncoding(2), EInvalidProofEncoding(3)
- Deterministic + unique: one valid output per (seed, key) pair, but unpredictable without the secret key
- Ristretto255 (Edwards) arithmetic — no pairings, so far cheaper than BLS/Groth16
- Useful for on-chain lotteries / commit-reveal where randomness must be proven, not trusted

WHY IT MATTERS

ECVRF is tangential to the core privacy-payment thesis (it reveals a public key and provides far less than a zero-knowledge proof), but it is the right tool whenever your design needs verifiable, unbiasable randomness without paying for pairings — e.g. selecting an auditor committee, randomizing a decoy set, or any commit-reveal sub-protocol where a party must prove its randomness was honestly derived from a committed seed.

THESIS ANGLE

If a Veil extension randomly samples which notes go into a privacy set (decoy selection) and must prove the sampling was unbiased, a coordinator with a registered Ristretto255 public key produces output = VRF(seed) and the contract calls ecvrf_verify(&output, &seed, &pubkey, &proof) before accepting the derived selection — guaranteeing the set was not grinded to deanonymize a target.

TECHNICAL

`sui::ecvrf` verifies an **Elliptic Curve Verifiable Random Function** over **Ristretto255**. A single native, `ecvrf_verify`, checks that a claimed 64-byte output is the unique correct VRF output for a given input seed and public key. Ristretto255 (a prime-order quotient of Curve25519) means no pairings and cheap verification.

File: `sui-framework/sources/crypto/ecvrf.move` (23 lines).

ECVRF_VERIFY AND ABORT CODES (ECVRF.MOVE:6-23)

```
 const EInvalidHashLength:         u64 = 1;  // output ≠ 64 bytes
const EInvalidPublicKeyEncoding:  u64 = 2;  // not a valid Ristretto255 point
const EInvalidProofEncoding:      u64 = 3;  // malformed proof

public native fun ecvrf_verify(
    hash:         &vector<u8>,   // VRF output, 64 bytes
    alpha_string: &vector<u8>,   // input seed
    public_key:   &vector<u8>,   // Ristretto255 public key, 32 bytes
    proof:        &vector<u8>,   // VRF proof, 80 bytes
) : bool;
```

The native reconstructs the VRF proof point from `hash`, `proof` and `public_key`, checks it against the challenge derived from `alpha_string`, and returns `true` iff all checks pass. Because the output is unique per (seed, key), even the secret-key holder cannot bias it — the basis for unbiasable on-chain randomness.

HISTORY Micali, Rabin, Vadhan (VRF concept); IRTF CFRG draft (ECVRF) (1999). *Unlike a plain signature, a VRF output is unique: even the secret-key holder cannot produce two different valid outputs for the same input, which is what makes it unbiasable.*

LIMITATIONS

- Reveals the prover's public key — provides verifiability, not anonymity, so it is a poor fit for the privacy core of the thesis
- Randomness quality depends on the secret key staying secret; a leaked key makes future outputs predictable
- Strict encodings: 64-byte hash, 32-byte key, 80-byte proof, or it aborts
- A single trusted producer can still withhold (refuse to reveal) — VRF prevents biasing the value but not censorship of it
- Lower-level than Sui's native on-chain randomness (sui::random); for general dApp randomness that beacon is usually the better default

EXERCISES

1. **conceptual** — What property does ECVRF give you that a normal digital signature over the same seed does not, and why does that property matter for fair randomness?
2. **design** — Why is ECVRF a weaker privacy tool than Groth16 for the thesis, and in what narrow role could it still appear in a Veil-style system?

sui::ecdsa_k1 — secp256k1 ECDSA & Recovery

INTUITIVE

A passport-control desk for Bitcoin/Ethereum signatures. secp256k1 is the curve those chains sign with, and this module lets a Sui contract check such signatures natively. ecrecover is the clever party trick: from a signature plus a one-byte recovery hint it reconstructs the signer's public key (and hence their Ethereum address) directly — no separately-supplied key needed.

KEY POINTS

- secp256k1_ecrecover(sig, msg, hash): 65-byte sig (r,s,v) + raw msg → 65-byte uncompressed pubkey
- Hash selector: KECCAK256 = 0, SHA256 = 1 (msg is hashed inside, not pre-hashed)
- secp256k1_verify(sig, pubkey, msg, hash): 64-byte (r,s) sig + 33-byte compressed key → bool (silent false)
- decompress_pubkey: 33-byte compressed (02/03 parity) → 65-byte uncompressed; aborts EInvalidPubKey
- Aborts: EFailToRecoverPubKey(0), EInvalidSignature(1), EInvalidPubKey(2)
- secp256k1_sign and secp256k1_keypair_from_seed are test-only (#[test_only]) — never for production
- Ethereum-compatible: recover address by hashing the recovered pubkey

WHY IT MATTERS

ecdsa_k1 is minor for the confidential-payment core (ECDSA reveals the public key and offers no privacy), but central to interoperability features your thesis may need: account abstraction and social recovery using existing Bitcoin/Ethereum keys, or bridged attestations where an off-chain or other-chain signature must be proven on Sui. It is the bridge between Sui's native identity model and the secp256k1 world.

THESIS ANGLE

If Veil offers social recovery where a guardian signs with their existing Ethereum wallet, the recovery entry function calls secp256k1_ecrecover(&guardian_sig, &recovery_msg, KECCAK256) and compares the Keccak hash of the recovered pubkey against the registered guardian address — letting users reuse MetaMask keys to authorize a Sui-side action without exposing any payment secrets.

TECHNICAL

`sui::ecdsa_k1` provides **ECDSA over secp256k1** (Bitcoin/Ethereum-compatible): public-key recovery from a signature, signature verification against a known key, and public-key decompression. Messages are hashed inside the native (selector: `KECCAK256 = 0` or `SHA256 = 1`), not pre-hashed.

File: `sui-framework/sources/crypto/ecdsa_k1.move` (115 lines).

CONSTANTS AND ABORT CODES

```
const KECCAK256: u8 = 0;
const SHA256:    u8 = 1;

const EFailToRecoverPubKey: u64 = 0;
const EInvalidSignature:    u64 = 1;
const EInvalidPubKey:       u64 = 2;
```

ECRECOVER, DECOMPRESS, VERIFY (ECDSA_K1.MOVE:49-75)

```
// 65-byte (r,s,v) signature + raw msg → 65-byte uncompressed pubkey (04||X||Y)
public native fun secp256k1_ecrecover(
    signature: &vector<u8>,   // 65 bytes (r,s,v), v in {0,1,2,3}
    msg:       &vector<u8>,
    hash:      u8,             // KECCAK256 | SHA256
) : vector<u8>;  // aborts EFailToRecoverPubKey | EInvalidSignature

// 33-byte compressed (02/03 parity) → 65-byte uncompressed
public native fun decompress_pubkey(
    pubkey: &vector<u8>,
) : vector<u8>;  // aborts EInvalidPubKey

// 64-byte (r,s) signature + 33-byte compressed key → bool (silent false)
public native fun secp256k1_verify(
    signature:  &vector<u8>,   // 64 bytes (r,s), non-recoverable
    public_key: &vector<u8>,   // 33 bytes compressed
    msg:        &vector<u8>,
    hash:       u8,
) : bool;
```

TEST-ONLY SIGNING PRIMITIVES (ECDSA_K1.MOVE:87-114)

These are gated `#[test_only]` and must never appear in a production contract.

```
#[test_only]
public native fun secp256k1_sign(
    private_key: &vector<u8>,  // 32 bytes
    msg:         &vector<u8>,
    hash:        u8,
    recoverable: bool,
) : vector<u8>;

#[test_only]
public native fun secp256k1_keypair_from_seed(
    seed: &vector<u8>,
) : KeyPair;
```

Note: `secp256k1_verify` returns `false` on a bad signature rather than aborting — callers must assert on the boolean. ECDSA is malleable (s vs -s) and has no native aggregation.

HISTORY Certicom (secp256k1 parameters); ANSI/SEC ECDSA standard (2000). *ecrecover is why Ethereum can derive 'who signed this' from a signature alone — the public key is literally recovered from the (r, s, v) tuple plus the message.*

LIMITATIONS

- ECDSA reveals the signer's public key — no privacy, so unsuitable for the confidential core of a privacy payment system
- secp256k1_verify fails silently (returns false) rather than aborting — easy to forget to check the bool
- No native signature aggregation (unlike BLS) — N signers means N verifications
- ECDSA is malleable (s and -s); enforce low-s if you key any logic on signature uniqueness
- secp256k1_sign / keypair_from_seed are test-only — they must never appear in a production contract

EXERCISES

1. **comparison** — Contrast secp256k1_ecrecover and secp256k1_verify: what inputs differ, what do they return, and which failure modes must you guard against in each?
2. **conceptual** — Why is ecdsa_k1 essentially irrelevant to the privacy core of the thesis, yet still worth including for account-abstraction features?

CHAPTER 7

Private Payments

Blockchain Privacy (Zcash, Sui, Private Payments)

Blockchain Transparency Problem

INTUITIVE

Publishing your bank statements on a billboard. Every transaction amount, sender, and receiver is visible to the entire world forever. Anyone with a browser can trace your financial life just by looking up your address.

KEY POINTS

- Pseudonymity is NOT anonymity: addresses are linkable via chain analysis
- Companies like Chainalysis de-anonymize users by clustering addresses
- Once identity is linked to one address, entire history is exposed
- Privacy is a fundamental right, not a tool for illicit activity
- Even "private" L2s leak metadata (timing, gas, access patterns)

WHY IT MATTERS

The transparency problem is the core motivation for the thesis. On Sui, all objects and transactions are public. The thesis builds a privacy layer using anonymous credentials and ZKP so users can transact without revealing identity or amounts.

THESIS ANGLE

In your thesis, this is the problem your system solves for Sui. Every Sui transaction today is fully transparent: if Alice pays Bob 100 SUI, the sender address, receiver address, amount, and Move function call are all visible forever. Your system adds a privacy layer where the same payment reveals only a ZK proof of validity — the Sui state records a commitment, not the actual payment details.

TECHNICAL

A blockchain transaction $\text{tx} = (\text{sender}, \text{receiver}, \text{amount}, \text{data})$ is a tuple where all fields are publicly visible on the ledger. The state of the chain $S = \{\text{tx}_1, \ldots, \text{tx}_n\}$ is a totally ordered, append-only sequence accessible to every participant.

TRANSACTION GRAPH ANALYSIS

The transaction history induces a directed multigraph $G = (V, E)$ where V is the set of addresses and E is the set of transactions. Each edge $e = (u, v, w, t) \in E$ carries a value w and timestamp t. Chain analysis exploits structural properties of G to de-anonymize users.

PSEUDONYMITY MODEL

An address is derived deterministically from a public key: $\text{addr} = H(\text{pk})$ where H is a hash function (e.g., Keccak-256 for Ethereum, SHA3-256 for Sui). Pseudonymity provides unlinkability only if no address is ever associated with a real-world identity. Once a single link is established, the entire transaction subgraph reachable from that address is de-anonymized.

COMMON-INPUT HEURISTIC

If a transaction spends multiple UTXOs $\{u_1, \ldots, u_k\}$, the common-input heuristic assumes all inputs belong to the same entity:

$$\text{Cluster}(\text{tx}) = \{\text{addr}(u_i) \mid u_i \in \text{inputs}(\text{tx})\}$$

This heuristic, combined with change-address detection, allows clustering of addresses into identity groups. Change detection identifies the output that returns surplus value to the sender: if $\sum v_{\text{in}} - v_{\text{out},1} = v_{\text{out},2}$ and $v_{\text{out},2}$ is a round number, $v_{\text{out},1}$ is likely the change.

FORMAL PRIVACY METRIC

The privacy of a transaction can be measured by the entropy of the sender distribution conditioned on observable data:

$$\text{Privacy}(\text{tx}) = H(\text{sender} \mid \text{observed}) = -\sum_{i=1}^{k} p_i \log_2 p_i$$

where $p_i = \Pr[\text{sender} = i \mid \text{observed}]$ and k is the anonymity set size. Perfect privacy yields $H = \log_2 k$ (uniform distribution). A system provides k-anonymity if the sender is indistinguishable among at least k users, i.e., $\max_i p_i \leq 1/k$.

HISTORY Satoshi Nakamoto (pseudonymous) (2008). *Chainalysis, founded in 2014 by Michael Gronager (ex-Kraken COO), reached a $8.6B valuation by 2022 — an entire industry built on the transparency that Nakamoto warned about. Their tools have been used by the IRS, FBI, and Europol to trace billions in illicit crypto.*

LIMITATIONS

- Pseudonymity is fragile: a single KYC exchange deposit links a real identity to an entire transaction graph — Meiklejohn et al. (2013) deanonymized 40% of Bitcoin transactions using basic heuristics
- Even "privacy-enhanced" L2 solutions (rollups, state channels) leak metadata: transaction timing, gas patterns, and access frequencies can be statistically correlated by a passive observer
- The transparency problem is structural, not incidental — public blockchains require transaction data for consensus validation, so privacy must be cryptographically constructed rather than assumed

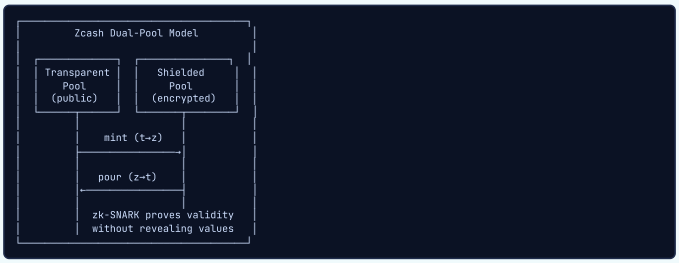
EXERCISES

1. **analysis** — Using a block explorer (e.g., suiscan.xyz), look up any recent Sui transaction. List every piece of information visible to the public. Then describe which of these your thesis system would hide and how.
2. **conceptual** — Explain why "just encrypting transaction data" is insufficient for blockchain privacy. What additional cryptographic properties are needed beyond confidentiality?

Zcash & Shielded Transactions

INTUITIVE

A magical accounting book where entries are encrypted. Auditors can verify the books balance without seeing any individual transaction. Deposits create sealed envelopes (note commitments), and spending reveals nothing except that a valid envelope was consumed.



KEY POINTS

- Sapling protocol uses Jubjub curve for efficient in-circuit arithmetic
- Note commitment tree (Merkle tree) stores encrypted transaction outputs
- Nullifiers prevent double-spending without revealing which note was spent
- Groth16 proofs: ~200 bytes, verify in ~10ms, but trusted setup required
- Optional transparency: users choose per-transaction (most stay transparent)

WHY IT MATTERS

Zcash proves that ZK-based private payments work at scale. The thesis adopts a similar commitment/nullifier model for private payments on Sui, but adds credential-gated access so that privacy does not preclude compliance.

THESIS ANGLE

For your system, the payment layer borrows directly from Zcash's design. Like Zcash notes, your system uses note commitments (cm = PedersenHash(pk, v, rho)) stored in a Sui Merkle tree object, nullifiers to prevent double-spending, and Groth16 proofs for spend authorization. The key difference: your system adds a credential check — users must prove they hold a valid identity credential before they can create shielded transactions, solving the compliance gap that led to Zcash's limited adoption.

TECHNICAL

Zcash implements a shielded UTXO model where transaction outputs (notes) are hidden behind commitments. A note is a tuple $\mathrm{note} = (\mathrm{pk}_d, v, \rho, \mathrm{rcm})$ consisting of a diversified payment address, a value, a unique nonce, and commitment randomness. The note commitment $\mathrm{cm} = \mathrm{PedersenHash}(\mathrm{repr}(\mathrm{note}))$ is added to a global Merkle tree. Spending a note requires publishing a nullifier $\mathrm{nf} = \mathrm{PRF}_{\mathrm{nsk}}(\rho)$ and a zero-knowledge proof that the note exists and is validly spent.

NOTE STRUCTURE & COMMITMENT

A Sapling note contains: diversified payment address pk_d (derived from recipient diversifier and key), value $v \in [0, 2^{64})$, unique nonce ρ (derived from nullifier of spent note), and randomness rcm. The commitment is computed as:

$$\mathrm{cm} = \mathrm{PedersenHash}(\mathrm{repr}(\mathrm{note}))$$

using the Jubjub curve for in-circuit efficiency. Each new commitment is inserted as a leaf into the global note commitment Merkle tree with root rt.

NULLIFIER DERIVATION

When spending a note, the owner computes a nullifier:

$$\mathrm{nf} = \mathrm{PRF}_{\mathrm{nsk}}(\rho)$$

where nsk is the nullifier secret key (part of the spending key) and ρ is the note nonce. The nullifier is published on-chain. Since PRF is a pseudorandom function, the nullifier reveals neither the note nor the spending key. The nullifier set prevents double-spending: a transaction is rejected if its nullifier already exists in the set.

SPEND CIRCUIT (GROTH16)

The spend proof demonstrates knowledge of a valid note without revealing it. The formal circuit statement is:

$$\exists\, (\mathrm{note}, \mathrm{path}, \mathrm{sk}) \text{ such that:}$$

$$\mathrm{cm} = \mathrm{Commit}(\mathrm{note})$$

$$\mathrm{MerklePath}(\mathrm{cm}, \mathrm{path}) = \mathrm{rt}$$

$$\mathrm{nf} = \mathrm{PRF}_{\mathrm{nsk}}(\rho)$$

$$v_{\mathrm{old}} \geq v_{\mathrm{new}}$$

The prover demonstrates: (1) knowledge of a valid note whose commitment is in the tree, (2) the Merkle path from that commitment to the published root rt is correct, (3) the nullifier is correctly derived, and (4) the note value is sufficient. None of the witness values (note, path, key) are revealed.

VALUE COMMITMENTS & BALANCE

Transaction values are hidden using Pedersen commitments on the Jubjub curve:

$$\mathrm{cv} = v \cdot \mathcal{V} + \mathrm{rcv} \cdot \mathcal{R}$$

where $\mathcal{V}, \mathcal{R}$ are independent generators. The homomorphic property enables balance verification without revealing amounts:

$$\sum_i \mathrm{cv}_{\mathrm{in},i} - \sum_j \mathrm{cv}_{\mathrm{out},j} = \mathrm{cv}_{\mathrm{fee}}$$

This equation is checked in the clear (on the commitments) and holds because Pedersen commitments are additively homomorphic: $\mathrm{Commit}(a, r_a) + \mathrm{Commit}(b, r_b) = \mathrm{Commit}(a + b, r_a + r_b)$.

SAPLING VS. ORCHARD

Sapling (2018): uses Groth16 proofs on BLS12-381 with Jubjub as the embedded curve. Proving time ~40ms, verification ~10ms, proof size ~192 bytes. Requires a trusted setup (powers-of-tau ceremony). Transaction structure: separate Spend and Output descriptions (more granular than the original 2-in-2-out JoinSplit). Orchard (2022): replaces Groth16 with Halo 2, a recursive proof system requiring NO trusted setup. Uses the Pallas/Vesta curve cycle for efficient recursion. Actions combine spend and output into a single description, improving privacy (spend and output counts are equal).

HISTORY Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, Madars Virza (2014). *The Zcash trusted setup ceremony for Sprout involved physically destroying the computers used to generate parameters. One participant, Peter Todd, incinerated his on a YouTube livestream. The Sapling ceremony ("Powers of Tau") had 87 participants to reduce the trust assumption. Halo 2 in Orchard finally eliminated trusted setup entirely.*

LIMITATIONS

- Opt-in privacy creates a small anonymity set: only ~29% of ZEC supply resides in shielded pools as of 2024 — Kappos et al. (2018, "An Empirical Analysis of Anonymity in Zcash") showed that statistical deanonymization is feasible when the shielded pool is small
- Groth16 trusted setup is a critical trust assumption: if any ceremony participant retained their toxic waste, they could forge proofs and create counterfeit ZEC undetectably (this motivated the move to Halo 2)
- Shielded transactions are ~10x more expensive than transparent ones (proving time ~2-4 seconds on mobile), which discourages adoption and further shrinks the anonymity set

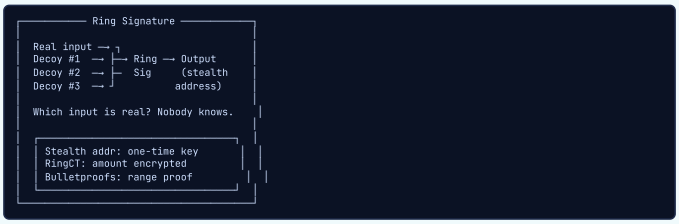
EXERCISES

1. **design** — Zcash's opt-in privacy led to only ~29% shielded adoption. Design an incentive mechanism for your Sui privacy layer that encourages users to use shielded transactions without making them mandatory.
2. **conceptual** — Explain how Zcash's nullifier scheme prevents double-spending without revealing which note was spent. Why is this better than simply marking a UTXO as "spent" in a public ledger?
3. **comparison** — Compare the Zcash Sprout, Sapling, and Orchard upgrade path. What was the key improvement at each stage, and which stage is most relevant to your thesis?

Monero Privacy

INTUITIVE

Sending a letter through 10 identical mailboxes simultaneously. Observers see all 10 but cannot tell which one contains your real letter. Every transaction automatically includes decoys, so privacy is the default, not an opt-in feature.



KEY POINTS

- Mandatory privacy: every transaction uses ring signatures (no opt-in choice)
- Ring signatures mix real input with decoy inputs (currently ring size 16)
- Stealth addresses: receiver gets a one-time address per transaction
- RingCT hides transaction amounts using Pedersen commitments
- Bulletproofs provide efficient range proofs without trusted setup

WHY IT MATTERS

Monero shows that mandatory privacy improves anonymity sets dramatically compared to opt-in (Zcash). The thesis considers mandatory privacy for its payment layer but needs compliance hooks that Monero deliberately avoids.

THESIS ANGLE

For your system, Monero proves that mandatory privacy (no transparent option) maximizes anonymity sets. Optional privacy (like Zcash) leads to low shielded adoption (~5%), shrinking the anonymity set. For your low-value payment tier on Sui, consider making privacy the default — all payments below the threshold are automatically shielded, ensuring a large anonymity set without requiring user opt-in.

TECHNICAL

Monero enforces mandatory privacy through three complementary mechanisms: ring signatures (sender ambiguity), stealth addresses (receiver privacy), and RingCT with Pedersen commitments (amount hiding). Every transaction uses all three, ensuring the anonymity set equals the full transaction set rather than an opt-in subset.

STEALTH ADDRESS DERIVATION

A Monero address is a pair of public keys $(A, B) = (aG, bG)$ where a is the view key and b is the spend key. To send to this address, the sender picks random r and computes a one-time destination key:

$$P = H_s(rA) \cdot G + B$$

The sender also publishes $R = rG$ (the transaction public key). The recipient scans transactions by computing $P' = H_s(aR) \cdot G + B$ and checking if $P' = P$. Only the recipient (knowing a) can detect incoming payments. To spend, the recipient derives the private key:

$$x = H_s(aR) + b$$

Each transaction output gets a unique one-time address, unlinking the recipient from their public address.

RING SIGNATURES (CLSAG)

The Compact Linkable Spontaneous Anonymous Group (CLSAG) signature allows signing with private key x_π among a ring of public keys $\{P_1, \ldots, P_n\}$. The verifier is convinced that one of the ring members signed, but cannot determine the index π. The signature satisfies: (1) Unforgeability: no adversary can produce a valid signature without knowing a private key in the ring, (2) Anonymity: the index π is computationally hidden under the Decisional Diffie-Hellman (DDH) assumption, (3) Linkability via key images.

KEY IMAGE (DOUBLE-SPEND PREVENTION)

Each input includes a key image:

$$I = x_\pi \cdot H_p(P_\pi)$$

where H_p maps a point to another curve point. The key image is deterministic for a given key pair — spending the same output twice produces the same image I. The network maintains a set of used key images and rejects duplicates. Crucially, I does not reveal π (which ring member is the real signer) because computing π from I requires solving the discrete log $x_\pi = \log_G P_\pi$.

RINGCT & PEDERSEN COMMITMENTS

RingCT hides transaction amounts using Pedersen commitments:

$$C_i = a_i G + b_i H$$

where a_i is the amount and b_i is a blinding factor, with G, H independent generators (discrete log of H w.r.t. G is unknown). Balance is verified homomorphically:

$$\sum C_{\text{in}} - \sum C_{\text{out}} = z \cdot H$$

If the transaction is valid, the excess z is zero (the blinding factors cancel). Miners verify this without learning any amounts.

BULLETPROOFS+ RANGE PROOFS

To prevent negative amounts (which would allow inflation), each output includes a range proof demonstrating $0 \leq a_i < 2^{64}$ without revealing a_i. Bulletproofs+ achieve logarithmic proof size: $O(\log n)$ where $n = 64$ is the bit-length. For m outputs, an aggregated proof has size $2\lceil \log_2(64m) \rceil + 9$ group elements, significantly smaller than the original Borromean ring signature range proofs. No trusted setup is required.

RING SIZE & ANONYMITY

The current mandatory ring size is 16, meaning each transaction input references 16 possible source outputs (1 real + 15 decoys). The effective anonymity set grows with ring size but also increases transaction size linearly. CLSAG reduces signature size compared to the previous MLSAG scheme: a CLSAG signature for ring size n requires $n + 1$ scalars and 1 key image, vs. $2n + 1$ scalars for MLSAG.

HISTORY Nicolas van Saberhagen (pseudonym, CryptoNote); Riccardo Spagni led Monero development (2014). *The IRS offered a $625,000 bounty in 2020 to anyone who could crack Monero tracing. CipherTrace (now Mastercard) and Chainalysis both claimed partial Monero tracing capabilities, but peer-reviewed analysis by Moser et al. (2018) showed that pre-2017 Monero (before RingCT) was significantly vulnerable, while post-RingCT Monero remains largely resistant to statistical deanonymization.*

LIMITATIONS

- Ring signatures provide plausible deniability, not cryptographic unlinkability: with ring size 16, a passive observer has a 1/16 chance of guessing the real input per transaction — statistical analysis across multiple transactions can narrow this (Moser et al., "An Empirical Analysis of Traceability in the Monero Blockchain," PoPETs 2018)
- No programmability: Monero is a pure payment chain with no smart contracts, so privacy-preserving DeFi (lending, DEX, etc.) is impossible — this is the gap Sui-based privacy fills
- Transaction size is ~2KB (with Bulletproofs+), roughly 8x larger than Bitcoin transactions, limiting throughput to ~1,700 TX/block vs. Bitcoin's ~4,000 — scalability is traded for privacy

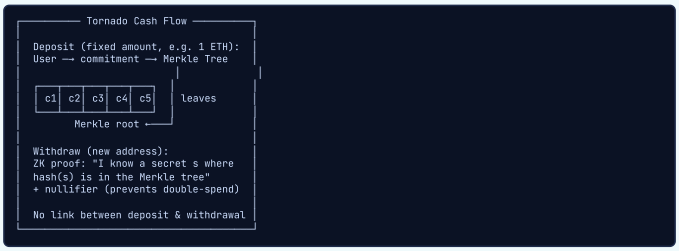
EXERCISES

1. **comparison** — Compare Monero's ring signature approach with Zcash's zk-SNARK approach for sender privacy. Which provides stronger anonymity guarantees, and what are the tradeoffs?
2. **analysis** — Monero's stealth addresses generate a one-time address per transaction. Explain how this works using Diffie-Hellman and why it prevents address reuse without requiring sender-receiver coordination.

Tornado Cash & Mixer Model

INTUITIVE

A coin laundry: you put your traceable coins in, wait, and withdraw unlinkable coins. The anonymity set is everyone who used the laundry at the same time. The bigger the crowd, the better your privacy.



KEY POINTS

- Merkle tree of commitments: each deposit adds a leaf hash(secret, nullifier)
- Withdrawal proves membership via ZK proof without revealing which leaf
- Nullifier hash published on withdrawal to prevent double-spending
- Fixed denominations (0.1, 1, 10, 100 ETH) maximize anonymity set overlap
- OFAC sanctions (2022) raised critical questions about protocol-level privacy vs. compliance

WHY IT MATTERS

Tornado Cash demonstrates the commitment/nullifier pattern the thesis reuses. However, its regulatory troubles highlight why the thesis includes compliance mechanisms (viewing keys, credential-gated access) that Tornado Cash lacked.

THESIS ANGLE

In your thesis, Tornado Cash demonstrates both the power and the regulatory risk of on-chain privacy. Your system directly addresses TC's failure mode: TC had no compliance mechanism, leading to OFAC sanctions. Your system solves this with credential-gated access — only KYC'd users (proven via BBS+ credential) can create private payments, and auditors with viewing keys can decrypt transaction details when legally required. Privacy WITH compliance, not privacy OR compliance.

TECHNICAL

Tornado Cash is a non-custodial zero-knowledge mixer on Ethereum. It implements a two-phase protocol: $\text{Deposit}(\text{commitment}) \to \text{Withdraw}(\text{proof}, \text{nullifier})$. Users deposit a fixed denomination by submitting a commitment $\text{cm} = H(s \| n)$ where s is a secret and n is a nullifier secret. Withdrawal proves knowledge of a valid commitment in the Merkle tree without revealing which one.

DEPOSIT PHASE

The user generates random values: secret s and nullifier secret n, both 248-bit field elements. The commitment is:

$$\text{cm} = H(s \| n)$$

where H is the Pedersen hash (or MiMC in earlier versions). The user sends a transaction to the Tornado Cash contract with the commitment and a fixed ETH amount (0.1, 1, 10, or 100 ETH). The contract inserts cm as a leaf into an incremental Merkle tree and stores the updated root R. Fixed denominations are critical: they ensure all deposits of the same size are indistinguishable.

MERKLE TREE STRUCTURE

The contract maintains an incremental Merkle tree of depth 20, supporting up to $2^{20} \approx 1,048,576$ deposits. The tree uses MiMC or Poseidon as the hash function (SNARK-friendly). Historical roots are stored (up to 100) to allow concurrent withdrawals against recent roots. The insertion cost is $O(\text{depth}) = O(20)$ hashes on-chain.

WITHDRAWAL CIRCUIT (GROTH16)

The withdrawal ZK circuit proves:

$$\exists\, (s, n, \text{path}) :$$

$$\text{leaf} = H(s \| n)$$

$$\text{MerklePath}(\text{leaf}, \text{path}) = R$$

$$\text{nullifier} = H(n)$$

Public inputs: root R, nullifier $H(n)$, recipient address, relayer address, fee. Private inputs (witness): secret s, nullifier secret n, Merkle path. The circuit has ~28,000 R1CS constraints. The nullifier $H(n)$ is published on-chain to prevent double-withdrawal: the contract rejects any nullifier that has been seen before.

ANONYMITY SET & RELAYER NETWORK

The anonymity set equals the number of unspent deposits of the same denomination since contract deployment. For the 0.1 ETH pool, this reached ~12,000 deposits, giving $\log_2(12000) \approx 13.5$ bits of anonymity. The relayer network solves the gas payment linkage problem: without a relayer, the withdrawer must pay gas from a funded address, potentially linking the withdrawal. Relayers submit the withdrawal transaction on behalf of the user, deducting a fee from the withdrawn amount.

HISTORY Alexey Pertsev, Roman Semenov, Roman Storm (2019). *Tornado Cash processed over $7.6 billion in deposits before sanctions. Vitalik Buterin publicly confirmed he used Tornado Cash to donate to Ukraine — demonstrating the legitimate privacy use case. After the sanctions, Ethereum validators running OFAC-compliant MEV relays began censoring TC transactions, briefly reaching 72% censorship rate before declining to ~30% by mid-2023.*

LIMITATIONS

- Fixed denomination pools (0.1, 1, 10, 100 ETH) limit usability: odd amounts require multiple deposits/withdrawals, creating timing correlation patterns that Chainalysis exploits for partial deanonymization
- No compliance mechanism whatsoever: Tornado Cash could not distinguish legitimate privacy users from money launderers, which directly led to OFAC sanctions and criminal prosecution of its developers
- Anonymity set is bounded by pool size and deposit/withdrawal timing: if only 5 people deposited 10 ETH in a given week, the anonymity set is just 5 — making correlation attacks practical for low-volume pools

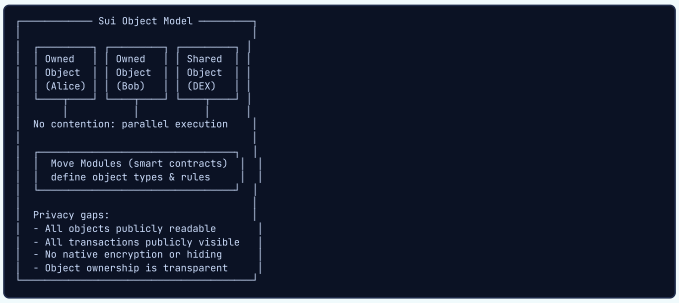
EXERCISES

1. **design** — Tornado Cash was sanctioned because it had zero compliance mechanisms. Design a "compliant mixer" that provides the same privacy guarantees but satisfies regulatory requirements. How does your thesis system differ from this approach?
2. **conceptual** — Explain how Tornado Cash's Merkle tree commitment scheme works. Why does the user need to remember their secret and nullifier, and what happens if they lose them?

Sui Architecture & Privacy Gaps

INTUITIVE

Sui is like a post office with numbered lockers (objects) instead of named mailboxes (accounts). Each locker is independent, so you can open multiple lockers simultaneously (parallel execution). But everything inside is still visible to anyone walking by.



KEY POINTS

- Object-centric model: state is objects, not global accounts
- Parallel execution on owned objects (no locks needed)
- Move language enforces resource safety (no duplication, no loss)
- No native privacy layer: all objects and transactions are public
- Opportunity: build a privacy-preserving layer on top using ZKP + TEE

WHY IT MATTERS

Sui is the target chain for the thesis. Its object model is ideal for representing credentials as objects, and its parallel execution means privacy operations on owned objects do not bottleneck the chain. The gap is privacy, which the thesis fills.

THESIS ANGLE

For your system, Sui's object-centric model is actually ideal for your privacy layer. Each private payment note becomes a Sui object (owned, encrypted). The object model's natural parallelism means private transactions on different note objects don't conflict — unlike account-based chains where all private transactions touch the same global state. Your Move module defines Note, Nullifier, and Credential objects with ZK verification as the access control mechanism.

TECHNICAL

Sui is an object-centric blockchain using the Move virtual machine with a DAG-based consensus protocol. State is represented as typed objects with unique identifiers, owners, and versions. Unlike account-based chains, Sui enables parallel execution on independent objects and single-writer fast-path for owned objects, achieving sub-second finality.

OBJECT MODEL

Everything in Sui is an object with a globally unique ID $\text{id} \in \{0,1\}^{256}$, an owner (address, another object, or "shared"), a version number, and typed contents defined by a Move struct. Objects are the fundamental unit of storage, ownership, and access control. This differs from the account model (Ethereum) where state is a global mapping $\text{address} \to \text{storage}$.

OWNED VS. SHARED OBJECTS

Owned objects have a single writer (the owner). Transactions on owned objects skip consensus entirely and use Byzantine Consistent Broadcast, achieving finality in ~200ms. Shared objects may be accessed by multiple writers and require full consensus via the Mysticeti protocol (DAG-based BFT), with finality in ~390ms. A transaction is specified as:

$$\text{tx} = (\text{sender}, [\text{input\_objects}], \text{Move\_call}, \text{gas\_budget})$$

where the input objects determine whether consensus is needed.

MOVE TYPE SYSTEM & RESOURCE SAFETY

Move enforces linear types: resources cannot be duplicated (copy is restricted) or implicitly dropped (drop must be explicit). This means a Coin object cannot be cloned (no double-spend at the language level) and cannot be lost (must be explicitly transferred or destroyed). Formally, if r is a resource, every execution path must consume r exactly once. The borrow checker enforces reference safety at compile time.

PRIVACY GAPS

Sui provides zero native privacy: (1) All objects are publicly readable via RPC: $\text{getObject}(\text{id}) \to \text{full\_contents}$, (2) All transactions are publicly visible: sender, called function, arguments, gas, and effects, (3) Object ownership is transparent: $\text{owner}(\text{obj})$ is always known, (4) Move module source and bytecode are public, (5) No native encrypted storage, confidential transactions, or private state. However, the object model is structurally suited for privacy: per-object encryption, per-object ZK proofs, and owned objects as private credential containers are all natural extensions.

HISTORY Mysten Labs: Evan Cheng, Sam Blackshear, George Danezis, Adeniyi Abiodun, Kostas Chalkias (2021). *Sui achieved 297,000 TPS on a test network in 2023, making it one of the fastest L1s. The Fastpay protocol (Danezis et al.) that underpins Sui's simple transaction path can finalize owned-object transactions in ~480ms without full consensus — this is why the thesis targets Sui for real-time payments.*

LIMITATIONS

- Zero native privacy: all Sui objects are publicly readable by any full node, all transaction details (sender, function calls, arguments, created objects) are permanently visible on-chain — there is no equivalent of Zcash's shielded pool or Monero's mandatory encryption
- Shared objects (e.g., DEX pools, the thesis's nullifier set) require Narwhal/Bullshark consensus (~2-3s), negating Sui's fast path advantage — the thesis's nullifier set is a shared object, so settlement latency is consensus-bound
- Move's type system prevents dynamic dispatch and complex generics, making ZK verifier circuits harder to implement natively — Groth16 verification in Move requires a dedicated native function or precompiled module (Sui added groth16::verify_groth16_proof in framework v0.28)

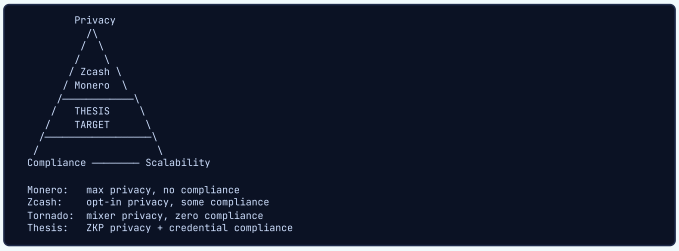
EXERCISES

1. **design** — Sui's object model distinguishes between owned objects (single-owner, fast path) and shared objects (multi-owner, consensus path). For your thesis privacy layer, which components should be owned objects and which should be shared? Justify each choice.
2. **analysis** — Compare Sui's object model with Ethereum's account model for implementing a private payment system. What are the advantages and disadvantages of each for privacy?

Private Payment Requirements

INTUITIVE

The privacy trilemma: like building a house that is cheap, fast, AND beautiful, you cannot have full privacy, full compliance, AND full scalability at once. Every system makes tradeoffs. The thesis seeks the sweet spot where enough privacy, enough compliance, and enough performance coexist.



KEY POINTS

- Sender privacy: hide who is paying
- Receiver privacy: hide who is being paid
- Amount hiding: conceal transaction values via Pedersen commitments
- Compliance hooks: viewing keys let authorized auditors decrypt selectively
- Low-value threshold: lighter ZK verification for small payments, heavier proofs for large ones

WHY IT MATTERS

This concept frames the exact design space of the thesis. The goal is to achieve sender/receiver/amount privacy on Sui while retaining compliance via anonymous credentials (KYC without identity disclosure) and viewing keys for auditors.

THESIS ANGLE

In your thesis, you define three payment tiers on Sui: (1) Micro-payments (<EUR10): TEE verification only, no on-chain proof, sub-second settlement. (2) Low-value (EUR10-EUR1000): standard ZK proof with simplified credential check (just 'valid credential exists'). (3) High-value (>EUR1000): full credential disclosure (KYC_level >= 3), detailed compliance memo, on-chain Groth16 verification. Each tier balances privacy, compliance, and performance differently.

TECHNICAL

A private payment system must satisfy four privacy properties: sender privacy (who paid), receiver privacy (who was paid), amount privacy (how much), and transaction graph privacy (unlinkability of payment flows). Formally, for each property, no probabilistic polynomial-time adversary should distinguish between any two valid scenarios with non-negligible advantage.

SENDER PRIVACY

The adversary cannot identify the sender from the transaction with probability better than random guessing within the anonymity set:

$$\Pr[\text{identify sender} \mid \text{tx}] \leq \frac{1}{|\text{anonymity set}|}$$

This can be achieved via ring signatures (Monero, $k = 16$), mixing (Tornado Cash, $k = |\text{pool}|$), or zero-knowledge proofs of Merkle membership (Zcash, $k = |\text{commitment tree}|$).

RECEIVER PRIVACY

The receiver should not be identifiable from the transaction. Stealth addresses generate a fresh one-time address per payment. In Zcash, the receiver address is encrypted inside the note ciphertext. The formal requirement: given tx and any set of candidate receivers $\{R_1, \ldots, R_m\}$, no adversary can determine the true receiver with advantage greater than $1/m$.

AMOUNT PRIVACY VIA PEDERSEN COMMITMENTS

Transaction amounts are hidden using Pedersen commitments:

$$C = vG + rH$$

where v is the value, r is randomness, and G, H are independent generators with unknown discrete log relation. The commitment is computationally hiding (given C, v is indistinguishable from random under the DDH assumption) and perfectly binding (no two (v, r) pairs produce the same C, given the discrete log hardness of H w.r.t. G). Range proofs (Bulletproofs) ensure $v \geq 0$ without revealing v.

GRAPH PRIVACY & UNLINKABILITY

Deposits and withdrawals must be unlinkable:

$$\Pr[\text{link}(\text{deposit}_i, \text{withdraw}_j) \mid \text{chain}] \leq \frac{1}{|\text{anonymity set}|}$$

This requires: (1) no timing correlation (random delay between deposit/withdraw), (2) no amount correlation (fixed denominations), (3) no address reuse (fresh addresses per withdrawal), (4) no metadata leakage (use relayers to avoid gas payment linking).

COMPLIANCE HOOKS

Viewing keys enable selective auditability: an auditor with key vk can decrypt transaction memos:

$$\text{Dec}(\text{vk}, \text{encrypted\_memo}) \to (\text{sender}, \text{receiver}, v)$$

This preserves privacy for regular users while enabling regulatory compliance. The encryption must be IND-CCA2 secure so that non-auditors learn nothing. For volume thresholds, a user can prove compliance without revealing individual transactions:

$$\pi_{\text{volume}} : \text{"my total monthly volume"} \sum v_i < X$$

using a ZK proof over committed values.

LOW-VALUE OPTIMIZATION

For transactions below a compliance threshold (e.g., $v < 1000$ EUR, aligned with EU simplified due diligence), lighter verification reduces latency: smaller range proofs (e.g., 32-bit instead of 64-bit), TEE-based verification instead of on-chain ZKP, or reduced anonymity set requirements. This mirrors real-world AML rules where low-value transactions face less scrutiny.

HISTORY FATF (Financial Action Task Force) / EU regulatory framework (2019). *The EUR1,000 travel rule threshold was chosen somewhat arbitrarily — the original FATF recommendation was $3,000 in 1996 (for traditional wire transfers), lowered to $1,000 for crypto in 2019 due to "higher risk." The thesis's tiered approach directly maps to these regulatory thresholds.*

LIMITATIONS

- The privacy-compliance tradeoff is fundamentally zero-sum at the extremes: perfect privacy makes compliance impossible, and perfect compliance requires full transparency — the thesis must find a Pareto-optimal point, not eliminate the tradeoff
- Viewing keys create a single point of trust: if the auditor's viewing key is compromised, all transactions encrypted to that key become transparent — key rotation and multi-party threshold decryption partially mitigate this but add complexity
- Low-value threshold creates a gaming surface: adversaries can split a large payment into many small ones below the threshold (structuring/smurfing), requiring additional heuristic detection that partially undermines privacy guarantees

EXERCISES

1. **design** — Design an anti-structuring mechanism for your thesis payment system that detects attempts to split a EUR5,000 payment into 50 x EUR99 payments to stay below the threshold. How can you do this without breaking privacy for legitimate small payments?
2. **conceptual** — Explain the difference between "privacy from the public" and "privacy from the state." Which does your thesis provide, and why is this distinction important for regulatory acceptance?

ZKP+TEE Integration + Thesis Architecture

Why Combine ZKP + TEE

INTUITIVE

Belt AND suspenders. ZKP is the belt: a mathematical guarantee that cannot break regardless of computational power. TEE is the suspenders: a hardware guarantee that is fast but could theoretically fail if the chip is compromised. Together, even if one fails, your security holds.

KEY POINTS

- ZKP = trustless but slow: Groth16 proving can take seconds to minutes
- TEE = fast but trust-dependent: computation in ms, but assumes hardware integrity
- Hybrid reduces trust assumptions while maintaining performance
- If TEE is compromised, ZKP verification still catches invalid proofs
- Fallback to pure ZKP if TEE attestation fails (graceful degradation)

WHY IT MATTERS

The thesis uses this hybrid model as its core architecture. TEE handles the expensive witness generation (credential decryption, balance checks) while ZKP provides the on-chain verification that requires no trust in hardware.

THESIS ANGLE

In your thesis, this is the core insight. Pure ZKP on Sui: ~500ms proof generation + 390ms finality = ~1s per payment. Too slow for coffee. Pure TEE: ~1ms verification, but trust Intel. Your hybrid: TEE verifies credentials at payment terminals (fast path), ZKP settles batches on-chain (trust path). If Intel SGX is compromised tomorrow, all payments verified on-chain remain secure — the TEE was only a performance optimization, not a security assumption.

TECHNICAL

A hybrid system Π_{hybrid} is secure under the disjunction of trust assumptions: $\text{Assumption}_{\text{ZKP}} \vee \text{Assumption}_{\text{TEE}}$. If the TEE hardware is intact, the system achieves full privacy with low latency. If the TEE is compromised, ZKP verification still guarantees computational soundness. The system is secure as long as at least one assumption holds.

PURE ZKP COST MODEL

ZKP proving for a circuit C with N constraints costs:

$$T_{\text{prove}} = O(N \log N)$$

dominated by multi-scalar multiplication (MSM) and number-theoretic transform (NTT) operations over the scalar field. For a credential verification circuit with ~100K constraints (BBS+ signature verification + selective disclosure), this translates to ~500ms on desktop, ~2s on mobile. Verification is fast: $T_{\text{verify}} = O(1)$ for Groth16 (3 pairings), but the proving bottleneck limits real-time applications.

PURE TEE COST MODEL

TEE execution is near-native speed:

$$T_{\text{TEE}} = O(|\text{computation}|) \cdot c_{\text{overhead}}$$

where $c_{\text{overhead}} \approx 1.1\text{-}1.5\times$ for SGX enclave context switches and memory encryption. A credential verification inside a TEE takes ~1-5ms. However, TEE security relies on hardware integrity (Intel/AMD attestation), which has been broken by side-channel attacks (Foreshadow, Plundervolt, AEPIC Leak). The trust model is: Intel is honest AND the hardware is not physically tampered with.

HYBRID ARCHITECTURE

The hybrid approach partitions computation: TEE handles expensive pre-computation (witness generation), ZKP handles the final proof:

$$\text{TEE} : z = f(w) \quad (\text{fast}, \text{private})$$

$$\text{ZKP} : \text{Prove}(g(z) = \text{output}) \quad (\text{trustless}, \text{public})$$

The ZKP circuit is over g (a simpler function operating on intermediate values z), not over $f \circ g$ (the full computation). This reduces circuit size and proving time by 10-100x while maintaining trustless verification.

FORMAL SECURITY: UC FRAMEWORK

In the Universal Composability (UC) framework, define ideal functionality $\mathcal{F}_{\text{hybrid}}$ that realizes the desired privacy and correctness properties. The real-world protocol Π_{hybrid} UC-realizes $\mathcal{F}_{\text{hybrid}}$ if:

$$\text{REAL}_{\Pi_{\text{hybrid}},\mathcal{A}} \approx_c \text{IDEAL}_{\mathcal{F}_{\text{hybrid}},\mathcal{S}}$$

for all environments $\mathcal{Z}$, under the assumption that EITHER the TEE functionality $\mathcal{F}_{\text{TEE}}$ is honestly implemented OR the ZKP soundness holds. This gives graceful degradation: TEE compromise does not break soundness, and ZKP assumption failure does not break confidentiality (TEE still protects).

COMPLEMENTARY GUARANTEES

TEE provides confidentiality (private data never leaves the enclave in plaintext) while ZKP provides verifiability (anyone can check the proof without trusting the prover). Neither alone suffices: ZKP alone is too slow for real-time credential checks, TEE alone requires trusting hardware vendors. The hybrid provides both properties simultaneously, with each compensating for the weakness of the other.

HISTORY Multiple teams: Secret Network (Enigma MPC, 2017), Oasis Labs (Dawn Song, UC Berkeley, 2018), Phala Network (2019) (2017). *Secret Network's SGX dependency became a liability when Intel disclosed the "AEPIC Leak" vulnerability (CVE-2022-21233, August 2022), which could read enclave memory from unprivileged code. Secret Network had to perform an emergency key rotation affecting all encrypted state. This real-world incident validates the thesis's "belt AND suspenders" approach: never rely on TEE alone.*

LIMITATIONS

- The hybrid model inherits complexity from both worlds: the system must implement and maintain both a ZK circuit (Groth16 with ~100K constraints) and a TEE enclave (SGX SDK, attestation infrastructure, key management), doubling the attack surface and development effort
- TEE attestation verification on-chain adds gas cost and latency: verifying an Intel DCAP attestation quote (~4KB) on Sui requires parsing and validating a certificate chain, which is expensive in Move (~500K gas units per attestation check)
- The hybrid model creates a two-tier security guarantee that may confuse users and auditors: TEE-verified transactions have weaker guarantees than ZKP-settled ones, but both appear as "confirmed" in the wallet — the UX must clearly communicate the difference

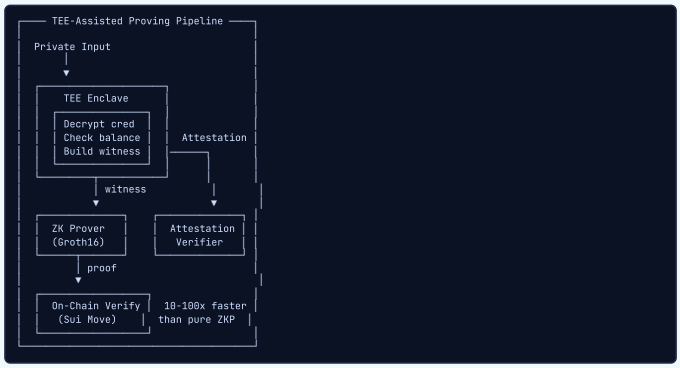
EXERCISES

1. **analysis** — Secret Network relies entirely on SGX enclaves for privacy. When the AEPIC Leak (CVE-2022-21233) was disclosed in 2022, what were the consequences? How does your thesis's hybrid model avoid this failure mode?
2. **design** — Design the fallback mechanism for your thesis system when TEE attestation fails. What should happen at the wallet level, the merchant terminal, and the on-chain contract?

Hybrid Architecture Patterns

INTUITIVE

A courtroom with a secure deliberation room. The jury (TEE) deliberates privately and quickly, then presents a verdict (proof). The judge (ZKP verifier) can independently verify the verdict is correct without seeing the private deliberation.



KEY POINTS

- TEE performs witness generation: decrypt credentials, verify balances, compute intermediate values
- ZKP proves the computation was correct without revealing private inputs
- Attestation provides evidence the TEE ran genuine code on genuine hardware
- Latency reduction of 10-100x compared to pure ZKP proving
- If TEE is compromised, system degrades to pure ZKP (slower but still secure)

WHY IT MATTERS

This pipeline is the thesis proving architecture. User private data enters the TEE, gets processed into a witness, and the ZK proof is generated and verified on Sui. The TEE never exposes private data; the ZKP ensures correctness publicly.

THESIS ANGLE

For your system, this implements Pattern 3 (TEE real-time + ZKP settlement). The merchant's TEE enclave verifies 100 credential-gated payments in real-time (~100ms total). Every 10 minutes, the TEE generates a batch Groth16 proof: 'I honestly verified 100 payments, here are the note commitments and nullifiers.' This single proof settles all 100 payments on Sui for the gas cost of one. The TEE attestation is verified on-chain once per session, not per payment.

TECHNICAL

Three canonical patterns for ZKP+TEE integration form a pipeline: $\text{Input} \xrightarrow{\text{TEE}} \text{Witness} \xrightarrow{\text{ZKP}} \text{Proof} \xrightarrow{\text{Chain}} \text{Verified}$. Each pattern trades off trust assumptions against performance, with different suitability for real-time vs. batch workloads.

PATTERN 1: TEE-ASSISTED WITNESS GENERATION

The TEE receives encrypted private inputs, decrypts them inside the enclave, computes the ZKP witness w, and seals the intermediate state. The ZKP prover (which may run outside the TEE) uses the witness to generate a proof π such that $C(w) = 1$. The key benefit: private inputs never leave the TEE in plaintext, yet the resulting proof is verifiable by anyone without trusting the TEE. The TEE attestation certifies that the witness was honestly computed from the correct inputs.

PATTERN 2: TEE-ACCELERATED PROVING

The TEE runs the ZKP prover internally, taking advantage of hardware-accelerated operations (AES-NI for field arithmetic, AVX-512 for MSM). The enclave produces both the proof π and an attestation σ_{att} certifying honest generation. This is less trustless than Pattern 1 because the verifier must check the attestation (trusting Intel SGX remote attestation), but it is much faster: the entire proving pipeline runs in protected memory. Verification can still be done on-chain without TEE trust if the ZKP itself is verified.

PATTERN 3: TEE REAL-TIME + ZKP SETTLEMENT

For latency-sensitive operations (credential verification at point of service), the TEE performs real-time verification (~1-5ms). Periodically, a batch ZKP is generated proving that the TEE honestly performed N verifications:

$$\pi_{\text{batch}} : \text{"TEE correctly verified } \text{tx}_1, \ldots, \text{tx}_N\text{"}$$

This batch proof settles on-chain (e.g., every 100 verifications or every 10 minutes). The ZKP proves the entire batch was honest, amortizing the proving cost over N transactions. Cost per transaction: T_{prove}/N.

COMMITMENT SCHEME BRIDGE

The TEE commits to intermediate values using a commitment scheme:

$$\text{com} = \text{Commit}(z, r)$$

where z is the intermediate computation result and r is randomness. The ZKP then proves properties about z by opening the commitment selectively. This bridges the TEE (which knows z in plaintext) and the ZKP (which proves statements about z without revealing it). The commitment must be binding (TEE cannot change z after committing) and hiding (the commitment reveals nothing about z).

HISTORY TEE-assisted proving was formalized by Tramer and Boneh (Stanford) (2019). *Dan Boneh's Stanford Applied Cryptography lab has produced an extraordinary number of privacy protocol researchers: both Zcash (Eli Ben-Sasson's PhD students collaborated with Boneh's group) and many TEE+crypto hybrid papers come from his lab. Boneh also co-designed the BLS signature scheme (Boneh-Lynn-Shacham, 2001) that underlies BLS12-381, the curve used in both Zcash Sapling and the thesis.*

LIMITATIONS

- Batch proving introduces latency: if the TEE batches 100 payments into one proof every 10 minutes, individual payments have up to 10 minutes of settlement uncertainty — during this window, the TEE is the sole trust anchor, and a TEE compromise could allow double-spending within the batch
- Witness generation inside the TEE requires the enclave to hold sensitive data (decrypted credentials, plaintext amounts) — SGX enclave memory is limited to ~94MB (EPC size), constraining the number of concurrent payment verifications
- The pattern requires tight coupling between the TEE enclave code and the ZK circuit: any change to the Groth16 circuit requires a matching update to the TEE witness generation code, creating a fragile deployment dependency

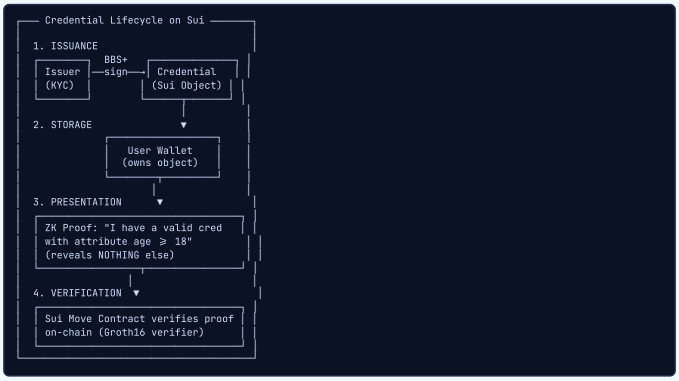
EXERCISES

1. **design** — The thesis uses a "batch proving" pattern where the TEE accumulates 100 payments and generates one Groth16 proof. Design the data structure the TEE maintains during a batch window. What happens if the TEE crashes mid-batch?
2. **comparison** — Compare three hybrid patterns: (1) TEE proves, ZKP verifies; (2) ZKP proves, TEE accelerates witness generation; (3) TEE real-time + ZKP batch settlement. Which does the thesis use and why?

Anonymous Credentials on Sui

INTUITIVE

A digital ID card system built on Sui objects. Each credential is a Sui object, issuance is a transaction, and verification uses ZKP so the credential object itself never reveals its contents. Like showing a bouncer a hologram that proves you are over 21 without showing your birthdate or name.



KEY POINTS

- Credential as Sui object: leverages Move resource safety (cannot duplicate or destroy)
- BBS+ signatures enable selective disclosure of individual attributes
- On-chain verification via Groth16: ~200 byte proof, cheap to verify in Move
- Privacy-preserving presentation: prove properties without revealing credential contents
- Issuer cannot track where or when credentials are used (unlinkability)

WHY IT MATTERS

This is the credential layer of the thesis. BBS+ issuance creates Sui objects that users own. Selective disclosure via ZKP allows proving KYC compliance without revealing identity, which gates access to the private payment layer.

THESIS ANGLE

In your thesis, this IS your implementation. A Sui Move module defines the credential schema as a Move struct. The issuer (e.g., SuiNS identity provider) issues a BBS+ signature on (did, name, age, country, kyc_level). The credential lives as an encrypted Sui object in the user's wallet. At payment time: user generates a Groth16 proof wrapping the BBS+ selective disclosure, submits to the Sui verifier contract, contract checks proof against issuer's on-chain public key, and payment note is created in the shielded pool.

TECHNICAL

The credential system on Sui is defined as a tuple of protocols: $\Pi_{\text{SuiCred}} = (\text{Setup}, \text{Issue}, \text{Present}, \text{Verify}, \text{Revoke})$. Credentials are BBS+ signed attribute vectors stored as encrypted Sui objects. Presentation uses zero-knowledge proofs for selective disclosure, verified on-chain by Move contracts.

SETUP & SCHEMA

A Move module is deployed defining the credential schema: attribute types $(t_1, \ldots, t_L)$, issuer public key ipk, and verification parameters. The issuer generates a BBS+ key pair (isk, ipk) where $\text{ipk} = (w, h_0, h_1, \ldots, h_L)$ with $w = g_2^{\text{isk}}$ and h_i are random generators in $\mathbb{G}_1$. The setup also initializes on-chain data structures: the revocation accumulator and the credential registry.

ISSUANCE (BBS+ SIGNATURE)

The issuer creates a BBS+ signature on the attribute vector $\mathbf{a} = (a_1, \ldots, a_L)$:

$$\sigma = (A, e, s) \quad \text{where} \quad A = \left(g_1 \cdot h_0^{s} \cdot \prod_{i=1}^{L} h_i^{a_i} \right)^{1/(\text{isk}+e)}$$

The signature is verified by checking:

$$e(A, w \cdot g_2^{e}) = e\left(g_1 \cdot h_0^{s} \cdot \prod_{i=1}^{L} h_i^{a_i},\ g_2 \right)$$

The signed credential $(\sigma, \mathbf{a})$ is encrypted and stored as a Sui object owned by the user.

CREDENTIAL STORAGE AS SUI OBJECT

The credential is stored as an encrypted Sui object: $\text{EncObj}(\sigma, \mathbf{a})$ with the user as owner. Using an owned object ensures single-writer semantics (no consensus needed for credential operations) and Move resource safety (the credential cannot be duplicated or implicitly destroyed). The encryption key is derived from the user's wallet key, ensuring only the user can decrypt the credential contents.

SELECTIVE DISCLOSURE (BBS+ PROOF)

To present a credential with selective disclosure set $D \subseteq [L]$ (indices of revealed attributes), the user derives a BBS+ proof of knowledge:

$$\pi_{\text{BBS}} = \text{SPK}\{(\sigma, \{a_i\}_{i \notin D}) : \text{Verify}(\text{ipk}, \sigma, \mathbf{a}) = 1\}$$

The proof reveals $\{a_i\}_{i \in D}$ and hides $\{a_i\}_{i \notin D}$. Each presentation uses fresh randomization of the signature σ, ensuring unlinkability: two presentations of the same credential produce statistically independent proofs.

GROTH16 WRAPPING FOR ON-CHAIN EFFICIENCY

Direct BBS+ proof verification requires pairing operations that are expensive in Move. Instead, the BBS+ proof is wrapped in a Groth16 proof: the Groth16 circuit verifies the BBS+ proof internally, and the on-chain Move contract only needs to verify a single Groth16 proof (3 pairings, ~200 bytes). The wrapping circuit for a 10-attribute credential has ~100K R1CS constraints. On-chain verification:

$$\text{Verify}_{\text{Groth16}}(\text{vk}, \pi, [\text{ipk}, \{a_i\}_{i \in D}]) = 1$$

REVOCATION & SCOPED PSEUDONYMS

Revocation uses a cryptographic accumulator stored as a shared Sui object. The issuer updates the accumulator to exclude revoked credentials. During presentation, the user proves non-membership of their credential in the revocation set. For scoped linkability, the user derives a pseudonym:

$$\text{nym} = \text{PRF}(\text{sk}, \text{scope})$$

Presentations within the same scope (e.g., same service provider) produce the same pseudonym (enabling rate-limiting or Sybil resistance), while presentations across different scopes produce independent pseudonyms (unlinkable).

HISTORY Concept extends Camenisch-Lysyanskaya (2001) to Sui; BBS+ by Au, Susilo, and Mu (2006) (2025). *Jan Camenisch (IBM Research Zurich) and Anna Lysyanskaya (Brown University) met at CRYPTO 1999. Their CL signature scheme has been cited over 3,000 times and forms the basis of every major anonymous credential system in production today, including Microsoft U-Prove, IBM Idemix, and Hyperledger Indy.*

LIMITATIONS

- Credential as Sui object creates a metadata leakage risk: even if the credential contents are encrypted, the object's creation timestamp, issuer transaction, and ownership transfer history are publicly visible — an observer can infer "this user received a KYC credential from issuer X on date Y"
- BBS+ selective disclosure inside a Groth16 circuit is expensive: proving a BBS+ signature over 5 attributes in Groth16 requires ~40K R1CS constraints, which translates to ~2-4 second proving time on a mobile device — this is the main bottleneck the TEE acceleration addresses
- On-chain credential revocation via accumulators (e.g., RSA accumulators or Merkle-based exclusion proofs) adds O(n) witness update cost for n credentials — if the system has 1M credentials, each revocation requires updating 1M witnesses or using a centralized update server

EXERCISES

1. **design** — Design the Move struct for a privacy-preserving credential object on Sui. What fields should it contain, and which should be encrypted vs. plaintext? How does the object handle selective disclosure?
2. **conceptual** — Explain why credential unlinkability is crucial for the thesis system. What would happen if an observer could link two credential presentations from the same user at different merchants?
3. **analysis** — Compare two approaches to on-chain credential verification: (1) verify BBS+ signature directly in Move, (2) wrap BBS+ verification inside a Groth16 proof and verify the Groth16 proof in Move. What are the tradeoffs?

Private Payment Flow

INTUITIVE

A vending machine with a privacy screen. You prove you have enough money (ZKP), insert the right amount (encrypted), and the machine gives you the item. Nobody in line sees how much you paid or your balance. But a compliance officer with a special key could audit the transaction if required.

KEY POINTS

- Credential-gated access: must prove valid KYC credential via ZKP before transacting
- Amount hiding via Pedersen commitments: C = g^v * h^r (additively homomorphic)
- Nullifiers prevent double-spending without revealing which commitment was consumed
- Viewing keys allow authorized auditors to decrypt specific transactions
- Low-value threshold: transactions below a limit use lighter proofs for speed

WHY IT MATTERS

This is the payment layer of the thesis. It combines the credential check (Block 2, concept 3) with the commitment/nullifier model (from Zcash/Tornado Cash in Block 1) and adds compliance via viewing keys. This flow runs on Sui with TEE-accelerated proving.

THESIS ANGLE

For your system, the complete flow on Sui: (1) User's wallet reads their BBS+ credential object, (2) generates ZK proof: 'I have valid credential + sufficient balance + correct nullifier + amount in range,' (3) submits transaction with proof + commitments + nullifier + encrypted auditor memo, (4) Sui Move contract verifies Groth16 proof in ~50ms, (5) updates note Merkle tree + nullifier set as shared objects, (6) receiver detects incoming note via encrypted memo. Total user-facing latency: ~1s (proof gen) + ~390ms (Sui finality).

TECHNICAL

A private payment $\text{Pay}(\text{sender}, \text{receiver}, v)$ is a credential-gated transaction that hides the sender identity, receiver identity, and amount while proving validity on-chain. The payment combines a credential proof (KYC compliance), a value commitment (amount hiding), a nullifier (double-spend prevention), and an encrypted memo (auditability).

STEP 1: CREDENTIAL VERIFICATION

The sender proves possession of a valid, non-revoked credential satisfying access predicates. The ZK proof π_1 states:

$$\pi_1 : \exists\, (\sigma, \mathbf{a}) : \text{BBS.Verify}(\text{ipk}, \sigma, \mathbf{a}) = 1 \wedge a_{\text{age}} \geq 18 \wedge a_{\text{country}} \in \text{EU} \wedge \text{NotRevoked}(\sigma, \text{acc})$$

The proof reveals no attributes beyond the truth of the predicates. The credential is not consumed (can be reused for future payments).

STEP 2: AMOUNT COMMITMENT

The sender creates a Pedersen commitment to the payment value:

$$C_{\text{send}} = v \cdot G + r_s \cdot H$$

The receiver creates a matching commitment:

$$C_{\text{recv}} = v \cdot G + r_r \cdot H$$

Both use the same value v but independent blinding factors r_s, r_r. The difference $C_{\text{send}} - C_{\text{recv}} = (r_s - r_r) \cdot H$ commits to zero value, which is proven in the balance proof.

STEP 3: BALANCE & RANGE PROOF

The ZK proof π_2 establishes:

$$\pi_2 : v_{\text{send}} = v_{\text{recv}} \wedge 0 \leq v < 2^{64}$$

The equality proof shows no value is created or destroyed (no inflation). The range proof (Bulletproofs or Groth16-embedded) ensures no negative values that could exploit the modular arithmetic of the commitment scheme. For efficiency, the range proof uses a binary decomposition: prove each bit $v = \sum_{i=0}^{63} b_i \cdot 2^i$ with $b_i \in \{0, 1\}$.

STEP 4: NULLIFIER (DOUBLE-SPEND PREVENTION)

The sender computes a nullifier for the spent note:

$$\text{nf} = \text{PRF}(\text{sk}_{\text{sender}}, \text{note\_id})$$

The nullifier is published on-chain. The contract maintains a nullifier set (sparse Merkle tree) and rejects transactions with duplicate nullifiers. The PRF ensures the nullifier is deterministic (same note always produces same nullifier) but pseudorandom (nullifier reveals neither the sender key nor the note ID).

STEP 5: VIEWING KEY & ENCRYPTED MEMO

For compliance, the sender encrypts transaction details under the auditor public key:

$$\text{memo} = \text{Enc}(\text{vk}_{\text{auditor}}, (\text{sender\_id}, \text{receiver\_id}, v, \text{timestamp}))$$

The encryption uses a hybrid scheme (ECIES or similar): $\text{Enc}(\text{pk}, m) = (rG, \text{AES}(H(r \cdot \text{pk}), m))$. Only the auditor holding the corresponding private key can decrypt. The memo is stored on-chain alongside the transaction.

COMBINED PROOF & LOW-VALUE FAST PATH

For gas efficiency, π_1 (credential) and π_2 (payment) are bundled into a single Groth16 proof π with combined public inputs. The on-chain Move contract verifies one proof instead of two, saving ~50% gas. For low-value payments ($v < \text{threshold}$), a fast path uses TEE verification (~5ms) instead of on-chain ZKP verification. The TEE attests to the validity and produces a signed receipt. Periodic batch ZKP proofs settle these fast-path transactions on-chain.

ON-CHAIN SETTLEMENT

The Sui Move contract executes: (1) verify combined proof π against verification key, (2) check nullifier is not in the nullifier set, (3) insert new note commitment into the Merkle tree (shared object), (4) insert nullifier into the nullifier set (shared object), (5) store encrypted memo, (6) emit payment event. The note commitment tree and nullifier set are shared Sui objects requiring Mysticeti consensus (~390ms finality).

HISTORY Builds on Zerocash (Ben-Sasson et al., 2014) + Zcash Sapling (Hopwood et al., 2018) + credential-gated access (thesis contribution) (2025). *The Zcash Sapling specification (ZIP 32) defines hierarchical deterministic key derivation for shielded addresses. The thesis adapts this pattern for Sui: a single master seed derives both the credential encryption key and the payment note keys, so one backup phrase recovers all privacy state.*

LIMITATIONS

- The combined credential + payment proof circuit has ~80K-120K R1CS constraints (BBS+ verification + Pedersen commitment + Merkle path + range proof), requiring ~2-4 seconds of client-side proving — this is the primary motivation for TEE acceleration in the fast path
- Receiver note detection requires scanning all new commitments: without a centralized notification server, the receiver must attempt decryption of every new encrypted memo on-chain — O(n) in the number of transactions per epoch, which does not scale beyond ~10K TX/epoch without optimization
- The encrypted compliance memo creates a permanent audit trail: even if the auditor's viewing key is well-protected today, a future key compromise (or quantum attack on the memo encryption) could retroactively reveal all historical transaction details

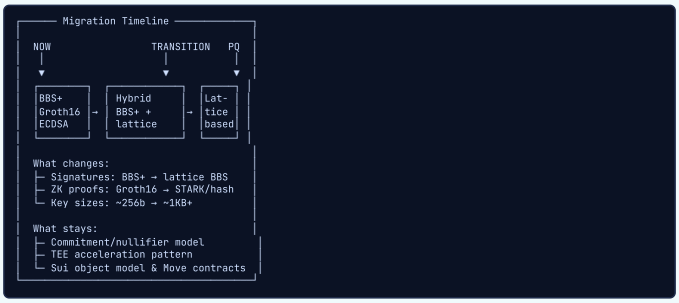
EXERCISES

1. **design** — Design the Groth16 public inputs and private inputs for the thesis payment circuit. What does the verifier (Sui Move contract) see, and what stays hidden?
2. **analysis** — The receiver needs to detect incoming payments without a notification server. Compare three approaches: (1) scan all commitments, (2) use a tag derived from a shared secret, (3) use a TEE scanning service. Evaluate privacy and performance for each.

Post-Quantum Considerations

INTUITIVE

Preparing for a flood before it rains. Quantum computers do not exist at scale yet, but harvest-now-decrypt-later attacks mean data encrypted today could be broken tomorrow. Credentials issued now must still be unforgeable in 10 years.



KEY POINTS

- Harvest-now-decrypt-later: adversaries store encrypted data to break later with quantum computers
- Lattice-based alternatives (e.g., Lether) offer post-quantum credential signatures
- Hash-based signatures (SPHINCS+) provide quantum-safe but larger proofs
- NIST PQC standardization (ML-KEM, ML-DSA) provides migration targets
- Thesis architecture is designed for modular crypto swap: change primitives, keep structure

WHY IT MATTERS

The thesis addresses post-quantum readiness by designing a modular architecture where cryptographic primitives (BBS+, Groth16) can be swapped for lattice-based alternatives without changing the overall system design. The Lether paper (Day 3 reading) provides a concrete post-quantum payment scheme.

THESIS ANGLE

In your thesis, you include a migration section. BBS+ signatures on BLS12-381 will be broken by quantum computers (Shor's algorithm solves the q-SDH problem). Lether (ePrint 2026/076) shows lattice-based private payments are practical (~68KB proof, sub-second). Your migration path: (1) start issuing dual credentials (BBS+ AND lattice-based) today, (2) verifier contracts accept both proof types, (3) phase out BBS+ when quantum threat materializes. TEE layer is unaffected by quantum — hardware isolation does not depend on computational hardness.

TECHNICAL

Post-quantum security requires resilience against adversaries with bounded quantum polynomial-time (BQP) computation. Quantum algorithms (Shor, Grover) break or weaken many classical cryptographic assumptions, necessitating migration to lattice-based, hash-based, or code-based alternatives for long-lived credentials and payment systems.

SHOR'S ALGORITHM IMPACT

Shor's algorithm factors integers and computes discrete logarithms in polynomial time:

$$\mathrm{Factor}(n) \in O((\log n)^2 (\log\log n)(\log\log\log n))$$

on a quantum computer (simplified: $O((\log n)^3)$ with quantum Fourier transform). This breaks: RSA (integer factoring), ECDSA/EdDSA (elliptic curve discrete log), BLS signatures (pairing-based discrete log), BBS+ signatures (relies on q-SDH in pairing groups), and Groth16 (relies on pairings over BLS12-381). Essentially, all elliptic curve and pairing-based cryptography used in the thesis is vulnerable.

GROVER'S ALGORITHM IMPACT

Grover's algorithm provides a quadratic speedup for unstructured search:

$$\mathrm{Search}(f, N) \in O(\sqrt{N}) = O(2^{n/2})$$

This halves the effective security of symmetric primitives: AES-256 provides 128-bit security against quantum, AES-128 provides only 64-bit. Hash functions are similarly affected: SHA-256 provides 128-bit collision resistance against quantum (vs. 128-bit classically via birthday attack). Grover does not break symmetric crypto but requires doubling key sizes.

LATTICE-BASED ALTERNATIVES

The Learning With Errors (LWE) problem is the foundation of most post-quantum schemes:

$$\mathrm{LWE}_{n,q,\chi} : \text{Given } (\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}), \text{find } \mathbf{s}$$

where $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \in \mathbb{Z}_q^n$, and $\mathbf{e} \leftarrow \chi$ is a small error vector. The Module-LWE variant (used in Kyber/ML-KEM) operates over polynomial rings $\mathbb{Z}_q[X]/(X^n + 1)$ for efficiency. The Short Integer Solution (SIS) problem provides hash function security:

$$\mathrm{SIS}_{n,q,\beta} : \text{Given } \mathbf{A}, \text{ find } \mathbf{x} \neq 0 \text{ s.t. } \mathbf{A}\mathbf{x} = 0 \text{ and } \|\mathbf{x}\| \leq \beta$$

Best known quantum attacks on LWE/SIS remain exponential in n.

POST-QUANTUM CREDENTIALS

Replacing BBS+ with lattice-based alternatives is challenging because efficient selective disclosure requires algebraic structure. Current approaches: (1) Dilithium-based credentials (hash-then-sign): efficient signing and verification but no native selective disclosure — requires combining with general-purpose ZK proofs, (2) Lattice-based anonymous credentials using relaxed proofs of knowledge over Module-LWE commitments: larger but provide native selective disclosure, (3) Hash-based signatures (SPHINCS+/SLH-DSA): quantum-safe and well-understood, but signatures are ~8-40KB and no algebraic structure for selective disclosure.

LETHER PROTOCOL

Lether is a lattice-based Anonymous Zether protocol providing post-quantum private payments. It uses Module-LWE encryption for confidential amounts and lattice-based zero-knowledge proofs for balance and range verification. Key metrics: ~68KB per transaction (vs. ~1.5KB for pairing-based), sub-second proving time, security under Module-LWE assumption. The proof system uses "relaxed" extractability (the extractor recovers a witness with small norm, not exact), which is sufficient for payment security.

MIGRATION STRATEGY

The recommended migration path is: (1) Hybrid classical+PQ during transition: run both BBS+ and a lattice-based scheme in parallel, accept either proof. (2) Harvest-now-decrypt-later defense: credentials with long lifetimes (identity documents, 10+ years) should use PQ signatures now. (3) NIST PQC standardized algorithms: ML-KEM (Kyber) for key encapsulation, ML-DSA (Dilithium) for digital signatures, SLH-DSA (SPHINCS+) for stateless hash-based signatures. (4) The thesis architecture is modular: cryptographic primitives (signature scheme, commitment scheme, hash function) can be swapped without changing the protocol structure.

HISTORY Peter Shor (Bell Labs/MIT) for quantum threat; NIST PQC competition (2016-2024) for defenses (1994). *In 2023, IBM unveiled the 1,121-qubit Condor processor, but error correction means a cryptographically relevant quantum computer (needing ~4,000 logical qubits for RSA-2048) likely requires millions of physical qubits. Estimates range from 2035 to 2050+. However, the "harvest now, decrypt later" threat means encrypted data captured today must resist quantum attacks if it has a long confidentiality lifetime.*

LIMITATIONS

- Lattice-based alternatives have dramatically larger proof/key sizes: ML-DSA (Dilithium) signatures are ~2.4KB vs. BBS+'s ~112 bytes; lattice-based ZK proofs (e.g., from Lether) are ~68KB vs. Groth16's ~200 bytes — this impacts on-chain storage costs and verification gas on Sui
- No standardized post-quantum anonymous credential scheme exists yet: Lether (2026) is the closest, but it has not undergone the multi-year scrutiny that BBS+ and CL signatures have received — deploying it in production carries cryptanalysis risk
- The TEE layer is quantum-resistant (hardware isolation does not depend on computational hardness), but the TEE attestation protocol uses ECDSA signatures — a quantum attacker could forge attestation quotes, undermining the TEE trust chain

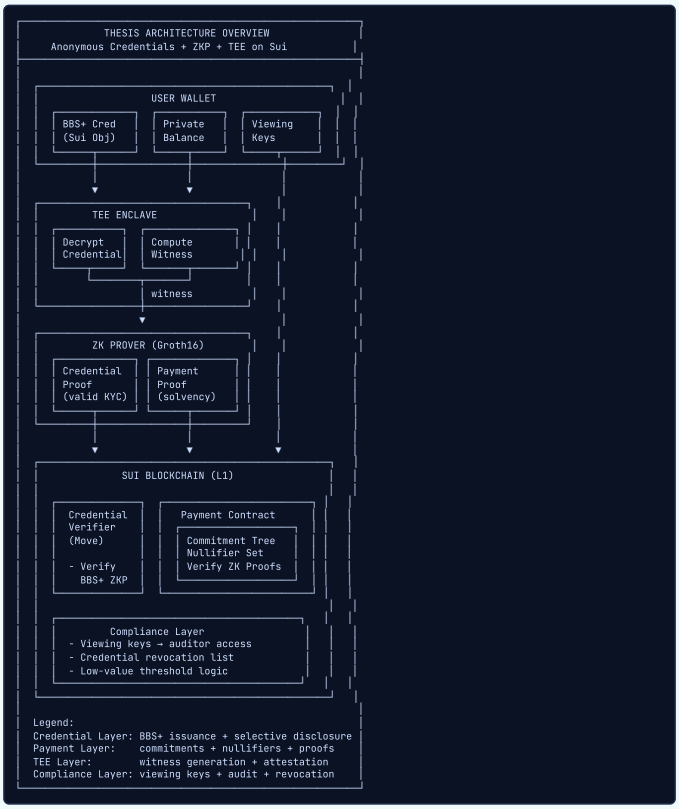
EXERCISES

1. **design** — Design a "crypto-agile" architecture for the thesis that allows swapping BBS+ for a post-quantum alternative without a hard fork. What abstraction layer is needed?
2. **conceptual** — Explain the "harvest now, decrypt later" threat as it applies to the thesis system. Which specific data on-chain is vulnerable, and what is its confidentiality lifetime?

Full Thesis Architecture

INTUITIVE

Building a privacy-preserving city on Sui. Credentials are your ID cards (BBS+), payments go through private channels (ZKP), and TEEs are the fast-track security checkpoints. The city has rules (compliance) but respects your right to walk the streets without being tracked.



KEY POINTS

- Layered architecture: Sui L1 base + credential layer + payment layer + TEE acceleration + compliance
- Credential layer: BBS+ issuance by trusted authorities, selective disclosure via ZKP, stored as Sui objects
- Payment layer: Pedersen commitments hide amounts, nullifiers prevent double-spend, ZK proofs ensure validity
- TEE acceleration layer: enclave computes witnesses for fast proof generation, attestation verifies enclave integrity
- Compliance layer: viewing keys for authorized auditors, credential revocation for banned users, low-value threshold for lighter verification

WHY IT MATTERS

This diagram IS the thesis. Every concept from all three study days feeds into one component: elliptic curves power BBS+ and Pedersen commitments, ZKP systems (Groth16) verify credentials and payments, TEEs accelerate proving, and Sui provides the programmable base layer. The thesis contribution is integrating these into a coherent privacy-preserving payment system with compliance.

THESIS ANGLE

In your thesis, this is the architecture diagram brought to life. Layer 1 (Credential): BBS+ issuance by identity provider, credential as encrypted Sui object, selective disclosure via Sigma protocols. Layer 2 (Payment): Pedersen commitments + nullifiers + Merkle tree stored as Sui shared objects, Groth16 proof for on-chain settlement. Layer 3 (TEE): SGX enclaves at payment terminals, remote attestation verified on-chain, real-time credential verification, batch proof generation. Layer 4 (Compliance): viewing keys distributed to regulators, encrypted memos per transaction, threshold volume proofs for AML. All four layers compose into a single Sui transaction package.

TECHNICAL

The complete thesis system is a multi-layer privacy protocol on Sui: $\Pi = (\Pi_{\text{cred}}, \Pi_{\text{pay}}, \Pi_{\text{TEE}}, \Pi_{\text{comply}})$, integrating anonymous credentials (BBS+), private payments (Pedersen commitments + nullifiers), TEE acceleration (witness generation + batch proving), and compliance mechanisms (viewing keys + threshold reporting) into a unified protocol verified by on-chain Sui Move contracts.

LAYER 1: CREDENTIAL LAYER (BBS+)

The credential layer handles issuance, storage, and privacy-preserving presentation:

$$\text{Issue}(\text{isk}, \mathbf{a}) \to \sigma = (A, e, s)$$

$$\text{Present}(\sigma, \mathbf{a}, D) \to \pi_{\text{cred}}$$

where $D \subseteq [L]$ is the disclosure set. The issuer signs the attribute vector with BBS+, the user stores the credential as an encrypted Sui object, and presentation generates a Groth16-wrapped BBS+ proof. The proof π_{cred} demonstrates: valid signature from a recognized issuer, satisfaction of access predicates (age, jurisdiction, etc.), and non-revocation against the on-chain accumulator.

LAYER 2: PAYMENT LAYER (PEDERSEN + NULLIFIER)

The payment layer handles value transfer with amount hiding and double-spend prevention:

$$\text{Pay}(v, \text{sk}) \to (C, \text{nf}, \pi_{\text{pay}})$$

where $C = vG + rH$ is the value commitment, $\text{nf} = \text{PRF}(\text{sk}, \text{note\_id})$ is the nullifier, and π_{pay} proves correct balance (no inflation), range validity ($0 \leq v < 2^{64}$), and correct nullifier derivation. The note commitment is added to the on-chain Merkle tree; the nullifier is added to the nullifier set.

LAYER 3: TEE ACCELERATION

The TEE layer provides two modes of operation:

$$\text{TEE.Verify}(\pi_{\text{cred}}) \to \{0, 1\} \quad (\text{real-time},\ \text{1-5ms})$$

$$\text{TEE.BatchProve}([\text{tx}_1, \ldots, \text{tx}_n]) \to \pi_{\text{batch}} \quad (\text{periodic settlement})$$

In real-time mode, the TEE verifies credential proofs for low-latency applications (point of sale, API access). In batch mode, the TEE aggregates multiple transaction witnesses and generates a single Groth16 proof covering the entire batch, reducing on-chain verification costs by a factor of n. The TEE attestation certifies enclave integrity for both modes.

LAYER 4: COMPLIANCE LAYER

The compliance layer enables regulatory adherence without compromising user privacy:

$$\text{ViewKey}(\text{auditor\_pk}, \text{tx\_details}) \to \text{encrypted\_memo}$$

$$\text{ThresholdReport}(\text{user\_sk}, \text{period}) \to \pi_{\text{volume}}$$

Viewing keys allow designated auditors to decrypt transaction metadata. Threshold reports let users prove compliance (e.g., "my monthly volume is below the reporting threshold") without revealing individual transactions. The compliance layer is modular: jurisdictions can define different policies (threshold amounts, required predicates, auditor keys) without modifying the core protocol.

SUI INTEGRATION DETAILS

The on-chain components are implemented as Sui Move modules: (1) Credential schema: a Move struct defining attribute types with issuer public key stored as a shared object. (2) Groth16 verifier contract: accepts serialized proofs and public inputs, performs pairing checks, returns boolean. (3) Note registry: a shared Sui object containing the Merkle tree of note commitments (Poseidon hash, depth 32). (4) Nullifier set: a shared Sui object implementing a sparse Merkle tree for efficient membership and non-membership proofs. (5) Compliance registry: stores encrypted memos and auditor public keys.

SYSTEM PARAMETERS

The protocol uses: BLS12-381 curve (for BBS+ and Groth16 pairings), Poseidon hash function (SNARK-friendly, ~8x faster than SHA-256 in-circuit), Groth16 proving system (192-byte proofs, 3-pairing verification), Ed25519 for Sui transaction signatures, AES-256-GCM for credential encryption, ECIES for viewing key encryption. The Merkle tree uses Poseidon with arity 2 and depth 32, supporting $2^{32} \approx 4 \times 10^9$ notes.

COMPLETE TRANSACTION FLOW

End-to-end flow for a credential-gated private payment: (1) User decrypts credential from Sui object (locally or in TEE), (2) TEE verifies credential validity (fast path) OR user generates ZK proof π_{cred} (trustless path), (3) User creates payment proof π_{pay} with value commitment C, nullifier nf, and range proof, (4) Proofs π_{cred} and π_{pay} are combined into a single Groth16 proof π, (5) Sui transaction submitted: Move contract verifies π, checks nullifier freshness, updates note tree, records nullifier, stores encrypted memo, (6) Event emitted for off-chain indexers. Total on-chain: one proof verification + two shared object mutations + one event.

PERFORMANCE BUDGET

End-to-end latency breakdown: credential decryption (~5ms), witness generation (~50ms in TEE), Groth16 proving (~500ms mobile, ~100ms desktop), transaction submission + consensus (~390ms Mysticeti), on-chain verification (~10ms execution). Total: ~700ms with TEE assist on mobile, ~100ms with TEE fast path (skipping on-chain ZKP for low-value). Gas cost: ~300K gas units (~$0.01 at current Sui prices). Transaction size: ~1.5KB (192B proof + commitments + nullifier + encrypted memo + Sui tx overhead).

HISTORY Thesis contribution: Alexandre Mourot, supervised at Mysten Labs (Sep 2026 - Mar 2027) (2027). *The thesis touches work from at least 5 Turing Award laureates: Adi Shamir (2002, ring signatures), Shafi Goldwasser and Silvio Micali (2012, zero-knowledge proofs), and the theoretical foundations of both commitments and proofs trace to Goldreich, Micali, and Wigderson (1986). The system literally stands on the shoulders of the most decorated researchers in computer science.*

LIMITATIONS

- System complexity is the primary risk: the architecture spans 4 layers (credential, payment, TEE, compliance), 3 cryptographic proof systems (BBS+, Groth16, Pedersen), and 2 trust models (cryptographic + hardware) — each layer introduces potential failure modes and the interaction between layers creates emergent attack surfaces not present in simpler systems
- The credential issuer is a centralized trust anchor: if the KYC issuer is compromised, malicious credentials can be created that pass all on-chain verification — the system inherits the trust assumptions of the traditional KYC provider, just moving the trust from per-transaction to per-credential
- Gas costs for full verification (Groth16 credential proof + Groth16 payment proof + Merkle tree update + nullifier set update) may exceed 2-5M gas units on Sui (~$0.01-0.05 at current prices), which is acceptable for individual transactions but becomes significant at high volume — batch proving amortizes this but adds latency

EXERCISES

1. **synthesis** — Draw a complete sequence diagram for a private payment in your thesis system, covering all 4 layers. Start from "user opens wallet" to "receiver sees funds." Include every cryptographic operation, on-chain interaction, and TEE call.
2. **analysis** — Identify the three most critical attack vectors against the full thesis architecture. For each, describe the attack, its impact, and the mitigation the system provides.
3. **comparison** — Compare the thesis architecture with three existing systems: Zcash (ZKP only), Secret Network (TEE only), and Aztec (ZK rollup). What does the thesis provide that none of these individually offer?

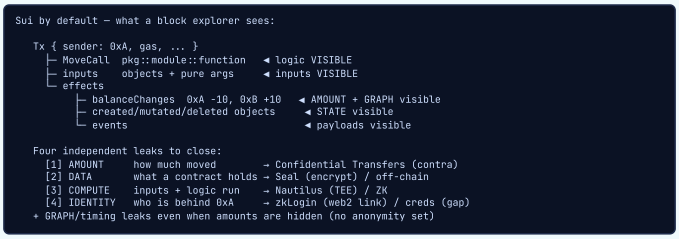
CHAPTER 8

Private Transactions on Sui — Synthesis

What's Public by Default on Sui — The Privacy Threat Model

INTUITIVE

Sui is a glass building. Every account is a glass room; every object (a coin, an NFT, a position) sits on a labelled shelf with its type, owner, and contents visible to anyone walking past with a block explorer. When you transact, you don't whisper — you rearrange the shelves in full view: which coins moved, how much, from whom, to whom, calling which function, are all recorded as plaintext in the transaction and its effects. There is no shielded pool, no default mixing, no encrypted mempool. So before you can claim ANY privacy property, you must name exactly which pane of glass you are frosting. Four things leak independently: the AMOUNT of a transfer, the DATA a contract stores, the LOGIC/inputs of a computation, and the IDENTITY behind an address (plus the GRAPH linking addresses over time). A privacy design is only as strong as the leakiest pane you forgot to cover.



KEY POINTS

- Sui's object model is fully transparent: object type, owner, version, and contents are public; transactions expose sender, MoveCall target, inputs, and effects (balanceChanges, created/mutated objects, events)
- There is NO native shielded pool, encrypted mempool, or default mixing on Sui — privacy is opt-in, per-facet, and application-level
- Validators see transaction contents during execution; privacy schemes hide data from the CHAIN STATE / public, not necessarily from a validator running TEE-less code
- Four orthogonal leaks: amount, stored data, computation inputs/logic, and identity/transaction-graph — each needs a different tool
- Even with amounts hidden, sender + receiver + timing of a confidential transfer stay public (the anonymity-set / unlinkability gap)
- Gas is always paid by a visible address (unless sponsored) — gas payment itself is a deanonymization vector

WHY IT MATTERS

This is the threat-model section your thesis MUST open with. Reviewers (Ford, Troncoso, Vaudenay) will judge the work by whether you state the adversary and the exact leak each mechanism closes. Naming the four panes (amount/data/compute/identity) + the graph leak gives you a checklist to evaluate every design against, and prevents the classic mistake of claiming 'private' when only the amount is hidden.

THESIS ANGLE

Write a single threat-model table: rows = {amount, stored data, computation, identity, transaction graph/timing}, columns = {visible by default?, which Sui tool hides it?, residual leak?}. Fill it from this part. That table becomes Figure 1 of your background chapter and the rubric for your evaluation.

TECHNICAL

Sui's state is a set of **objects**, each with a public `(id, version, type, owner, contents)`. A transaction is a **Programmable Transaction Block** (PTB) whose `sender`, `MoveCall` targets, input objects/pure args, and **effects** (`balanceChanges`, created/mutated/deleted objects, emitted events) are all recorded in plaintext and served by RPC/explorers. There is no native shielded pool or encrypted mempool.

Formally, model the adversary `A` as a passive observer with full read access to chain state + effects (optionally also a validator with execution-time visibility, or a registered auditor). Privacy goals are stated per leak:

- **Amount hiding:** `A` cannot learn transfer values.
- **Data hiding:** `A` cannot read contract-held blobs.
- **Compute hiding:** `A` cannot learn inputs/intermediate state.
- **Identity/graph hiding:** `A` cannot link address↔real-identity nor link an address's transactions (unlinkability).

WHAT AN EXPLORER READS FROM ONE TRANSFER

```
TransactionBlockResponse {
  transaction.data.sender: 0xA            // identity (pseudonym)
  transaction.data.commands: [MoveCall{ package, module, function }] // logic
  transaction.data.inputs:   [Object(id), Pure(bytes)]               // inputs
  effects.status, gasUsed, gasObject (payer)                         // gas → payer
  balanceChanges: [{owner:0xA, -10},{owner:0xB,+10}]                 // AMOUNT+GRAPH
  objectChanges, events                                              // state
}
```

Each line maps to a leak category. A privacy claim must specify which line(s) are redacted and which remain — anything unredacted is in `A`'s view.

RESIDUAL GRAPH LEAK (FORMAL)

Even with amounts hidden, define the public transfer graph `G = (V, E)` where `V` = addresses and `(u,v,t) ∈ E` for each confidential transfer at time `t`. Amount-hiding redacts edge *weights* but not edge *existence*. Standard graph deanonymization (degree, timing correlation, wrap/unwrap boundary events) recovers structure from `G` alone — hence amount privacy ≠ payment privacy, and an **anonymity set** (edge-existence hiding) is a separate, unmet goal.

HISTORY Sui object model by Mysten Labs (transparency is a deliberate design choice for parallel execution + verifiability) (2023). *On a transparent chain, hiding the amount can paradoxically make you MORE identifiable if you're the only one using the confidential token — privacy needs a crowd, which is why the anonymity-set gap matters as much as the encryption.*

LIMITATIONS

- Per-facet privacy means a single uncovered leak (e.g. a public gas payer, or a unique transfer timing) can unravel the rest
- Transaction graph analysis works even against hidden amounts — wrap/unwrap events and timing correlate addresses
- Validator-side visibility during execution limits what 'on-chain privacy' can mean without TEE or pure ZK
- No base-layer anonymity set means k-anonymity must be engineered by the application

EXERCISES

1. **concept** — List, for a single ordinary Sui coin transfer, every field a block explorer can read. Then mark which of the four leak categories each field falls under.
2. **concept** — Argue why hiding the amount alone does NOT give payment privacy. Construct a deanonymization attack using only senders, receivers, and timestamps of confidential transfers.
3. **design** — Draft your thesis threat model: who is the adversary (passive chain observer? validator? auditor?), and which of the four panes must your design frost to meet your privacy goal for low-value payments?

The Sui Privacy Stack — Four Layers of Confidentiality

INTUITIVE

Think of privacy on Sui as four independent tinting films you can apply to the glass building, each made by a different supplier and each tinting a different window. AMOUNT film = Confidential Transfers (`contra`): encrypts balances/transfer amounts with twisted ElGamal + Bulletproofs. DATA film = Seal: encrypts arbitrary blobs so only those who satisfy an on-chain access policy (`seal_approve`) can decrypt, with the keys split threshold-style across key servers. COMPUTE film = Nautilus: runs your logic inside an AWS Nitro TEE and lets a Move contract verify a hardware attestation, so the inputs and intermediate state stay sealed while the OUTPUT is trustworthy. IDENTITY film = zkLogin: lets a user authenticate with a web2 account (Google, etc.) and act on-chain without publishing the link between their real identity and their address, via a Groth16 proof. They're composable — you can stack films — but each has a distinct trust assumption (math vs threshold vs hardware vs OAuth provider), and choosing the right film per window is a core thesis decision.

```
Four privacy layers, four trust roots, four hidden facets:

  LAYER        HIDES              PRIMITIVE               TRUST ROOT
  ──────────────────────────────────────────────────────────────────
  Amount       balances/amts      Confidential Transfers  DDH + DL(h)
               (contra)           twisted ElGamal+BPs     (pure crypto)
  Data         stored blobs       Seal (IBE threshold)    t-of-n key
               + access ctrl       seal_approve policy     servers
  Compute      inputs + logic     Nautilus (AWS Nitro)    AWS CA +
               (output signed)    attestation in Move      silicon
  Identity     web2↔address       zkLogin (Groth16 BN254) OAuth issuer
               link               ephemeral key + salt    + prover

  Composable:  zkLogin user ── confidential transfer ── of value
               whose KEY is stored via Seal, issued by a
               credential minted inside a Nautilus TEE.
```

KEY POINTS

- Amount layer = Confidential Transfers / `contra`: twisted ElGamal over Ristretto255 + Bulletproofs; hides balances & transfer amounts, NOT sender/receiver (devnet beta, UNAUDITED) — see the dedicated Ch 2.2 chapter
- Data layer = Seal: identity-based threshold encryption; data encrypted to an on-chain policy, decryptable only if a Move `seal_approve` check passes; keys held t-of-n by key servers (trust = threshold of servers)
- Compute layer = Nautilus: AWS Nitro Enclave runs logic, signs outputs with an attested ephemeral key, Move verifies the Nitro attestation (trust = AWS CA + silicon) — see the Nautilus chapter
- Identity layer = zkLogin: Groth16 (BN254) proof binds an OAuth JWT to an ephemeral key; on-chain address = hash over (salt, iss, aud, sub) so the web2↔address link stays private (trust = OAuth issuer + prover + salt secrecy)
- Different trust models: pure crypto (amounts) > threshold (data) > hardware (compute) > federated OAuth (identity) — your thesis should rank these for its threat model
- The layers compose but each adds its own residual leak and failure mode; privacy is the MIN over all layers used, not the max

WHY IT MATTERS

This four-layer map is the organizing frame for your entire background section, and each layer already has its own deep chapter in this study plan (Confidential Transfers in Ch 2.2, Seal/Nautilus in Ch 2.3, zkLogin in Ch 2.6/crypto). Your contribution sits at the seam between AMOUNT (contra) and IDENTITY (anonymous credentials): you add unlinkable authorization on top of confidential value, the one combination none of the four layers delivers alone.

THESIS ANGLE

Use this as your related-work taxonomy. For each layer, write one paragraph: what it hides, its trust assumption, its residual leak, and whether your design uses or extends it. Conclude with the gap statement: 'no existing Sui primitive provides anonymous authorization over confidential amounts; this thesis composes contra's amount-hiding with credential-based anonymous authorization to close it.'

TECHNICAL

Four orthogonal primitives, each a different hardness assumption. Composition privacy is the **conjunction** of guarantees and the **union** of trust assumptions — i.e. privacy = MIN over layers, trust = SUM over layers.

Layer	Primitive	Assumption
Amount	Confidential Transfers (twisted ElGamal + Bulletproofs)	DDH on Ristretto255; DL of `h` unknown
Data	Seal (IBE threshold encryption)	< t-of-n key servers collude; BLS/IBE hardness
Compute	Nautilus (AWS Nitro TEE)	AWS root CA + Nitro silicon honest; no side-channel
Identity	zkLogin (Groth16 BN254)	OIDC issuer honest; salt secret; prover correct

SEAL — ACCESS-GATED THRESHOLD DECRYPTION

Data is IBE-encrypted to an *identity* = a policy id; the master key is shared `t-of-n` across key servers. A client obtains a decryption key only if a Move `seal_approve` function (a dry-run access check) does not abort, then combines `t` server responses. Trust degrades gracefully: privacy holds unless `≥ t` servers collude.

```
encrypt(data) → ciphertext bound to policy P
decrypt: dry-run seal_approve(P, ctx) must pass → fetch t key shares → combine
```

(Deep dive: the Seal material in Ch 2.3 / `/sui-seal`.)

NAUTILUS — ATTESTED COMPUTE (OUTPUT INTEGRITY, INPUT SECRECY)

Logic runs in a Nitro Enclave that emits a COSE_Sign1 attestation binding PCRs (code measurement) to an ephemeral public key; a Move verifier checks the AWS-rooted cert chain via `sui::nitro_attestation`, then trusts outputs signed by that key. Inputs/intermediate state stay sealed; only the OUTPUT is published and verifiable. (Deep dive: the Nautilus chapter.)

ZKLOGIN — IDENTITY-LINK HIDING

```
address = Blake2b( salt, iss, aud, sub )   // no visible web2 tie
Groth16 proof (BN254) attests:
  • a valid, unexpired OIDC JWT for (iss, aud, sub)
  • commitment to an ephemeral key + max epoch
verified on-chain by sui::groth16 (same verifier family the thesis reuses)
```

HISTORY Mysten Labs (Seal, Nautilus, zkLogin, Confidential Transfers) — a coordinated 2023–2026 privacy/identity product line (2026). *These four were built by overlapping teams and reuse each other: the Confidential Transfers demo wallet explicitly plans to move secret-key storage from browser localStorage to Seal — a real in-the-wild composition of two of the four layers.*

LIMITATIONS

- No single layer gives full transaction privacy; you must combine and the combination inherits every trust assumption
- Trust roots are heterogeneous — a thesis must justify each (e.g. is depending on AWS acceptable for your model?)
- Seal and Nautilus add off-chain infrastructure (key servers, enclaves) the user/issuer must run or trust
- zkLogin reveals nothing about identity on-chain but the OAuth provider + salt service can still link if compromised

EXERCISES

1. **concept** — For each of the four layers, write its trust assumption in one sentence and the single event that would break it (e.g. 'Seal breaks if t key servers collude').
2. **concept** — The contra demo plans to store viewing keys in Seal. Which two layers compose here, and what new combined trust assumption results?
3. **design** — Pick the minimal subset of layers your low-value private payment system actually needs. Justify each inclusion and each exclusion against your threat model.

Amount Privacy — Confidential Transfers (`contra`) in One Page

INTUITIVE

This is the AMOUNT film, and it is the part of the stack closest to your thesis core, so here is the one-page version (the full deep dive is the Confidential Transfers chapter in Ch 2.2). You take an ordinary Sui `Coin<T>` and WRAP it into a confidential balance: the coin's value goes into a shared pool, and you get encrypted credit. Inside the confidential domain you can TRANSFER to others with the amount hidden (twisted ElGamal ciphertexts, proven correct by Bulletproofs range proofs + Schnorr-style equality proofs, so nobody inflates or overspends). To exit, you UNWRAP back to a public coin. The catch, repeated because it matters for composition: wrap and unwrap REVEAL the amount (they touch the public coin layer), and even private transfers reveal sender, receiver, and timing. So contra is precisely an amount-confidentiality layer — and the natural substrate your anonymous-credential work sits on top of, via its `Auth`/policy hook.

```
contra in one page (| = amount revealed at the border):

   public Coin<T> ──wrap──|──> confidential balance ──transfer(HIDDEN)──>
                                        |                    receiver
                                        └──unwrap──|──> public Coin<T>

   Hidden:   transfer AMOUNTS, balances
   Public:   sender, receiver, timing, wrap/unwrap amounts

   Crypto:   twisted ElGamal (c=r·g+m·h, d=r·pk) + 4×u16 limbs
             Bulletproofs range + DDH/ElGamal sigma proofs
   Hook:     Auth<T> + Policy gate register/wrap/unwrap  ◄ thesis seam
   Compliance: per-account auditor, freeze, seize, selective disclosure

   Status: devnet public beta, UNAUDITED. pkg 0xe0f1b22e…0c271
```

KEY POINTS

- Wrap (public→confidential) and unwrap (confidential→public) reveal the amount; only intra-domain transfers hide it — top-level README 'Privacy boundary'
- Amounts are four u16 twisted-ElGamal limbs; correctness via Bulletproofs (sui::rangeproofs) + sigma NIZKs (nizk.move) — see Ch 2.2 deep chapter
- Authorization is an Auth<T> minted via authorize_as_sender / authorize_as_object / authorize_with_witness (contra.move:216-237) — the witness path is the credential-integration seam
- Built-in compliance: per-account auditors (selective visibility), freeze/seize levers, and user-initiated selective disclosure — already a partial identity/compliance story
- It is an AMOUNT layer only: it gives zero sender/receiver unlinkability — the anonymity-set gap is unaddressed by contra
- UNAUDITED devnet beta — usable as a research substrate, not production

WHY IT MATTERS

contra is the substrate your thesis extends. Its `authorize_with_witness` lets an issuer's contract mint authorization after arbitrary checks — replace 'issuer checks a KYC list' with 'a ZK credential proof authorizes the witness' and you have anonymous authorization over confidential amounts. The per-account auditor is also your concrete baseline for the compliance-preserving-privacy comparison.

THESIS ANGLE

In your design chapter, present contra as the chosen amount-privacy layer and your credential module as the authorization layer above it. Show the exact call: your Move module verifies a Groth16 credential proof and, on success, calls authorize_with_witness (contra.move:223) to mint an Auth<T> for register/wrap/unwrap — composing private identity with private value.

TECHNICAL

The amount layer. An `EncryptedAmount` is four u16 twisted-ElGamal limbs (`c = r·g + m·h`, `d = r·pk`); a `WellFormedProof` = Bulletproofs (per-limb range, `sui::rangeproofs::verify_bulletproofs_ristretto255`) + per-limb ElGamal sigma proofs; a `DdhProof` proves balance equality on a spend. Value crosses the public/confidential boundary via `wrap`/`unwrap` (amount revealed) and moves hidden via `batched_transfer`. Full treatment in the Ch 2.2 Confidential Transfers chapter; this page is the integration view.

THE INTEGRATION SEAM (AUTH WITNESS)

```
// policy.move:35 — drop-only, phantom T, per-PTB capability
public struct Auth<phantom T> has drop { operations: u32, owner: address }

// contra.move:223 — issuer-gated mint: the thesis hook
public fun authorize_with_witness<T, W: drop>(
    ct: &ConfidentialToken<T>, operation: u8, owner: address, witness: W
): Auth<T>
// mints Auth iff policy.witness_type == W AND operation is permissioned

// PERMISSIONED_REGISTER=0  PERMISSIONED_WRAP=1  PERMISSIONED_UNWRAP=2 (contra.move:103)
```

Replace the issuer's KYC-list check with a credential ZK-proof check inside the module that holds `W`: verifying the proof, then calling `authorize_with_witness`, yields anonymous authorization over confidential value.

WHAT IT DOES / DOESN'T HIDE (PRECISE)

```
HIDES : transfer amounts, balances (active/pending)
LEAKS : sender, receiver, timing (every transfer);
        amount + counterparty of each wrap/unwrap (boundary)
EXTRAS: per-account auditor (selective visibility), freeze/seize,
        user selective disclosure (verifyDecryption, protocol-id 100)
STATUS: devnet beta, UNAUDITED, pkg 0xe0f1b22e…0c271
```

HISTORY Mysten Labs; twisted-ElGamal lineage from Zether (Bünz et al. 2019) and Solana confidential transfers (2026). *contra deliberately uses Bulletproofs + sigma proofs instead of a SNARK precisely so the payment path needs no trusted setup and deposits are free homomorphic adds — a different point in the design space from Zcash-style note systems.*

LIMITATIONS

- No anonymity set: hides amounts, not the who/when of transfers
- Wrap/unwrap leak the boundary amount and counterparties
- UNAUDITED, crypto 'not finalized', devnet only
- Decryption/aggregation limits (u16 limbs, merge cap) impose UX constraints on heavy receivers

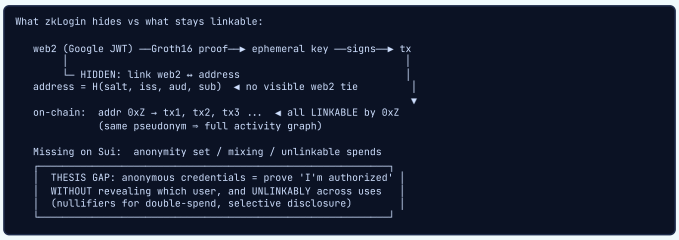
EXERCISES

1. **concept** — State precisely what contra hides and what it leaks. Why is it accurate to call it an 'amount-confidentiality layer' rather than a 'private payments system'?
2. **design** — Sketch the Move signature of a credential-gated wrapper that calls authorize_with_witness (contra.move:223). What does your witness type W represent, and what does the ZK proof attest?
3. **hands-on** — Open the dedicated Confidential Transfers chapter (Ch 2.2) and map each of its six concepts to one row of your four-pane threat-model table.

Identity & Sender Privacy — zkLogin and the Anonymity-Set Gap

INTUITIVE

zkLogin is the IDENTITY film: it lets a user log in with Google (or any OIDC provider) and act on Sui WITHOUT publishing the link between 'alice@gmail.com' and her on-chain address. Under the hood a Groth16 proof attests 'I hold a valid, unexpired JWT for this provider and committed to this ephemeral key', and the address is derived by hashing a user-specific SALT together with the token's issuer/audience/subject — so the chain sees a normal-looking address with no visible tie to the web2 account. But — and this is the crux for your thesis — zkLogin hides the web2↔address LINK, not the on-chain ACTIVITY of that address. Once you act, your address is as traceable as any other: every transaction it sends is linkable to every other by the shared address. Sui has no native mixer, no shielded pool, no ring signatures — there is NO anonymity set. So 'sender privacy' on Sui today means at best 'fresh pseudonym per login', not unlinkability. Closing that gap — unlinkable authorization so multiple actions can't be tied to one user or each other — is the anonymous-credentials problem your thesis tackles.



KEY POINTS

- zkLogin hides the web2-identity ↔ on-chain-address link via a Groth16 (BN254) proof over an OIDC JWT + an ephemeral key + a user salt; address = hash over (salt, iss, aud, sub)
- It does NOT anonymize on-chain activity: a zkLogin address is a stable pseudonym, so all its transactions are mutually linkable
- Sui has no native anonymity set: no shielded pool, no mixer, no ring signatures — sender unlinkability must be built at the application layer
- Trust in zkLogin = the OIDC provider (issues honest JWTs) + the prover service + salt secrecy; a salt or provider compromise can re-link identity
- Anonymous credentials (BBS+/Coconut/Semaphore-style) are the missing primitive: prove possession of an attribute/authorization without revealing identity AND without linking repeated uses
- Double-spend / sybil control without identity needs nullifiers + revocation-friendly accumulators — the hard part your thesis must design

WHY IT MATTERS

This is the precise statement of your thesis gap. The Sui stack gives amount privacy (contra), data privacy (Seal), compute privacy (Nautilus), and web2-link privacy (zkLogin) — but NO unlinkable authorization. Your anonymous-credential system (ZKP for the proof, TEE for issuance) supplies exactly that, and plugs into contra's witness hook so the authorized action can ALSO have a hidden amount. zkLogin is your closest relative and your strongest 'why not just use X' to address head-on.

THESIS ANGLE

Devote a related-work subsection to 'zkLogin vs anonymous credentials': both use Groth16, but zkLogin gives per-account web2 unlinkability while your credentials give per-USE unlinkability + selective disclosure + nullifier-based double-spend protection. Tabulate the difference and use it to motivate the contribution. Tie nullifier design to your revocation question (key thesis question #4).

TECHNICAL

zkLogin provides **web2-link unlinkability**: the map real-identity→address is hidden behind a Groth16 proof + a secret salt. It does NOT provide **per-use unlinkability**: a zkLogin address is a persistent pseudonym, so its transactions are mutually linkable, and Sui has no anonymity set (no mixing / shielded pool / ring signatures). Anonymous credentials supply the missing notion.

An **anonymous credential** scheme provides algorithms `(Issue, Prove, Verify, Revoke)` such that a holder can prove possession of a credential (and selectively disclose attributes) so that: (i) proofs reveal nothing beyond the disclosed predicate (zero-knowledge), (ii) two showings are *unlinkable*, and (iii) double-spend/sybil is prevented by a **nullifier** that is unique per (credential, context) yet reveals neither the credential nor links to other nullifiers.

ZKLOGIN VS ANONYMOUS CREDENTIALS

Property	zkLogin	Anon. credential
Hides web2 ↔ address	yes	n/a
Unlinkable across uses	**no** (stable pseudonym)	**yes**
Selective attribute disclosure	no	yes
Double-spend control	n/a	nullifier
On-chain verifier	sui::groth16	sui::groth16 (same)

Same Groth16 verifier, different statement — a clean feasibility/cost argument: benchmark the credential circuit against zkLogin's known on-chain verify gas.

NULLIFIER SKETCH (UNLINKABLE + UNIQUE)

```
nullifier = H( sk_credential , context )
  • unique:    deterministic in (sk_credential, context) ⇒ a 2nd spend repeats it
  • unlinkable: H hides sk_credential; different contexts ⇒ uncorrelated nullifiers
on-chain: maintain a nullifier set; reject if present (double-spend);
          ZK proof attests nullifier well-formed w.r.t. a committed credential
revocation: prove membership in a NON-revoked accumulator (Merkle / RSA acc.)
```

The subtlety (thesis question #4): revocation must not deanonymize — prove non-membership in a revocation set without revealing which credential.

HISTORY zkLogin by Mysten Labs (2023); anonymous credentials lineage: Chaum (1985), Camenisch–Lysyanskaya (2001), BBS+ , Coconut (Sonnino et al. 2018), Semaphore (2023). *zkLogin and your thesis credentials can share the SAME on-chain Groth16 verifier (Sui's sui::groth16) — the difference is entirely in the statement being proved, which is a clean way to argue feasibility/cost in your evaluation.*

LIMITATIONS

- zkLogin pseudonyms are linkable on-chain; rotating addresses fragments balances and UX
- No base-layer anonymity set means k-anonymity must be bootstrapped by the application (hard for low-volume tokens)
- Anonymous-credential nullifiers + revocation interact subtly with unlinkability — naive designs leak or fail to revoke
- Trust dependencies (OIDC provider, salt service, prover) remain even with perfect on-chain ZK

EXERCISES

1. **concept** — Explain the difference between 'web2-link unlinkability' (zkLogin) and 'per-use unlinkability' (anonymous credentials). Give a scenario where the first holds but the second fails.
2. **design** — Design a nullifier scheme that prevents double-spend of a one-time credential WITHOUT linking the two attempted uses to each other or to the holder. What does the nullifier commit to?
3. **concept** — Why does hiding amounts (contra) plus fresh pseudonyms (zkLogin) still NOT give unlinkable payments? Identify the residual linkage and which thesis primitive removes it.

Composing Private Payments on Sui — The Thesis Design Space

INTUITIVE

Now assemble the films into a window that's actually private. A complete private low-value payment on Sui needs four things at once: (1) the AMOUNT hidden — use contra; (2) the AUTHORIZATION anonymous and unlinkable — your anonymous-credential proof, verified on-chain (Groth16) and minting contra's Auth<T> via the witness hook; (3) the CREDENTIAL issued without a trusted central database leaking identity — issue it inside a Nautilus TEE (stateless, attested) and/or store viewing keys with Seal; (4) double-spend & revocation handled by NULLIFIERS + an accumulator, so a spent or revoked credential can't be reused, all without deanonymizing honest users. The art is in the seams: the credential proof must bind to the contra operation it authorizes (so it can't be replayed elsewhere), the nullifier must be unlinkable yet unique, and the compliance path (selective disclosure / per-account auditor) must coexist with anonymity. That composition — confidential value + anonymous authorization + TEE-anchored issuance + accountable privacy — IS your thesis.

KEY POINTS

- Full private payment = confidential AMOUNT (contra) + anonymous AUTHORIZATION (credential ZK proof) + private ISSUANCE (Nautilus TEE) + double-spend/revocation (nullifiers + accumulator)
- Integration point is contra's authorize_with_witness (contra.move:223): a Move module verifying a Groth16 credential proof mints the Auth<T> that gates wrap/transfer/unwrap
- On-chain verification reuses Sui's native Groth16 (sui::groth16, BN254) — the same verifier family as zkLogin, so feasibility/cost is benchmarkable
- TEE issuance (Nautilus) gives stateless, attested credential minting so the issuer holds no linkable identity database — pairs with Seal for key storage
- Accountable privacy: contra's per-account auditor + user selective disclosure let regulators open specific facts without breaking default anonymity — a core thesis selling point
- Open research gaps to claim: bootstrapping an anonymity set, unlinkable-yet-unique nullifiers, revocation without deanonymization, gas-payer privacy (sponsored tx), and the TEE-vs-pure-ZK trust trade
- Evaluation axes (thesis question #5): proof size, prover time, on-chain Groth16 verify gas, end-to-end latency, and practicality for LOW-VALUE payments

WHY IT MATTERS

This concept is the blueprint for your design + evaluation chapters and directly answers the five thesis questions: on-chain issuance without linking usage (TEE + accumulator), attribute verification without revealing (selective-disclosure ZK), TEE's role (private attested issuance), revocation without breaking anonymity (nullifier + revocation accumulator), and gas/proof cost on Sui (native Groth16 benchmark). Every other concept in this part feeds into this synthesis.

THESIS ANGLE

Make this diagram your Figure 2 (system architecture). Then structure the implementation chapter exactly along its boxes: (a) Nautilus issuance module, (b) on-chain credential accumulator + Groth16 verifier, (c) the contra witness adapter, (d) nullifier registry. Each box gets a benchmark in the evaluation chapter, rolled up against the low-value-payment practicality bar.

TECHNICAL

A complete private low-value payment composes the layers: confidential AMOUNT (contra) + anonymous unlinkable AUTHORIZATION (credential ZK proof → `Auth<T>`) + private ISSUANCE (Nautilus TEE, optionally Seal for key storage) + double-spend/REVOCATION (nullifier + accumulator). Verification reuses `sui::groth16`. The binding requirement: the credential proof must be bound to the specific contra operation it authorizes (no replay), and the nullifier must be unique-yet-unlinkable.

END-TO-END PTB (ONE PRIVATE PAYMENT)

```
PTB:
  1. cred::verify_and_authorize<T, W>(ct, proof, public_inputs, op)
       • sui::groth16 verifies: valid non-revoked credential
         + selective-disclosure predicate + well-formed nullifier
       • assert nullifier ∉ NullifierSet ; insert it
       • bind proof to (op, this PTB) to prevent replay
       • return Auth<T> via contra::authorize_with_witness(ct, op, owner, W{})
  2. contra::wrap | batched_transfer | unwrap (auth, ...)   // amount HIDDEN
  // Auth is drop-only ⇒ single-use authorization enforced by the type system
```

MAPPING TO THE FIVE THESIS RESEARCH QUESTIONS

```
Q1 issue without linking usage  → TEE issuance + on-chain accumulator commitment
Q2 verify attributes hidden     → selective-disclosure Groth16 circuit
Q3 role of TEE                  → stateless attested issuance (no identity DB)
Q4 revocation w/o deanon        → non-membership proof in revocation accumulator
Q5 cost on Sui / low-value      → benchmark sui::groth16 verify gas + prover time +
                                  e2e latency vs payment value
```

OPEN GAPS & EVALUATION AXES

Gaps to claim as contributions / future work: bootstrapping an anonymity set on a transparent chain; unlinkable-yet-unique nullifiers with practical revocation; gas-payer privacy (sponsored / Enoki transactions so the gas address doesn't deanonymize); and the TEE-vs-pure-ZK trust trade (justify the hardware root to Ford/Troncoso). **Evaluation axes:** proof size, prover time, on-chain Groth16 verify gas, end-to-end latency, and the practicality threshold for low-value payments. Caveat: contra is UNAUDITED devnet beta — the prototype is research-grade.

HISTORY ZKP+TEE hybrids are an active research line; this specific composition (anonymous credentials over Confidential Transfers on Sui) is the thesis contribution (2026). *Because contra's Auth<T> is a drop-only, phantom-typed capability minted per-transaction, it is an almost perfect plug for a credential check: the proof authorizes exactly one operation in exactly one PTB and cannot be stored or replayed — the type system enforces single-use authorization for free.*

LIMITATIONS

- Composition inherits ALL four layers' trust assumptions plus the new credential/TEE assumptions — the privacy is the minimum across them
- Anonymity set must still be bootstrapped; with few users, hidden amounts + unlinkable proofs still leak via timing/volume
- TEE issuance reintroduces a hardware trust root that pure-ZK purists (and some reviewers) will challenge — justify it explicitly
- Revocation + nullifiers + unlinkability is genuinely hard; a flawed scheme silently breaks either privacy or double-spend protection
- contra is UNAUDITED devnet beta, so an end-to-end prototype is research-grade, not production

EXERCISES

1. **design** — Draw your full system as a PTB: which calls verify the credential proof, mint the Auth<T>, and execute the confidential transfer, and in what order? Where does the nullifier get checked and recorded?
2. **concept** — Map each of the five thesis research questions to a specific box in the architecture diagram and name the artifact that answers it.
3. **design** — Justify (or reject) the TEE issuance box against a pure-ZK alternative. State the threat each stops, the cost of each, and which you'd defend to Bryan Ford / Carmela Troncoso.

CHAPTER 9

IOP & PCP — Probabilistic Proofs

PCP (Probabilistically Checkable Proofs)

INTUITIVE

A 10,000-page proof where a verifier randomly reads 3 sentences and catches any error with 99% probability. The proof is written in a special format so that any lie corrupts enough of the text to be detectable by spot-checking.

KEY POINTS

- PCP Theorem: NP = PCP[O(log n), O(1)] — every NP proof can be rewritten so a verifier checks O(1) bits with O(log n) randomness
- Query complexity q: number of proof positions read (constant for PCP theorem)
- Randomness complexity r: number of random bits (O(log n) for PCP theorem)
- Soundness error: probability verifier accepts false proof (amplified by repetition)
- Perfect completeness: honest proofs always accepted (no false negatives)
- 3-SAT has a PCP with O(1) queries — verifier checks via clever algebraic encoding
- PCPs imply hardness of approximation (Hastad 1997)

WHY IT MATTERS

PCPs are the theoretical ancestor of all succinct proof systems. The Kilian-Micali construction compiles a PCP into a succinct argument using Merkle hashing. Modern SNARKs evolved from this: IOP generalizes PCP with interaction, then Fiat-Shamir removes the interaction.

THESIS ANGLE

PCPs explain WHY SNARKs can verify computation in O(1): the PCP theorem guarantees that any NP statement can be checked with constant queries. When your Sui verifier checks a credential proof in constant time, it is because the underlying proof system descends from this theorem.

TECHNICAL

A PCP system for language L is a pair (P, V) where P produces a proof string $\pi \in \Sigma^m$, and V is a probabilistic polynomial-time verifier with oracle access to π. The verifier uses $r(n)$ random bits and makes $q(n)$ queries to π.

PCP PARAMETERS

Define $\mathrm{PCP}[r(n), q(n)]$ as the class of languages with PCP verifiers using $r(n)$ random bits and $q(n)$ queries over alphabet Σ, with completeness c and soundness s:

$$x \in L \Rightarrow \exists \pi,\ \Pr[V^{\pi}(x) = 1] \geq c$$

$$x \notin L \Rightarrow \forall \pi,\ \Pr[V^{\pi}(x) = 1] \leq s$$

PCP Theorem: $\mathrm{NP} = \mathrm{PCP}[O(\log n), O(1)]$. Every NP statement has a proof checkable with $O(\log n)$ random bits and $O(1)$ queries.

QUERY COMPLEXITY & SOUNDNESS

Soundness error with q queries and repetition:

$$\varepsilon \leq 2^{-\Omega(q)}$$

Free-bit complexity: the number of "free" answer bits not determined by consistency checks. Lower free-bit complexity yields better soundness. The *parallel repetition theorem* (Raz) shows that k-fold parallel repetition reduces soundness error exponentially, though not perfectly: $s^k \leq \varepsilon(k) \leq s^{\Omega(k)}$.

PCP OF PROXIMITY

A PCP of Proximity (PCPP) relaxes the standard PCP: the verifier also has oracle access to the input x, and accepts if (x, w) is δ-close to the language L in relative Hamming distance:

$$\Delta((x, w),\ L) \leq \delta \Rightarrow \Pr[V^{x,\pi}(|x|) = 1] \geq c$$

$$\Delta((x, w),\ L) > \delta \Rightarrow \forall \pi,\ \Pr[V^{x,\pi}(|x|) = 1] \leq s$$

PCPPs are the key building block for constructing IOPs of proximity.

HISTORY Arora, Lund, Motwani, Sudan, Szegedy (1992). *The PCP theorem was initially motivated by hardness of approximation, not cryptography. Its application to succinct proofs came later via Kilian (1992) and Micali (1994), eventually spawning the entire SNARK/STARK ecosystem.*

LIMITATIONS

- Classical PCPs have enormous proof length (polynomial blowup) — impractical without further compilation
- The constant in O(1) queries hides large constants; practical systems use IOPs instead
- PCP constructions are complex to implement; no production system uses raw PCPs directly

EXERCISES

1. **conceptual** — Why does the PCP theorem imply that NP-hard problems are hard to approximate? Sketch the connection between query complexity and approximation ratio.
2. **conceptual** — Explain why O(log n) random bits suffice for the PCP verifier. What does this imply about the number of possible query patterns?

IOP (Interactive Oracle Proofs)

INTUITIVE

A multi-round courtroom where each round the witness submits a sealed evidence box. The judge randomly opens a few drawers from any box submitted so far. The witness cannot change previously submitted boxes, but can adapt future submissions to the judge's questions.

KEY POINTS

- Combines IP interactivity with PCP oracle access — the best of both worlds
- Rounds k: number of prover messages (each is an oracle)
- Query complexity: total number of positions read across all oracles
- Public-coin IOPs: verifier messages are just random challenges (enables Fiat-Shamir)
- BCS transformation: IOP + Merkle hash tree = non-interactive argument in ROM
- Strictly more powerful than PCPs: interaction allows shorter proofs
- Every modern STARK and SNARK is compiled from an IOP at its core

WHY IT MATTERS

IOPs are the information-theoretic core of all modern proof systems. A STARK is an IOP compiled with Merkle trees (BCS transform). A SNARK like PLONK is a polynomial IOP compiled with KZG commitments. The IOP abstraction cleanly separates the "math" from the "crypto".

THESIS ANGLE

Every STARK/SNARK is compiled from an IOP. When you choose between Groth16, PLONK, or STARK for your credential verification on Sui, you are choosing which IOP to compile and with which commitment scheme. The IOP determines round structure, prover complexity, and soundness.

TECHNICAL

A k-round IOP: in round i, the prover sends an oracle f_i, and the verifier sends a challenge r_i. At the end, V makes q queries total across all oracles $f_1, \ldots, f_k$. IOPs combine the round structure of interactive proofs (IP) with the oracle access model of PCPs.

IOP PARAMETERS

Key parameters: round complexity k, query complexity q, soundness error ε, proof length $l(n)$.

Comparison of proof models:

- **IP:** k rounds of interaction, no oracle access — verifier reads full messages.
- **PCP:** 0 rounds of interaction, oracle access to static proof.
- **IOP:** k rounds of interaction + oracle access to each prover message — strictly generalizes both IP and PCP.

BCS TRANSFORMATION

IOP + collision-resistant hash $\rightarrow$ non-interactive argument in the Random Oracle Model (ROM):

1. Merkle-commit each oracle f_i to obtain root rt_i.
2. Apply Fiat-Shamir: $r_i = H(\mathrm{rt}_1, \ldots, \mathrm{rt}_i)$.
3. Proof includes Merkle authentication paths for queried positions.

Theorem (BCS): If the IOP has soundness ε, the compiled argument has soundness $\varepsilon + \mathrm{negl}(\lambda)$ in the ROM.

PUBLIC-COIN IOPS

A public-coin IOP is one where all verifier messages are uniformly random coins (no secret state). This property enables the Fiat-Shamir transform. Most efficient IOPs in practice (FRI, Aurora, STARK) are public-coin.

Arthur-Merlin characterization: $\text{public-coin IOP} \subseteq \text{IOP}$, but for constant-round IOPs with polynomial proof length, public-coin suffices (analogous to the GS theorem for IP).

HISTORY Ben-Sasson, Chiesa, Spooner (2016). *The IOP abstraction was "obvious in hindsight" — researchers had been building IOP-like protocols for years (FRI, Aurora) before the formal definition crystallized the concept.*

LIMITATIONS

- IOP is an information-theoretic model — it does not directly give a concrete protocol until compiled with a commitment scheme
- The BCS transformation requires the random oracle model; real hash functions may not perfectly instantiate it
- Multi-round IOPs increase the prover communication complexity linearly with rounds

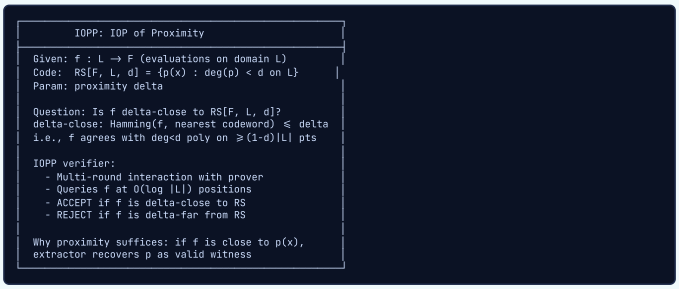
EXERCISES

1. **comparison** — Compare PCP, IP, and IOP along three axes: verifier access model, interaction, and known proof length. Why is IOP strictly more powerful?
2. **conceptual** — Explain the BCS transformation. How does it convert an IOP into a non-interactive argument?

IOPP (IOP of Proximity)

INTUITIVE

Testing if a painting is "close enough" to a masterpiece by examining random brushstrokes. You cannot inspect every pixel, but you can catch a forgery that differs in more than a few percent of strokes. Proximity is the key relaxation that makes sublinear verification possible.



KEY POINTS

- Proximity vs membership: tests if f is CLOSE to a codeword, not necessarily IN the code
- Reed-Solomon proximity: is f close to a low-degree polynomial?
- Relative Hamming distance: fraction of disagreeing positions with nearest codeword
- Proximity parameter delta: typically 1 - sqrt(rate)
- Why proximity suffices: close to valid polynomial => extraction works
- Composition: IOPP + polynomial IOP = full proof system
- FRI is the most efficient known IOPP for Reed-Solomon codes

WHY IT MATTERS

IOPP is the "type-checking" layer in STARK construction. The polynomial IOP assumes prover messages are low-degree polynomials; the IOPP actually verifies this assumption. Without IOPP, a cheating prover could send arbitrary functions that pass polynomial identity checks at queried points.

THESIS ANGLE

IOPP is the bottleneck component in STARKs. Its efficiency determines proof verification time and therefore gas costs on Sui. A better IOPP (like FRI with tight soundness bounds) directly reduces the cost of verifying your credential proofs on-chain.

TECHNICAL

An IOPP for code $C \subseteq \mathbb{F}^n$: given oracle access to $f : L \to \mathbb{F}$, the verifier tests whether f is δ-close to C in relative Hamming distance:

$$\Delta(f, C) = \min_{c \in C} \frac{|\{i : f(i) \neq c(i)\}|}{n}$$

PROXIMITY PARAMETER

The proximity parameter $\delta \in (0, 1 - R)$ where R is the code rate. Key radii:

- **Unique decoding radius:** $\delta < (1 - R)/2$ — at most one codeword within distance δ.
- **List decoding radius:** $\delta < 1 - \sqrt{R}$ (Johnson bound) — polynomially many codewords within distance δ.

The Johnson bound: for a code of distance d and block length n, the list decoding radius is $1 - \sqrt{1 - d/n} = 1 - \sqrt{R}$ for Reed-Solomon codes (MDS).

REED-SOLOMON IOPP

Test proximity to the Reed-Solomon code:

$$\mathrm{RS}[\mathbb{F}, L, d] = \{(p(\alpha))_{\alpha \in L} : \deg(p) < d\}$$

This is the set of evaluations of polynomials of degree $< d$ over the evaluation domain $L \subseteq \mathbb{F}$. Testing proximity to this code is the central problem that FRI solves.

SOUNDNESS ANALYSIS

Correlated agreement theorem: If f is δ-far from $\mathrm{RS}[\mathbb{F}, L, d]$, then the verifier rejects with probability:

$$\Pr[\text{reject}] \geq 1 - \varepsilon(\delta, |\mathbb{F}|, d, n)$$

The correlated agreement theorem shows that agreement of f with a low-degree polynomial on a random subspace implies global proximity — this is the key lemma enabling recursive proximity testing in FRI.

HISTORY Ben-Sasson, Bentov, Horesh, Riabzev (2017). *The relaxation from membership to proximity testing is what makes sublinear verification possible. Testing exact membership in RS requires reading the entire function; testing proximity only needs O(log n) queries.*

LIMITATIONS

- Proximity parameter delta must be chosen carefully — too small and soundness degrades, too large and completeness fails for "almost valid" proofs
- Current soundness analyses (e.g., for FRI) rely on complex combinatorial arguments that are still being refined (BCIKS 2020)
- IOPP guarantees proximity to the code, not exact membership — the extraction step adds a small soundness loss

EXERCISES

1. **calculation** — A Reed-Solomon code RS[F, L, d] has |L| = 1024 and d = 256. What is the rate? What is the relative minimum distance? If delta = 0.5, how many positions can f disagree with the nearest codeword and still be accepted?
2. **conceptual** — Why does proximity testing suffice for soundness in a STARK? Why not require exact membership?

FRI Protocol (Fast Reed-Solomon IOP of Proximity)

INTUITIVE

Testing if someone is a real pianist by repeatedly asking them to play "half the song." Each round, you fold the music in half and check consistency. After log(d) rounds, you are left with a single note that you can verify directly. A fake pianist cannot maintain consistency through all the folding steps.

KEY POINTS

- Recursive degree halving: f(x) = g(x^2) + x*h(x^2) where deg(g),deg(h) < deg(f)/2
- Domain squaring: L_{i+1} = {x^2 : x in L_i} — domain halves each round
- Each round halves BOTH degree AND domain size — log(d) rounds total
- Final check: is f_final a constant? (degree-0 = single field element)
- Query complexity: O(log d) — exponentially better than reading full function
- Soundness: each fold catches cheating w.p. >= (1 - rate); amplified over rounds
- FRI as polynomial commitment: commit to Merkle root of evals, open via FRI
- Batching: combine multiple polynomials with random linear combination

WHY IT MATTERS

FRI is the polynomial commitment scheme behind all STARKs. It provides transparent (no trusted setup) and post-quantum polynomial commitments using only hash functions. The BCS-compiled FRI gives the STARK verifier.

THESIS ANGLE

FRI is the polynomial commitment behind STARKs (transparent, post-quantum). If you use a STARK backend for credential proofs on Sui, FRI's efficiency directly impacts gas costs. The blowup factor (domain/degree ratio) controls the proof size vs. soundness tradeoff.

TECHNICAL

FRI tests proximity to $\mathrm{RS}[\mathbb{F}, L_0, d]$ via recursive folding. At round i: given f_i on domain L_i of degree $< d/2^i$, decompose:

$$f_i(x) = g(x^2) + x \cdot h(x^2)$$

Verifier sends α_i, prover sends $f_{i+1}(x) = g(x) + \alpha_i \cdot h(x)$ on $L_{i+1} = \{x^2 : x \in L_i\}$.

FRI FOLDING ALGEBRA

Split $f(x)$ into even and odd components:

$$f(x) = f_{\text{even}}(x^2) + x \cdot f_{\text{odd}}(x^2)$$

After folding with challenge α:

$$f'(y) = f_{\text{even}}(y) + \alpha \cdot f_{\text{odd}}(y)$$

Degree halves: $\deg(f') < \deg(f)/2$. Domain squaring: $L' = L^2 = \{x^2 : x \in L\}$, so $|L'| = |L|/2$.

FRI SOUNDNESS

Johnson bound application yields per-round soundness. Conjectured soundness (used in practice) vs proven soundness (BCIKS20):

$$\varepsilon \leq O\left(\frac{d}{|\mathbb{F}|}\right)^{1/2} \text{ per round} \times \text{number of rounds}$$

Batched FRI: test multiple polynomials simultaneously via random linear combination $f = \sum_j \beta_j f_j$. Soundness amplified by independent folding challenges across rounds.

FRI AS POLYNOMIAL COMMITMENT

Commit phase: Merkle tree of evaluations $\{f(\alpha)\}_{\alpha \in L}$.

Open phase: FRI folding + consistency checks between layers.

Batch opening: random linear combination of polynomials.

Comparison with KZG:

- FRI: transparent (no trusted setup), post-quantum, proof size $O(\log^2 n)$ field elements.
- KZG: smaller proofs (1 group element), but requires trusted setup and pairing-based assumptions (not post-quantum).

FRI VARIANTS

DEEP-FRI: out-of-domain sampling — verifier picks a point $z \notin L$ and checks $f(z)$ via quotient polynomial. Improves soundness from list-decoding to unique-decoding regime.

FRI with code switching: change rate ρ between rounds to optimize proof size vs soundness tradeoff.

ethSTARK optimizations: grinding (proof-of-work on first Fiat-Shamir challenge), optimized Merkle caps, and coset-based evaluation domains.

HISTORY Ben-Sasson, Bentov, Horesh, Riabzev (2018). *FRI was directly motivated by making STARKs practical. The name encodes its purpose: Fast Reed-Solomon IOP of Proximity. It reduced STARK proof generation from hours to seconds for realistic circuit sizes.*

LIMITATIONS

- Blowup factor: domain must be larger than degree by factor 1/rho (typically 4-16x), increasing proof size
- Field must have multiplicative subgroups of size 2^k (requires specific field choices like the Goldilocks field or BabyBear)
- FRI soundness proofs are non-trivial — tight bounds proved only recently (BCIKS 2020, conjectured bounds still used in practice)
- Proof size is O(log^2(d)) field elements — larger than KZG O(1) but avoids trusted setup

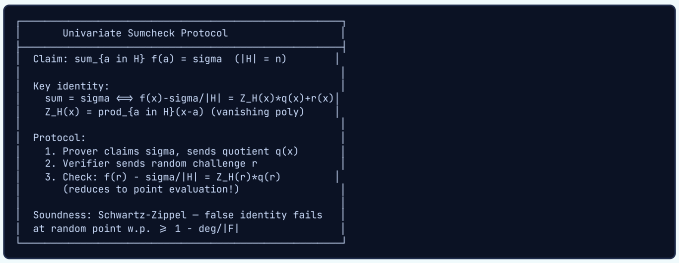
EXERCISES

1. **calculation** — Walk through one FRI fold for f(x) = 3x^3 + 2x^2 + x + 5 with random challenge alpha = 2. Decompose f into even/odd parts and compute f1.
2. **conceptual** — Why does FRI require the field to have large multiplicative subgroups of order 2^k? What happens if the subgroup is not large enough?

Univariate Sumcheck & Functional IOP

INTUITIVE

Instead of checking every cell in a spreadsheet, verifying that the SUM formula is correct by evaluating a random weighted combination. If the sum claimed is wrong, a random challenge will catch the discrepancy with high probability over a large field.



KEY POINTS

- Univariate sumcheck: prove sum_{a in H} f(a) = sigma by reducing to point evaluation
- Vanishing polynomial Z_H(x) = prod_{a in H}(x-a): zero on all of H
- Zero-on-H testing: f vanishes on H iff Z_H divides f
- Functional oracle: evaluates linear functions of the proof (generalizes point queries)
- Multivariate vs univariate: LFKN is multivariate; Aurora/Marlin use univariate variant
- Schwartz-Zippel: non-zero poly of degree d has at most d roots over F (soundness basis)

WHY IT MATTERS

Sumcheck is the "engine" inside many modern SNARKs. Aurora, Marlin, and Fractal use univariate sumcheck to reduce constraint satisfaction to polynomial evaluations. Spartan and HyperPlonk use multivariate sumcheck directly. The GKR protocol chains sumchecks for delegated computation.

THESIS ANGLE

Sumcheck helps evaluate which proof system fits Sui's verification model. Systems using sumcheck (Spartan, HyperPlonk) have different tradeoffs than systems using polynomial commitments directly (PLONK). For your credential circuit, the sumcheck-based approach may offer better prover time at the cost of slightly larger proofs.

TECHNICAL

Functional IOP: the prover sends a functional oracle F that the verifier can evaluate on linear functions (not just pointwise).

Sumcheck protocol: prove $H = \sum_{x \in \mathcal{H}^m} f(x)$ via m rounds of degree reduction.

UNIVARIATE SUMCHECK

Claim: $\sum_{a \in H} f(a) = \sigma$. This is equivalent to:

$$Z_H(x) \mid \left(f(x) - \frac{\sigma}{|H|} \right)$$

where $Z_H(x) = \prod_{a \in H}(x - a)$ is the vanishing polynomial. The verifier checks at random r:

$$f(r) = h(r) \cdot Z_H(r) + \frac{\sigma}{|H|}$$

where $h(x)$ is the quotient polynomial sent by the prover.

MULTIVARIATE SUMCHECK

m rounds, each fixing one variable. In round i:

1. Prover sends univariate $g_i(x_i) = \sum_{x_{i+1}, \ldots, x_m \in \{0,1\}} f(r_1, \ldots, r_{i-1}, x_i, \ldots, x_m)$.
2. Verifier checks $g_i(0) + g_i(1) =$ previous claim.
3. Verifier sends random r_i, new claim $= g_i(r_i)$.

Final check: query $f(r_1, \ldots, r_m)$ directly. Total communication: $O(m \cdot d)$ field elements for degree-d polynomial.

AURORA/MARLIN/FRACTAL PARADIGM

Encode R1CS constraints as univariate sumcheck over evaluation domain H. The approach uses:

- **Rowcheck:** verify that a polynomial vanishes on H — i.e., $Z_H(x) \mid p(x)$.
- **Colcheck:** verify a column relation via rational constraints and the sumcheck over H.

This paradigm enables succinct verification of R1CS satisfiability with $O(\log n)$ verifier time after a polynomial commitment setup.

HISTORY Lund, Fortnow, Karloff, Nisan (1992). *The sumcheck protocol is arguably the most important "trick" in complexity theory and proof systems. Almost every interactive proof or argument system uses some form of sumcheck at its core.*

LIMITATIONS

- Univariate sumcheck requires the evaluation domain H to be a structured subgroup (roots of unity) for efficient Z_H computation
- Soundness error is deg(f)/|F| per round — requires large fields for negligible error
- The quotient polynomial q(x) has high degree, requiring the prover to commit to it (adds communication)

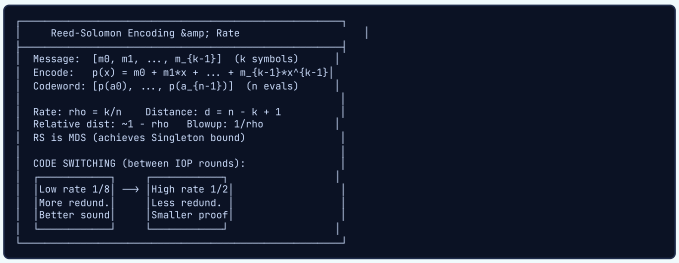
EXERCISES

1. **calculation** — Let H = {1, w, w^2, w^3} be 4th roots of unity in F_17 (w = 4). Compute Z_H(x) = x^4 - 1 evaluated at a random point r = 3. If f(x) = x^2 and sigma = sum_{a in H} f(a), find sigma and verify the sumcheck identity.
2. **conceptual** — Explain why the vanishing polynomial Z_H(x) is central to both zero-testing and sumcheck in SNARKs. How does it encode constraint satisfaction?

Code Switching & Rate of the Code

INTUITIVE

Choosing between a thick textbook (low rate: lots of redundancy, easy to spot errors) and a compressed summary (high rate: less redundancy, harder to verify but shorter). In STARKs, you pick a redundancy level that balances proof size against soundness guarantees.



KEY POINTS

- Linear codes: generator matrix G maps messages to codewords (C = m*G)
- RS codes = polynomial evaluations: k coefficients encoded as n point evaluations
- Rate rho = k/n: higher rate = more efficient but less redundancy
- MDS: RS achieves Singleton bound d = n - k + 1 exactly
- Code switching: changing rate between IOP rounds (trade soundness vs proof size)
- Blowup factor = 1/rho: STARK with blowup 4 has 4x evaluation domain vs degree
- Interleaved codes: stack multiple codewords, test together for amortized efficiency

WHY IT MATTERS

The rate of the Reed-Solomon code directly determines STARK proof size and soundness. FRI operates over RS codes; its blowup factor is 1/rho. Plonky2/3 use aggressive parameters (blowup 2-4) for efficiency. Code switching between rounds allows optimizing each stage independently.

THESIS ANGLE

The blowup factor directly affects proof size in STARKs. A blowup of 4 (rate 1/4) vs 8 (rate 1/8) doubles the proof size but improves soundness. Plonky2/3 use smaller blowup for efficiency — relevant if generating proofs on mobile for your credential system where bandwidth and storage are constrained.

TECHNICAL

Reed-Solomon code: $\mathrm{RS}[\mathbb{F}, L, k] = \{(p(\alpha))_{\alpha \in L} : \deg(p) < k\}$ with rate $\rho = k/|L|$ and minimum distance $d = |L| - k + 1$. Code switching refers to changing ρ between IOP rounds.

REED-SOLOMON PARAMETERS

Rate $\rho = k/n$, relative distance $\delta = 1 - \rho$ (MDS property: RS codes achieve the Singleton bound with equality). Blowup factor $\beta = 1/\rho$.

For FRI: domain L must be a multiplicative subgroup of $\mathbb{F}$, with $|L| = 2^l$ for binary folding (each squaring halves the domain).

CODE SWITCHING IN IOPS

Tradeoff between rate and soundness:

- **High rate** ($\rho \to 1$): small proofs but weak per-round soundness (distance $\delta \to 0$).
- **Low rate** ($\rho \to 0$): large proofs but strong per-round soundness (distance $\delta \to 1$).

Strategy: start with low rate for strong base soundness, switch to high rate in later rounds where soundness is amplified by composition across rounds.

INTERLEAVED CODES & BATCHING

Stack m codewords row-wise to form an interleaved code. Test proximity of all m codewords simultaneously using a single random linear combination:

$$f = \sum_{j=1}^{m} \beta_j \cdot f_j$$

Proximity of the interleaved code $\approx$ proximity of individual codes (with high probability over β_j). Enables batch FRI for multiple polynomial commitments with sublinear overhead.

HISTORY Reed and Solomon (1960). *RS codes were originally designed for satellite communications and are used in CDs, DVDs, QR codes, and deep-space probes. Their reuse in ZK proofs is one of the most elegant cross-pollinations between coding theory and cryptography.*

LIMITATIONS

- Lower rate means larger proofs: blowup factor 8 (rate 1/8) gives 8x overhead in evaluation domain size
- RS codes require the field size to exceed the codeword length (|F| > n), constraining field choices
- Code switching adds complexity to soundness analysis — must account for rate changes between rounds
- Interleaved testing provides amortized (not worst-case) soundness — rare pathological cases exist

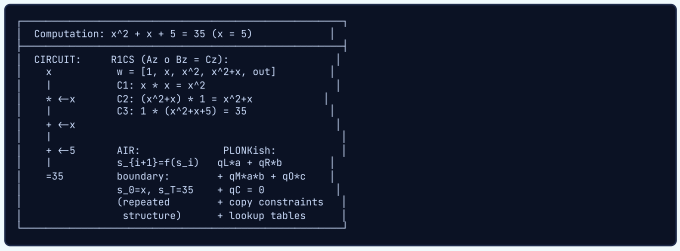
EXERCISES

1. **calculation** — Compare two STARK configurations: (A) blowup factor 4 (rate 1/4) and (B) blowup factor 8 (rate 1/8). For a degree-1024 polynomial, what is the evaluation domain size and relative distance for each?
2. **conceptual** — Why is Reed-Solomon an MDS code, and why does this property matter for proof systems?

Arithmetic Circuits & Constraint Systems

INTUITIVE

Converting a cooking recipe into a factory assembly line. Each step becomes a labeled gate with specific wiring. Different factory layouts (R1CS, AIR, PLONKish) suit different recipes — a repetitive recipe works best on a conveyor belt (AIR), while a complex one-off dish needs a flexible kitchen (PLONKish).



KEY POINTS

- Arithmetic circuits: add/multiply gates over finite field F
- R1CS: Az o Bz = Cz — Groth16/Spartan; degree-2 only
- AIR: transition + boundary constraints — STARKs; great for repeated structures
- PLONKish: custom gates + copy constraints + lookups — Halo2/PLONK; most flexible
- SHA-256 in R1CS: ~22k constraints (bitwise ops expensive over arithmetic fields)
- Choice of constraint system determines which proof system you can use downstream

WHY IT MATTERS

Constraint systems are the "assembly language" of proof systems. Your computation must be expressed in one of these formats before any proof can be generated. The choice of arithmetization cascades through the entire pipeline: R1CS leads to Groth16, AIR leads to STARKs, PLONKish leads to PLONK/Halo2.

THESIS ANGLE

Your BBS+ credential verification will be expressed as a circuit. R1CS (for Groth16) gives the smallest proofs (192 bytes) ideal for Sui gas costs. AIR (for STARKs) gives transparent setup but larger proofs. PLONKish (for PLONK) gives universal trusted setup with medium proofs. The choice cascades through your entire system design.

TECHNICAL

An arithmetic circuit C over field $\mathbb{F}$ is a DAG with input gates, constant gates, addition gates ($+$), and multiplication gates ($\times$). Size = number of gates, depth = longest input-to-output path.

R1CS (RANK-1 CONSTRAINT SYSTEM)

System of constraints:

$$(A \cdot z) \circ (B \cdot z) = C \cdot z$$

where $A, B, C \in \mathbb{F}^{m \times n}$, $z = (1, x, w)$ is the extended witness, and $\circ$ denotes the Hadamard (entry-wise) product. Each constraint is degree-2:

$$\left(\sum_i a_i z_i\right)\left(\sum_i b_i z_i\right) = \sum_i c_i z_i$$

Expressiveness: any fan-in-2 arithmetic circuit maps to R1CS with approximately the same number of constraints as multiplication gates.

AIR (ALGEBRAIC INTERMEDIATE REPRESENTATION)

Execution trace matrix $T \in \mathbb{F}^{N \times w}$ with:

- **Transition constraints:** $p(T[i], T[i+1]) = 0$ for all $i \in [N-1]$.
- **Boundary constraints:** $T[0, j] = v_j$ for specified positions.

Degree of transition polynomial d_t determines prover complexity. AIR is natural for iterated computations (hash functions, VM execution) where the same transition relation repeats N times.

PLONKISH ARITHMETIZATION

Gate equation with selector polynomials:

$$q_L \cdot a + q_R \cdot b + q_O \cdot c + q_M \cdot (a \cdot b) + q_C = 0$$

Additional features:

- **Copy constraints:** permutation argument (grand product check) ensures wire consistency across gates.
- **Lookup tables:** Plookup protocol for range checks and non-arithmetic operations.
- **Custom gates:** degree-d constraints for specialized operations (e.g., Poseidon S-box as degree-5 gate).

Most flexible: subsumes both R1CS and AIR as special cases.

COMPARISON TABLE

- **R1CS:** degree 2, simple structure, used by Groth16 and Spartan.
- **AIR:** arbitrary degree, good for repetitive structure, used by STARKs.
- **PLONKish:** configurable degree, custom gates + lookups, used by PLONK/Halo2.

Choice depends on computation structure: R1CS for general circuits, AIR for iterative computations, PLONKish for maximum flexibility with domain-specific optimizations.

HISTORY Gennaro, Gentry, Parno, Raykova (R1CS); Ben-Sasson et al. (AIR); Gabizon, Williamson, Ciobotaru (PLONKish) (2012). *The name "arithmetization" comes from complexity theory where Boolean circuits are converted to arithmetic form. In practice, the biggest engineering challenge is expressing non-arithmetic operations (comparisons, bit manipulation) efficiently in field arithmetic.*

LIMITATIONS

- R1CS is limited to degree-2 constraints; expressing higher-degree operations requires auxiliary variables (quadratic blowup)
- AIR requires the computation to have regular structure (uniform transition function); irregular computations need padding
- PLONKish custom gates require careful design — poorly designed gates can increase prover time without reducing constraint count
- All systems incur overhead for non-arithmetic operations: comparisons need O(log p) constraints, bitwise ops need bit decomposition

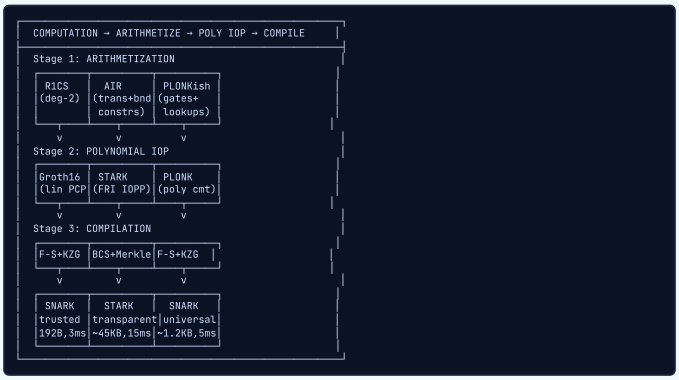
EXERCISES

1. **design** — Express the computation "prove I know x such that x^3 + x + 5 = y" in R1CS. How many constraints and variables do you need? Then sketch how it would look in AIR.
2. **comparison** — Compare the constraint count for SHA-256 in R1CS vs PLONKish with lookup tables. Why do lookup tables dramatically reduce the constraint count?

The Full Pipeline — Computation to SNARK

INTUITIVE

Manufacturing a diamond from raw carbon. Each stage compresses and transforms while preserving the essential truth. Raw computation is bulky and unstructured; the pipeline refines it into a tiny, verifiable gem that anyone can inspect in milliseconds.



KEY POINTS

- Three stages: arithmetization, polynomial IOP, compilation (make non-interactive)
- Three compilers: Kilian/Micali (PCP), BCS (IOP), polynomial commitment (poly-IOP)
- Setup: trusted (Groth16), universal (PLONK), transparent (STARK)
- Proof size: Groth16 ~192B, PLONK ~1.2KB, STARK ~45KB
- Post-quantum: only STARK path (hash-based, no elliptic curves)
- Recursion: wrap one proof inside another (STARK inside Groth16 for small on-chain proof)
- No single path dominates all metrics — design requires tradeoffs

WHY IT MATTERS

This pipeline is the master map connecting all concepts in this chapter. PCP/IOP provides the information-theoretic foundation, FRI/IOPP the polynomial commitment, constraint systems the arithmetization.

THESIS ANGLE

Master map for thesis decisions. Sui gas cost favors small proofs (Groth16), no trusted setup on-chain favors STARK/PLONK, mobile prover favors fast systems. Practical approach: STARK proofs (transparent, fast) wrapped in Groth16 (small) for on-chain verification — recursion gives best of both worlds.

TECHNICAL

A SNARK for NP language L is a triple $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ satisfying: **completeness** (honest prover always convinces), **(knowledge) soundness** (no efficient cheating prover), and **succinctness** (proof size and verification time are sublinear in witness size $|w|$).

THREE-STAGE CONSTRUCTION

The modular SNARK construction pipeline:

1. **Stage 1 — Arithmetization:** computation $\to$ constraint system (R1CS, AIR, or PLONKish).
2. **Stage 2 — Information-theoretic proof:** constraint system $\to$ IOP/PCP (polynomial IOP with ideal oracles).
3. **Stage 3 — Compilation:** IOP $\to$ argument via commitment scheme + Fiat-Shamir (replace oracles with commitments).

COMPILER FAMILIES

Three main compiler strategies:

1. **PCP + Kilian/Micali:** succinct argument using collision-resistant hash functions.
2. **IOP + BCS:** transparent argument via Merkle commitments + Fiat-Shamir. No trusted setup, post-quantum.
3. **Polynomial IOP + polynomial commitment:** (zk-)SNARK using KZG, IPA, or FRI as the commitment backend.

PROPERTY COMPARISON

- **Groth16:** 3 $\mathbb{G}_1$ + 1 $\mathbb{G}_2$ elements (192 bytes), trusted setup (per-circuit), not post-quantum.
- **STARK:** $O(\log^2 n)$ field elements (~45 KB), transparent, post-quantum (hash-based).
- **PLONK (KZG):** 9–15 $\mathbb{G}_1$ elements (~1.2 KB), universal trusted setup, not post-quantum.
- **Recursive SNARKs (Nova/Supernova):** fold instances incrementally, amortize verification cost to $O(1)$ per step.

HISTORY Kilian (1992), Micali (1994), Groth (2016), Ben-Sasson et al. (2018) (2016). *The field went from theory to billions in production in ~5 years (2017-2022). Zcash, StarkNet, zkSync, Scroll all deploy different paths through this pipeline.*

LIMITATIONS

- No single path dominates: smallest proofs need trusted setup, transparent proofs are large
- Recursion adds prover overhead: verifying STARK inside Groth16 circuit is expensive
- Pipeline complexity: bugs can hide at any stage (arithmetization, soundness, field arithmetic)
- Trusted setup ceremonies are logistically complex and a single point of failure

EXERCISES

1. **design** — For your thesis credential system on Sui, trace "prove knowledge of BBS+ credential" through each pipeline variant. Consider: proof size (affects gas), setup requirement, prover time (mobile device), and post-quantum security. Which path would you choose?
2. **comparison** — Compare the three compilation strategies (Fiat-Shamir+KZG, BCS+Merkle, Fiat-Shamir+KZG for PLONK). What security model does each assume, and what are the post-quantum implications?